

ESP32 EN EL AULA

ACTIVIDADES PARA REALIZAR CON EL KIT ESP32+ TDRSTEAM

- [Introducción](#)
 - [Objetivos y Contenidos](#)
 - [Kit ESP32 en el Aula](#)
 - [Pensamiento computacional](#)
 - [¿Qué es ESP32?](#)
 - [Hardware ESP32 Plus STEAMakers](#)
 - [Sensores](#)
 - [Actuadores y otras salidas](#)
 - [Sensores y actuadores de la placa Imagina TDR STEAM](#)
 - [Sensores en la placa ESP32 plus STEAMakers](#)
 - [Software](#)
- [STEAMAKERBLOCKS](#)
 - [¿Qué es Steamakerblocks?](#)
 - [Crear cuenta en STEAMAKERBLOCKS](#)
 - [Cuentas alumnos](#)
 - [Connector](#)
 - [Empezando un proyecto](#)
- [Prácticas con sensores digitales](#)
 - [Actividad-01. LED](#)
 - [Actividad-02. LEDs rojo y azul](#)
 - [Actividad-03. Parpadeo intermitente. Multitarea](#)
 - [Actividad-04. LED RGB](#)

- [Actividad-05. Zumbador](#)
- [Actividad-06. Pulsadores](#)
- [Prácticas con sensores analógicos](#)
 - [Actividad-07. El potenciómetro y Plotter](#)
 - [Actividad-08. La fotorresistencia o LDR](#)
 - [Actividad-09. El sensor de temperatura LM35D](#)
 - [Actividad-10. El sensor de temperatura y humedad DHT-11](#)
 - [Actividad-11. Emisor y receptor de infrarrojos](#)
- [Prácticas con otros elementos del kit](#)
 - [Actividad 12 Puerto de expansión I2C. La LCD de 2x16](#)
 - [Actividad 13 Sensor de sonido o micrófono](#)
 - [Actividad 14 Sensores internos de ESP32](#)
 - [Avanzado: Multitarea, interrupciones, memoria Flash y tarjeta SD](#)
- [Bluetooth](#)
 - [Un poco de teoría Bluetooth](#)
 - [APP Serial Bluetooth Terminal](#)
 - [Actividad 15 APP->ESP32APP Serial Bluetooth Terminal](#)
 - [ACTIVIDAD 15 con otra APP: Arduino Bluetooth Control](#)
 - [Actividad 16 APP<->ESP32 Enunciado y programa STEAMAKERBLOCKS](#)
 - [Actividad 16 con la APP Serial Bluetooth Terminal](#)
 - [Actividad 16 con APP INVENTOR](#)
- [Que es IoT](#)
- [IoT-Wifi-Telegram](#)
 - [Crear bot en Telegram](#)
 - [Encontrar tu ID en Telegram](#)
 - [Código para enviar y recibir](#)
 - [Actividad 17 Telegram](#)

- [IoT-Wifi-Blynk IoT](#)
 - [¿Qué es Blynk.io?](#)
 - [Crear DASHBOARD en Blynk.io](#)
 - [Crear programa en STEAMAKERBLOCKS Actividad18 Blynk IoT](#)

- [Avanzado: Blynk Legacy local](#)
 - [Raspberry: Qué hay que hacer para empezar](#)
 - [Raspberry: Poner un Sistema Operativo](#)
 - [Raspberry: Cambiar usuario y contraseña](#)
 - [Raspberry: Conectarlo a la red](#)
 - [Raspberry: Conexión SSH](#)
 - [Raspberry: Apagar](#)
 - [Raspberry: VNC](#)
 - [Raspberry: Instalar Java 8](#)
 - [Raspberry: Instalar y ejecutar el servidor Blynk local](#)
 - [En Windows Instalar Java 11](#)
 - [En Windows Instalar Blynk local](#)
 - [En Windows Ejecutar Blynkserver](#)
 - [En Linux: Insyalar Blynk Legay local](#)
 - [Blynk Legacy: La APP](#)
 - [Blynk Legacy: Crear cuentas](#)
 - [Blynk Legacy: El panel de control](#)
 - [Blynk Legacy: En la APP](#)
 - [Blynk Legacy: Poner el TOKEN en Steamakerblocks](#)
 - [Blynk Legacy: 1programa: Encender los pines rojo y verde](#)
 - [Blynk Legacy: 2programa RGB y el potenciómetro](#)
 - [Blynk Legacy: 3programa Medir la luz del LDR](#)
 - [Blynk Legacy: 4programa: Medir T y H con el DHT11](#)
 - [Blynk Legacy: 5programa: Lectura sensor LM35 y receptor IR](#)
 - [Blynk Legacy: 6programa Leer eventos del TDR Steam](#)



- [Blynk Legacy: 7programa: El timbre](#)
- [Blynk Legacy: ¿Todo junto?](#)

- [Muro](#)
- [Para saber más ...](#)
- [Créditos](#)

Introducción

Introducción

Objetivos y Contenidos

Objetivos

- Capacidad de desarrollar retos **STEAM** con un amplio nivel educativo: primaria hasta secundaria
 - Retos **STEAM** de nivel principiante con manejo de sensores
 - Retos **STEAM** de nivel medio adentrándose en las comunicaciones Bluetooth y wifi y almacenamiento de datos
 - Retos **STEAM** de nivel avanzado con prácticas IoT
- Realizar retos STEAM en un entorno **ARDUINOBLOCKS** amigable, sin complicaciones técnicas, multiplataforma y con lenguaje gráfico por bloques
 - Conocer **ARDUINOBLOCKS** una plataforma web que permite programar por bloques gráficos
 - Conocer las diferentes posibilidades de la programación por bloques de **ARDUINOBLOCKS**
 - Conocer las posibilidades didácticas de **ARDUINOBLOCKS** y su gestión de proyectos en el aula
- Conocer el kit robótico **TDR STEAM ESP32** que se propone, con un coste económico bajo pero de calidad para que un centro educativo lo pueda asumir para su aplicación en el aula (mínimo 12 por aula)
 - Conocer la placa **TDR STEAM ESP32**, sus sensores y actuadores
 - Aplicaciones prácticas del **TDR STEAM ESP32**

Contenidos

- **PLACA ESP32**
 - Placa ESP32, hardware y software
 - Sensores
 - Actuadores
 - Shield TDR Steam
- **ArduinoBlocks**
 - **Retos**
 - **Telecomunicaciones**
 - **Bluetooth**
 - **IOT -WIFI**
 - MQTT
 - BLYNK IOT



- BLYNK LEGACY
 - Blynk en una raspberry local

Introducción

Kit ESP32 en el Aula

Se ha buscado un kit entre todos los comerciales que cumpla los siguientes objetivos :

- Que sea fácil de utilizar en el aula, sin necesidad de utilizar excesivo cableado, con el objetivo de realizar el máximo número de retos.
- Utilizar un lenguaje de programación por bloques fácil de desarrollar y sin complejidades técnicas multiplataforma.
- Con buena relación calidad/precio.
- Con un amplio abanico de prácticas y niveles educativos, desde primaria hasta secundaria.
- Facilidad de crear actividades en entorno IoT

El equipo pedagógico de CATEDU ha elegido para este curso el [Kit TDR STEAM ESP32 de Innovadidactic](#), con el objetivo de alcanzar prácticas más avanzadas dentro del mundo del Internet de las cosas IoT.

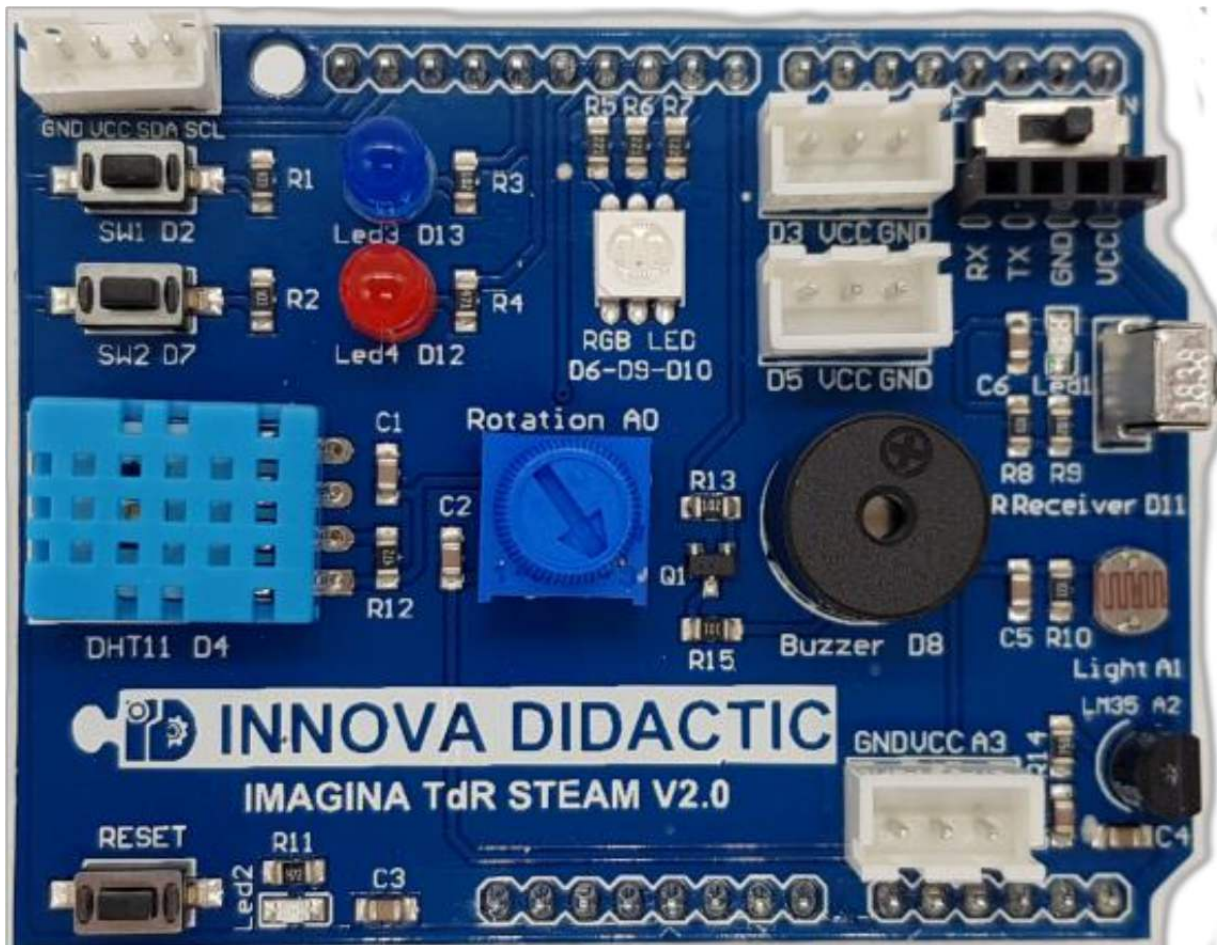
El kit esta pensado **para utilizar dentro del aula** y se ha establecido un número de 12 grupos, consideramos que es un número óptimo para llegar a la ratio del aula y que el profesor llegue a atender correctamente a todos los grupos.



Partes del kit Imagina TdR STEAM

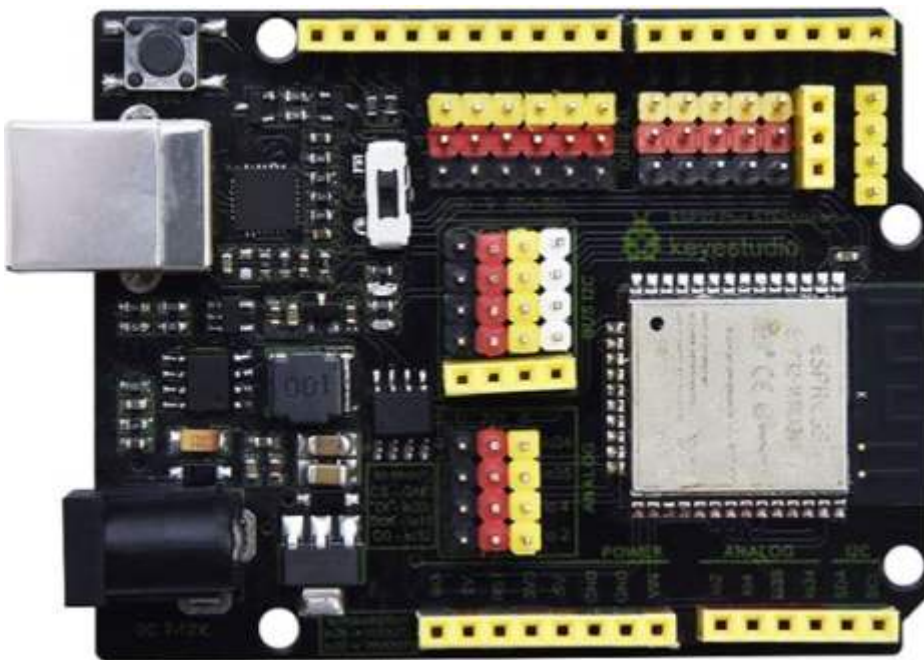
Este apartado está extraído de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Una placa Imagina TdR STEAM versión 2.0 como la de la imagen siguiente:



Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Una placa ESP32 Plus STEAMakers.



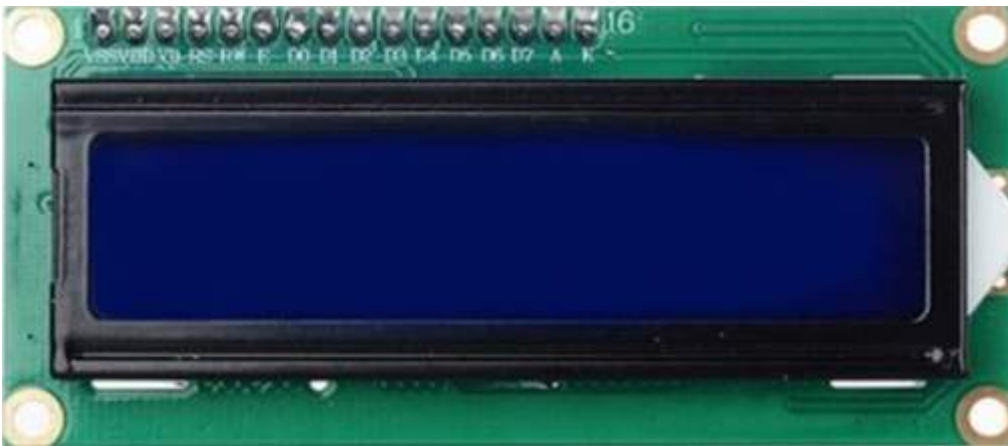
Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Mando de control remoto por infrarrojos para utilizar en conjunto con el sensor de infrarrojos integrado en la placa TdR-STEAM.



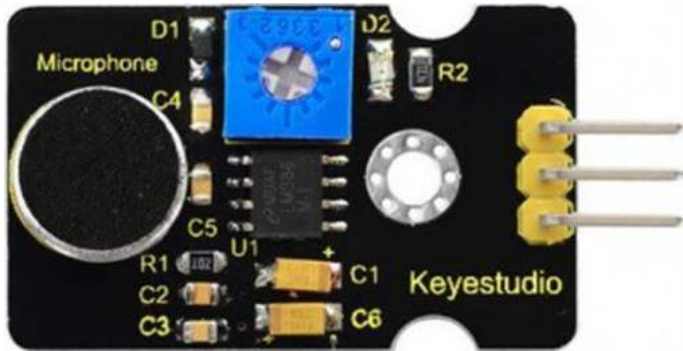
Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Pantalla LCD 1602 IIC (I2C) de 2 línea de 16 caracteres. Nos va a permitir mostrar mensajes de texto e irá conectada al conector I2C de la placa TdR-STEAM.



Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Sensor de sonido analógico (con potenciómetro). Se utiliza básicamente para detectar el nivel sonoro ambiental. El potenciómetro permite ajustar el nivel de ganancia.



Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Todo el kit se presenta en una caja de plástico, pero una opción práctica para no abrir y cerrar tantas veces la caja, es utilizar una caja impresa 3D, las partes impresas e instrucciones se pueden conseguir en <https://fgcoca.github.io/ESP32-STEAMakers/necesidades/> el resultado es :



Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Introducción

Pensamiento computacional

¿Dónde se encaja este robot? ¿Se puede comparar este robot con otros robots de otros cursos que hacemos desde CATEDU?

Esta es la hoja de ruta que proponemos, no se tiene que tomar al pie de la letra, pero intenta ayudar al profesorado que tenga una visión global de tanta oferta robótica:

Como se puede ver **ESP32 EN EL AULA** tiene la ventaja de tener un precio razonable, y dentro del rango de programación en bloques y de prácticas avanzadas IoT con la ventaja que es un kit muy amigable sin apenas conexiones y con muchas posibilidades.

Guía orientativa

https://docs.google.com/presentation/d/e/2PACX-1vQHIZvv1cGHet7eXVy-QcECY4Lj0k0I7ntDi8MevRWHQX-9myA0bfR5IofMeuGZkWD0Hw-Ob-MGoco_/embed?start=trueloop=true&delayms=3000

Tenemos un **grupo Telegram Robótica Educativa en Aragón**,
<https://t.me/roboticaeducativaaragon>



Introducción

¿Qué es ESP32?

Primero : ¿Qué es Arduino?

Arduino es una tarjeta electrónica que integra básicamente a un microcontrolador y un conjunto de pines de conexión de entradas y salidas que permiten, mediante un determinado programa, interaccionar con el medio físico mediante sensores y actuadores electrónicos. De esta forma podrás crear tus propios proyectos tecnológicos, dotarlos de sensores que detecten magnitudes físicas como luz, calor, fuerza, etc... y en base a esa información, escribiendo un programa, activar otros dispositivos (actuadores) como pequeñas bombillas, ledes, servomotores, pequeños motores DC, relés, etc... Los sensores se conectan a los pines de entrada y los actuadores a los de salida.

¿Sabías que.... ? Uno de los co-creadores de Arduino es Español, de Zaragoza: **David Cuartielles** [+info](#)

Segundo ¿Qué es un microcontrolador?

Es un circuito integrado que se puede programar, o sea que puede ejecutar las órdenes que tenga almacenadas en su memoria. Tiene las tres funciones principales de un computador: la unidad central de proceso, memoria y entradas y salidas.

Arduino utiliza la marca ATMEL, y el modelo de microcontrolador depende del tipo de tarjeta, por ejemplo la tarjeta Arduino Uno utiliza el micro ATMEL MEGA 328P. Si quieres saber las entrañas de esta placa [aquí](#)

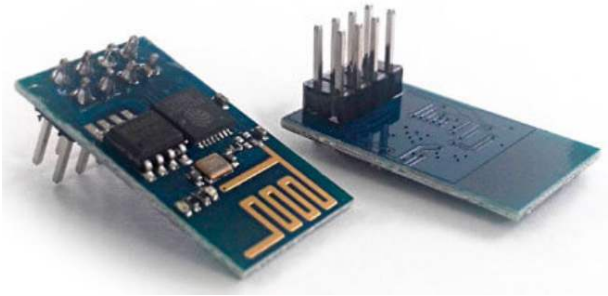
Tercero ¿Arduino tiene wifi?

Arduino (el clásico) no tiene wifi, y es importante esto para conseguir hacer prácticas IoT. Hay shields que permiten una conexión Ethernet o Wifi pero resultan caras. Otra opción es utilizar la versión del Arduino MKR1000 pero también resulta cara. A partir del 2023 lanzaron Arduino R4 Wifi pero realmente era un ESP32. Lo mejor es utilizar el ESP8266 para que a través de él nuestro Arduino pueda volcar o recibir datos a través de una Wifi.

Resumiendo brevemente, el **ESP8266** es un chip microcontrolador, es decir, no es un sensor, no es un actuador, no es una entrada/salida del Arduino sino que es, igual que el Arduino, es una placa electrónica montado en un módulo que tiene un microcontrolador (Tensilica L106 de 32bits) capaz de hacer cosas pero que tiene una característica importante: **Que tiene Wifi**, pero no lo

veas como un módulo Wifi para Arduino, sino como una placa electrónica completa, como el Arduino, incluso es su competencia.

El chip **ESP8266** se vende montado en un módulo, el más vendido es el **ESP01** y es el que se proporciona en el kit del curso **Arduinoblocks en el aula** de CATEDU.



Fuente Luis Llamas CC-BY-NC-SA [ver](#)

Por eso se habla indistintamente ESP8266 o el ESP01

Su bajo precio y su software libre permitió al mundo maker utilizar este chip. No sólo se puede utilizar para que el Arduino tenga acceso a Internet, sino también se han desarrollado multitud de módulos con el ESP8266, como veremos más adelante, el más interesante en su evolución es el módulo ESP-12E o el ESP32.

Pero sigamos con el ESP8266 montado en el módulo ESP01. Tiene unas pegas... no se diseñó para montarlo en el Arduino : **PRIMERO** La alimentación es 3V a 3.6V con picos de 200mA por lo que no puede conectarse directamente a la alimentación 3.3V y 50mA de Arduino.

SEGUNDO Consecuencia del máximo de 3.6V es que las entradas y salidas del ESP8266 no conviene conectarlas directamente a las entradas y salidas del Arduino que van a 5V.

Cuarto: ¿El ESP32 es?

Es una placa sucesora del ESP8266 de bajo coste y consumo que tiene el microprocesador Tensilica 32bits (variante el Xtensa LX6) que tiene Wifi y Bluetooth integrados. Como dice Luis Llamas en <https://www.luisllamas.es/esp32/> el ESP32 es el hermano mayor del ESP8266 con Wifi y Bluetooth.



Módulo ESP32 Fuente Luis Llamas CC-BY-NC-SA <https://www.luisllamas.es/esp32/>

Es posible programarlo usando código IDE de Arduino, pero también se puede programar con MicroPython.

Su principal característica es su potencial de uso en aplicaciones IoT

- Para saber más
 - Consideraciones sobre ESP32 <https://fgcoca.github.io/ESP32-STEAMakers/conesp32/>

“ Evolucion de búsquedas en Google de Arduino, Raspberry PI, ESP32 y MicroBit

- Arduino y Rpi tuvieron máximo en 2017-18. Desde entonces bajan.
- Arduino aguanta tipo
- Rpi cuesta abajo sin frenos
- ESP32 despegue
- Microbit no, pese a que tuvo buen arranque (inmerecido?)

pic.twitter.com/A2PI18qecZ

— Luis Llamas (@LuisLlamas) [August 9, 2023](#)

Introducción

Hardware ESP32 Plus STEAMakers

Esta placa está basado en el ESP32 que hemos visto que tiene :

- Wifi integrado
- Bluetooth integrado
- Puede trabajar con el código IDE de Arduino

Pero, esta [versión de Innova Didactic](#) además incorpora :

- Zócalo para tarjetas microSD para almacenamiento de datos.
- Conexiones iguales que cualquier tarjeta Arduino UNO

Esto permite usar cualquier Shield de Arduino, como la TDR STEAM Imagina de este curso.
Para saber las diferentes Shields de Arduino mira esta página

<https://libros.catedu.es/books/programa-arduino-mediante-codigo/page/hardware>

En la figura ¿Puedes localizar el ESP32 famoso?

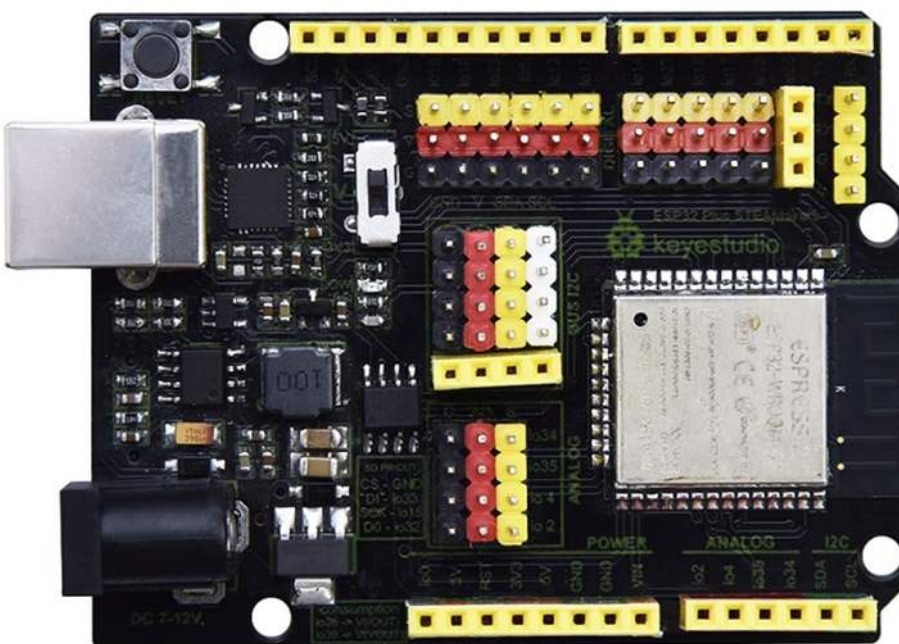


Imagen Fuente Innovadidactic

El contenido mostrado a continuación es de [la web de Federico Coca](#) Licencia CC-BY-SA

La placa ESP32 Plus STEAMakers nos ofrece una gran cantidad de prestaciones al estar basada en un microcontrolador de 32 bits con conectividad WiFi y Bluetooth integradas en la propia placa y también un zócalo para tarjetas μ SD para el almacenamiento de datos. También dispone de conexiones para todas las entradas y salidas con posibilidad de tener la alimentación adjunta y puertos de expansión I2C para poder conectar diferentes dispositivos directamente en la placa.

La placa está basada en el microcontrolador ESP32-WROOM-32 y sus principales especificaciones técnicas son:

- Microcontrolador Tensilica Xtensa 32-bit LX6 a 160MHz.
- Conectividad WiFi 802.11 b/g/n/e/i.
- Conectividad Bluetooth 4.2 y modo BLE.
- Zócalo para tarjetas μ SD.
- 14 entradas y salidas digitales con alimentación.
- Conector serie hembra con alimentación.
- Conector I2C para conectar hasta 5 dispositivos a la vez sobre la misma placa.
- Conector hembra I2C para conexión de una pantalla OLED.
- Botón de Reset.
- Conector de 5V
- Conector de 3.3V
- Interruptor 3.3-5V para cambiar entre estas dos tensiones en algunos pines de alimentación.
- Entradas y salidas analógicas.
- Sensor Hall y de temperatura integrado.
- 2 convertidores Digital-Analógico (DAC) de 8 bits.
- 16 convertidores Analógico-Digital (ADC) de 12 bits.
- 16 canales PWM.
- 2 UART.
- 2 canales I2C.
- 4 canales SPI.
- 448Kb ROM.
- 520 KB SRAM.
- 8KB+8KB SRAM en RTC.
- 1kbit eFUSE.
- 512 bytes Memoria Flash (EEPROM).
- 10 sensores táctiles.

- 4 temporizadores internos de 64 bits.

No están disponibles todas las características del controlador ESP-WROOM-32, ya que algunos pines tienen funciones dobles y se utilizan en la placa de forma específica (como, por ejemplo, para controlar la tarjeta SD). Pero la mayoría de funciones se pueden utilizar, además de disponer la placa ESP32 Plus STEAMakers de una mejor conexión de elementos debido a los pines para conectores tipo Dupont de entrada y salida, de I2C y de alimentación. Además, algunos pines de alimentación pueden cambiar su valor (3,3V o 5V) mediante un interruptor en función de nuestras necesidades.

<https://www.youtube.com/embed/SEAhjQx-e4A?list=PL1pKD-Bz2QBAgy580m8OaQ2Z60v6DOhCindex=2>

A continuación vemos una imagen en la que se compara el potencial de la placa ESP32 Plus STEAMakers En la figura siguiente vemos los elementos que componen la placa ESP32 Plus STEAMakers:

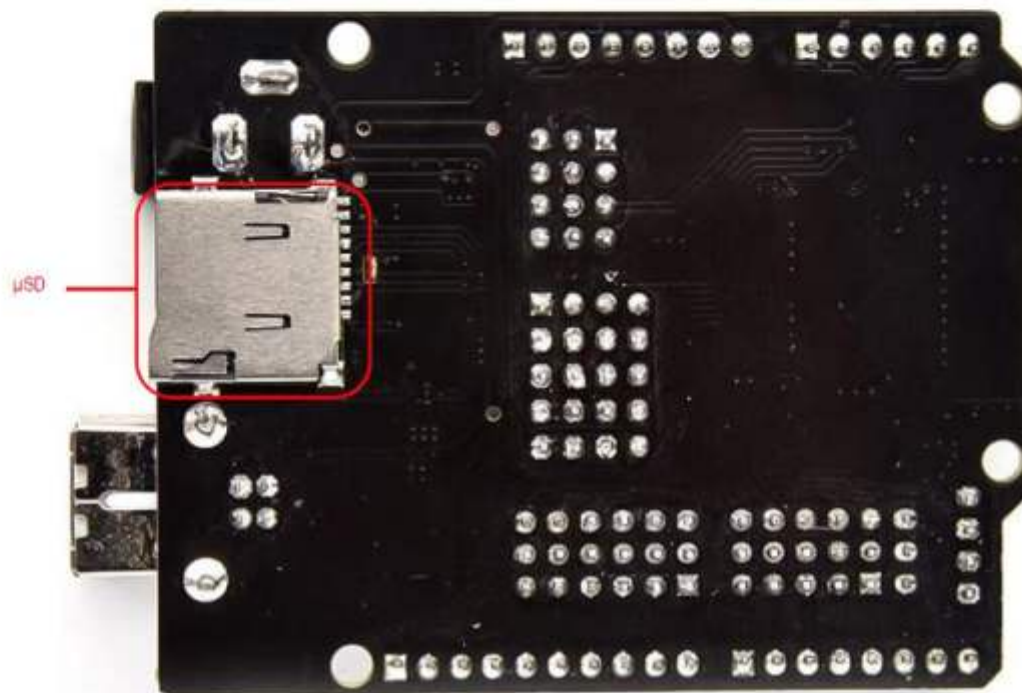
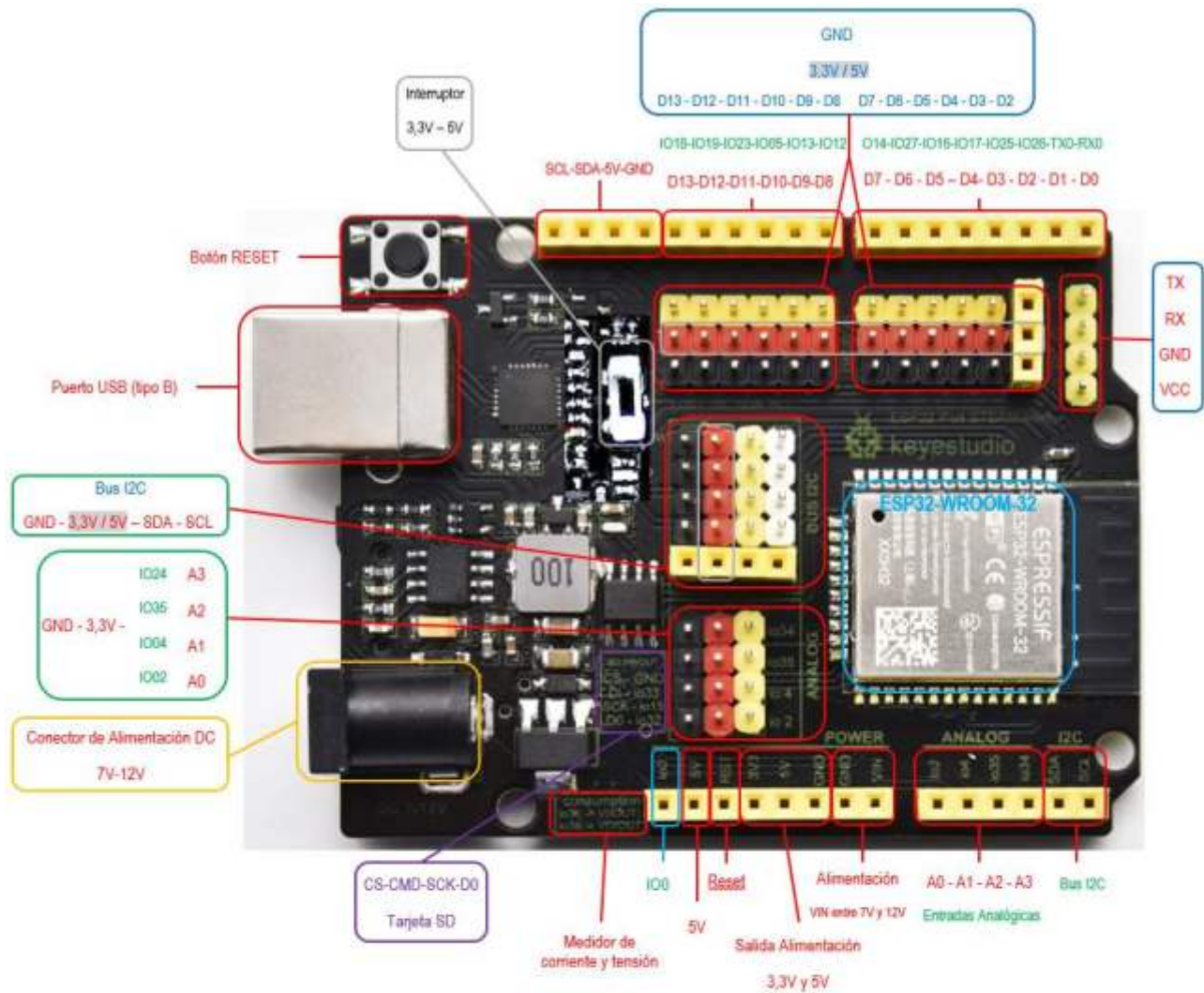


Imagen [Federico Coca](#) CC-BY-SA

Las conexiones de la placa Imagina TDR STEAM con la placa ESP32 Plus STEAMakers son las mismas que si utilizamos cualquier placa compatible con Arduino UNO.

Importante: Todos los pines IOxx son entradas y salidas digitales, algunas con más funciones. Utilizando la comunicación WiFi no funciona el ADC2.

En la tabla siguiente tenemos relacionados todos los pines entre los tipos de placas UNO, Imagina TdR STEAM y ESP32 STEAMakers.

UNO	TdR STEAM	ESP32		
Pin	Función	Pin	Función	Ampliación
D0	Rx	IO03	Rx	UART 0 RX
D1	Tx	IO01	Tx	UART 0 TX
D2	Pulsador SW1	IO26	ADC2 CH9	DAC2
D3	Libre	IO25	ADC2 CH8	DAC1
D4	DHT11	IO17		UART 2 TX
D5	Libre	IO16		UART 2 RX
D6	Color rojo del LED RGB	IO27	ADC2 CH7	ADC2-7 / TOUCH7
D7	Pulsador SW2	IO14	ADC2 CH6	ADC2-6 / TOUCH6
D8	Zumbador o buzzer	IO12	ADC2 CH5	ADC2-5 / TOUCH5
D9	Color verde del LED RGB	IO13	ADC2 CH4	ADC2-4 / TOUCH4
D10	Color azul del LED RGB	IO05		VSPI CS0
D11	Sensor IR	IO23		VSPI MOSI
D12	LED rojo	IO19		VSPI MISO
D13	LED azul	IO18		VSPI CLK
GND		GND		
AREF		Reset		
SDA	I2C	IO21		

UNO	TdR STEAM	ESP32		
SCL	I2C	IO22		
A0	Potenciómetro	IO02	ADC2 CH2	
A1	Sensor de luz (LDR)	IO04	ADC2 CH0	
A2	Sensor de temperatura (LM35)	IO36	ADC1 CH0	
A3	Libre	IO34	ADC1 CH6	
A4	I2C	IO38		
A5	I2C	IO39	ADC1 CH3	
VIN		VIN		
GND		GND		
GND		GND		
5V		5V		
3.3V		3.3V		
RST		Reset		
5V		5V		
		IO00	¡ No conectar !	
-		IO32	D0 - uSD	
-		IO15	CLK - uSD	
-		IO33	CMD - uSD	
-		IO35	IOUT	Medidor de corriente
-		IO37	VOUT	Medidor de tensión

- Para saber más :
 - Notas sobre ESP32 STEAMakers <https://fgcoca.github.io/ESP32-STEAMakers/hsteamakers/>

Introducción

Sensores

Esta sección es una visión rápida de las posibles **entradas** de los robots.

NO LEAS TODOS SINO LOS QUE TIENE TU ROBOT

Un poco de teoría...

Cualquier sistema de control podríamos decir que funciona de una manera similar a un ser humano, salvando las distancias. Nosotros recibimos la información del mundo exterior gracias a nuestros sentidos (oído, olfato, gusto, vista y tacto), nuestro cerebro procesa esa información y a través de nuestros músculos o de nuestra voz realizamos diferentes acciones. Pues lo mismo sucede con los sistemas de control, reciben información del exterior gracias a los diferentes **SENSORES**, procesan esa información en sus PLACAS CONTROLADORAS (sus cerebros) tales como Arduino y dan una respuesta utilizando sus diferentes **ACTUADORES**.



Un sensor es un objeto capaz de detectar magnitudes físicas o químicas y transformarlas en variables eléctricas. Los sensores o periféricos de entrada nos permiten obtener información del mundo real para utilizarla desde el programa de Arduino.

En la actualidad la cantidad de sensores disponibles es tan extensa como las variables que queramos medir, desde sensores de temperatura, humedad, luminosidad,... hasta acelerómetros, giroscopios, GPS,... pasando por detectores de gases, de pulsos cardiacos, sensores de efecto HALL,...

Tipos de sensores

- **DIGITAL:** un sensor digital sólo tiene dos estados: activado/desactivado, ON/OFF, 1/0, Alto/Bajo, ... En este caso conectaremos el sensor a una de las entradas digitales de Arduino para leer el estado.

Ejemplo: un pulsador es un tipo de sensor sencillo que sólo nos da dos estados, “pulsado o no pulsado”. Conectado a la placa Arduino debe generar 0v en reposo y 5v al pulsarlo. De esta forma desde el programa de Arduino podremos leer el estado del botón.



- **ANALÓGICO:** el sensor nos puede dar un rango de valores, normalmente se traduce en un valor de tensión o de corriente variable en función de la señal captada al sensor. En este caso conectaremos el sensor a una de las entradas analógicas de Arduino (A0,..., A5). El rango de entrada será una tensión entre 0v (GND) y 5v.

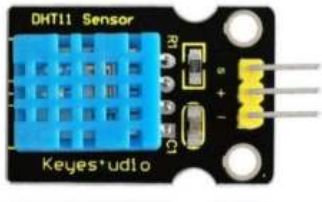
Ejemplo: Una fotorresistencia es un componente electrónico cuya resistencia disminuye con el aumento de intensidad de luz incidente. Su valor varía entre 0 y 5 v. la cantidad de valores que pueden leer las entradas analógicas de Arduino son de 10 bits es decir 1024 valores. De tal modo que $0 = 0 \text{ v.}$ y $1023 = 5 \text{ V.}$



- **DATOS:** el sensor ofrece su información a través de una interfaz de comunicación. La forma de comunicación puede ser por sistemas estándar como **I2C** o SPI o algunos

sensores usan su propio protocolo para codificar la información y debemos realizar desde el software la decodificación correcta para interpretar los datos del sensor (normalmente los desarrolladores de este tipo de sensores ofrecen una librería software para Arduino que hace todo el trabajo).

Ejemplo: el sensor DHT11. Por un solo pin envía los datos de temperatura y humedad.



Sensores modulares.

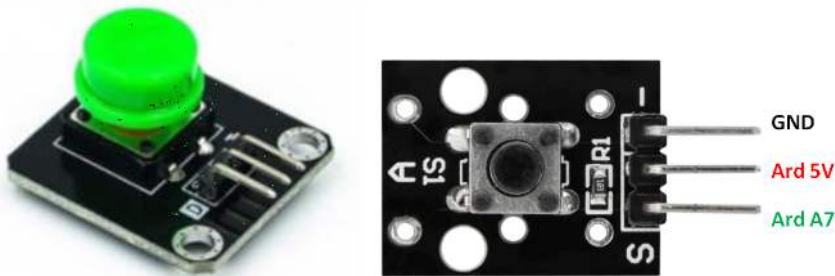
En la actualidad existen infinidad de sensores que los fabricantes presentan en forma modular. Esto hace que su conexión y utilización sea mucho más sencilla que la tradicional, olvidándonos de resistencias, polaridades, cableados,... para su correcto funcionamiento.

Sensor pulsador

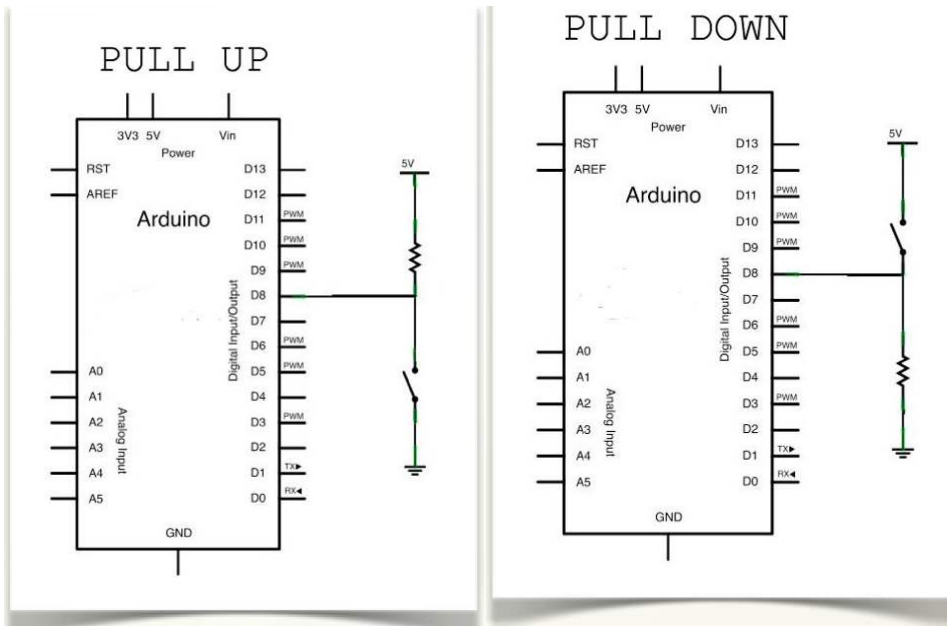
Es un sensor digital, que presenta dos estados; cuando se presiona el botón, emite una señal de bajo (0V), cuando suelta el botón, emite una señal de bajo alto (5V). [Datasheet](#)

Un ejemplo de uso

• en el robot mClon



Otra manera más "barata" de sustituir este módulo pulsador es poner un pulsador normal y una resistencia ($\pm 10k$), al pulsar se produce una entrada en el Arduino, hay dos configuraciones, que al pulsar se emita un 0 lógico (configuración **Pull up**) o que al pulsar emita un 1 lógico



Lo "normal" es que al pulsar se emita un '1' configuración **Pull down**, pero hay pulsadores que funcionan **Pull up** y los llaman **lógica invertida**, por eso en la programación por bloques podemos encontrar esto:



Sensor Táctil Capacitivo.

Este pequeño sensor puede "sentir" a las personas y el tacto y la retroalimentación de metales a un nivel de voltaje alto / bajo. Incluso aislado por alguna tela y papel, todavía puede sentir el tacto. Su sensibilidad disminuye a medida que la capa de aislamiento se hace más gruesa. En nuestra opinión lo preferimos frente al *Sensor pulsador* pues es muy económico, duradero y fiable.

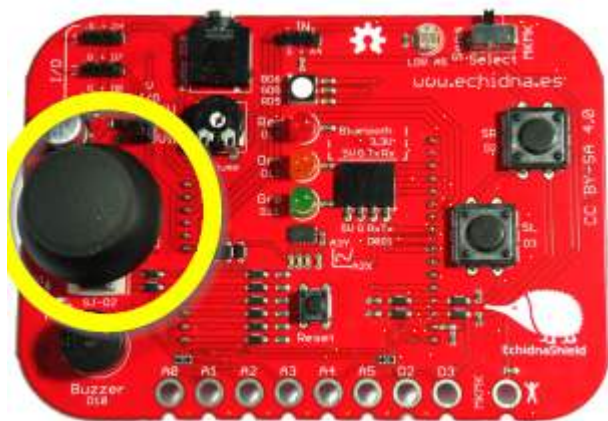
Un ejemplo de uso en

- [Disparo láser en Arduino con ArduinoBlocks](#)
- [Apertura de puerta en Domotica con Arduino](#)



Potenciómetro y joystick

Un potenciómetro es una resistencia variable, es decir, cambia de valor mecánicamente, lo tenemos en multitud de dispositivos. El **joystick** es internamente dos potenciómetros con un pulsador integrado en un solo mando.



Este sensor es **analógico**, su salida puede ser cualquier valor entre Vcc y GND (si está en divisor de tensión como en la placa Edubásica no llega a esos valores extremos), por lo tanto hay que conectarlo a una entrada analógica de Arduino y como cualquier entrada analógica, proporcionará valores entre 0 y 1023.

Ejemplos de uso:

- Arduino con código: [Mapeo del potenciómetro](#)
- Arduino con código: [Regular la luz con potenciómetro](#)
- [Arduinoblocks en el aula](#)
- En [Arduino con Echidna](#), con joystick
- [Domótica con Arduino con joystick](#)

Sensor Fotocélula LDR.

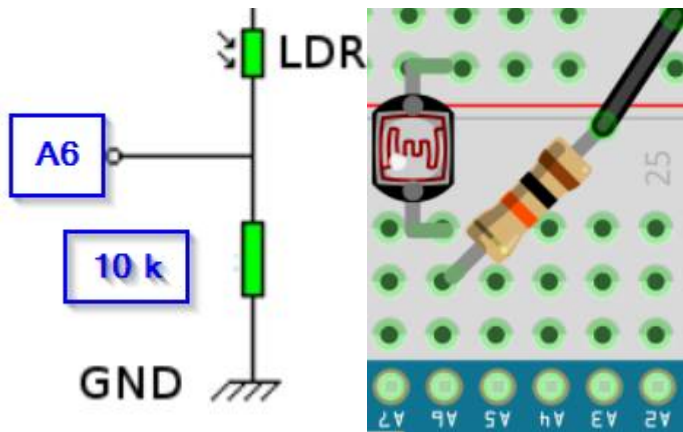
El uso de fotocélulas es muy común en nuestras vidas, las encontramos en el encendido automático de farolas, apertura de puertas,... La fotocélula es un semiconductor. Es ampliamente utilizado en campos de interruptores de control automático como cámaras, luces solares de jardín, lámparas de césped, detectores de dinero, relojes de cuarzo, tazas de música, cajas de regalo, mini luces nocturnas, interruptores de control de luz y sonido, etc. Es un sensor analógico dando valores entre 0 y 5V y como entrada analógica de un Arduino se traduce en un rango de 0 a 1023 valores.

Un ejemplos de uso :

- [el interruptor crepuscular del curso Arduino con ArduinoBlocks](#)
- [Medir la luz en Rover con Arduino](#)
- [Medir la oscuridad en Arduino con mBlock](#)
- [Hinchar un balón en Arduino con mBlock](#)

Una manera más económica de montar este sensor es utilizar una resistencia y un LDR:

- El LDR cuando más oscuridad, más resistencia
- En una configuración **PULL DOWN**, cuanto **más** luz, la resistencia del LDR baja, por lo tanto **más** tensión en A6



Los módulos LDR que se venden suelen esta configuración Pull down, es decir, cuanto **más** luz, **más** tensión:



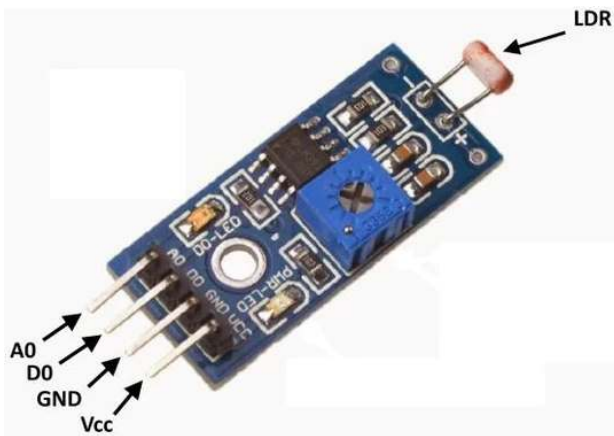
La instrucción con Arduinoblocks ya cuenta con esta configuración Pull downUp de que cuando **más** luz, **más** valor tiene la entrada analógica.



Hay módulos LDR ya montados, que tienen componentes **activos** es decir, llevan circuitos electrónicos, transistores que amplifican etc... y dan la salida **digital** con un potenciómetro para definir el rango de luz que cambia de estado lógico. Puedes ver en la figura que tiene una salida digital **D0**.



O hay [algunos que tienen 4 pines](#) como en la figura que ofrecen las dos cosas: salida analógica **A0** y digital **D0**.



Nosotros aconsejamos el divisor de tensión por tres razones: más barato, no implica gran circuitería y es visible su funcionamiento frente a estos encapsulados.

Sensor de Ultrasonidos.

Es un sensor digital de distancias por ultrasonidos capaz de detectar objetos y calcular la distancia a la que se encuentra en un rango de 2 a 350 cm. Su uso es tan sencillo como enviar el pulso de arranque y medir la anchura del pulso de retorno.

No es un sensor preciso, con una ligera inclinación de la superficie ya da lecturas erróneas pero es muy barato

El más común es el [HC-SR04](#) que tiene 4 pines de conexión: **VCC** **Trig** (Disparo del ultrasonido)

Echo (Recepción del ultrasonido) y **GND** aunque en algunos modelos como el de [ElecFreaks](#) tiene 3 pines. Integra Trig y Echo en uno sólo.

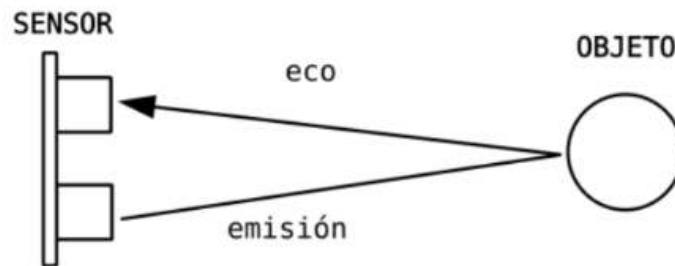
La distancia se calcula con esta fórmula:

Distancia en cm = {(Tiempo en segundos entre Trig y el Echo) * (V.Sonido 34000 en cm/s)} / 2

Si programas en código, tienes que utilizar la fórmula anterior, previamente tienes que programar

el cálculo del tiempo entre una emisión de un pulso en Trg y la respuesta en Echo.

loques, no es necesario, seguro que hay un bloque que lo hace todo



Ejemplos de uso:

- [Alarma en Domótica con Arduino](#)
- [Piano invisible en Arduino con ArduinoBlocks,](#)
- [Sensor parking en Arduino con ArduinoBlocks](#)
- [Piano invisible en Arduino con mBlock](#)
- [Sensor parking en Arduino con mBlock](#)
- [Sensor de distancia de ultrasonidos con Picobricks](#)

Sensor DHT11 (Temperatura y Humedad).

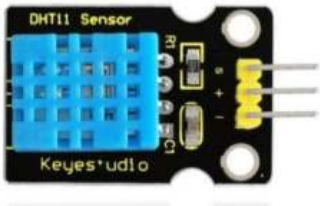
Este sensor de temperatura y humedad **DHT11** nos permite [determinar las zonas de confort](#) para un rango de temperaturas entre 0°C y 50°C con un error de $\pm 2^\circ\text{C}$ y un rango de humedad entre 20 y 90 % $\pm 5\%$. Una salida digital para dos variables cómo lo hace? Tiene dentro un pequeño microprocesador que lanza por el bit de datos 40 bits en serie, los 16 primeros son la humedad (en BCD) y los 16 restantes es la temperatura (en BCD) los 8 restantes son de comprobación

Checksum (en binario) como la letra del DNI. Por ejemplo 0100 0111 0000 0011 0001 1001 0000 0000 0001 1000 es 0100 0111 0000 0011 = 47.03% de humedad y 0001 1001 0000 0000 = 19.00°C y la comprobación es la suma de 4+7+0+3+1+9+0+0=24=11000

Ejemplos de uso:

- [Medir H y T con Blink en Rover con Arduino](#)
- [Estación meteorológica Arduino con Arduinoblocks](#)

- [Arduinoblocks en el aula](#)
- [SMART HOME con Micro:bit](#)

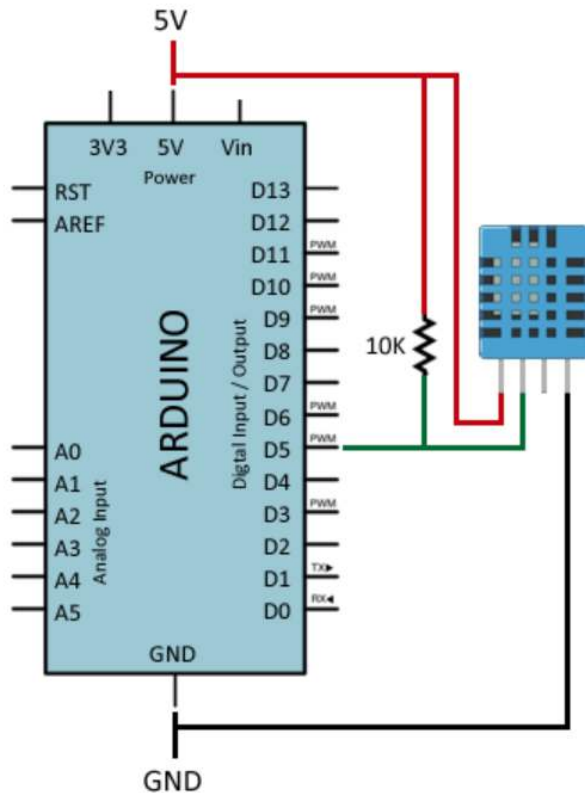


No es un sensor con gran sensibilidad, pero para propósitos educativos cumple sus funciones. Por dentro tiene una resistencia NTC que decrementa su resistencia si aumenta la temperatura. Hay otros que van al revés, los PTC. Tanto los NTC como los PTC se llaman **termistores**. Para la humedad, mide la capacidad de un condensador que es sensible a la humedad, o sea, un **sensor capacitivo**.

Tenemos dos opciones comerciales: **Encapsulado** que lo tienes preparado para conectar la alimentación y leer por el pin de datos, o **sin encapsular**, que hay que colocar una resistencia de aproximadamente 10k entre Vcc y Data

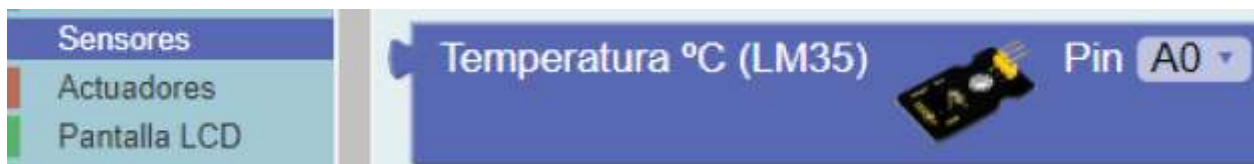


Ejemplo de uso de un DHT11 sin encapsular:



Fuente Luis LLamas CC-BY-NC-SA <https://www.luisllamas.es/arduino-dht11-dht22/>

Si queremos mejorar la sensibilidad, podemos utilizar el **DHT22** que es igual pero de color blanco y más caro. Si lo que queremos es sólo la temperatura es mejor utilizar el **LM35D** que tiene un rango de temperaturas desde 0°C a 100°C con una sensibilidad de 2mV/°C



Es un sensor bastante mediocre, si necesitas una precisión el doble, te recomendamos el DHT22 que funciona exactamente igual pero es de color blanco y más caro ~3€. Ver <https://www.luisllamas.es/arduino-dht11-dht22/>

Sensor IR

Es un sensor para distancias cortas hasta 2cm y no da la distancia, simplemente si hay o no hay obstáculo, pero son muy baratos, unos 0.30€. [Aquí tienes un ejemplo de evita obstáculos en un rover marciano con Raspberry](#) Para saber más te recomendamos [esta página de Luis Llamas](#)

<https://sketchfab.com/models/6ad4f3afb83940fea95cd3846aa68a18/embed>

[IR Sensor Module for Arduino Projects | 3D Model](#) by [Veer AI](#) on [Sketchfab](#)

Sensor llama

Este sensor de llama se puede utilizar para detectar fuego u otras luces cuya longitud de onda se encuentra entre 760 nm ~ 1100nm.

Un ejemplo de su uso:

- [Alarma por fuego en Domótica con Arduino](#)

Radiación		Longitud de onda λ
Ultravioleta 100-400 nm	ultravioleta C	100 nm – 280 nm
	ultravioleta B	280 nm – 315 nm
	ultravioleta A	315 nm – 400 nm
Visible 400-780 nm	violeta	400 nm – 455 nm
	azul	455 nm – 490 nm
	verde	490 nm – 570 nm
	amarillo	570 nm – 590 nm
	anaranjado	590 nm – 620 nm
	rojo	620 nm – 780 nm
Infrarroja 780nm-1mm	infrarroja A	780 nm – 1400 nm
	infrarroja B	1400 nm – 3000 nm
	infrarroja C	3000 nm – 1 mm



Sensor de Gas (MQ2).

Detecta gases inflamables : GLP, I-butano, propano, metano, alcohol, hidrógeno, humo... con más sensibilidad en algunos que en otros. Siempre detecta el conjunto. Son usados en electrónica de consumo y mercados industriales.

- **Sensibilidad** Tiene alta sensibilidad y se puede ajustar girando el potenciómetro.
- **Tiempo de respuesta:** Internamente posee un calentador para aumentar su temperatura y que estos gases reaccionen con la resistencia interna que tiene, por lo

tanto tardan algo en responder la primera vez que se conectan, incluso horas en algunos modelos. Una vez calentados son rápidos en la respuesta.

- **Tipo de salida:** Analógico pero si tiene 4 pines como el de la figura, incorpora un pin digital.
- Ejemplos de uso:
 - [Smart Home Microbit](#)
 - [Smart Home ESP32](#)



Sensor de humedad de suelo.

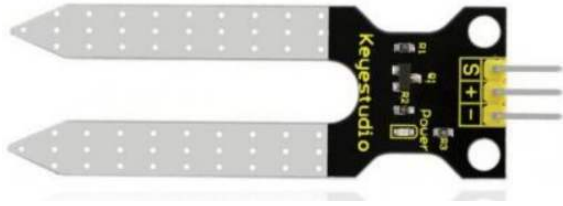
“ Un higrómetro de suelo FC-28 es un sensor que mide la humedad del suelo. Son ampliamente empleados en sistemas automáticos de riego para detectar cuando es necesario activar el sistema de bombeo. El FC-28 es un sensor sencillo que mide la humedad del suelo por la variación de su conductividad. No tiene la precisión suficiente para realizar una medición absoluta de la humedad del suelo, pero tampoco es necesario para controlar un sistema de riego. Los valores obtenidos van desde 0 sumergido en agua, a 1023 en el aire (o en un suelo muy seco). Un suelo ligeramente húmedo daría valores típicos de 600-700. Un suelo seco tendrá valores de 800-1023.

Luis Llamas CC-NC-BY-SA <https://www.luisllamas.es/arduino-humedad-suelo-fc-28/>

Se puede utilizar este sensor para hacer un dispositivo de riego automático, puede detectar si las plantas “tienen sed” y evitar que se marchiten.

La corriente de trabajo del sensor es menor de 20mA. El voltaje de salida es de 0V (en el aire) a 2,3V (totalmente sumergido en agua).

- [Smart Agriculture Kit micro:bit](#)



Sensor de humedad.

Este sensor **analógico** está diseñado para identificar y detectar la presencia de agua y su cantidad. Puede servir para detectar el nivel de agua, para disparar una alarma en caso de una fuga de agua, también para hacer un limpiapalabrisas automático.... puedes ver un ejemplo de uso en :

- Curso de [Domótica con Arduino](#)
- [Smart Agriculture Kit micro:bit](#)

Mide el volumen de agua caída a través de una serie de rastros de cables paralelos expuestos.



Sensor de efecto Hall.

Este es un sensor de inducción magnética. Detecta los materiales magnéticos dentro de un rango de detección de hasta 3 cm. El rango de detección y la fuerza del campo magnético son proporcionales. La salida es digital.

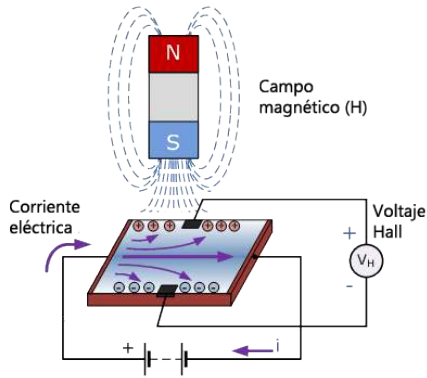


Sensor Hall.

Edwin Helber Hall De Desconocido - [Popular Science Monthly Volume](#)

[64, Dominio público](#)

[Edwin Helbert Hall](#) descubrió en 1879 que en presencia de un campo magnético, un conductor que conduzca una corriente se le producía un campo eléctrico porque las cargas eléctricas se desviaban de su trayectoria principal, nuestro sensor simplemente mide ese campo eléctrico:



De [Luis Llamas](#) CC-BY-NC

El sensor tiene un led de color rojo que indica que hay una lectura de campo magnético. Un ejemplo de uso lo puedes ver aquí: [medir rocas magnéticas con el Rover con Arduino](#)

Sensor inclinación

Este sensor funciona al hacerle vibrar, emitiendo una señal digital de todo o nada. El módulo del sensor viene provisto de un potenciómetro para poder regularlo.



Sensor de golpe

Es un sensor digital que al ser golpeado este sensor envía una señal momentánea.



Sensor de pulso cardíaco.

Este módulo utiliza un LED infrarrojo (IR) ultrabrillante y un fototransistor para detectar el pulso en el dedo. Principio de funcionamiento: Se debe colocar el dedo entre el LED infrarrojo ultrabrillante (parte superior) mientras que el fototransistor, que queda en el otro lado, recoge la cantidad de luz transmitida. La resistencia del fototransistor variará levemente a medida que la sangre pase a través de su dedo.



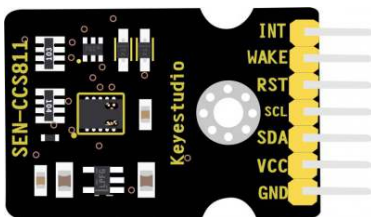
Sensor de Alcohol.

Este sensor de gas analógico MQ-3 es adecuado para detectar alcohol. Se puede usar en un analizador de aliento. También tiene una alta sensibilidad al alcohol y baja sensibilidad a la bencina (éter de petróleo). La sensibilidad se puede ajustar con el potenciómetro.



Sensor de CO2

Hay sensores que utilizan **el protocolo I2C**, este protocolo permite conexiones serie y pueden compartir el mismo cable pues cada elemento tiene una dirección diferente. Esto lo veremos en el Display LCD. Se identifican por los pines SDA y SCL

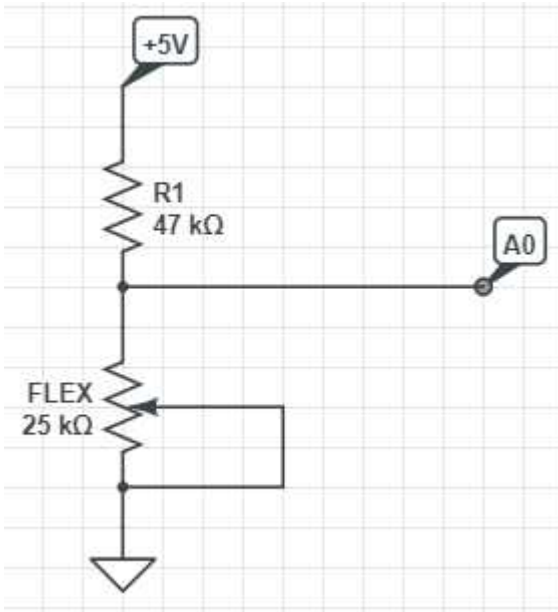


Resistencia Flex

Es una resistencia que cuanto más se dobla más resistencia ofrece, desde 25k hasta 125k

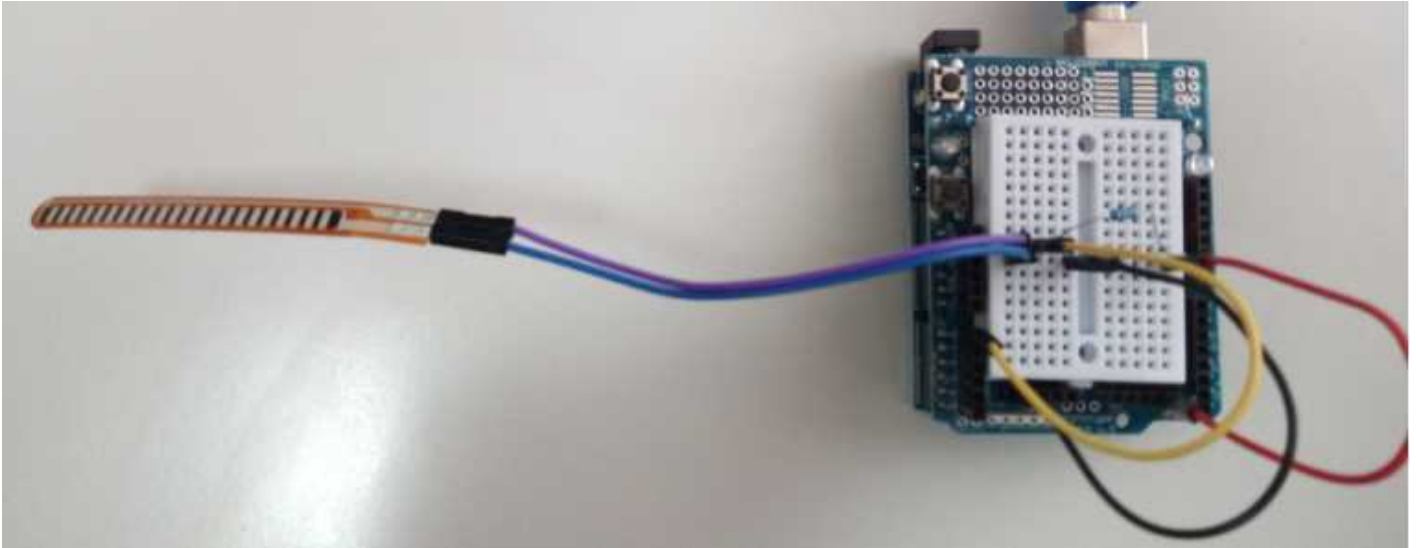


Para utilizar esta resistencia haremos un **DIVISOR DE TENSION** que consistirá en poner dos resistencias en serie y repartirá la tensión total entre 0V y 5V en las dos resistencias, **el punto medio** será un punto que tendrá una tensión variable en función de las dos resistencias, como la es variable, esa tensión es variable y ya tenemos la entrada **analógica**:



Es decir:

- La resistencia entre masa GND del ARDUINO (cable negro) y un punto en la placa protoboard
- ese punto medio conectarlo a una entrada analógica, por ejemplo A0 (cable amarillo)
- Una resistencia de valor parecida a la Flex de decenas de K entre ese punto y +5V (cable rojo en la foto)



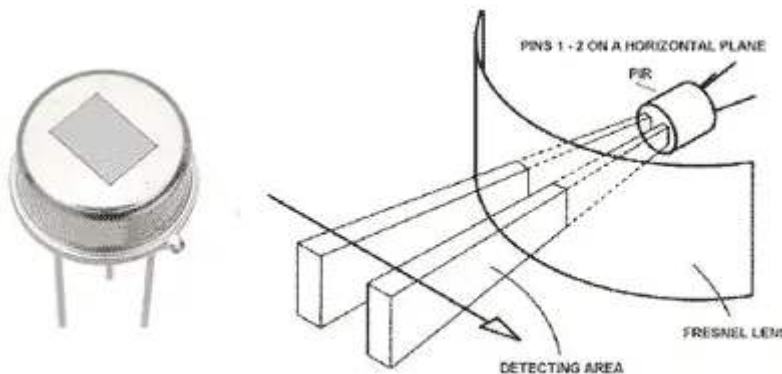
Este sensor tiene posibilidades para usarlo en "ropa inteligente".

Sensor de movimiento

“ Los sensores infrarrojos pasivos (PIR) son dispositivos para la detección de movimiento. Son baratos, pequeños, de baja potencia, y fáciles de usar. Por esta razón son frecuentemente usados en juguetes, aplicaciones domóticas o sistemas de seguridad.

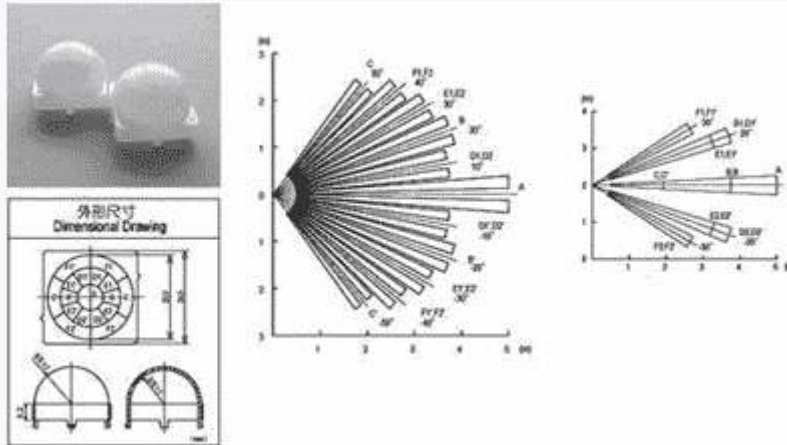
Los sensores PIR **se basan en la medición de la radiación infrarroja**. Todos los cuerpos (vivos o no) emiten una cierta cantidad de energía infrarroja, mayor cuanto mayor es su temperatura. Los dispositivos PIR disponen de un sensor piro eléctrico capaz de captar esta radiación y convertirla en una señal eléctrica. En realidad **cada sensor está dividido en dos campos** y se dispone de un circuito eléctrico que compensa ambas mediciones. Si ambos campos reciben la misma cantidad de infrarrojos la señal eléctrica resultante es nula. Por el contrario, si los dos campos realizan una medición diferente, se genera una señal eléctrica.

De esta forma, si un objeto atraviesa uno de los campos se genera una señal eléctrica diferencial, que es captada por el sensor, y se emite una señal digital.



El otro elemento restante para que todo funcione es **la óptica del sensor**. Básicamente es una cúpula de plástico formada por lentes de fresnel, que divide el espacio en zonas, y enfoca la radiación infrarroja a cada uno de los campos del PIR.

De esta manera, cada uno de los sensores capta un promedio de la radiación infrarroja del entorno. Cuando un objeto entra en el rango del sensor, alguna de las zonas marcadas por la óptica recibirá una cantidad distinta de radiación, que será captado por uno de los campos del sensor PIR, disparando la alarma.



Luis Llamas CC-BY-NC-SA <https://www.luisllamas.es/detector-de-movimiento-con-arduino-y-sensor-pir/>

Puedes ver ejemplos de uso en robótica en :

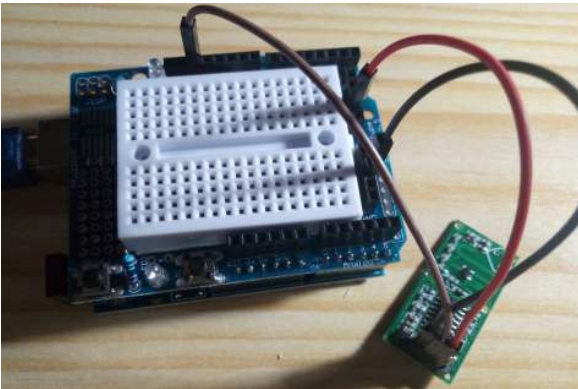
- [Smart Home para Microbit](#)
- [Smart Agriculture Kit para Microbit](#)



Más sensibles son los **sensores de microondas**. Son un radar que por efecto Doppler pueden captar cualquier objeto en movimiento dentro de un alcance de 5-7 metros en cualquier dirección e independiente de su temperatura. Es un buen sensor para alarmas, activación de luz por presencia..... Para saber más [ver la página de Luis Llamas](#)



Su conexión es muy sencilla, es un detector digital que hay que alimentarlo como el resto de sensores.



Curiosamente, la gran ventaja técnica de los de microondas **es un gran inconveniente para usarlo en el aula**, con cualquier movimiento se dispara, luego **para clase es mejor el sensor PIR**

Esta página esta adaptada de [este enlace](#). José Andrés Echevarría @cantabRobots CC-BY-NC-SA.

Introducción

Actuadores y otras salidas

Esta página es un repaso rápido de las posibles **salidas** de los robots
NO LEAS TODOS SÓLO LOS QUE TIENE TU ROBOT

¿Qué es un actuador?

“ Un actuador es el nombre genérico que empleamos para cualquier **dispositivo capaz de realizar acciones en el mundo físico** y que podremos controlar desde un autómata o procesador como Arduino. En particular, usamos el nombre actuador para los dispositivos que son capaces de generar movimiento.

[Luis LLamas](#) CC-BY-NC-SA

ACTUADORES

Motores baja potencia

Con el Arduino podemos usar motores de corriente continua de juguete como en la figura, aptos para poner un ventilador. Ejemplo de uso [Smart Home para microbit](#)

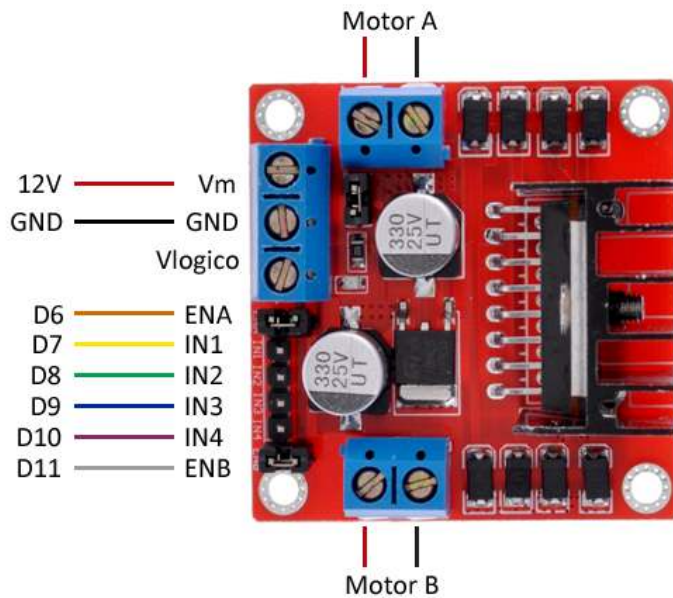


Motores con algo de potencia

Si quieres usar un motor para mover unas ruedas, ya pide más corriente, no se puede conectar directamente a la placa (Arduino, esp32 etc..). El de la figura es típico en robótica y lleva reductores dentro para bajar la velocidad :



Necesita una alimentación extra, como el LM298N y las pilas., o el B6612FNG o una shield específica.



CC-BY-NC [Luis Llamas](#)

Ejemplos de uso lo puedes ver en

- [Curso mClon con nanoArduino con el B6612FNG](#)
- [Curso Rover con Arduino con la shield L293D ESP-12E para el NODE MCU que internamente tiene el LM298N](#)
- [Curso Arduino con código con LM298N](#) y también con un transistor

Los motores pueden (y deben) de estar conectados a **salidas PWM** de tal manera que se pueda regular la potencia y por lo tanto la velocidad.

Servos

Un [servo convencional](#) es un motor donde fijamos el ángulo desde 0º a 180º, pero si queremos una rotación, existen [servos rotatorios](#) que simplemente tienen su velocidad de rotación controlada



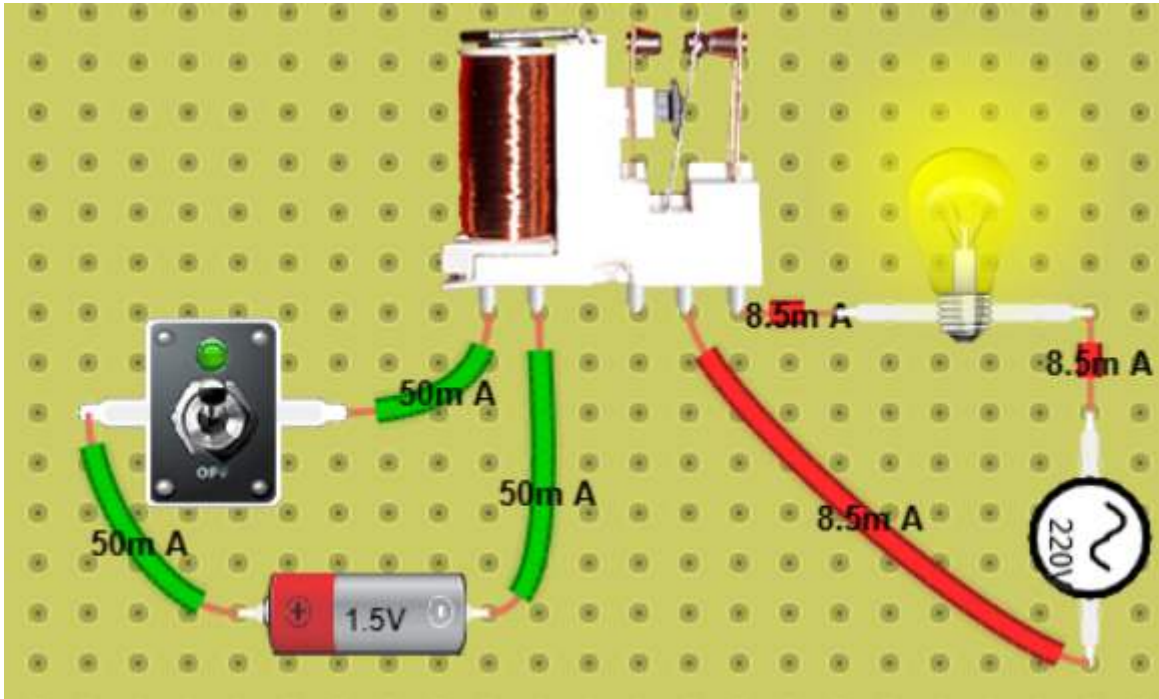
Ejemplos de uso de servos:

- [Curso Arduino con código](#)
- [Curso brazo robótico del mClon con nanoArduino](#)
- [Apertura de barrera por ultrasonidos en curso Arduino con ArduinoBlocks](#)
- [Tractor entrando en el corral Arduino con EchidnaShield](#)
- [Apertura de puerta Domótica con Arduino](#)
- [Servo con PicoBrick](#)
- [Apertura ventana y puerta en Smarth Home para microbit](#)
- [Smart Agriculture Kit para micro:bit](#)

Relés

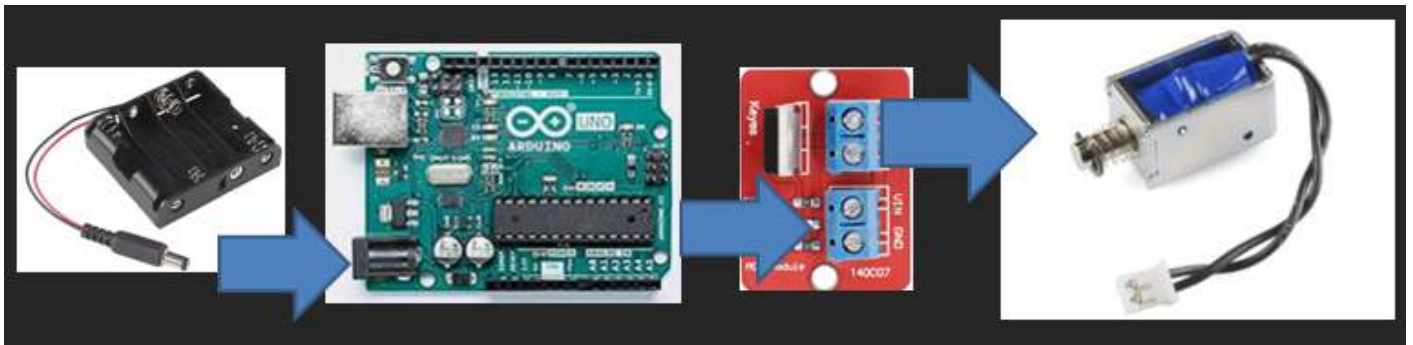
Un relé es un interruptor activado por un electroimán, lo que permite independizar los circuitos. En el dibujo se ve que el circuito rojo de 220V esta separado del verde, de sólo 1.5V. Pero es el circuito verde que al funcionar, hace que el electroimán mueva el interruptor del relé y encienda la bombilla. El objetivo es que he podido encender una bombilla de 220V sin tocar los 220V peligrosos. En el circuito verde, el interruptor puede ser un Arduino. Experimentalo en este [simulador](#).

- Un ejemplo con Arduino en [Luis LLamas](#)
- Un ejemplo de uso en [Picobricks](#)



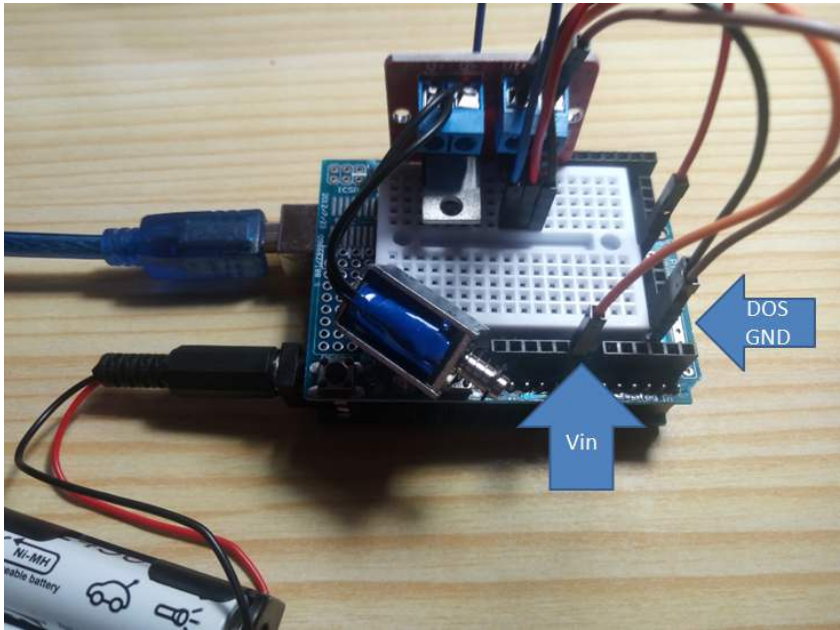
Electroimán

El electroimán no se puede conectar directamente, utilizaremos una etapa de potencia, por ejemplo el transistor el [IRF520N](#) que amplifique la señal del Arduino, pues las salidas de Arduino no tienen potencia para mover el electroimán. Para dar esa potencia utilizaremos otra fuente externa, unas pilas:



Las conexiones son :

- SIG del IRF520N a una salida digital por ejemplo D13
- VCC del IRF520N al 5V del ARDUINO
- Los dos GND del IRF520N a GND del ARDUINO
- V+ y V- del IRF520N al solenoide, da igual qué cable pues no tiene polaridad.
- VIN del IRF520N al VIN del ARDUINO (son los voltios de la pilas)



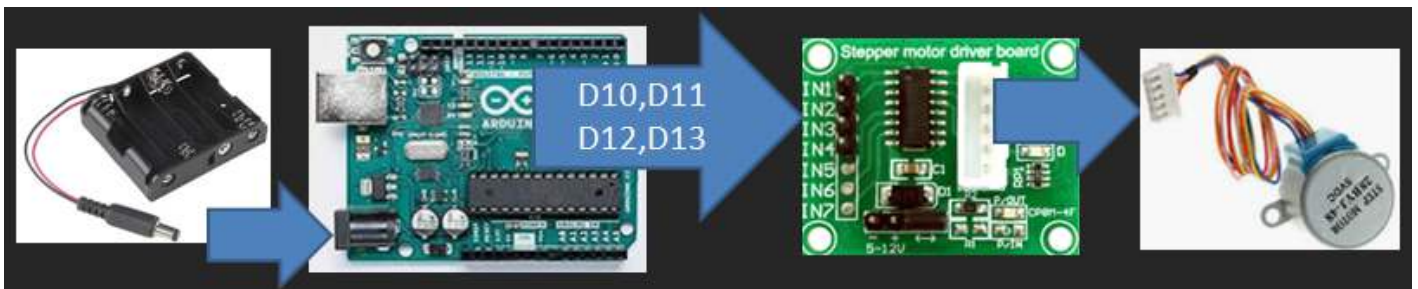
Motor paso a paso

Igual que el [electroimán](#), necesitamos un controlador que nos de la potencia necesaria para mover el motor, el ULN2003



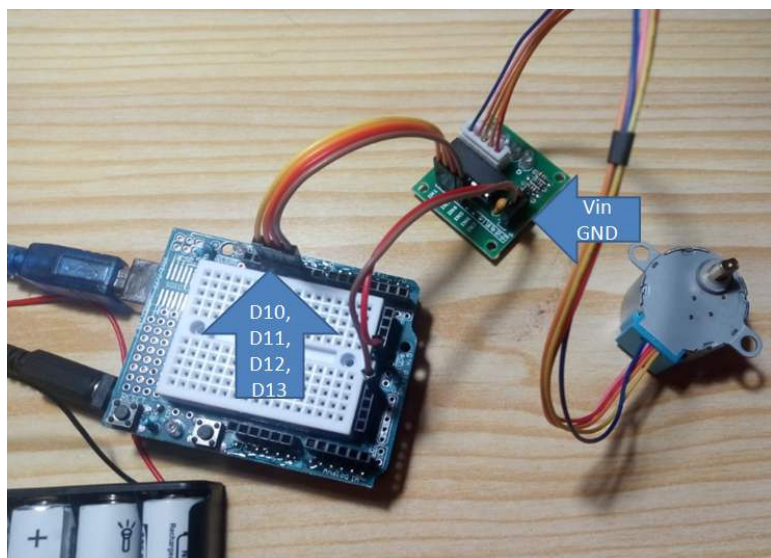
Imagen BY-NC-SA de [Luis Llamas](#)

También igual que el electroimán, necesitamos una potencia extra con las pilas:



Conexión :

- Cuatro pines digitales del ARDUINO al IN1,IN2,IN3,IN4 del ULN2003 por ejemplo D10,D11,D12,D13
- El conector blanco del ULN2003 al motor paso a paso
- El (+) del ULN2003 al Vin del ARDUINO
- El (-) del ULN2003 al GND del ARDUINO



La configuración más sencilla es la rotación simple en sentido horario (llamada fase1) :

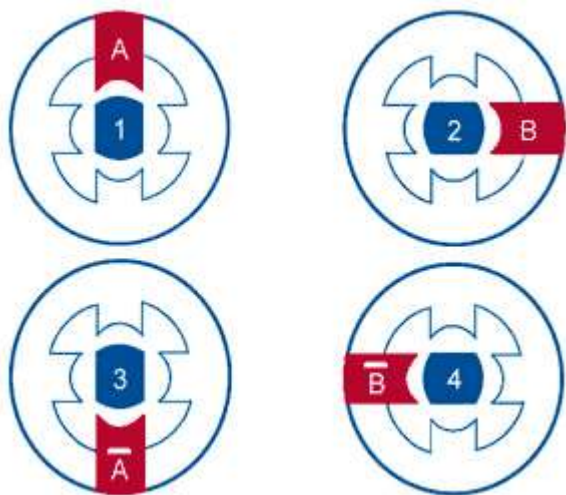


Imagen BY-NC-SA de [Luis Llamas](#)

Es decir:

Paso	IN1=D10	IN2=D11	IN3=D12	IN4=D13
Paso 1	ON	OFF	OFF	OFF

Paso	IN1=D10	IN2=D11	IN3=D12	IN4=D13
Paso 2	OFF	ON	OFF	OFF
Paso 3	OFF	OFF	ON	OFF
Paso 4	OFF	OFF	OFF	ON

Si has entendido este ejemplo serás capaz de realizar las configuraciones que desees.

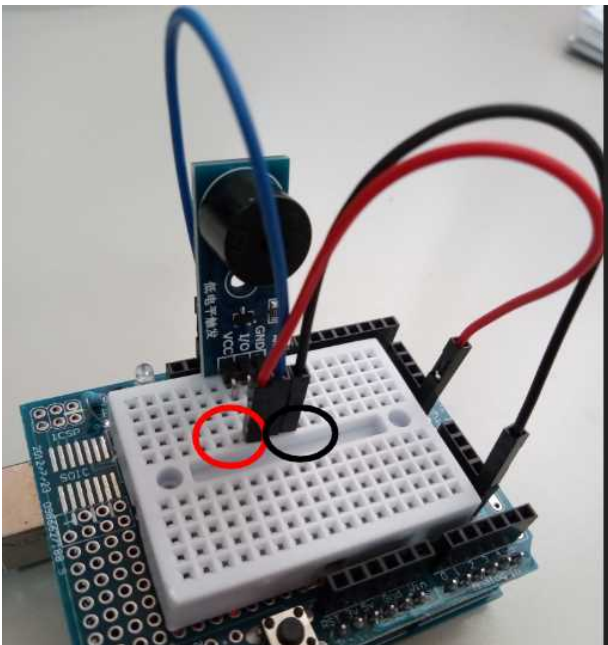
OTRAS SALIDAS

No son actuadores pues no representan movimiento pero son también **salidas** del Arduino como los actuadores, luego lo vemos en esta página

Buzzer activo

Reproduce un tono de una frecuencia determinada por el fabricante cuando recibe un '1' digital. Su conexión es muy simple:

- **GND** a GND y **VCC** a 5V (ojo que están a los dos extremos, marcados en rojo y en negro)
- **OUT** o también señalado como I/O a un pin digital, por ejemplo D13



Ejemplos de uso:

- Domótica con Arduino: [Apertura puerta con mBlock](#)

- Domótica con Arduino [Apertura puerta con código](#)

Buzzer pasivo

La diferencia de un buzzer activo con el pasivo es que el pasivo hay que mandar la onda que se tiene que reproducir, como Arduino no puede reproducir onda puras (senoidales analógicas), se le envía ondas cuadradas con la frecuencia que se pretende reproducir. Como se puede ver en la figura, no tiene elementos auxiliares para reproducir un tono, es simplemente un altavoz.



Ejemplos de uso

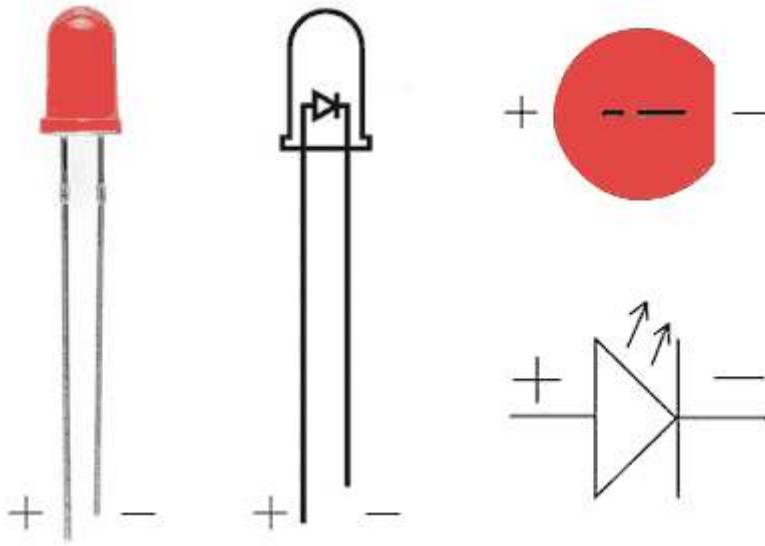
- [Timbre Arduino con Echidna](#)
- [Arduino blocks en el aula](#)

LEDS Y OTROS ELEMENTOS VISUALIZADORES

Led normal

Un diodo es la unión de dos semiconductores P-N que sólo permite la corriente en un sentido. Un diodo LED es un diodo, que al pasar esta corriente emite una luz. Hay de 3mm y de 5mm de grosor, transparentes, traslúcidos y de colores (realmente son traslúcidos tintados)

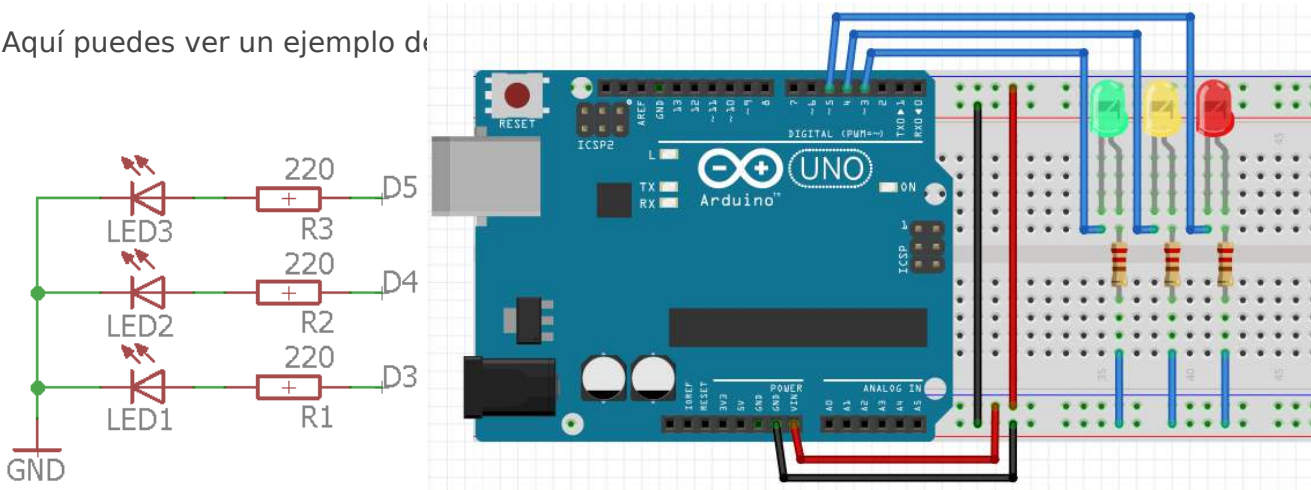
Tenemos pues que respetar su polaridad para que funcione:



Fuente Luis Llamas CC-BY-NC [Encender LED con Arduino](#)

Si alimentamos el LED con una tensión inferior a su tensión de polarización directa $V_d \pm 1,4V-3,8V$ el led no luce. si alimentamos con una tensión superior, la corriente que circula se dispara por lo que se rompe. Conclusión: Hay que poner una resistencia limitante. Para calcularla te recomiendo la página de Luis Llamas [Encender LED con Arduino](#)

Aquí puedes ver un ejemplo de



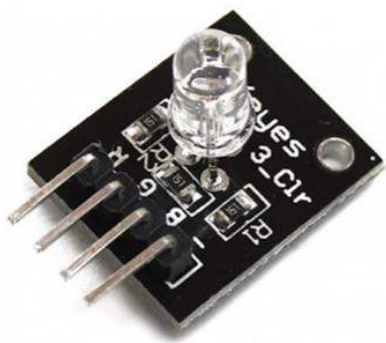
Se puede conectar directamente pues las salidas de Arduino tienen una limitación interna de 20mA como en este ejemplo <https://www.youtube.com/watch?v=EFFSLvIF9rY>

<https://www.youtube.com/embed/EFfSLvIF9rY>

Los LEDS igual que los motores pueden se encendidos o todo o nada con salidas digitales 0 y 1 o salidas PWM de esta manera conseguimos luminosidades medias, en esta práctica con el potenciómetro se regula la intensidad de la luz de un LED

Led RGB

Se trata de un encapsulado que integra 3 leds de los tres colores básicos. Cada color básico se puede regular desde 0 a 255 de intensidad, luego tenemos 16 millones de tonalidades diferentes.



Un ejemplo de su uso lo puedes ver en

- [Domótica con Arduino, interruptor crepuscular](#)
- [Intermitente RGB en Arduino con ArduinoBlocks](#)

Existen LEDS RGB que se comunican unos a otros, ([ver este ejemplo en el curso mClon](#)) como las tiras de leds RGB que cambian de color todos simultáneamente.

Neopixel

La cinta LED RGB se basa en el chip controlador WS2812B de Neopixel con el cual podemos tener el control de cada pixel y de cada color, ya sea rojo, verde y azul a través de un solo cable de datos.

“ Los WS2811, WS2812 y WS2812B son LED que disponen de lógica integrada, por lo que es posible variar el color de cada LED de forma individual (a

diferencia de las tiras RGB convencionales en las que todos los LED cambian de color de forma simultánea). Están basados en el LED 5050, llamado así porque tiene un tamaño de 5.0 x 5.0 mm.

Es un LED de bajo consumo y alto brillo, que incorpora en un único encapsulado los 3 colores RGB. La genial novedad del WS2812B (y resto de familia) es añadir un integrado dentro de cada LED, que permite acceder a cada pixel de forma individual. Por este motivo este tipo de LED se denominan “individual addressable”. Esto abre la puerta a un sinfín de aplicaciones y combinaciones, que van desde dotar de iluminaciones distintas zonas con una única tira, animaciones complejas, o incluso generar pantallas enteras de alta luminosidad....

A los LED WS2812B también se les denomina NeoPixel

Luis Llamas CC-BY-NS-SA <https://www.luisllamas.es/arduino-led-rgb-ws2812b/>



- [Smart Home para Microbit con 4 leds RGB Neopixel](#)
- [Smart Home para ESP32](#)
- [Smart Agriculture Kit micro:bit](#)

Láser

El diodo láser es un elemento motivador, barato y fácil de usar con el Arduino pues se activa digitalmente, si enviamos un 1 emite un rayo láser. Un ejemplo de uso lo puedes ver en la [Alarma por láser en Domótica con Arduino](#)



Si quieres saber más de este componente, te recomendamos [esta página de Luis Llamas](#).

Si tienes que comprar uno, es importante que no sea superior a 5mW, pues puede dañar permanentemente la retina del ojo [[+info](#)].

El modelo que proponemos es de **1mW**, no obstante, **EVITA SIEMPRE QUE EL LÁSER APUNTE A LOS OJOS** especialmente con niños.

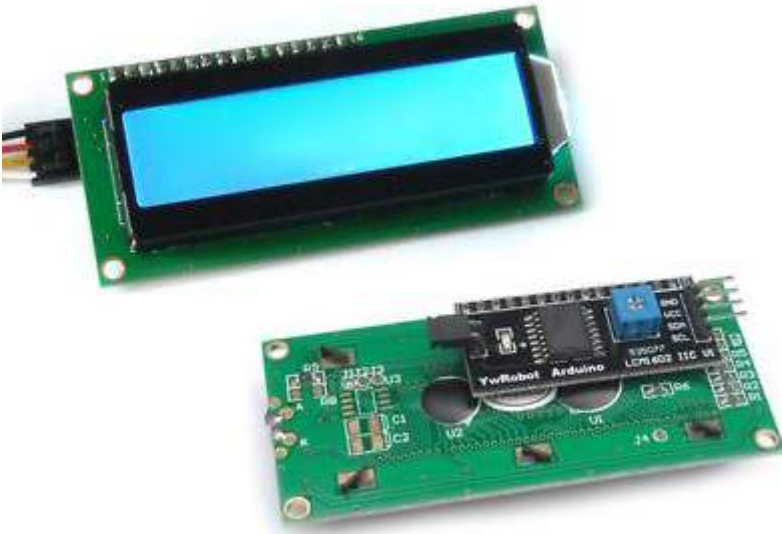


Display LCD

Si queremos visualizar datos de forma independiente del ordenador, existen matrices de led que puedes ver en los cursos de mBot o mClon pero lo mejor es una pantalla Liquid Crystal Display y la mejor forma de conectar esta interfaz es en serie utilizando el **protocolo I2C**.

Ejemplo de uso :

- en [Arduino con código](#)
- [Arduinoblocks en el aula](#)
- [Smart Home para Microbit](#)

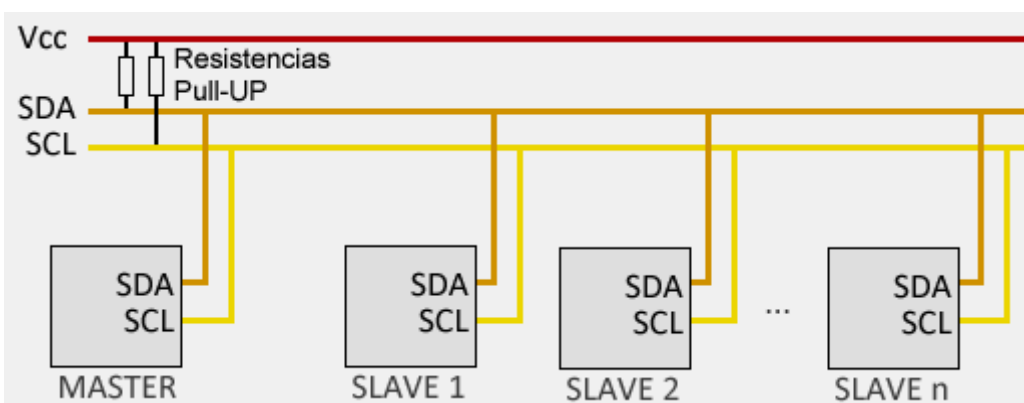


CC-BY-SA [Luis Llamas](#)

No confundas Display LCD con matriz de LEDs , o una OLED

Aclarando conceptos: Protocolo I2C

Es un protocolo de emitir los datos con sólo dos cables **SDA** (los datos) **y SCL** (reloj) y además pueden compartir varios dispositivos SLAVE) para ello la placa Arduino ESP32, microbit... (MASTER) tiene que saber **qué dirección tiene cada dispositivo IMPORTANTE:**



Fuente [Luis Llamas ArduinoI2C](#) CC-BY-SA

Aclarando conceptos: Lógica invertida

Igual que vimos en sensores, con las configuraciones Pull up y Pull down, las salidas de cualquier actuador **PUEDEN SER LÓGICA INVERTIDA**, mira en este NodeMCU (básicamente es un Arduino con wifi, aquí está conectado con el servidor Blynk) como funciona al revés, cuando apretamos, por lo tanto enviamos un uno a la salida, **se apaga la salida !!**

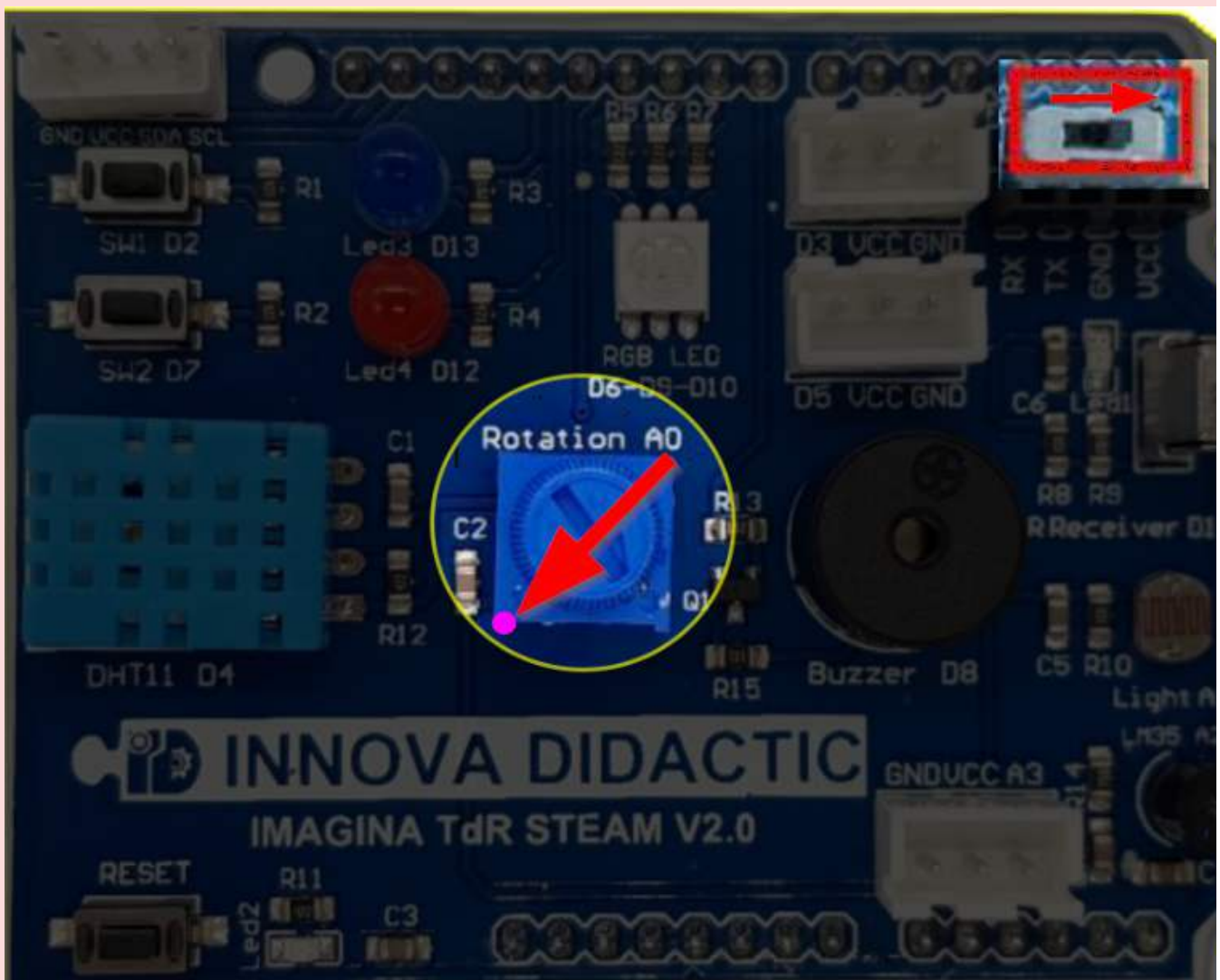
<https://www.youtube.com/embed/GLvTBdcfKAI>

Introducción

Sensores y actuadores de la placa Imagina TDR STEAM

ATENCIÓN, ANTES DE SUBIR UN PROGRAMA A TU PLACA TIENES QUE

- TENER EL POTENCIÓMETRO (8) A CERO, DE LO CONTRARIO PUEDE DAR ERROR AL SUBIRLA **FLECHA DEL POTENCIÓMETRO QUE APUNTE AL PUNTO ROJO**
- El interruptor del Bluetooth tiene que estar **hacia al borde**



Adaptado de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

- El motivo es que el potenciómetro comparte la conexión A0 (GPIO02) con el sistema de grabación del programa y si el mismo no está en su posición de cero resistencia se producirá un error en el envío del programa a la placa porque se entenderá que los pines de transmisión están ocupados con otra tarea. (Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA)
- El interruptor del Bluetooth tiene que estar **hacia al borde** de lo contrario no se comunica con el puerto COM

¿Y si no lo hago qué pasa? Pues esto:

🔔 arduinoblocks :: Info

✖ Error compilando o subiendo

("", None, "Error during Upload: uploading error: uploading error: exit status 2")

Esta página es extraída de Actividades con Imagina TDR STEAM y ArduinoBlocks

<https://github.com/arduinoblocks/libros>

Autoría: Equipo de Innova Didàctic y Robolot Team. Licencia **CC-BY-NC-ND**

La placa Imagina TDR STEAM es una placa didáctica desarrollada por el equipo ROBOLOT que presenta la gran ventaja de tener una gran cantidad de sensores, actuadores y conexiones de expansión incorporados directamente en ella. Únicamente hay que conectar esta placa a una placa Arduino UNO (en nuestro caso, una placa compatible llamada Keyestudio UNO) y ya está todo listo para empezar a programar.

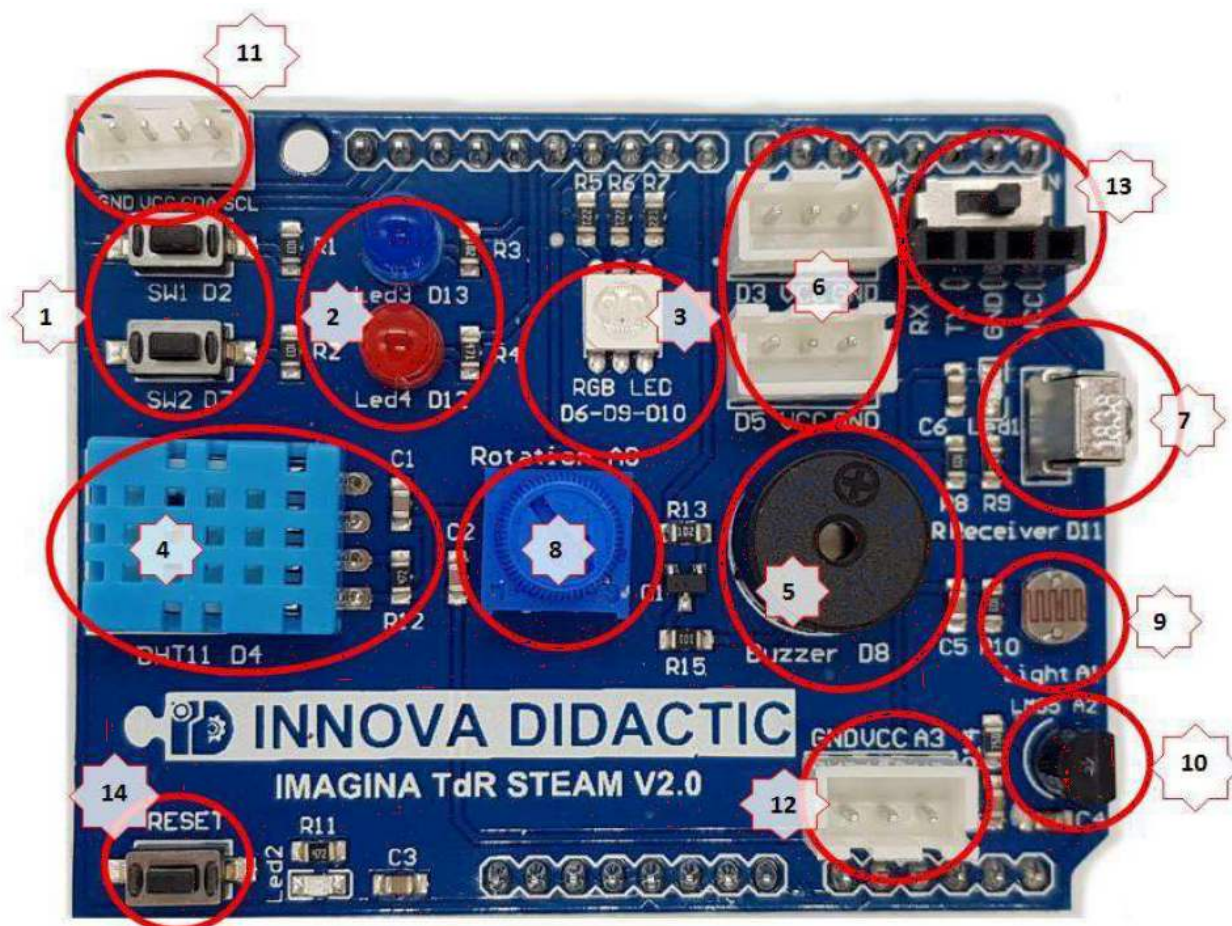


Tabla con la relación de elementos que hay en la placa Imagina TDR STEAM y sus conexiones:

	Sensor/ Actuador/ Módulo	Pin de conexión
1	Dos pulsadores (SW1, SW2)	D2 y D7
2	Dos leds (Azul Led3 y Rojo Led4)	D13 y D12
3	Led RGB	D6-D9-D10
4	Módulo DHT11 Sensor de Temperatura y Humedad	D4
5	Zumbador o Piezoeléctrico	D8
6	Dos puertos (Entradas/Salidas) digitales	D3 y D5
7	Módulo receptor de infrarrojos (IR)	D11
8	Módulo potenciómetro giratorio	A0
9	Sensor de luminosidad (LDR)	A1
10	Sensor de temperatura (LM35)	A2
11	Interface I2C compatible con sensores y módulos Keyestudio	SDA-A4 SCL-A5
12	Puerto Entrada Analógico	A3
13	Conexión de comunicaciones Bluetooth y Wifi (Swich On/Off)	Rx - Tx
14	Botón de reinicio.	-

Esta página es extraída de Actividades con Imagina TDR STEAM y ArduinoBlocks

<https://github.com/arduinoblocks/libros>

Autoría: Equipo de Innova Didàctic y Robolot Team. Licencia **CC-BY-NC-ND**

ATENCIÓN: EN LA PLACA ESP32, SI SE UTILIZA LA COMUNICACIÓN WIFI DEJA DE FUNCIONAR A0 Y A1 ES DECIR EL POTENCIÓMETRO Y EL LDR

<https://view.genial.ly/5ec3b33a8ba5700d1e8078ed>

Financiado por el Ministerio de Educación y Formación Profesional y por la Unión Europea - NextGenerationEU



Introducción

Sensores en la placa ESP32 plus STEAMakers

Página extraída de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

La placa ESP32 Plus STEAMakers se basa en el microcontrolador ESP32-WROOM-32 que dispone de dos sensores internos. Se trata de un **sensor de efecto hall** y un **sensor de temperatura**.

La placa ESP32 Plus STEAMakers lleva implementado un **medidor de tensión e intensidad** con el que podemos saber la tensión de alimentación en todo momento. Si la tensión de alimentación baja de aproximadamente 4,8V la placa no funcionará correctamente (sobre todo la transmisión WiFi). Es recomendable realizar una verificación de la tensión que está entregando el puerto USB del ordenador. Si la tensión no es próxima a 5V deberemos cambiar el cable USB, alimentar el puerto USB de forma externa o alimentar la placa con una fuente de alimentación. Es decir, dispone de un sistema para poder medir el consumo de energía.

Los bloques para trabajar con estos sensores están en 'Sensores', entrada 'Integrados', y son los que vemos en la figura siguiente:



Lógica	 Sensor de campo magnético
Control	
Matemáticas	
Texto	
Variables	
Listas	
Funciones	
TDR STEAM	
ESP	
▶ Tiempo	
▶ Entrada/Salida	
▼ Sensores	
Integrados	 Medidor de energía Amps (A)
Receptor IR	
	 Medidor de energía > Calibrar I=0
	 Medidor de energía > Reset Wh=0
	 Medidor de energía > Offset V= 0

Introducción

Software

Hemos visto que el ESP32 acepta código IDE Arduino, por lo tanto puede programarse como un Arduino, (acepta microPython, mBlock, etc...) pero nuestra recomendación es programarlo con **bloques gráfico** con dos opciones :

- **STEAMAKERSBLOCKS** (antes ARDUINOBLOCKS) es el programa que se usa en este tutorial
 - Es **lento** cargando el programa y hay que tener bien instalado Connector
 - Es sólo online
 - Permite programas **IoT**
 - Tiene una buena **gestión de alumnado** ideal para envío de proyectos y corregirlos
 - Tiene un canal de Telegram de apoyo al docente
https://t.me/innovadidactic_comunidad
- **MICROBLOCKS** El tutorial recomendado es:
<https://pedroruizf.github.io/steamakersmicroblocks>
 - No permite muchos programas IoT
 - Es muy rápido al ejecutar y no da problemas de configuración
 - Local o online

https://docs.google.com/presentation/d/e/2PACX-1vQb1Dv9wN9QK-F6V7yvwDoyzquqwWIGvlyVJr83Yk56kAoYD7bXLnYDm_tCQkeAgg/pubembed?start=false&loop=false&delayms=3000

Si quieres saber más sobre las diferencias entre programar con código, con bloques gráficos, en vivo, en carga... [aquí](#)

STEAMAKERBLOCKS

STEAMAKERBLOCKS

¿Qué es Steamakerblocks?

Esto no pretende ser un tutorial exhaustivo de STEAMAKERSBLOCKS, sino una guía rápida. STEAMAKERSBLOCKS es un programa que tiene muchas posibilidades. Si quieres saber más sobre ARDUINOBLOCKS tutoriales, ejemplos, foro.... te recomendamos

<http://arduinoblocks.didactronica.com/> o el libro [Arduino blocks - libros y tutoriales](#)

IMPORTANTE SABERLO: TENEMOS UN CHAT DE Ayuda en STEMAKERSBLOCKS

hay chat de **Telegram** con una comunidad de profesores y técnicos de la empresa que apoya Arduinoblocks donde puedes encontrar proyectos, enlaces interesantes y lo más importante: Puedes preguntar tus dudas o problemas

https://t.me/innovadidactic_comunidad



arduinoblocks + InnovaDidàctic

553 members

¿Por qué una programación con bloques?

Arduino se programa en lenguaje C++ (con algunas variaciones para simplificarlo). Para programar normalmente se utiliza el IDE ("Integrated Development Environment"/"Entorno de Desarrollo Integrado") de Arduino, que permite escribir el código, compilar el programa (crear el programa binario para el procesador Arduino) y grabarlo en la placa Arduino a través del puerto USB. El IDE de Arduino se puede descargar [desde la web oficial](#). Es totalmente libre (José Andrés Echevarría @cantabRobots CC-BY-NC-SA)

```

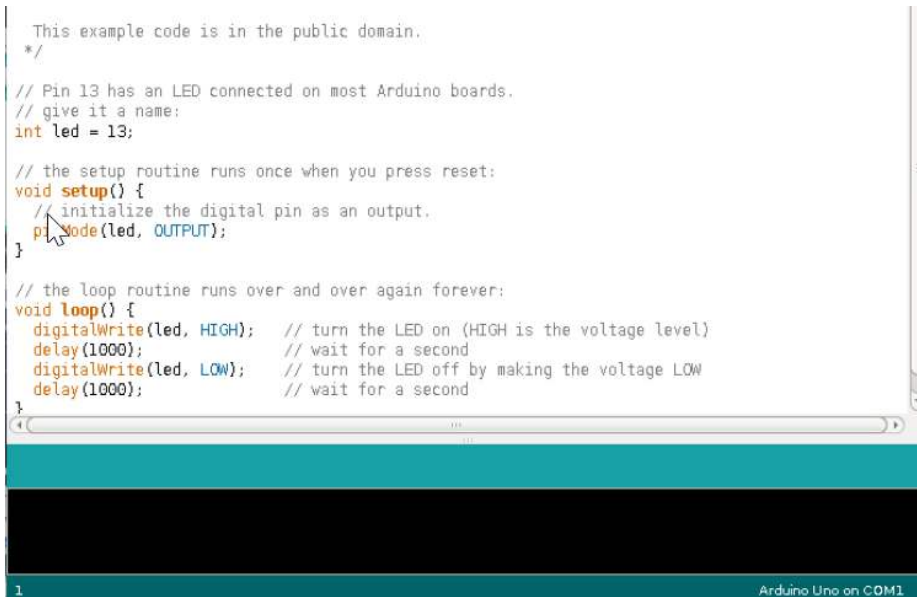
This example code is in the public domain.
*/

// Pin 13 has an LED connected on most Arduino boards.
// give it a name:
int led = 13;

// the setup routine runs once when you press reset:
void setup() {
  // initialize the digital pin as an output.
  pinMode(led, OUTPUT);
}

// the loop routine runs over and over again forever:
void loop() {
  digitalWrite(led, HIGH); // turn the LED on (HIGH is the voltage level)
  delay(1000);             // wait for a second
  digitalWrite(led, LOW);  // turn the LED off by making the voltage LOW
  delay(1000);             // wait for a second
}

```



(José Andrés Echevarría @cantabRobots CC-BY-NC-SA)

Sin embargo pensando en edades más tempranas se han desarrollado formas más sencillas e intuitivas de programar Arduino como son los **lenguajes de programación por bloques**. De todos estos lenguajes cabe destacar **STEAMAKERSBLOCKS**.

Gracias a este lenguaje visual podemos programar las placas Arduino sin necesidad de escribir ni una sola línea de código, de esta forma podemos empezar a realizar proyectos con Arduino de una forma muy rápida y a edades más tempranas. La única desventaja es que el lenguaje por código tiene todo el potencial que requiere la programación de un experto.

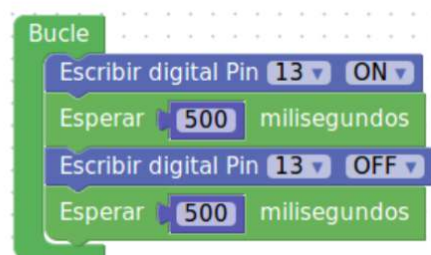
Mismo programa creado con el IDE de Arduino en C++ (imagen de la izquierda) y con Arduinoblocks (imagen de la derecha).

(José Andrés Echevarría @cantabRobots CC-BY-NC-SA)

```

void setup()
{
  pinMode(13, OUTPUT);
}
void loop()
{
  digitalWrite(13, HIGH);
  delay(500);
  digitalWrite(13, LOW);
  delay(500);
}

```



José Andrés Echevarría @cantabRobots CC-BY-NC-SA)

Para trabajar con Arduinoblocks debemos ir a su página web <http://www.arduinoblocks.com/> desde cualquier navegador y para cualquier sistema operativo (Windows, Linux, Mac). (José Andrés Echevarría @cantabRobots CC-BY-NC-SA)

STEAMAKERSBLOCKS (antes Arduinoblocks)



STEMAKERSBLOCKS es un programa creado por el profesor Juanjo López. Gracias a su entorno gráfico facilita la programación de placas Arduino a todos los niveles. Esta herramienta permite programar a personas sin conocimientos previos de programación, pero su versatilidad y potencia es tan grande que expertos programadores también pueden utilizarlo. (José Andrés Echevarría @cantabRobots CC-BY-NC-SA)



De [Juan José López Almendros](#) CC-BY-SA

La programación en **STEMAKERSBLOCKS** se realiza con bloques al estilo AppInventor o Scratch, se puede utilizar a partir de 8 años. No tenemos que escribir líneas de código y no nos permitirá unir bloques incompatibles evitando así posibles errores de sintaxis. La plataforma **STEMAKERSBLOCKS** genera, compila y sube el programa a la placa Arduino por medio de la conexión USB. Una vez subido el programa, la placa el Arduino no necesitará de la conexión al PC para funcionar pudiendo alimentarla con baterías o una fuente de alimentación para que funcione de forma autónoma.

STEMAKERSBLOCKS actualmente funciona con todos los navegadores de última generación: Mozilla Firefox, Google Chrome, Opera, Safari,... (José Andrés Echevarría @cantabRobots CC-BY-NC-SA)

Por otro lado, tal y como se describe en la [Wiki](#) de **Vitalinux**, **STEMAKERSBLOCKS** funciona perfectamente con todos los sistemas operativos, pudiendo ser fácilmente instalable en equipos individuales y a nivel de centro dentro del soporte de **Vitalinux**.

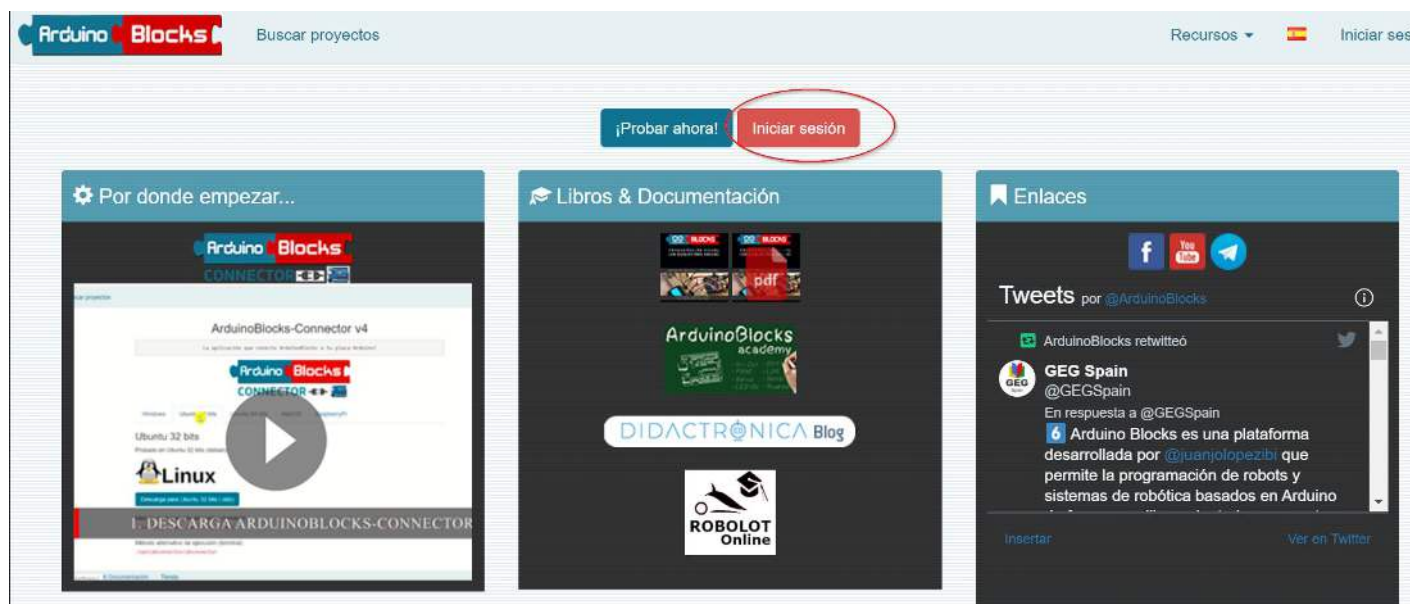
STEAMAKERBLOCKS

Crear cuenta en STEAMAKERBLOCKS

Registrándonos como usuarios de la plataforma **Steamakerblocks** podemos aprovechar todas estas posibilidades:

- Guardar tus proyectos en la nube de **Steamakerblocks**.
- Añadir información al proyecto: descripción, componentes utilizados, imágenes, etc.
- Añadir archivos adjuntos relacionados con el proyecto: esquemas, fotos, archivos para impresión 3D, aplicaciones, etc.
- Compartir proyectos con el resto del mundo.
- Importar proyectos compartidos por otros usuarios.
- Valorar y comentar proyectos.
- Programar directamente Arduino desde el propio navegador (con la aplicación: **Steamakerblocks -Connector**).
- Utilizar la consola serie desde el propio navegador.

Entramos en <https://www.steamakersblocks.com/> e iniciamos sesión



Y rellenamos el formulario

[Buscar proyectos](#)

Nuevo usuario

*** Recommended **GMail** accounts (Review SPAM folder) *** (Hotmail, Msn,... may not work due to spam filters)

Correo electrónico

Confirmación de correo
electrónico

Clave

Confirmación de clave

Nombre

Apellidos

País

Ciudad

Autor José Andrés Echevarría @cantabRobots CC-BY-NC-SA

STEAMAKERBLOCKS

Cuentas alumnos

Tal y como dice el tutorial de Juanjo López : *Permite a un usuario registrado con email, crear y administrar nuevas cuentas de usuario dentro de una organización, centro educativo o institución.*

- ♥ Permite crear usuarios alumnos sin necesidad de ceder ningún tipo de datos
- ☐ Puedes pasar proyectos a los alumnos, vacíos o empezados
- ☐ Tú puedes ver los proyectos de los alumnos y ponerles comentarios

<https://www.youtube.com/embed/zdzKX0NX60Y>

- Si lo quieres en papel, te recomendamos el tutorial de Juanjo López son 12 diapositivas muy bien explicados
 - [Repositorio Github](#)
 - [Repositorio en steamakerbloks.com](#)
- Prueba con Catedu:
 - <https://www.steamakersblocks.com/>
 - usuario alumnox.catedu donde x de 1 a 20
 - contraseña [donde esta catedu en minúsculas]

STEAMAKERBLOCKS

Connector

Espera !!! Aún no conectes tu placa (ESP32, Arduino, TDR STEAM...)

PRIMER PASO Descargar e instalar Connector

Para poder usar la herramienta **Steamakerblocks** tenemos que ejecutar antes **Connector**. Lo descargamos de la misma página de **Steamakerblocks** según el sistema operativo que usemos: Windows (**W7 E INFERIORES NO FUNCIONA**), Linux

Entra en <https://www.steamakersblocks.com/> y en Recursos, tienes la web para descargar este programa:



Lo descargamos y lo **instalamos**.

En el caso de tener equipos Vitalinux, es fácilmente accesible e instalable desde la aplicación **Vitalinux Play** o si se desea una instalación masiva en el centro a través de su página de [soporte](#):



SEGUNDO PASO: INSTALAR LOS DRIVERS

Si no hacemos estos pasos, cuando conectamos la placa, siempre sale en el COM1, le damos a subir y sale error

En <http://www.arduinoblocks.com/web/site/abconnector5> tenemos abajo ARDUINO SERIAL DRIVERS

RECOMENDAMOS EL PRIMER ENLACE Y EL TERCERO



AB-Connector v5

Windows

Ubuntu

MacOS

Chromebook



v5.3 (Windows 64)

v5.3 - Descarga para Windows 64 (Installer .exe)

v5.3 - Descarga para Windows 64 [Portable .zip]

v5.1 (Windows 32/64)

[Descarga para Windows \(Installer .exe\)](#)[Descarga para Windows \[Portable zip\]](#)

¡Desactiva el antivirus si la descarga falla!

Arduino serial drivers:[Arduino + FTDI usb-serial converter](#)[Arduino + CH340G usb-serial converter](#)[Arduino + CP2102 usb-serial converter](#)

En el primero el instalador está en este enlace

https://cdn.sparkfun.com/assets/learn_tutorials/7/4/CDM21228_Setup.exe



The screenshot shows the SparkFun website's 'How to Install FTDI Drivers' tutorial. A blue callout bubble with the text 'AQUI TIENES EL INSTALADOR' points to a red-bordered box containing the download link: 'WINDOWS FTDI VCP DRIVER EXECUTABLE - V2.12.28 (COM21228.SETUP.EXE)'. The page includes a sidebar with a 'Pages' list (Introduction, Meet the FT232RL, Windows - Quick and Easy, Windows - In Depth, Mac, Linux, Resources and Going Further) and a 'Comments' section with 14 comments.

El segundo sólo si quieres utilizar Arduinos no oficiales, de fabricantes chinos, que tiene el CH340g y hay que leerse la página, paciencia

El tercero es necesario el 2102 si utilizas el ESP32 el instalador esta en este enlace, es una carpeta comprimida, la descomprimes y está el ejecutable instalador

https://www.silabs.com/documents/public/software/CP210x_Windows_Drivers.zip



Download and Install VCP Drivers

Downloads for Windows, Macintosh, Linux and Android below.

*Note: The Linux 3.x.x and 4.x.x version of the driver is maintained in the current Linux 3.x.x and 4.x.x tree at www.kernel.org.

Software Downloads

Software (11)

Software · 11

CP210x Universal Windows Driver	v11.3.0 6/24/2023
CP210x VCP Mac OSX Driver	v6.0.2 10/26/2021
CP210x VCP Windows	v6.7 9/3/2020
CP210x Windows Drivers	v6.7.6 9/3/2020
CP210x Windows Drivers with Serial Enumerator	v6.7.6 9/3/2020

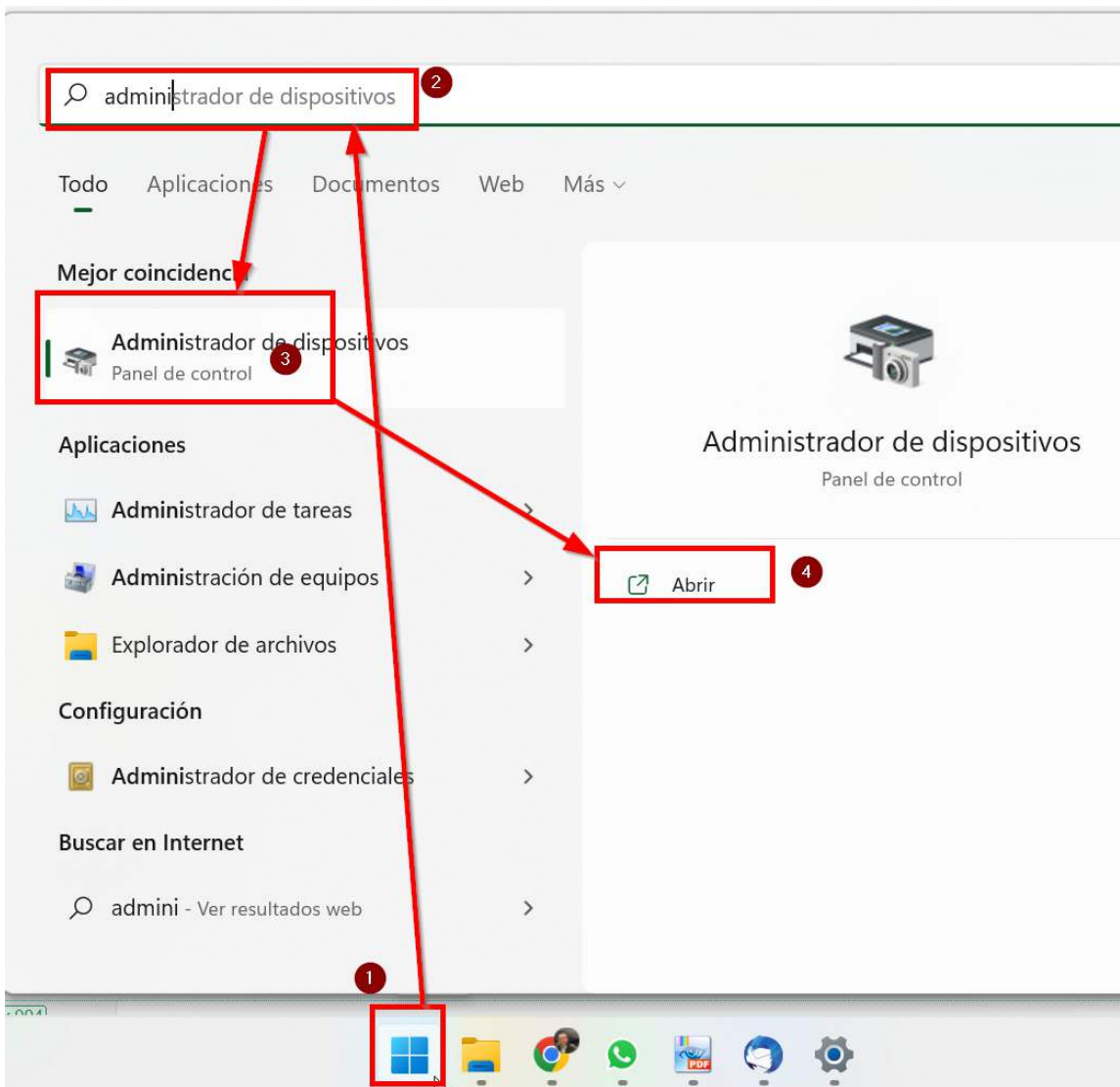
Show 6 more Software

concretamente hay que ejecutar este (al menos que el equipo sea muy viejo de 32bits)

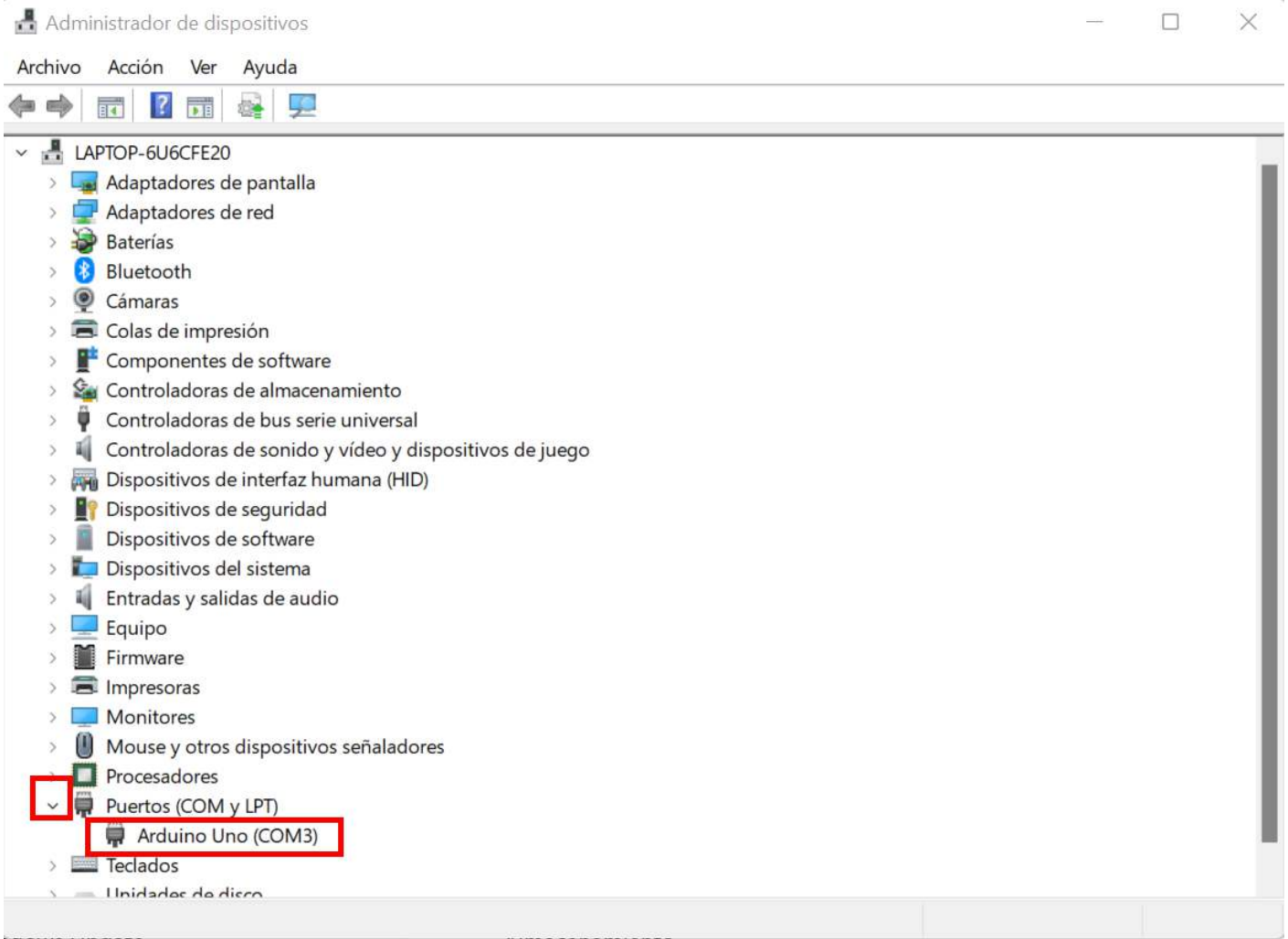
CP210xVCPInstaller_x64.exe	19/01/2026 8:53	Aplicación	1.026 KB
CP210xVCPInstaller_x86.exe	19/01/2026 8:53	Aplicación	903 KB
dpinst.xml	19/01/2026 8:53	Microsoft Edge H...	12 KB
SLAB_License_Agreement_VCP_Windows...	19/01/2026 8:53	Documento de tex...	9 KB
slabvcp.cat	19/01/2026 8:53	Catálogo de segur...	11 KB
slabvcp.inf	19/01/2026 8:53	Información sobre...	8 KB
v6-7-6-driver-release-notes.txt	19/01/2026 8:53	Documento de tex...	16 KB
x64	19/01/2026 8:53	Carpeta de archivos	
x86	19/01/2026 8:53	Carpeta de archivos	

COMPROBAR QUE DETECTA LA PLACA

Ahora **conectamos la placa** (ESP32, Arduino, NodeMCU, KeyStudio TDR STEAM...) a nuestro ordenador, y observamos si lo detecta, en Windows entramos en Administrador de dispositivos:



Y vemos que en los puertos COM se ha detectado correctamente la placa:



En el caso de que no aparezca, es que no se han instalado correctamente los *drivers* de Arduino. Entonces vamos a la página oficial de Arduino y descargamos el programa **ARDUINO IDE** : <https://www.arduino.cc/en/software> y lo instalamos. Al instalar este programa se instalan los drivers en nuestro ordenador. No hace falta ejecutarlo.

En el caso de equipos con sistema operativo Linux (como Vitalinux) el puerto serie tiene la forma **/dev/XXXX**

YA PUEDES EJECUTAR ARDUINOBLOCKS CONNECTOR

Ahora buscamos el programa ArduinoBlocks connector que hemos descargado e instalado en el primer paso y lo **ejecutamos**.



```
12:36:51> AB-Connector v5.3
12:36:51> Path: C:\Program Files\abconnector\bin
12:36:51> Port: 9987
12:36:51> Arduino-CLI: 0.35.3
12:36:51> ['arduino:avr', 'esp32:esp32', 'esp8266:esp8266']
12:36:51> Checking/updating libs...
12:36:51> Libraries version: 56
```

ATENCIÓN No podemos cerrar la ventana mientras utilizamos *Arduinoblocks*, la minimizamos simplemente.

En caso contrario, Arduinoblocks no se puede comunicar con nuestra placa Arduino, NodeMCU, KeyStudio, etc

YA PUEDES EJECUTAR ARDUINOBLOCKS

Entramos en la web ARDUINOBLOCKS <http://www.arduinoblocks.com/> nos logueamos e iniciamos un proyecto, Vemos que en el editor que aparece ya los puertos COM (si no te aparece, dale a la rueda actualizar)

Aparecen varios COM, **elegir el último que tiene que coincidir con el que has visto en el segundo paso**, no necesariamente es el COM más alto.

Si se queda una ruleta de espera demasiado tiempo, entonces, actualizar la página o darle a actualizar el botón 1 de la figura :



Una vez elegido el COM ya puedes darle al botón amarillo **SUBIR** cuando has realizado tu proyecto pero antes de subir, por si acaso dale a **guardar** el proyecto que has realizado.

En el caso de equipos con Linux veremos algo así:



¿Tengo que hacer los cuatro pasos cada vez?

No, sólo la primera vez para asegurar los drivers del Arduino, las siguientes veces que te conectes lo único que tienes que hacer es el tercer y cuarto paso

IMPORTANTE: TENER EL SOFTWARE ARDUINOBLOCKS ACTUALIZADO para que funcionen los nuevos bloques que se incorporan en Arudinoblocks

STEAMAKERBLOCKS

Empezando un proyecto

Entramos en Proyectos y podemos ver nuestros proyectos creados como también empezar uno.



Y nos aparece tres opciones :



En esta ventana podremos elegir que tipo de proyecto vamos a realizar:

- **Proyecto Personal:** Iniciar un nuevo proyecto que sólo será accesible para el usuario. Posteriormente se puede compartir al resto de la comunidad si se desea.
- **Proyecto Profesor:** Iniciar un proyecto como profesor. De esta forma no se inicia un proyecto como tal, sino que se especifican los datos del proyecto y se genera un código para que los alumnos se puedan suscribir al proyecto. El profesor podrá supervisar y valorar los proyectos de sus alumnos.
- **Alumno:** De esta forma nos unimos a un proyecto planteado por el profesor. Nosotros realizaremos el proyecto como si de un proyecto personal se tratara, pero

el profesor podrá supervisar y valorar nuestro trabajo.

Adaptado de [este enlace](#). José Andrés Echevarría @cantabRobots CC-BY-NC-SA

Lo primero que tenemos que elegir es para qué tipo de placa se hace el proyecto.

- En el caso de que estés con el kit de CATEDU **Rover marciano con Arduinoblocks** el tipo de proyecto es para **ESP8266 / NodeMCU**
- En el caso de que estés con el kit de CATEDU **Arduino con Arduinoblocks** el tipo de proyecto es para **Arduino UNO**
- En el caso de que estés con el kit de CATEDU **STEAMAKERSBLOCKS en el aula** tienes dos opciones totalmente válidas:
 - **ArduinoUno**
 - **ArduinoUno + Imagina TdR STEAM**
- En el caso de que estés con el kit de CATEDU **ESP32 en el aula** tienes dos opciones totalmente válidas:
 - **ESP32 STEAMakers**
 - **ESP32 STEAMakers + Imagina TdR STEAM**
- En el caso de que estés con el kit de CATEDU **SMARTHOME ESP32** tienes que elegir:
 - **ESP32 STEAMakers**
-

Nuevo proyecto personal

Tipo de proyecto	
Nombre	
Descripción	
Componentes	

Arduino Uno

Arduino Nano / ATmega328

Arduino Nano / ATmega328 (new bootloader)

Arduino Mega / 2560

Arduino Leonardo

Arduino UNO + Imagina TdR STEAM

3dBot / Imagina-Arduino

Keyestudio EasyPlug

Keyestudio KeyBot

Otto DIY / Nano

Otto DIY / Nano (new bootloader)

ESP8266 / NodeMCU v2

ESP8266 / WeMos D1

ESP32 / WROOM

ESP32 STEAMakers

ESP32 STEAMakers + Imagina TdR STEAM

ESP32 STEAMakers + 3dBot

ATENCIÓN luego **NO** se puede cambiar. Es decir, un proyecto realizado para un tipo de placa, no se puede cambiar a otro tipo de placa (la razón es simple: las instrucciones cambian)

Luego el nombre y el resto de campos es optativo pero **importante y buena costumbre** rellenarlos, sobre todo si el proyecto lo **compartimos**:

- Descripción
- Componentes
- Comentarios

Área de programación del proyecto

Este es el área sobre el que se trabaja en Arduinoblocks. En esta área arrastraremos y colocaremos los bloques que vamos a utilizar para crear nuestro programa.

En el área de trabajo hay un Zoom (2) para ampliar o reducir la imagen, un icono para centrar (1) y un icono donde podremos borrar los bloques que no utilicemos (3).



Adaptado de [este enlace](#). José Andrés Echevarría @cantabRobots CC-BY-NC-SA

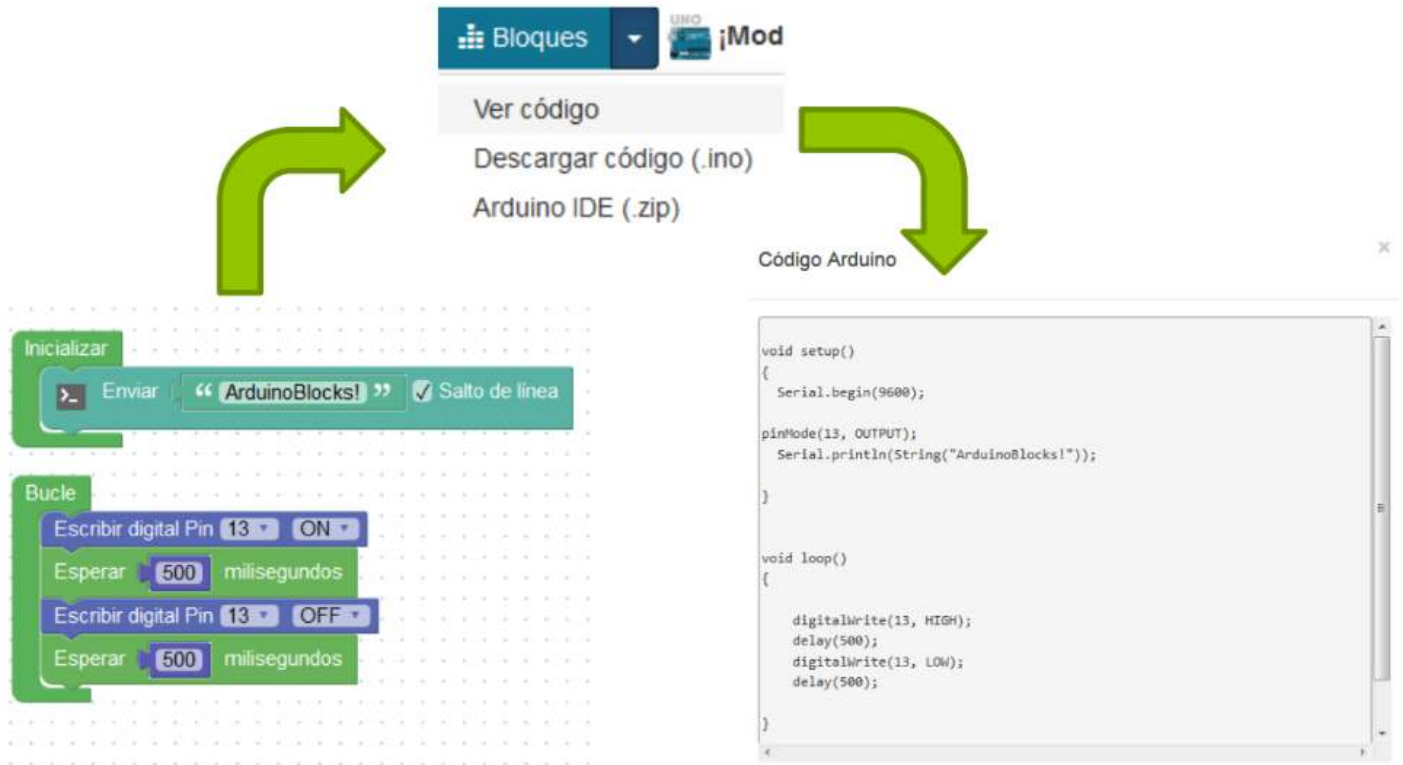
Las principales secciones del área de programación son las siguientes :

Herramientas	Área de programa	Opciones
- Lógica - Control - Matemáticas - Texto - Variables - Listas - Funciones - Entrada/Salida - Tiempo - Puerto serie - Bluetooth - Sensores - Actuadores - Pantalla LCD - Memoria - Motor - Motor-Shield - Keypad - Reloj RTC - GPS	Bloque de iniciación  Bloque de bucle del programa principal 	Subir el programa a la placa Arduino conectada:  Puerto de conexión de la placa Arduino:  Mostrar la consola serie: 

Adaptado de [este enlace](#). José Andrés Echevarría @cantabRobots CC-BY-NC-SA

Ver el código

ArduinoBlocks genera el código de Arduino a partir de los bloques. El programa se puede compilar y subir directamente a la placa Arduino gracias a la aplicación **ArduinoBlocks-Connector**, sin embargo si deseamos ver o descargar el código podemos realizarlo desde el área de bloques.



Adaptado de [este enlace](#). José Andrés Echevarría @cantabRobots CC-BY-NC-SA

Siempre, desde un **lenguaje de programación en bloques** podemos obtener su equivalente a **Código de Arduino IDE** (de hecho es lo que hacen los programas), y luego con las funciones de **Código de Arduino IDE** el software lo pasa a **lenguaje máquina** que es la que se graba el Arduino, **pero no al revés** es decir, no existen programas que dado un código máquina o código Arduino IDE lo pasen a bloques gráficos, (igual que no hay programas que lean el código máquina que hay grabado en un Arduino y lo pasen a código Arduino IDE). Esto no es del todo 100% verdadero pues la Ingeniería inversa en informática trata pues de eso: obtener la fuente aunque sea parcial, pues si obtienes el código legible, puedes alterar lo que quieras.

Cuando compras un programa comercial, te dan el lenguaje máquina ilegible. Mientras que los programas de software libre se publica el código fuente legible para que todo el mundo pueda mejorarlo.

Por ejemplo en la siguiente figura, el programa gráfico **mBlock** que se utiliza en Arduino, mBot, etc... pasa sus instrucciones de lenguaje de programación de bloques parecido a Scratch a lenguaje de **Código de Arduino IDE** y Arduino IDE graba instrucciones binarias de **lenguaje máquina** al Arduino.



¡¡A disfrutar!!

Consejo: Te recomendamos visitar el canal de Youtube de Arduinoblocks

<https://www.youtube.com/c/ArduinoBlocks>

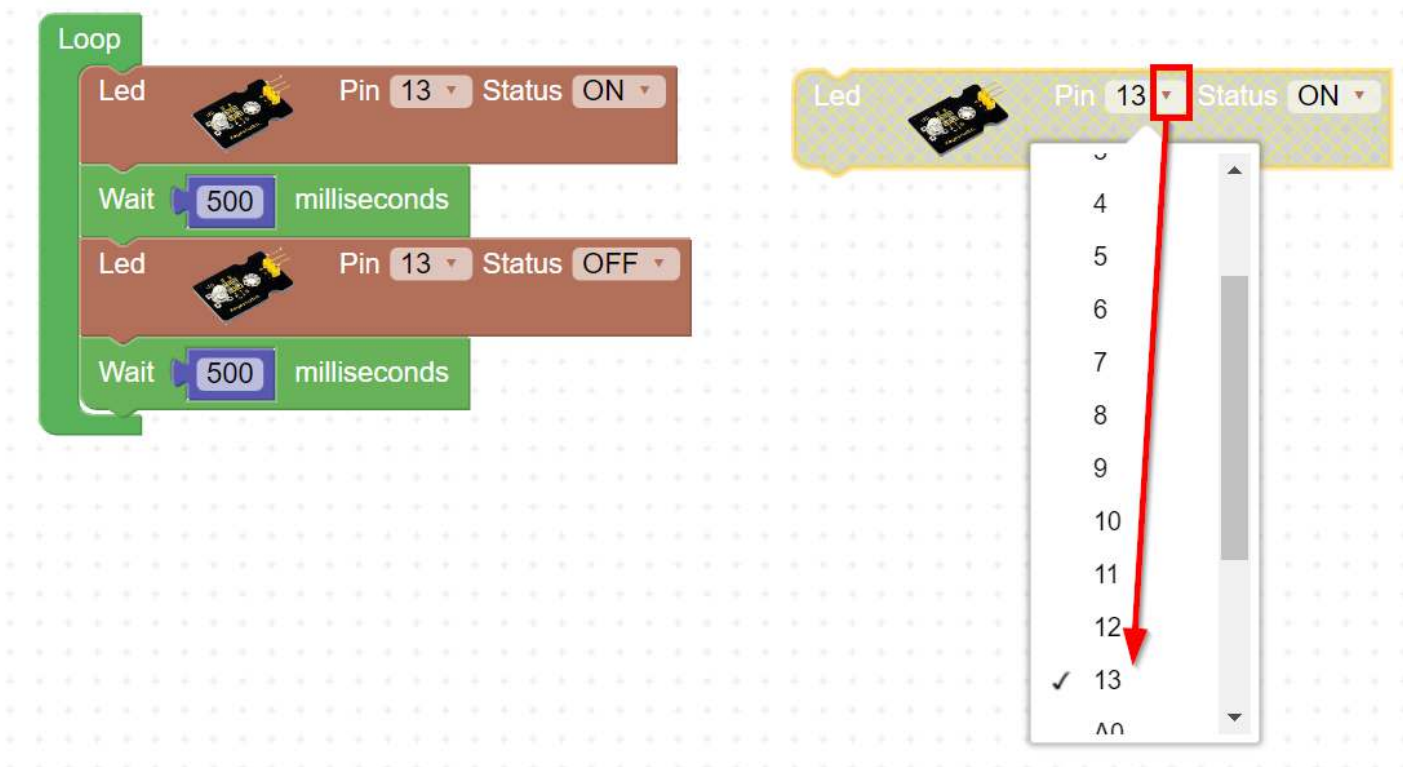
Con Imagina TdR STEAM ¿Qué tipo de proyecto elijo?

Para aclarar, las dos opciones son válidas para nuestros proyectos, pero la específica con Imagina TdR STEAM es más cómoda:

- **La placa sola (ESP32 STEAMakers)** donde te aparecerán todos los sensores y salidas con todas las posibilidades de entradas y salidas (E/S) de la placa (Arduino o ESP32)
- **La placa (ESP32 STEAMakers) + Imagina TdR STEAM** donde aparecen:
 - Los sensores y salidas específicas de la placa TdR STEAM ya conectadas en sus respectivos pines E/S del ESP32.
 - En contra, algunos sensores ya no aparecen todas las E/S del Arduino que están ocupadas por los elementos de la placa TdR STEAM.

Vamos a poner un ejemplo: Hacer una intermitencia con el LED AZUL del TdR STEAM

Si lo hacemos con el tipo de proyecto **ESP32 STEAMaker** tenemos que poner el Actuador LED y en el pin elegir el 13 porque el led Azul está conectado en el 13. Fíjate que en el desplegable del Pin están todas las E/S del ESP32



En cambio si elegimos un proyecto tipo **ESP32 STEAMaker+ Imagina TdR STEAM** veremos que tenemos añadidos unos actuadores específicos para esta placa TDR STEAM que es más fácil usarlos, donde en el desplegable sólo aparece ROJO o AZUL y el alumno **no necesita recordar** que ROJO es el 12 y AZUL es el 13

Pero si elegimos un led del desplegable de actuadores, veremos que en su desplegable de pines sólo aparece los números 3, 5 y A3 **porque son los que tiene libres, el resto están ocupados por la placa TdR STEAM**



Está claro que el tipo de proyecto **ESP32STAMaker+ Imagina TdR STEAM** es más cómodo por que los bloques ya tienen fijados qué pines tienen cada sensor y si eliges un sensor no incluido en la placa, ya no muestra los pines que están ocupados.

Si eliges **ESP32STEAMaker + Imagina TdR STEAM** es muy cómoda la programación y se evitan muchos errores.

Si eliges **ESP32STEAMaker** utilizas un entorno de programación como si la placa Imagina TdR STEAM no existiera y los alumnos se acostumbran a usar los elementos propios de ESP32, los pines, etc.... pero tienen que saber en cada momento qué pin utilizar.

Aconsejamos en este caso que los alumnos tengan impresa la página [Componentes de la placa TdR STEAM](#).

En los apuntes de este libro vamos a utilizar el proyecto indistintamente las dos formas de trabajar.

Prácticas con sensores digitales

Prácticas con sensores digitales

Actividad-01. LED

Página extraída de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Enunciado

Realizar un programa que encienda y apague el LED rojo conectado al pin D12 DE LA PLACA STEAMMAKER.

Teoría

El diodo LED (Light Emitting Diode) es un diodo semiconductor capaz de emitir luz, lo mas usuales dentro del espectro visible aunque también pueden ser de infrarrojos, laser, etc. Su uso mas habitual es como indicador y, últimamente cada vez mas frecuentes en iluminación. Sus principales ventajas frente a luces incandescentes son:

- Menor consumo de energía
- Mayor vida útil
- Menor tamaño
- Gran durabilidad y fiabilidad

En la imagen siguiente vemos el aspecto físico que tiene y su símbolo electrónico.



Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

El color de la cápsula es simplemente orientativo de la longitud de onda que define realmente el color de la luz emitida. Por ello el LED con la cápsula transparente puede emitir en cualquiera de los colores del espectro visible.

El LED es un dispositivo que tiene polaridad siendo su comportamiento el siguiente: En polarización directa (ánodo a positivo y cátodo a negativo) el LED emite luz y en polarización inversa (ánodo negativo y cátodo positivo) se comporta prácticamente como un interruptor abierto.

Para su correcto funcionamiento el diodo LED se polariza poniéndole en serie una resistencia que limita la corriente que pasa a través del mismo y, por tanto, determina el nivel de brillo de la luz emitida.

Sin entrar en detalles en la tabla siguiente se dan los valores de tensión directa (V_F) y corriente directa (I_F) para los colores mas habituales de LEDs. A partir de estos valores y el valor de tensión de alimentación de nuestro LED podemos calcular el valor de la resistencia serie sin mas que aplicar la formula indicada.

Color del LED	Tensión directa (V_F)	Corriente directa (I_F)	$R = \frac{V_{CC} - V_F}{I_F}$
Rojo	1,5 V	15 mA	
Verde	1,8 V	15 mA	
Amarillo	1,8 V	15 mA	
Azul	3 V	20 mA	
Blanco	2,7 V	20 mA	

Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Si quieres saber más de cómo calcular la Resistencia para un led [mira esta hoja de cálculo](#) pero sustituye la V_{CC} por 5 V

En la TdR STEAM

La placa Imagina TDR STEAM dispone de un LED rojo conectado al pin D12 tal y como se indica en la serigrafía de la propia placa y que podemos ver en la imagen siguiente:

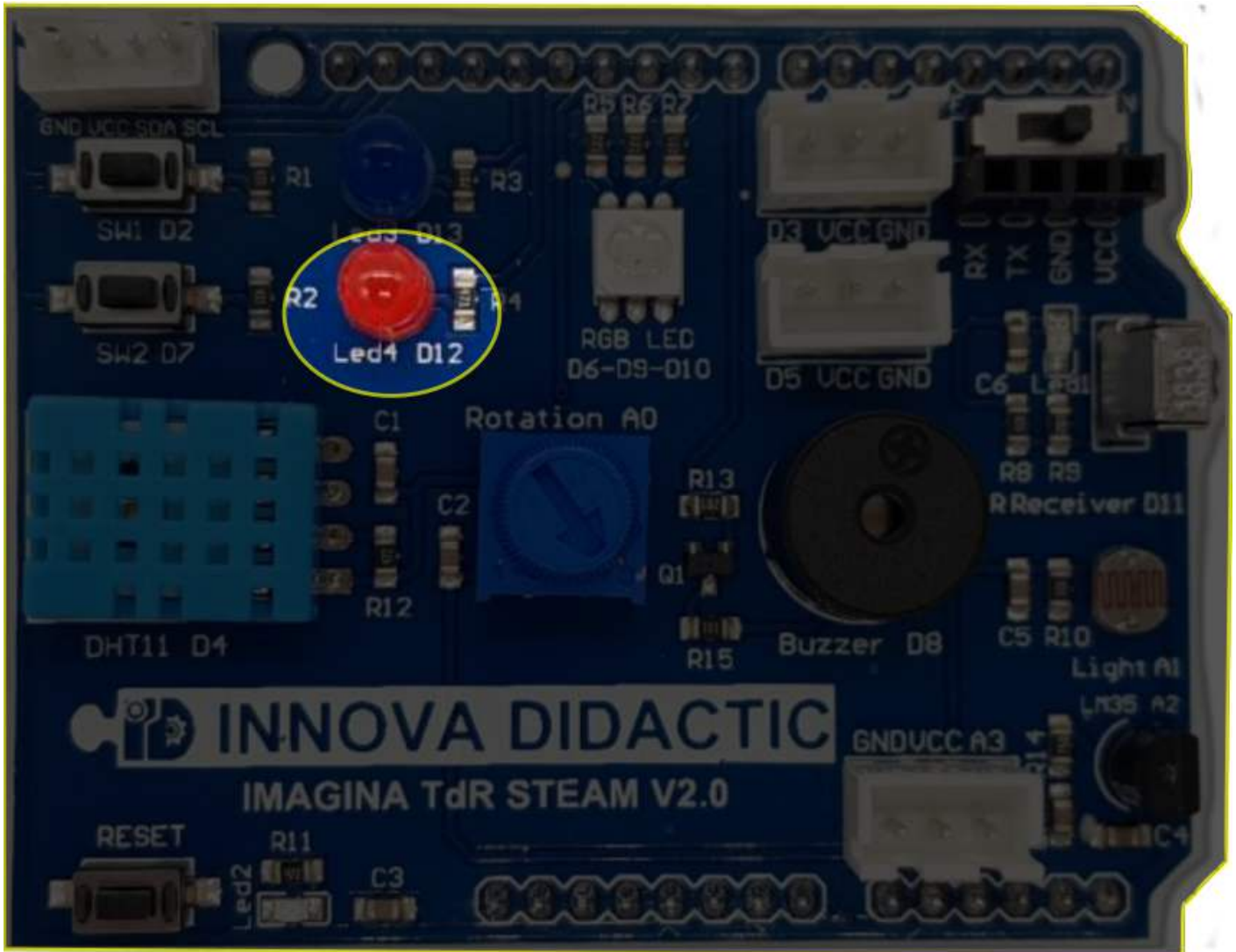


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Programando la Actividad

Entramos en ArduinoBlocks y nos identificamos convenientemente. Vamos a crear un nuevo proyecto para la placa ESP32 STEAMakers + TdR STEAM.

Escogemos el bloque LED de TDR STEAM y lo colocamos en el bucle, quedando algo similar a la imagen siguiente:

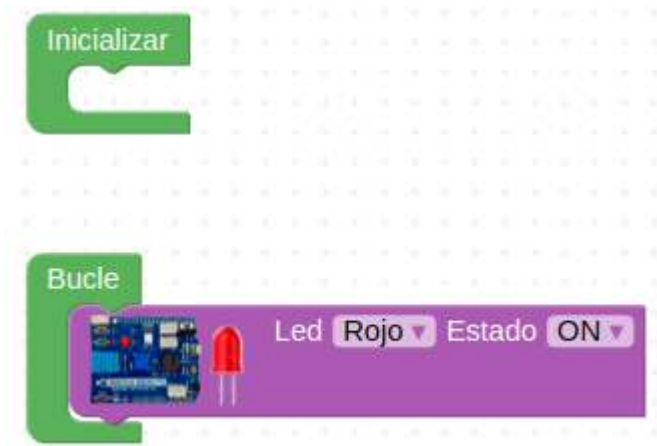


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Bloque LED de TDR STEAM

Podemos comprobar haciendo clic sobre las flechas como podemos cambiar de LED y también como podemos cambiar de estado al LED. Si solamente dejamos este bloque el LED permanecerá encendido de forma permanente y para que se acabe debemos ponerlo en estado OFF. En la imagen siguiente vemos el programa como quedaría.

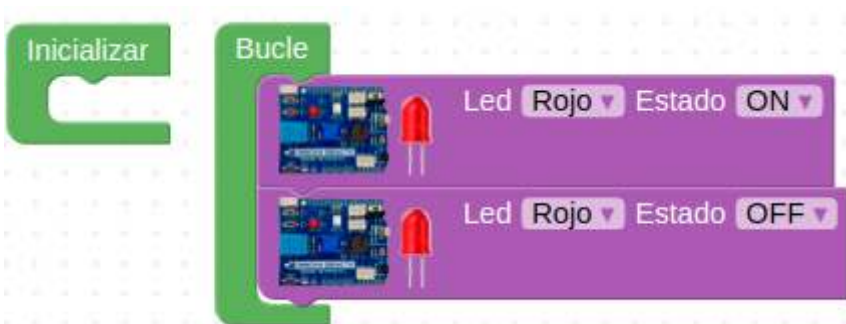


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

LED D12 a ON y OFF

Pero este programa no nos permite ver el efecto de encendido y apagado del LED debido a la velocidad de procesamiento que tiene la placa UNO que típicamente trabaja a 16 MHz. Esto supone que el micro tarda en ejecutar una instrucción algo mas de 0,06 microsegundos que es mucho menor que la persistencia visual humana de 0,1 segundo aproximadamente. Es decir, en realidad el LED se enciende y se apaga, pero nuestro ojo no puede apreciarlo y lo verá siempre encendido debido a la persistencia visual.

Tenemos por tanto que dejar el diodo un tiempo encendido y otro apagado (pueden ser el mismo tiempo) y para ello vamos a colocar un bloque Esperar desde el bloque de Tiempo. Si dejamos el tiempo por defecto en 100 milisegundos el diodo se encenderá y apagará cada segundo (1000 ms = 1s).

El programa final queda como vemos en la imagen siguiente y lo tenemos disponible en el enlace [ESP32-SM-Actividad-01](#).

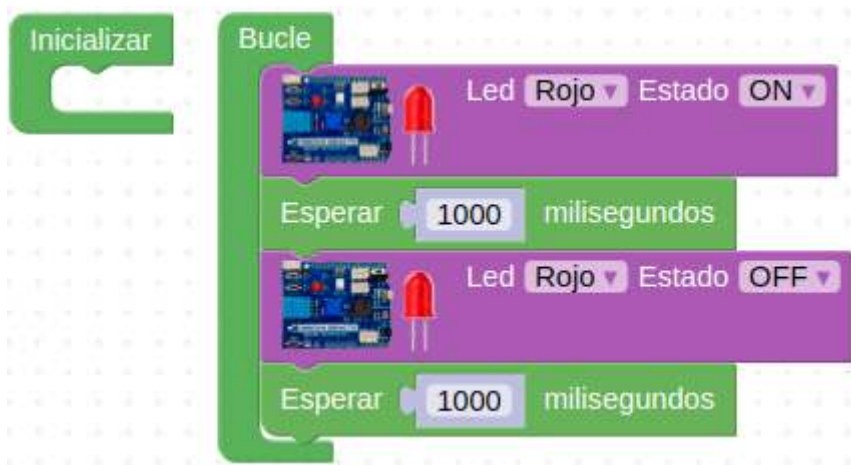


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Programa final para el LED rojo

Coloca la placa Imagina TDR STEAM encima del ESP32

Conectamos nuestra placa a un puerto USB del ordenador, ponemos en marcha el programa Connector y cargamos el programa. Podemos observar como el diodo LED rojo parpadea con un intervalo de un segundo.

De esta forma el programa queda grabado en la memoria de programa del microcontrolador y el ciclo se repetirá por tiempo indefinido o hasta que quitemos la alimentación a la placa. Si alimentamos la placa externamente con una fuente de alimentación se ejecutará el programa en memoria.

Retos de ampliación

R1.A1. Cambiar los tiempos para que el parpadeo sea más rápido, mas lento y que los tiempos de encendido y apagado no coincidan.

R2.A1. Repetir el ejercicio A1 utilizando el LED azul conectado al pin D13.

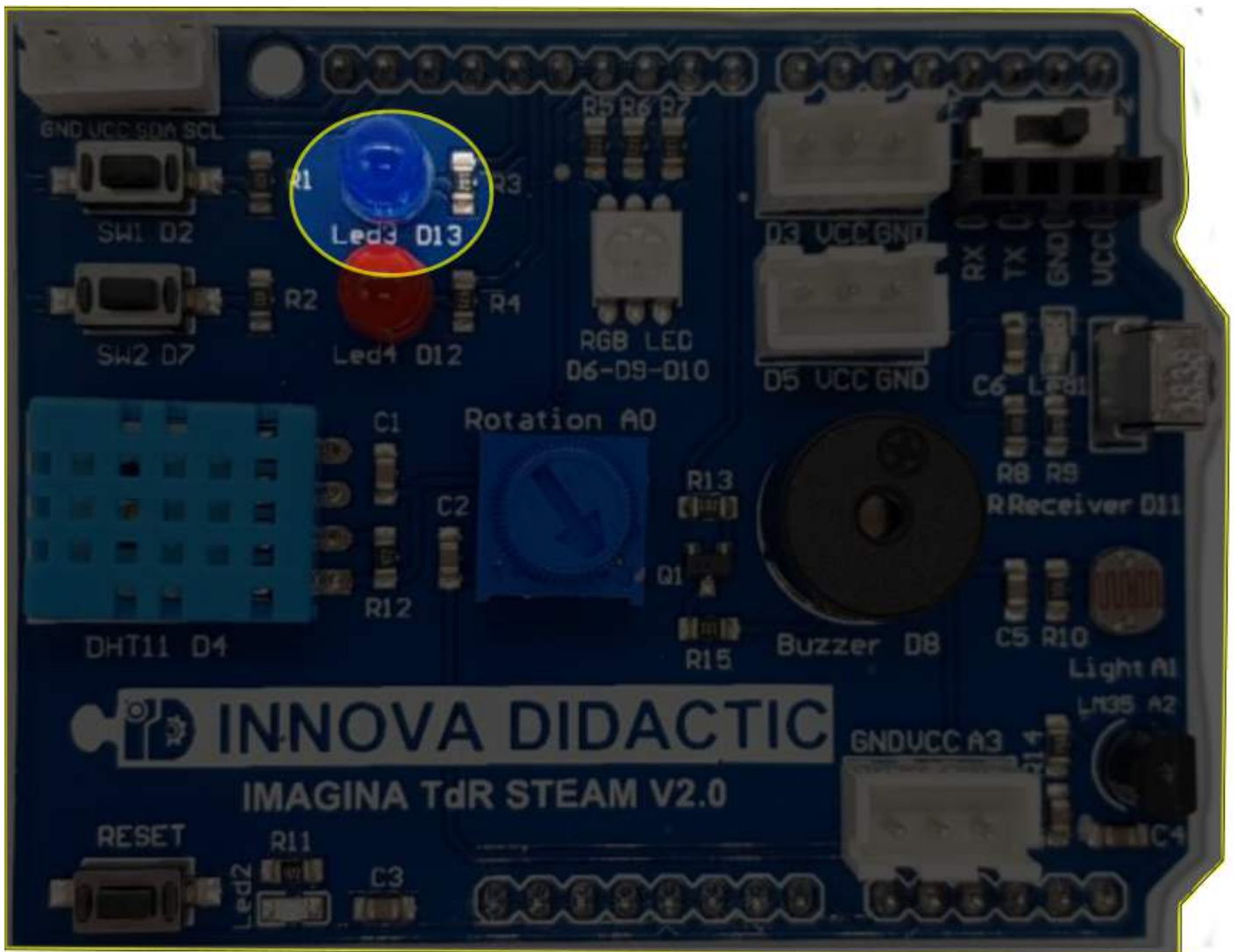


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Prácticas con sensores digitales

Actividad-02. LEDs rojo y azul

Página extraída de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

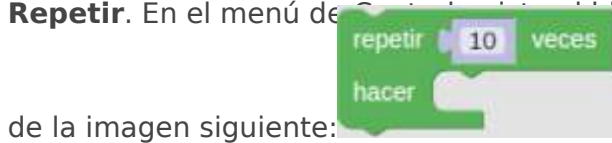
Enunciado

Como ya hemos visto anteriormente la placa dispone de dos LED (uno rojo y otro azul). Vamos a realizar un programa para que se vayan alternando en su encendido y apagado.

Teoría

Vamos a ver como se hacen los ciclos de repetición o bucles en ArduinoBlocks.

- **Repetir.** En el menú de **Ciclos** encontramos el bloque 'Repetir (valor) veces hacer...', como el



de la imagen siguiente:

Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Lo que pongamos en hacer se va a repetir tantas veces como indiquemos en el número de veces, que por defecto estará a 10.

En realidad lo que estamos haciendo es lo que en programación se conoce como bucle for.

- **Repetir según condición.** En la imagen siguiente vemos dos bloques que repiten su interior mientras, o hasta, que se cumpla una condición.



Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

- **Contador.** Realiza un bucle contando con un variable índice (normalmente i o j). Se define un valor de inicio, un valor de fin y los incrementos que se realizarán en cada iteración del bucle. Dentro del bucle podremos usar esta variable índice.



Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

En la TdR STEAM

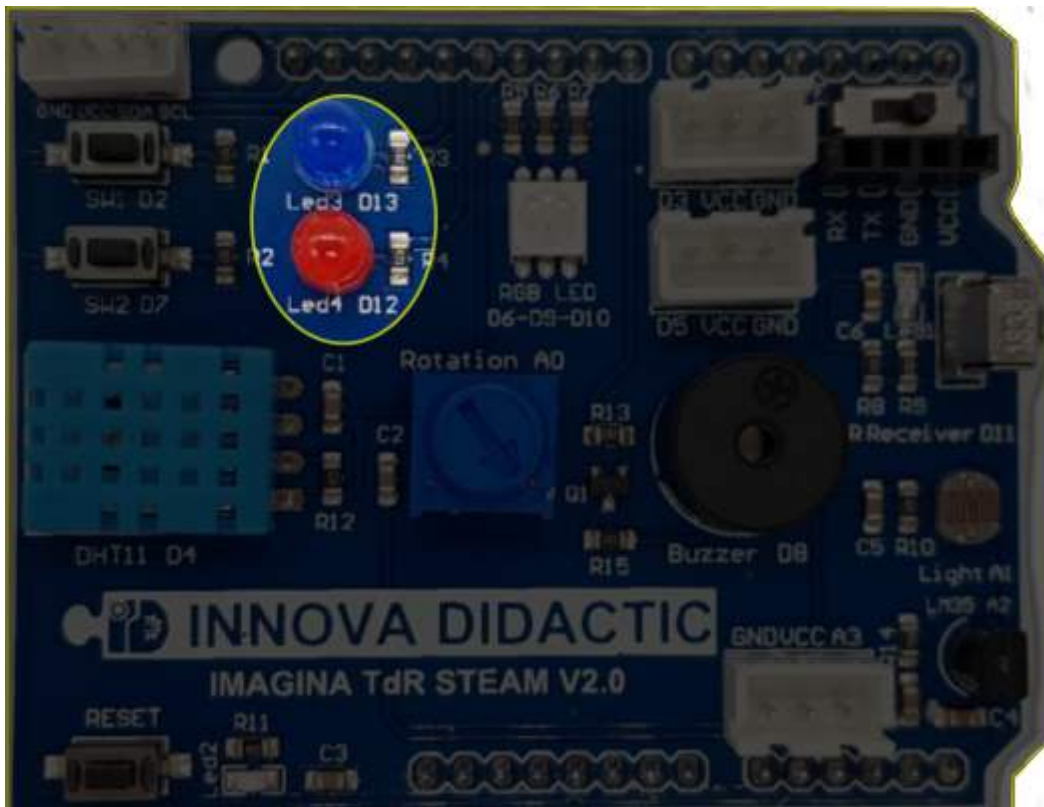


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Los dos LEDs de la TdR-STEAM

Programando la Actividad

Vamos a hacer que ambos diodos se enciendan y apaguen de forma simultanea con un programa como el siguiente, que lo tenemos disponible en [Actividad-02. LED-rojo-azul](#)

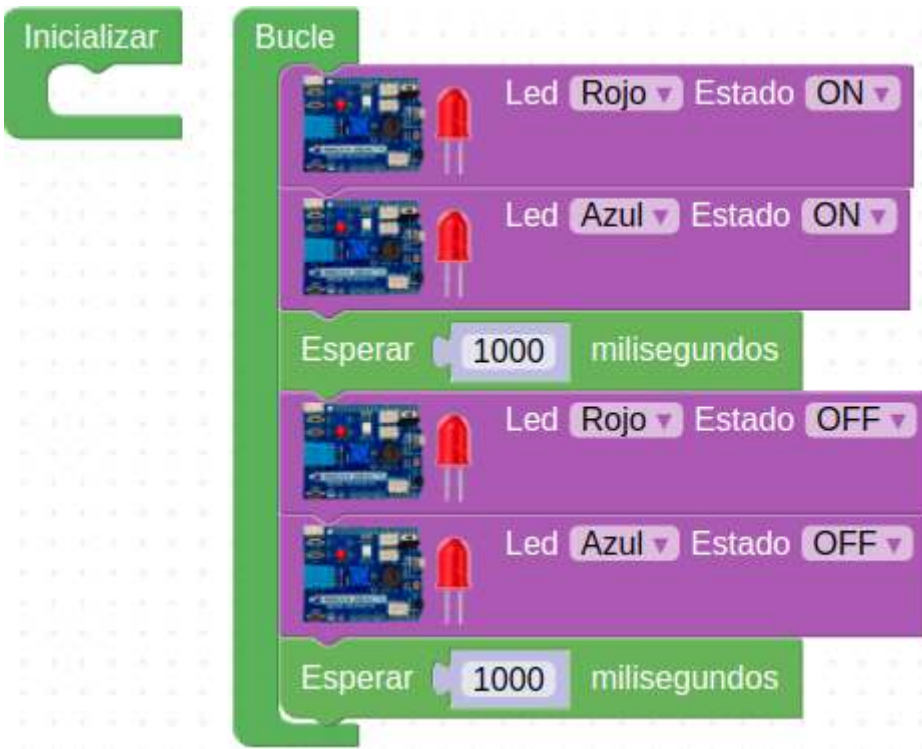


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Intermitencia LEDs rojo y azul

Retos de ampliación

A2.R1. Hacer que los LEDs rojo y azul se enciendan simultáneamente con tiempos de espera de 300ms y 150ms respectivamente.

A2.R2. Realizar 4 intermitencias de 500ms con el LED azul y cuando estas acaben dejar el LED rojo encendido durante 1.5 segundos. Esperar un segundo para iniciar de nuevo el proceso.

A2.R3. Realiza 5 intermitencias de 500ms con el LED azul cada vez que el LED rojo lo hace 3 veces a intervalos de 150ms. Esperar un segundo para iniciar de nuevo el proceso.

Solución A22.R2

El Programa es el de la imagen siguiente:

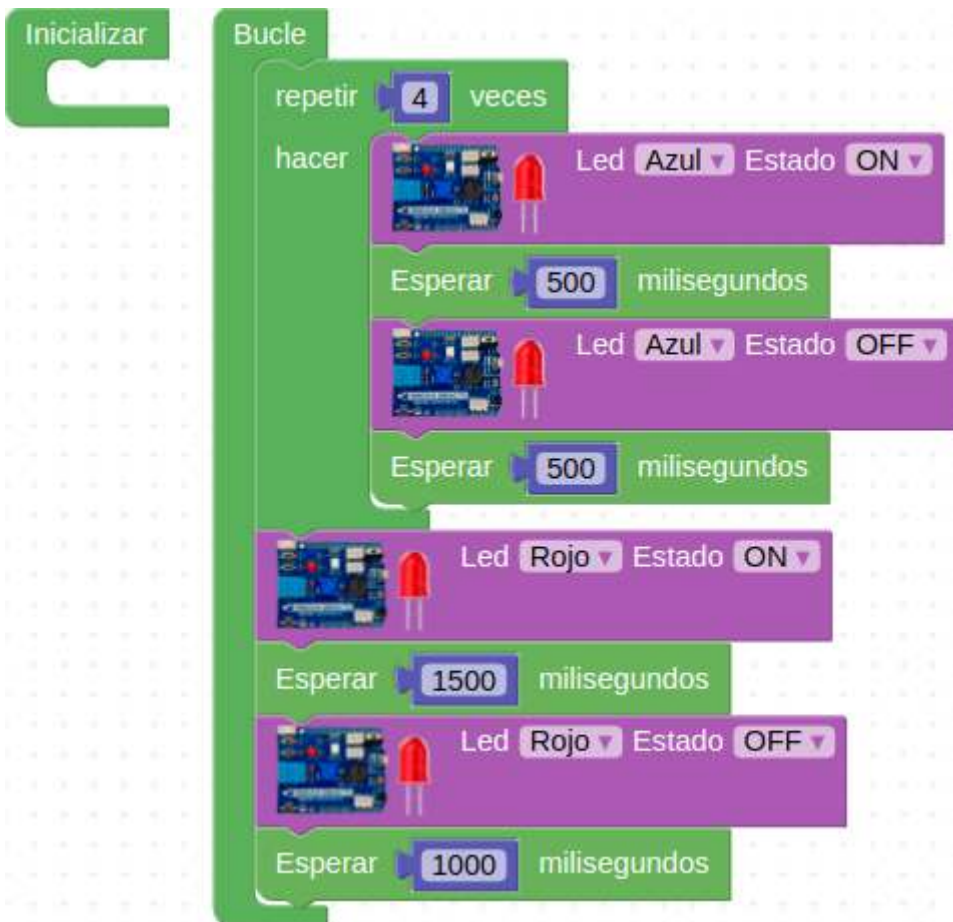


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Solución A2.R3

El Programa es el de la imagen siguiente:



Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Prácticas con sensores digitales

Actividad-03. Parpadeo intermitente. Multitarea

Página extraída de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Enunciado

Ya hemos visto como hacer funcionar a los LEDs de diferentes formas, pero ahora vamos a introducir un concepto que nos puede resultar bastante útil en el futuro, se trata de la multitarea en ArduinoBlocks.

Teoría

Este apartado se extrae de ArduinoBlocks - FreeBook disponible en Free Book (online & updated).

ArduinoBlocks nos permite utilizar una capa para implementar un sistema multitarea avanzado basado en FreeRTOS (del inglés Real Time Operating System), que es un sistema operativo de tiempo real kernel para dispositivos embebidos para plataformas de microcontrolador que se distribuye bajo licencia MIT. Este sistema permite crear tareas que se ejecutarán de forma paralela (virtualmente). En microcontroladores modestos como el Arduino UNO, Nano o incluso MEGA la multitarea con FreeRTOS es bastante limitada y consume gran parte de los recursos de nuestro Arduino, en caso de necesitar de un sistema multitarea más potente podemos optar por usarlo en placas basadas en ESP8266 o ESP32 con mucha más potencia y recursos (especialmente el ESP32 con doble núcleo y gran potencia de procesamiento y memoria interna)

Los sistemas software de multitarea utilizan un planificador o scheduler que se encarga de repartir el tiempo de procesamiento entre las distintas tareas, de forma que a cada una le toca un tiempo de microcontrolador para ejecutar un poquito de su parte de programa.

En las web de freeRTOS, en su entrada de menú Kernel podemos encontrar los conceptos básicos de multitarea y de programación que vamos a extraer seguidamente.

Conceptos básicos de multitarea

Un procesador convencional como el de Arduino UNO solo puede ejecutar una tarea a la vez, pero al cambiar rápidamente entre tareas, un sistema operativo multitarea puede hacer que parezca que cada tarea se ejecuta simultáneamente. Esto es lo que se representa en el diagrama de la Figura siguiente que muestra el patrón de ejecución de tres tareas con respecto al tiempo. Los nombres de las tareas están codificados por colores y escritos a la izquierda. El tiempo se mueve de izquierda a derecha y las líneas de colores muestran qué tarea se está ejecutando en un momento determinado. El diagrama superior demuestra el patrón de ejecución concurrente percibido, y el inferior el patrón de ejecución multitarea real.

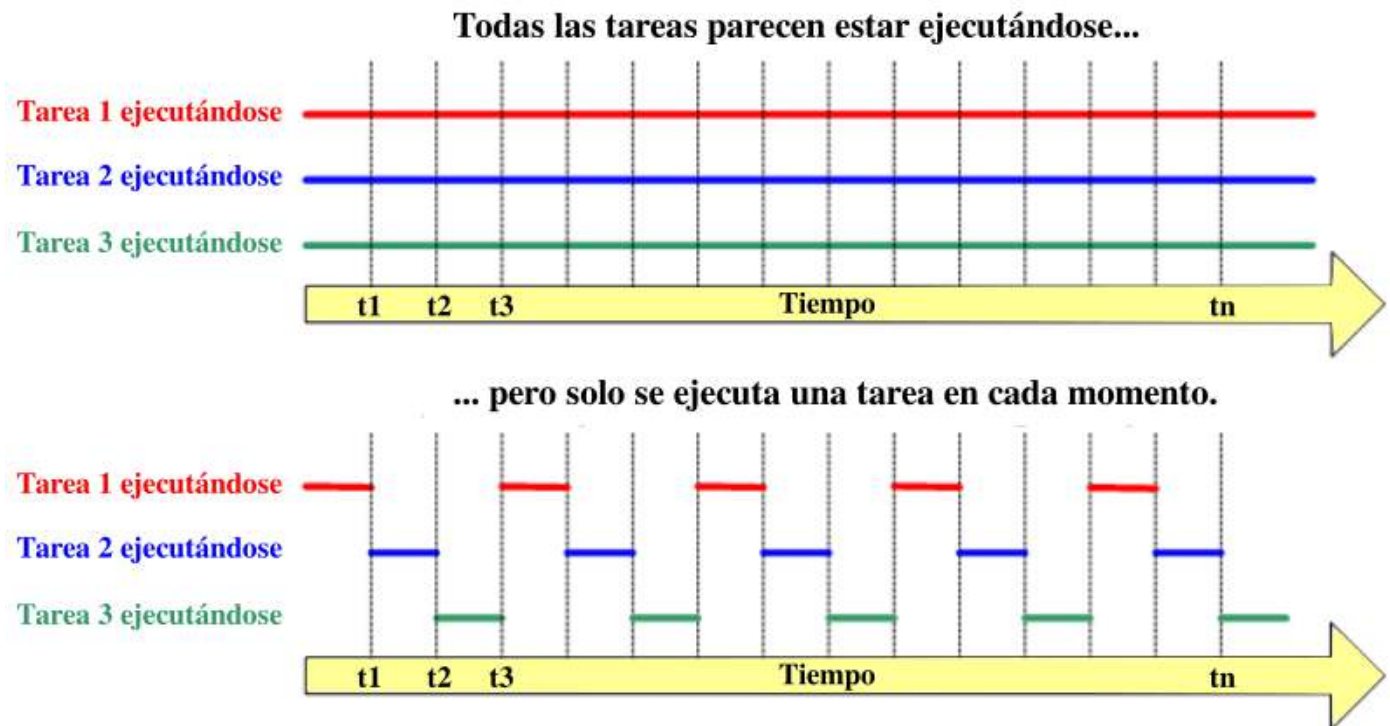


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Patrón de ejecución de tres tareas con respecto al tiempo

Programación

El programador es quien debe decidir qué tarea debe ejecutarse en un momento determinado. El kernel o núcleo puede suspender y luego reanudar una tarea muchas veces durante el tiempo de vida de la tarea.

Además de ser suspendida involuntariamente por el núcleo o kernel, una tarea puede optar por suspenderse a sí misma. Hará esto si desea retrasar (**dormir**) por un período fijo o esperar (**bloquear**) a que un recurso (por ejemplo, un puerto serie) esté disponible, o que ocurra un evento (por ejemplo, presionar una pulsador). Una tarea bloqueada o inactiva no se puede ejecutar y no se le asignará ningún tiempo de procesamiento.

En la Figura siguiente vemos un posible diagrama de ejecución de tres tareas analizado punto por punto en distintos instantes de tiempo. En los círculos se representan los instantes de tiempo t_1 a t_{10} .

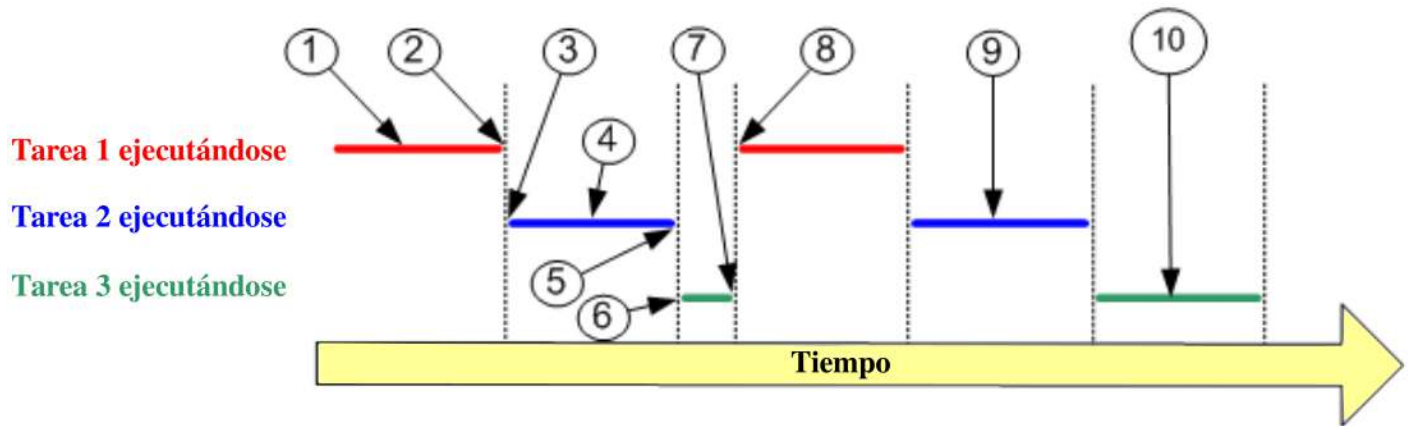


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Diagrama de ejecución de tres tareas en el tiempo

- t_1 : la tarea 1 se está ejecutando.
- t_2 : en el kernel se suspende, o mejor dicho se intercambia, la tarea 1.
- t_3 : se reanuda la tarea 2.
- t_4 : mientras se ejecuta la tarea 2 el procesador bloquea el puerto serie para su acceso exclusivo.
- t_5 : el kernel suspende la tarea 2.
- t_6 : el kernel reanuda la tarea 3.
- t_7 : la tarea 3 intenta acceder al puerto serie y lo encuentra bloqueado por lo que no puede continuar y se suspende.
- t_8 : el kernel reanuda la tarea 1.
- t_9 : al ejecutarse de nuevo la tarea 2 se desbloquea el puerto serie.
- t_{10} : la tarea 3 ahora si puede acceder al puerto serie y se ejecuta al completo

Planificadores

Los planificadores de multitarea permiten asignar a cada tarea una prioridad, para así darle preferencia a las tareas más críticas o que necesitan más tiempo de procesamiento. Si creamos muchas tareas con “alta” prioridad puede que afectemos a las demás dejando poco tiempo de procesamiento para ellas. En la Figura siguiente vemos un esquema de varias tareas con distintas prioridades, variando así su tiempo de microprocesador asignado.

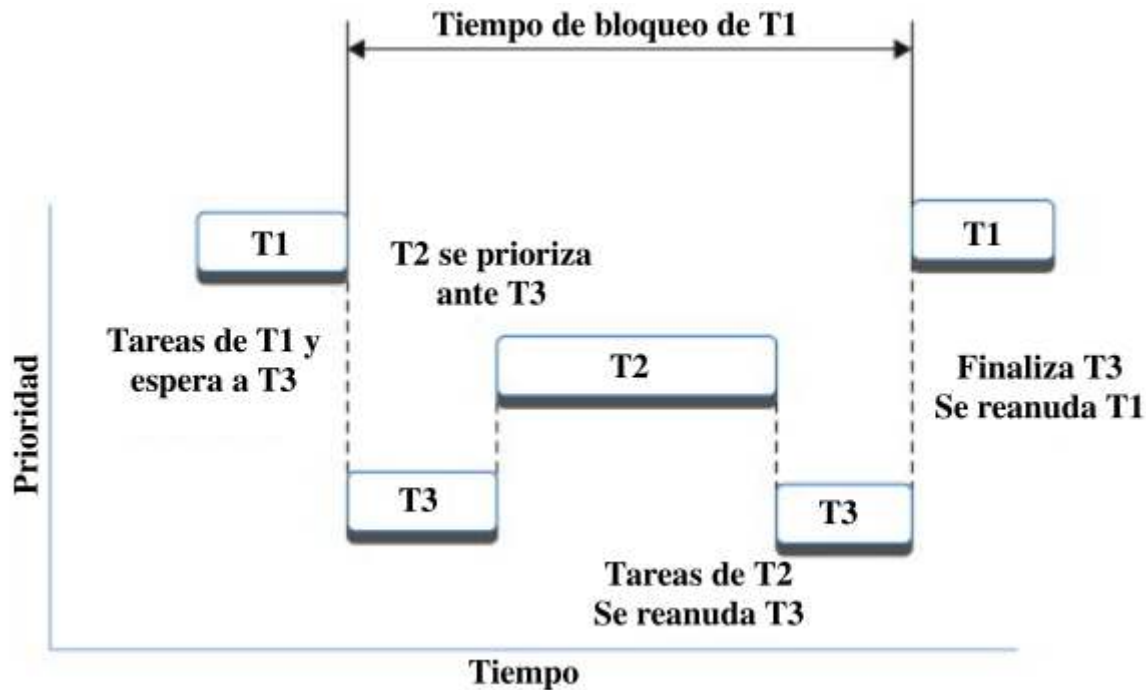


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Distintas prioridades en tres tareas

Cada tarea tiene su propio espacio de memoria, por lo que crear demasiadas tareas también puede dejarnos el procesador sin memoria. Si la memoria asignada a las tareas tampoco es suficiente para almacenar los datos se podría reiniciar de forma inesperada el Arduino, o funcionar incorrectamente, es decir que como siempre, hay que ser consciente de los limitados recursos de los que disponemos.

Semáforos

Con la introducción teórica a la multitarea vista, debemos hacernos otra pregunta: ¿Qué pasa si una tarea accede a un recurso o variable, y el sistema multitarea le da el control a otra tarea y por tanto ese proceso falla o quizás otra tarea acceda al mismo recurso y se solapen?

Para ese problema de convivencia entre tareas se inventaron los “semáforos”, en concreto el que más nos interesa es el semáforo “mutex” o de exclusión mutua, que permite que bloqueemos el sistema multitarea, hagamos lo que tengamos que hacer crítico, y luego liberemos el control. Por supuesto estas tareas críticas deben ser lo más cortas y atómicas posibles: una escritura crítica en una variable, un envío de un dato, una actualización de una pantalla LCD,... siempre cosas simples. Los semáforos debemos usarlos en casos que tengamos claro que se pueden crear conflictos, pues su abuso puede hacer que el sistema multitarea empiece a fallar.

En la Figura siguiente vemos el esquema de acceso a un recurso desde dos tareas diferentes.

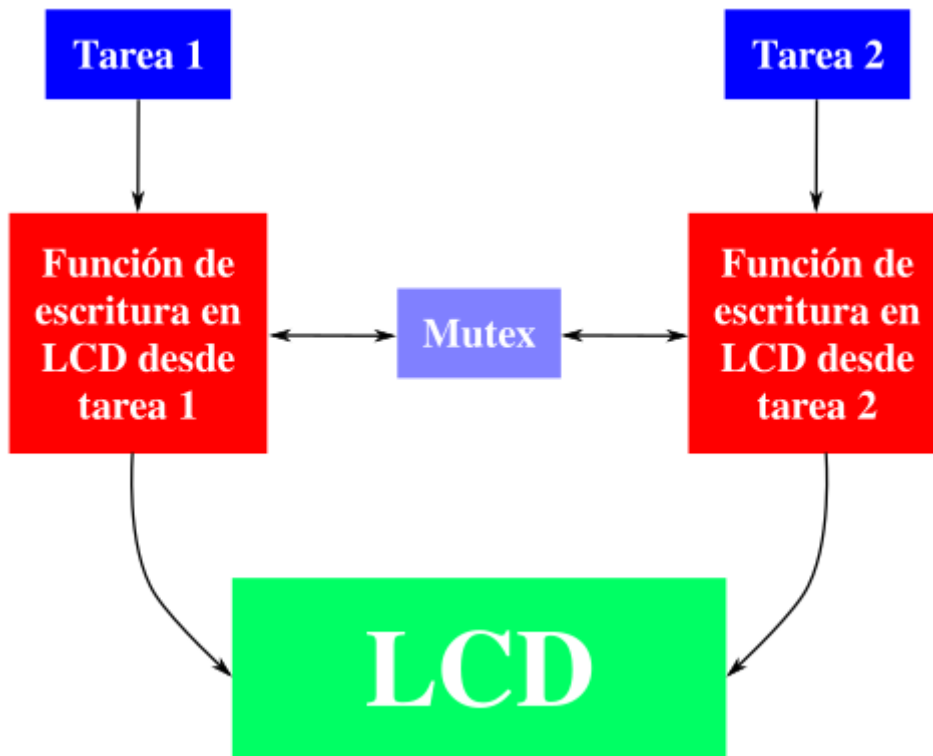


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Esquema de acceso a un mismo recurso por parte de 2 tareas diferentes

En la ESP32 STEAMakers

Una ventaja que posee la ESP32 Plus STEAMakers es que, al estar basado en un ESP32 con dos microcontroladores internos, podemos hacer que cada microcontrolador trabaje en una tarea y esto si sería multitarea real, o en varias tareas mezclando la multitarea real con la simulada. Esto también se puede hacer en las placas Arduino UNO, pero con el ESP32 tenemos más potencia para realizar estas tareas.

Esquema de acceso a un mismo recurso por parte de 2 tareas diferentes

Bloques esperar

¿Qué pasa con los bloques tipo “esperar” que estaban tan prohibidos en la programación de Arduino cuando queríamos simular una multitarea antes de tener estos bloques? Pues seguimos teniéndoles bastante tirria. Aunque en teoría podríamos usarlos, un bloque esperar hace pensar al microcontrolador que está haciendo algo útil, cuando en realidad no es así, por lo que el sistema multitarea querrá asignarle tiempo de procesamiento a la tarea, aunque sea para eso, ¡para no hacer nada!

Tenemos una solución, tenemos un nuevo bloque de esperar “task friendly” que en lugar de esperar sin hacer nada le dice al sistema: ¡voy a estar un rato sin hacer nada, permite ejecutar otras tareas mientras y luego vuelves!Mucho más “friendly”, claro que sí.

Bloques

Con toda esta información pasamos a ver los bloques disponibles para poner todo ésto en marcha.

Imágenes Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Bloque	Descripción
Bloque ejecutar tarea	Permite crear una nueva tarea con su bloque de “inicializar” y su “bucle” al igual que la tarea original de Arduino. Debemos asignar una prioridad a cada tarea, por defecto dejaría todas a “baja” y luego iría ajustando si hace falta. Para gestionar mejor las prioridades, es recomendable en algunos casos no utilizar el “inicializar” y “bucle” propio de Arduino que suele tener preferencia sobre todas estas tareas y es más difícil de equilibrar las prioridades.
Bloque esperar en esta tarea	El bloque esperar óptimo para tareas, pues deja funcionar al resto de tareas de forma más óptima mientras se espera en ésta. Este bloque tiene menos precisión que el bloque “esperar” original, si necesitamos hacer esperas muy precisas (o de menos de 20 ms) debemos usar el “esperar” tradicional. Pero nos servirá en la mayoría de casos.
Bloque bloqueo exclusivo	Si tenemos que hacer alguna acción crítica que no queremos que sea interrumpida internamente por el planificador del sistema multitarea podemos poner este bloque y dentro los bloques críticos. (no utilizar si no es estrictamente necesario)
Bloque establecer memoria	Cada tarea tiene su propio espacio de memoria reservado, esta es la cantidad por defecto para las tareas (192 bytes), si necesitamos ajustarla podemos utilizar este bloque en el “inicializar” principal y se ajustará para todas las tareas. Un mal ajuste puede provocar reinicios del microcontrolador o mal funcionamiento.
Bloque y destruir tarea	Las tareas en principio, igual que el bucle de Arduino, están pensadas para ejecutarse de forma indefinida, si en un caso una tarea deja de ser necesaria la forma de terminarla es con este bloque que parará la ejecución y liberará la memoria de la tarea en la que se ejecuta.

En la TdR STEAM

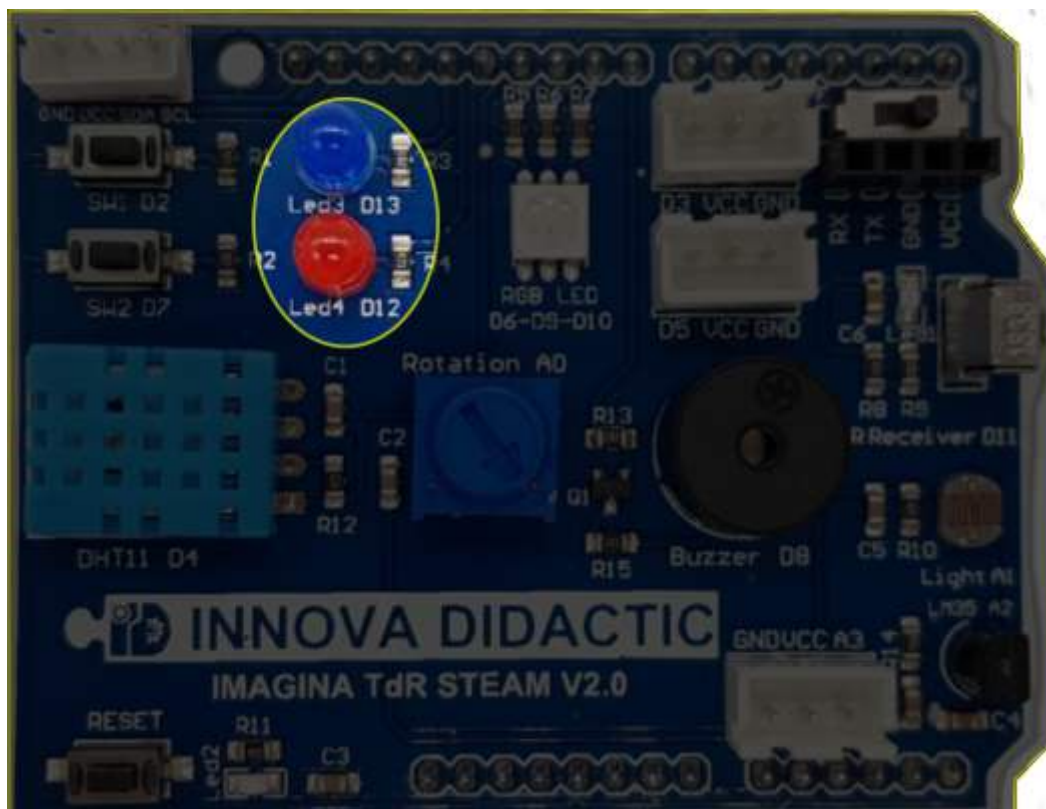


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Programando la Actividad

Vamos a hacer que los dos diodos parpadeen de forma independiente con un programa como el siguiente, que lo tenemos disponible en [Actividad-03. Parpadeo intermitente. Multitarea](#)



Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Retos de ampliación

A3.R1. Hacer que los LEDs rojo y azul trabajen en multitarea modificando los tiempos.

Prácticas con sensores digitales

Actividad-04. LED RGB

Página extraída de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Enunciado

La actividad básica consistirá en ver los colores primarios y su mezcla mediante el uso del LED RGB y también habrá una actividad secundaria sobre el control de intensidad mediante PWM.

Teoría

LED RGB

Un LED RGB es en realidad un encapsulado que incorpora tres diodos LED, uno por cada color fundamental. Los colores primarios en óptica son el rojo, el verde y el azul y la correcta combinación, en términos de intensidad, de ellos originará cualquiera de los colores secundarios. Las siglas RGB son el acrónimo de Red, Green y Blue. En el caso de la TDR-STEAM se utiliza un LED RGB con los tres LEDs unidos por su cátodo o terminal negativo, es decir es un LED RGB de cátodo común. En la imagen siguiente vemos el modelo aditivo de los colores rojo, verde y azul.

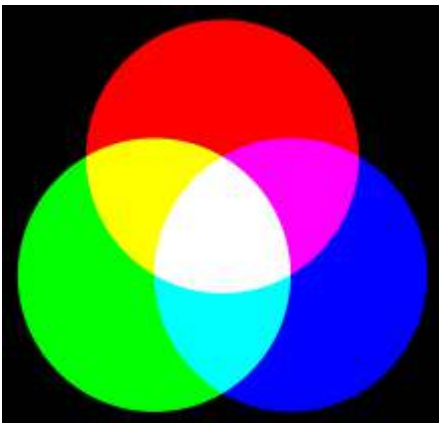


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

El símbolo y la representación mas comunes de este componente lo vemos en la imagen siguiente:

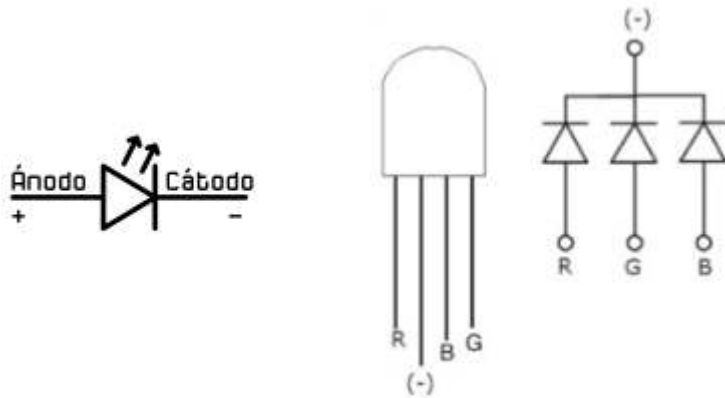


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

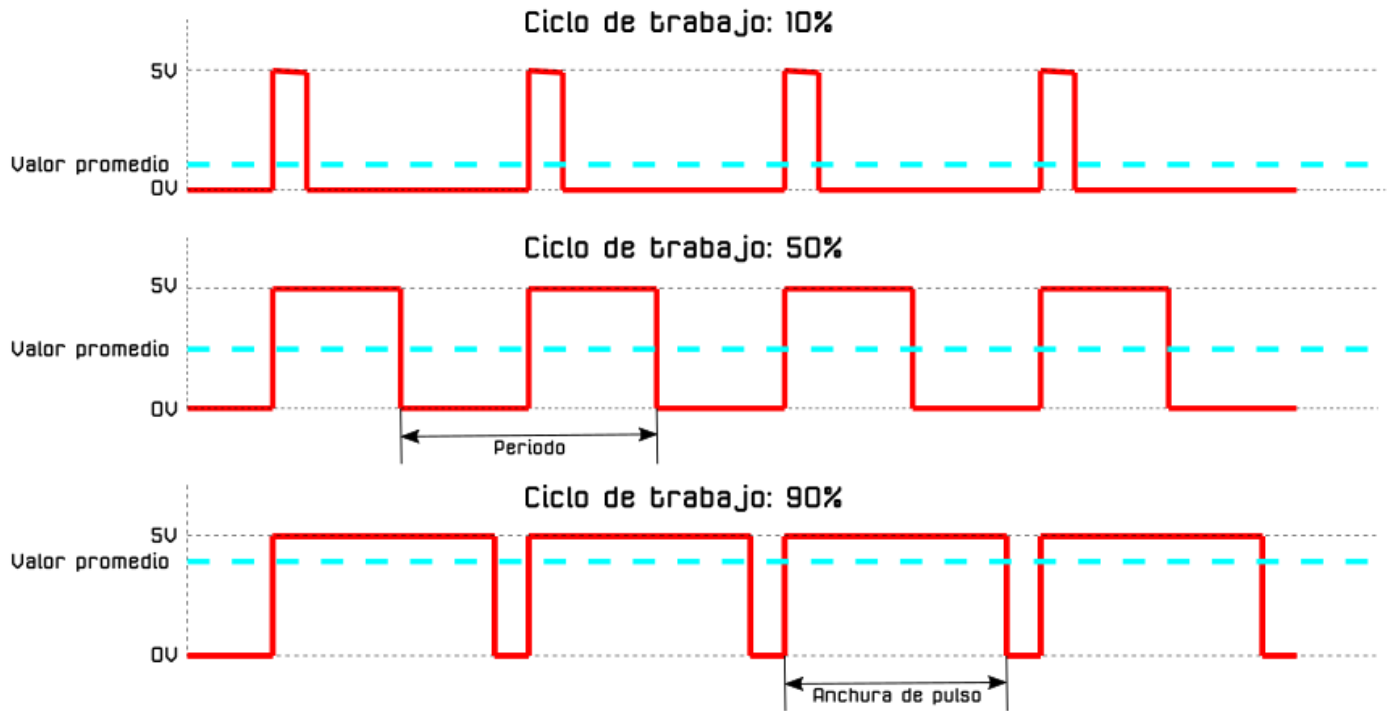
Teóricamente en Arduino, cada uno de esos LEDs podría adoptar 256 (valores entre 0 y 255) colores diferentes, es decir, un total de 16.777.216 posibles colores diferentes con un LED RGB.

PWM

PWM son siglas en inglés que significan Pulse Width Modulation y que lo podemos traducir a español como Modulación de ancho de pulso. Los pines PWM permiten generar una señal analógica mediante una salida digital mapeada con 8 bits, o lo que es lo mismo, valores del 0 al 255, es decir mediante una salida PWM podemos emular una señal analógica.

En realidad una placa tipo UNO no es capaz de generar una salida analógica y lo que se hace es emplear un truco que consiste en activar una salida digital durante un tiempo y el resto del tiempo del ciclo mantenerla desactivada. El valor promedio de la salida es el valor analógico. En el tipo de modulación PWM mantendremos constante la frecuencia, o lo que es lo mismo, el tiempo entre pulsos y lo que se hace es variar la anchura del pulso.

La proporción de tiempo que está encendida la señal, respecto al total del ciclo, se denomina ciclo de trabajo o Duty cycle, y generalmente se expresa en tanto por ciento. En la imagen siguiente vemos señales con distintos ciclos de trabajo.



Distintos Duty cycle Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Es importante recordar que en una salida PWM el valor de la tensión es 5V por lo que si alimentamos un dispositivo de 3V a partir de una salida de 5V lo dañaremos de forma irreversible.

Las señales PWM emula una señal analógica para aplicaciones como variar la luminosidad de un LED y variar la velocidad de motores de corriente continua.

La placa ESP32 Pus STEAMakers tiene muchas salidas PWM, pero en la placa Imagina TDR STEAM sólo se puede controlar por PWM el led RGB (pines 6, 9 y 10) y los tres pines que quedan libres para conectar elementos externos.

En la TdR STEAM

En la placa existe un LED RGB 5050 de 6 pines como el de la imagen siguiente conectado a los pines D6 (Red), D9 (Green) y D10(Blue). Estos tres pines son PWM y nos van a permitir regular su intensidad.



RGB 5050 Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

En la placa TdR STEAM se localiza donde vemos en la imagen siguiente:

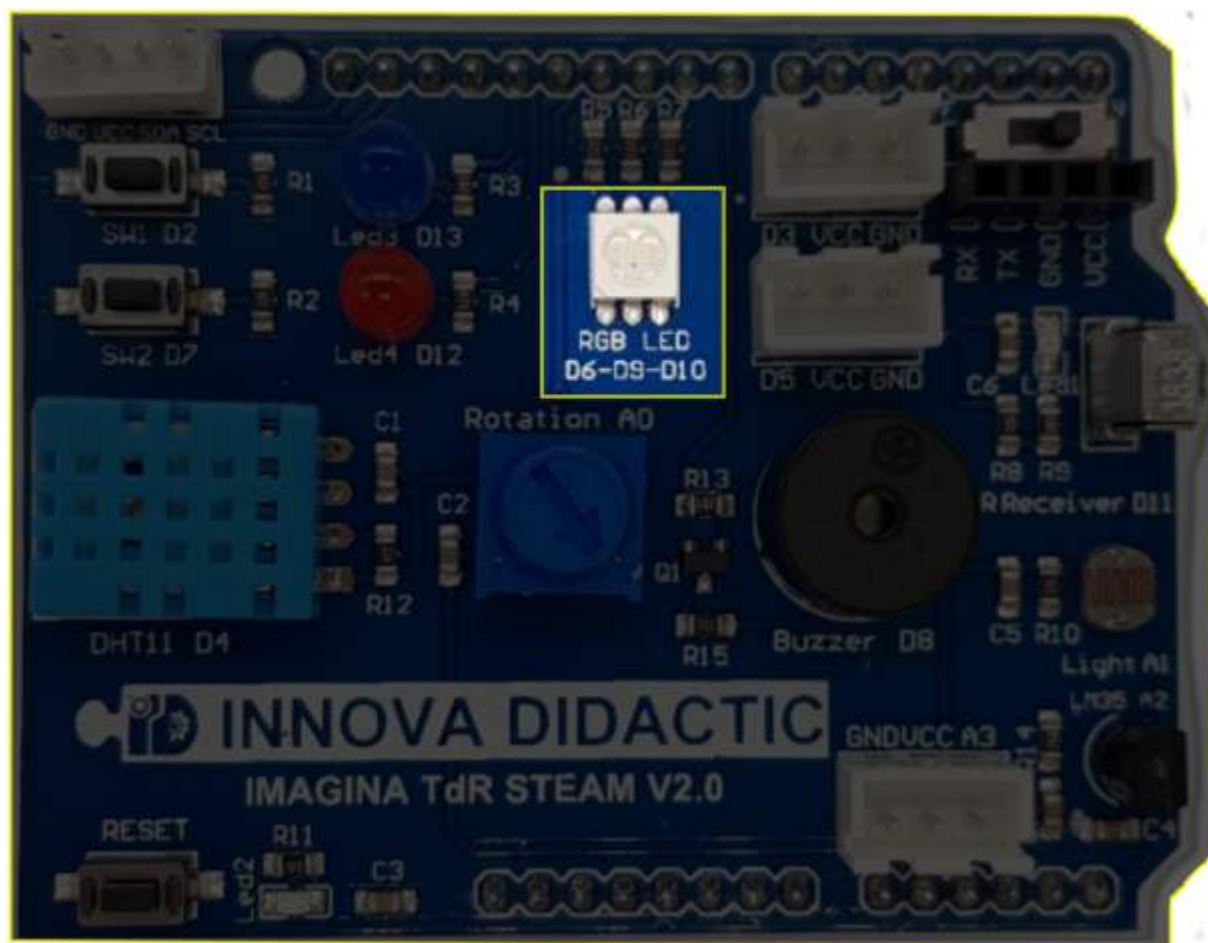


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Programando la actividad

En ArduinoBlocks disponemos de los dos bloques que vemos en la imagen siguiente para el control PWM del LED RGB.

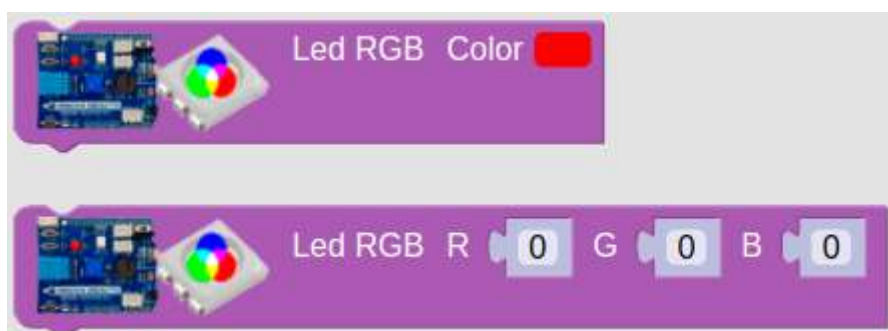
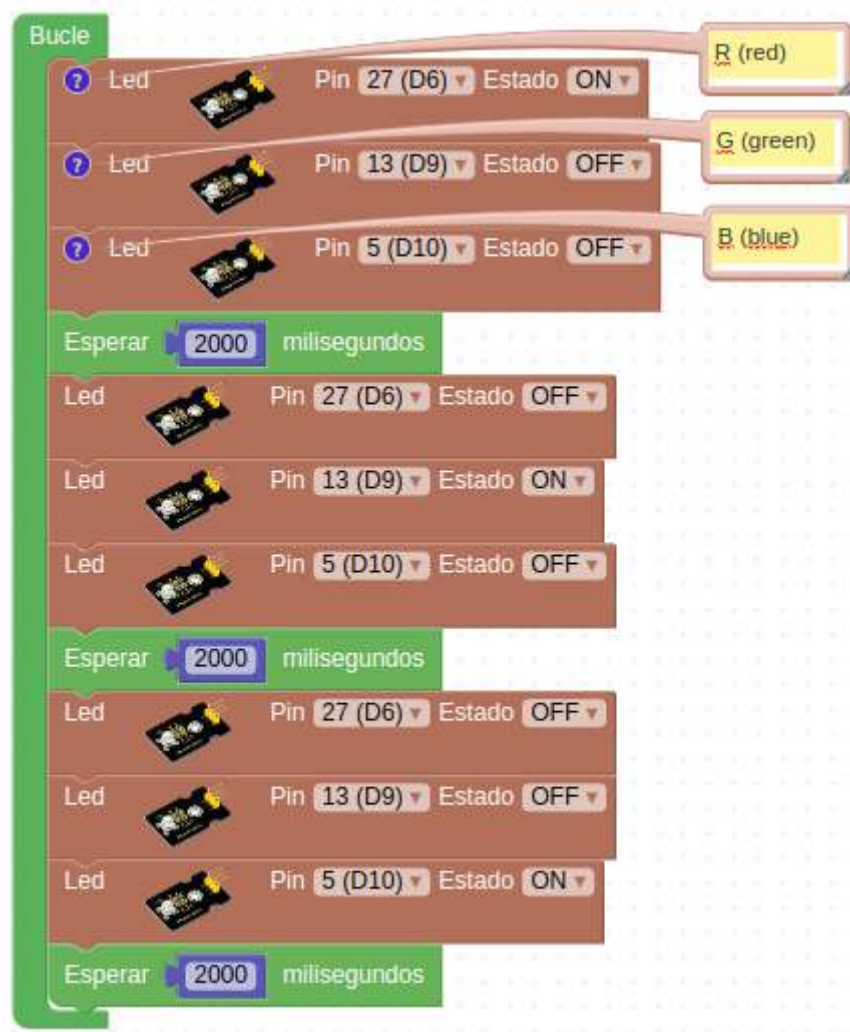


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

El bloque superior permite asignar el color a partir de la paleta que se despliega al hacer clic sobre el cuadrado de color y en el inferior debemos introducir el valor numérico (entre 0 y 255) correspondiente a cada color primario en cada uno de los tres colores RGB. Con estos bloques no tenemos que preocuparnos por saber las conexiones de cada diodo ya que están asignadas internamente en el bloque.

También hemos visto que el LED RGB tiene asociados tres pines y por tanto podemos tratar a cada LED de forma individual. Pero para poder hacerlo debemos crear un tipo de proyecto "ESP32 STEAMakers" y no como hasta ahora "ESP32 STEAMakers + Imagina TdR STEAM". Esto nos va a permitir disponer del bloque LED que está dentro de "Actuadores" con todos los pines digitales configurables.

De esta forma un programa como el de la imagen siguiente nos va a permitir activar de forma individual cada diodo LED. La solución a esta actividad la tenemos disponible en [Actividad-04](#).



Actividad 04 Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Retos de ampliación

A4.R1. A partir de la idea del reto realizar un programa que muestre sucesivamente los siguientes colores: magenta o violeta, cian o azul claro, amarillo y blanco. Ayúdate del gráfico que muestra el modelo aditivo de colores que hemos puesto al principio de la teoría.

A4.R2. Realizar un programa que nos muestre los tres colores primarios a partir de un proyecto tipo "ESP32 STEAMakers + Imagina TdR STEAM" y el bloque con la paleta de colores.

A4.R3. Realizar un programa que nos muestre los tres colores primarios a partir de un proyecto tipo "ESP32 STEAMakers + Imagina TdR STEAM" y el bloque con el valor numérico de cada color.

A4.R4. Realizar un programa que muestre de manera secuencial los colores del arcoiris en el orden que vemos en la imagen siguiente.



Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Solución A4.R2

El [Programa](#) es el de la imagen siguiente:

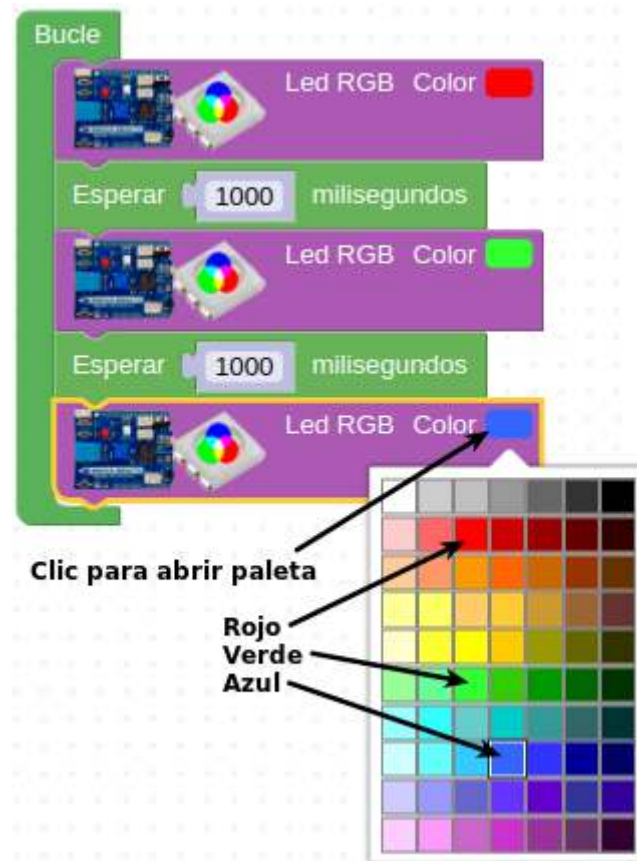


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Solución A4.R3

El Programa es el de la imagen siguiente:

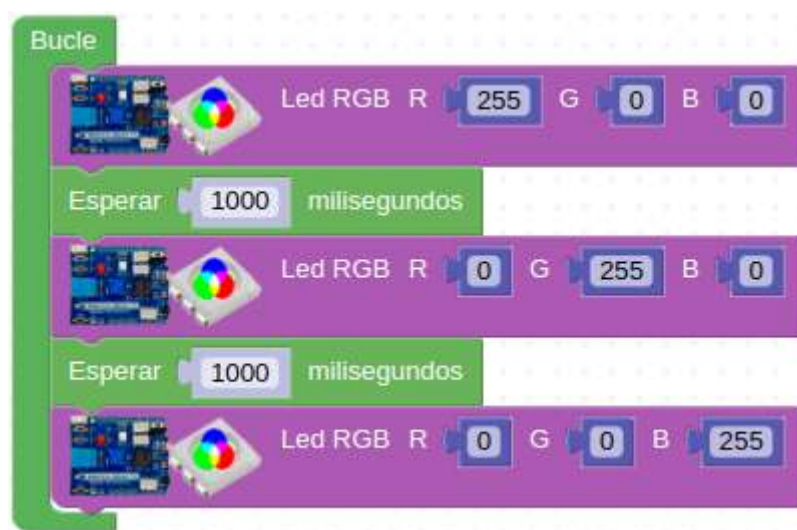


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Prácticas con sensores digitales

Actividad-05. Zumbador

Página extraída de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Enunciado

Trabajaremos con el buzzer o zumbador partiendo de la reproducción de sonidos básicos hasta llegar a la reproducción de melodías completas.

Teoría

El buzzer, zumbador o altavoz es un transductor electroacústico (convierte una señal eléctrica en una onda de sonido) que produce un determinado zumbido o sonido. Existen de dos tipos:

- Pasivos: no disponen de electrónica interna, por lo que tenemos que proporcionar una señal eléctrica para conseguir el sonido deseado.
- Activos: disponen de un oscilador interno, por lo que únicamente tenemos que alimentar el dispositivo para que se produzca el sonido.

El zumbador que incorpora la placa TdR STEAM es de tipo pasivo y está conectado al pin D8.

Una de los parámetros que caracterizan a un sonido es su frecuencia de emisión, siendo la frecuencia el número de veces que se repite por unidad de tiempo (segundo). La transmisión del sonido se realiza por ondas a través en cualquier medio (sólido, líquido o gaseoso) excepto en el vacío. La frecuencia de un sonido nos indica cuantos ciclos por segundo tiene una onda.

En la imagen y la tabla siguientes vemos un dibujo con un fragmento de las teclas de un piano estando todo referido a una nota estándar, la nota "La" central que tiene una frecuencia de 440 Hz. Podemos ver la nota musical que reproduce, en las dos notaciones más comunes de los sonidos (Inglés: C D E F G A B, Alemán: C D E F G A H, Español, italiano y francés: Do Re Mi Fa Sol La Si) y además se encuentra la frecuencia que produce esa nota musical.

	Si	Sib (La#)	Si	B	493.88
	La	Lab (Sol#)	La # (Si b)	A# (Bb)	466.16
	Sol	Solb (Fa#)	La	A	440.00
	Fa		Sol # (La b)	G# (Hb)	415.30
	Mi	Mib (Re#)	Sol	G	392.00
	Re	Reb (Do#)	Fa # (Sol b)	F# (Gb)	369.99
	Do		Fa	F	349.23
			Mi	E	329.63
			Re # (Mi b)	D# (Eb)	311.13
			Re	D	293.66
			Do # (Re b)	C# (Db)	277.18
			Do	C	261.63

Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Bloques de programación

En ArduinoBlocks disponemos de un bloque que nos permite reproducir cualquier melodía RTTTL (del inglés, Ring Tone Text Transfer Language) y es el que vemos en la imagen siguiente:

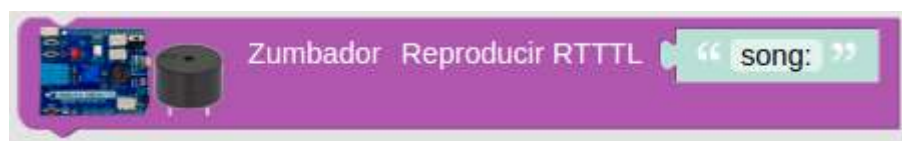


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Este bloque permite reproducir una melodía a partir de un texto con formato RTTTL, formato desarrollado por Nokia para ser usado para transferir tonos de llamada a teléfonos móviles. El formato RTTTL es una cadena dividida en tres secciones: nombre, valor predeterminado y datos. Por ejemplo, la siguiente cadena de texto se corresponde con la Intro de Donkey Kong:

Lo único que tenemos que hacer para reproducir la melodía es pegar esta cadena en la zona de texto del bloque.

```
d=4,o=5,b=140:8a#,8p,8d6,16p,16f.6,16g.6,16f.6,8a#,8p,8d6,16p,16f.6,16g.6,16f.6,8a#,8p,8d6,16p,16f.6,16g.6,16f.6,8a#,8p,8d6,16p,16f.6,16g.6,16f.6
```

ArduinoBlocks nos suministra información y enlaces referentes al tema accediendo desde nuestro proyecto:



Imagen Federico

Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

En la imagen siguiente tenemos desplegada la información que nos ofrece esta herramienta.



RTTTL - Ring Tone Text Transfer Language

Información sobre el formato RTTTL

RTTTL Format Info:

https://en.wikipedia.org/wiki/Ring_Tone_Transfer_Language

RTTTL Arcade List:

<http://arcadetones.emuunlim.com/arcade.htm>

Listados de melodías

RTTTL Movies/Songs

<https://ringtonessgalore.co.uk/popular-ringtones.php>

RTTTL Ringtones Packs:

<http://www.picaxe.com/RTTTL-Ringtones-for-Tune-Command/>

Reproductor online de RTTTL

RTTTL Online Player:

<https://adamonsoon.github.io/rtttl-play/>

Información RTTTL *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

También podemos acceder a esta información haciendo clic derecho sobre el bloque y escogiendo la opción "Ayuda" de entre las mostradas en la ventana emergente.

Zumbador activo

Existe otro tipo de zumbador que incluye un oscilador que genera una frecuencia audible fija y que se conoce como zumbador pasivo y en realidad es el que de forma correcta se puede denominar como zumbador. Este es mucho mas sencillo de usar ya que basta con conectarlo a un pin digital y cuando pongamos a nivel alto este pin el zumbador generará su zumbido característico durante el tiempo que establezcamos. Con este tipo de zumbador no se pueden generar melodías.

Cuando está nuevo se distingue del zumbador pasivo o altavoz por la pegatina de protección que lo acompaña, pero esta hay que quitarla para oír el zumbido y sin ella es difícil distinguir un tipo del otro, así que en este caso deberemos crear un programa con alguna melodía y si no se reproduce correctamente es que tenemos conectado el zumbador activo en lugar del pasivo. Lo mejor es marcar alguno de los dos cuando podemos distinguirlos. Una buena idea puede ser utilizar la propia pegatina en el lateral del mismo.

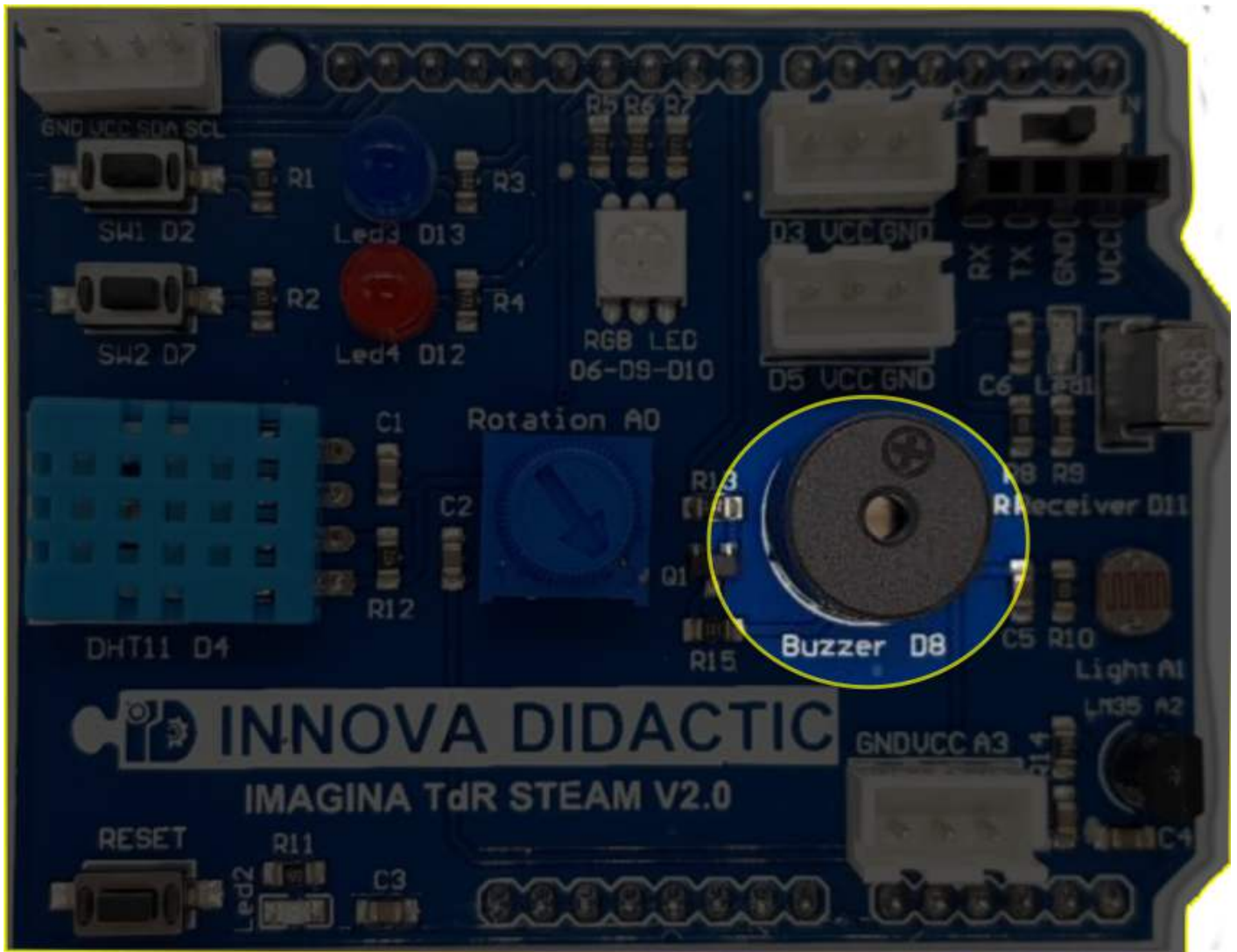


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Programando la actividad

Los cuatro bloques destinados a trabajar con el zumbador los vemos en la imagen siguiente:

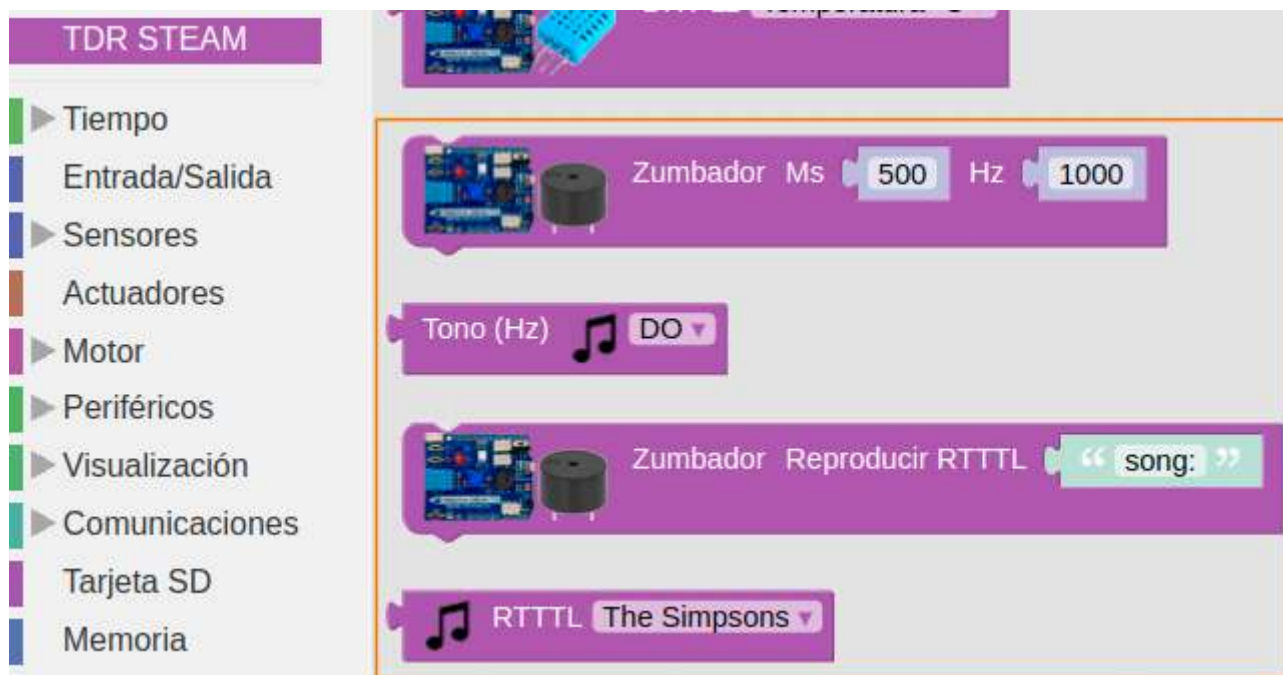


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

En el bloque Zumbador podemos modificar dos parámetros, el tiempo que dura cada sonido expresado en milisegundos (campo Ms) y la frecuencia en Hz a la que reproducirá el sonido el zumbador (campo Hz).

Vamos a comenzar haciendo un programa que reproduzca tres de las notas de la escala musical vista anteriormente. La solución al reto la tenemos disponible en [Actividad-05](#).



Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Retos de ampliación

A5.R1. Reproducir la escala musical con las notas básicas utilizando los bloques Zumbador y Tono

A5.R2. Reproducir la melodía de la imagen siguiente sabiendo que las negras tienen una duración de 500ms, las negras con un puntito 750ms y las blancas 1000ms.



Himno a la alegría Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

A5.R3. Reproducir diferentes melodías a partir de los bloques RTTTL (Ring Tone Text Transfer Language o lenguaje de tonos de llamada).

A5.R4. Reproducir alguna otra melodía que descarguemos de las páginas web propuestas.

Prácticas con sensores digitales

Actividad-06. Pulsadores

Página extraída de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

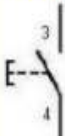
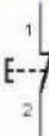
Enunciado

Utilizaremos uno, o ambos pulsadores, para llevar a cabo determinadas tareas como respuesta al accionamiento de los mismos.

Teoría

El pulsador

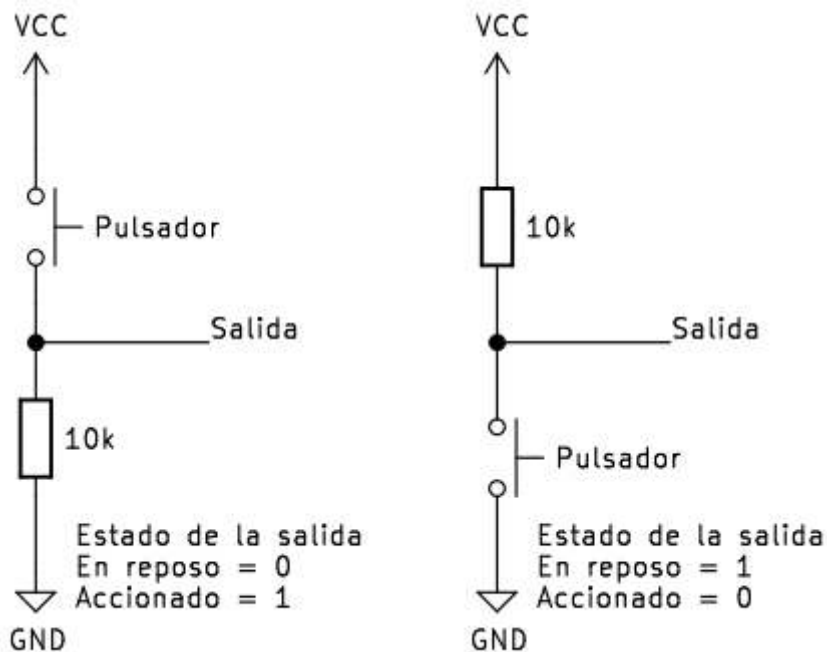
En la tabla siguiente vemos la simbología y algunos ejemplos del aspecto físico de estos elementos.

Tipo	Símbolo americano	Símbolo europeo	Aspecto
Normalmente abierto (NO)			
Normalmente cerrado (NC)			

Símbolos y aspecto real de un pulsador *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Se trata de un dispositivo que es capaz de abrir o cerrar el paso de la corriente eléctrica mientras permanece accionado, es decir, su función no queda anclada como por ejemplo en los interruptores de la luz de casa, en donde el accionamiento permanece hasta que no se vuelve a accionar. En un pulsador, por contra, su activación dura mientras lo mantenemos pulsado y vuelve a su estado de reposo en cuanto dejemos de pulsarlo.

Mediante la configuración adecuada podemos convertir un pulsador en un elemento de entrada a algún pin de nuestra placa UNO. Las configuraciones más básicas posibles con pulsadores las podemos ver en la imagen siguiente.



Configuración circuito elemental con pulsador *Imagen Federico Coca* [Notas sobre ESP32](#)

[STEAMakers](#) CC-BY-SA

*Esto lo hemos dado en **Sensores**, son dos configuraciones: la primera PULL DOWN o lógica normal y la segunda PULL UP o lógica inversa. Los pulsadores Tdr Steam son PULL DOWN o lógica normal.*

En el caso de la TdR STEAM los pulsadores se han configurado para que en reposo pongan a cero su entrada digital correspondiente y que se ponga a uno cuando son accionados.

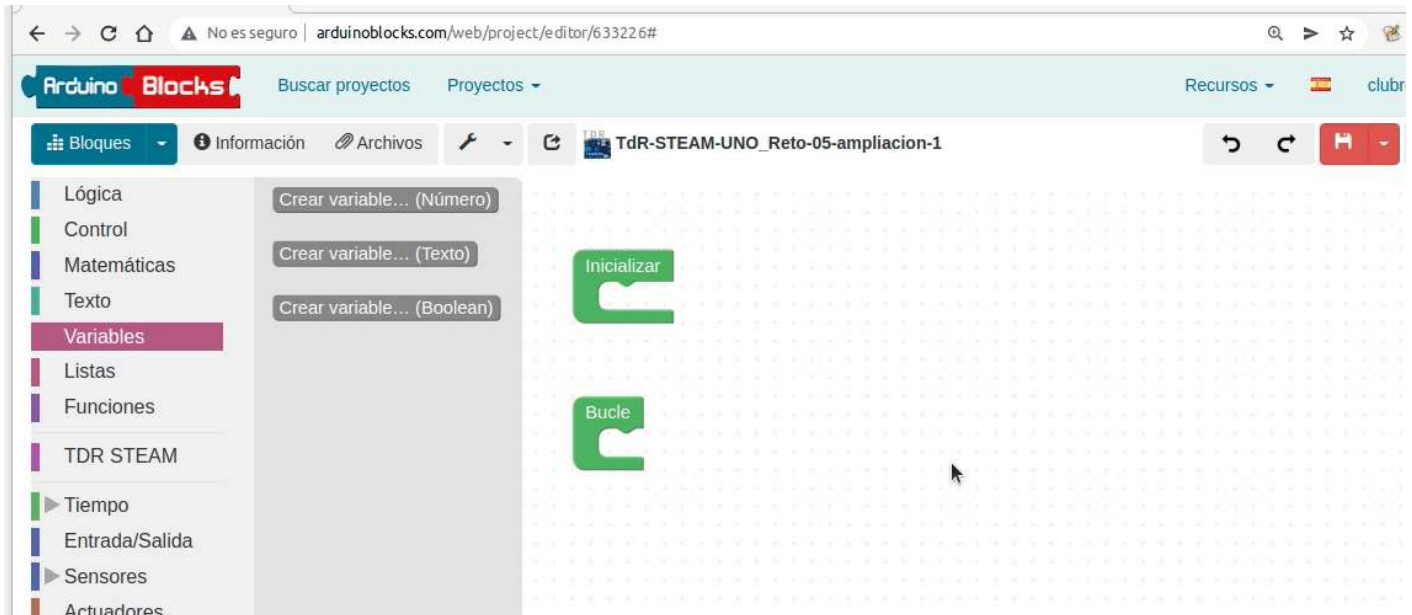
Concepto de variable y de contador

El concepto de variable en programación consiste simplemente en asignarle un nombre significativo a un espacio de memoria donde almacenar determinada informa

ción durante la ejecución normal del programa. El concepto es muy amplio y complejo y en nuestro caso no vamos a entrar en detalles sobre el mismo, pero si indicar que no se debe confundir con el concepto de variable matemática, ya que una expresión como $x = x + 1$ que es una aberración en

matemáticas tiene todo el sentido en programación. Lógicamente en matemáticas no se puede cumplir pero en programación significa que a la variable x se le suma uno y el resultado se vuelve a guardar en la misma variable.

En ArduinoBlocks podemos crear tres tipos de variables, numéricas, de texto o booleanas. En la animación siguiente podemos ver como se pueden crear, eliminar y renombrar variables.



Crear, renombrar y eliminar variables *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

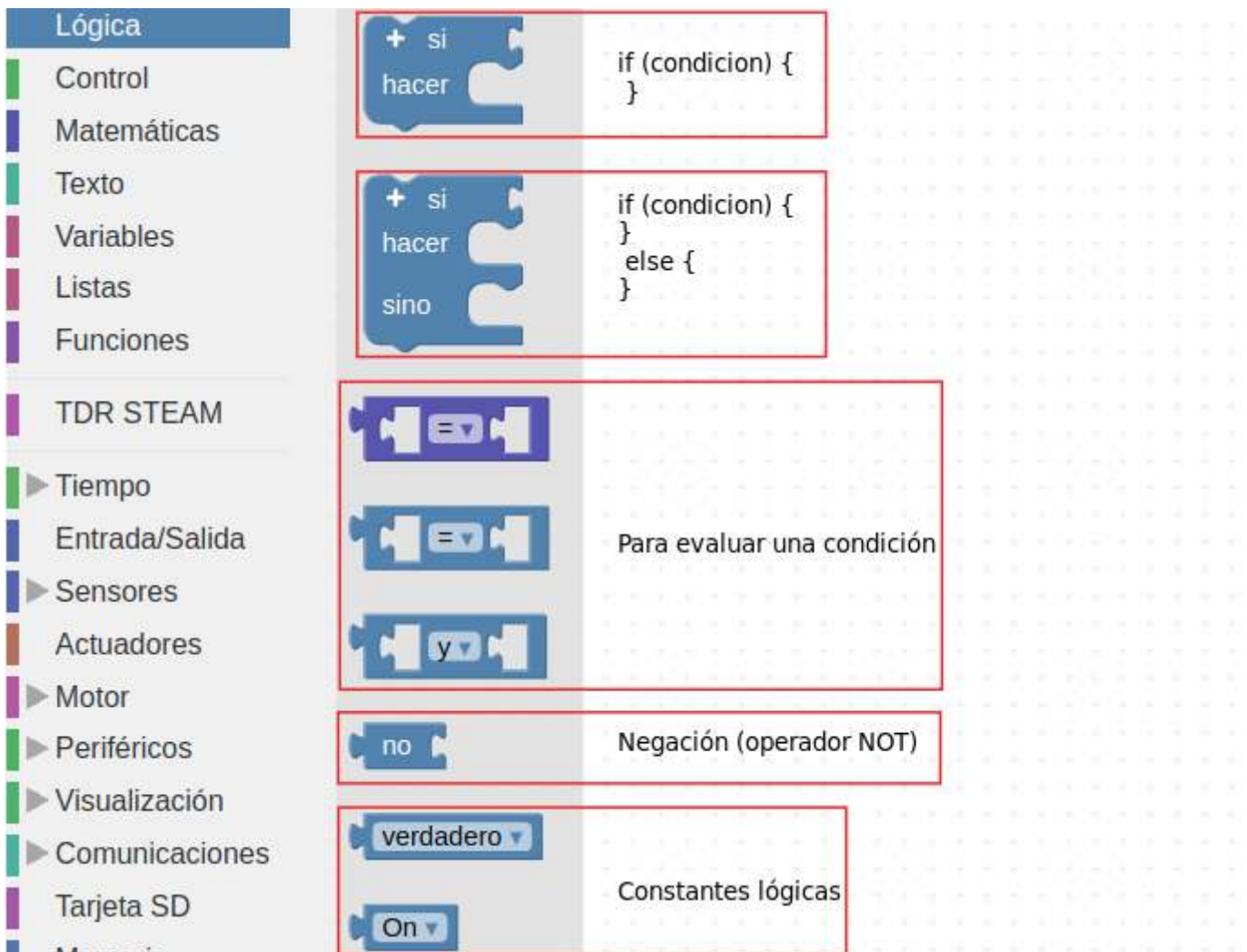
En programación, llamamos contador a una variable cuyo valor se incrementa o decrementa en un valor fijo para cada iteración del bucle para el que se ha definido. El uso habitual de un contador es simplemente contar el número de veces que itera un bucle en general o de forma mas extensa contar, solamente, aquellas iteraciones en las que se cumpla una determinada condición.

Por ejemplo, supongamos que tenemos una variable de nombre Estado de valor inicial cero y que se incremente cada vez que accionamos un pulsador, de esta forma si en un bucle vamos incrementando la variable de uno en uno, tenemos:

- Estado = 0 // valor inicial
- Estado = 1 // Estado = Estado + 1
- Estado = 2 // Estado = Estado + 1
- ...

Condicionales

Las sentencias condicionales son aquellas que nos permiten tomar decisiones en función de si ocurre o no ocurre determinada cosa. En el caso de ArduinoBlocks estas las podemos encontrar en el bloque lógica. Este bloque contiene los elementos que vemos en la imagen siguiente:



Bloques de lógica Imagen Federico Coca Notas sobre ESP32 STEAMakers CC-BY-SA

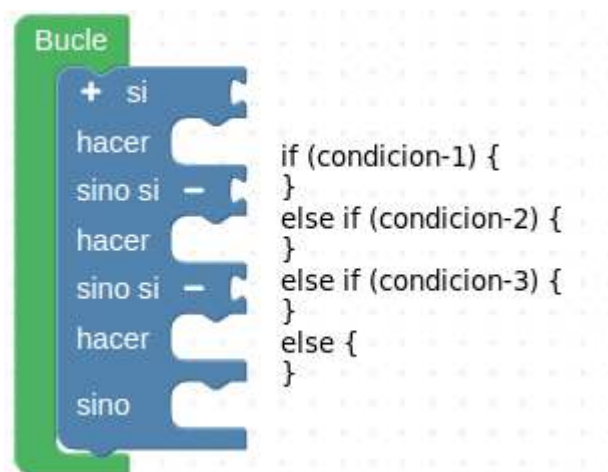
El funcionamiento es el siguiente: se evalúa la condición que ponemos en "si" y si el resultado es verdadero, o sea condición cierta, se realizan las acciones que pongamos en "hacer" y si no es cierta dichas acciones no se realizan. En el apartado condición se pueden poner infinidad de factores como pueden ser el estado de sensores, realizar comparaciones, hacer operaciones matemáticas, etc. Lógicamente el resultado de evaluar la condición debe ser verdadero o falso lo que se resuelve en el caso de la programación en Arduino diciendo que si el resultado es 0, el resultado de la evaluación es falso y si es 1 o cualquier otro valor es verdadero.

Si hemos sido observadores hemos visto en los bloques "si" de "Lógica" un signo mas (+) en la parte superior izquierda tanto del condicional "if" como la del "if ... else" tal y como se destaca en la imagen siguiente.



Añadir opciones else if Imagen Federico Coca Notas sobre ESP32 STEAMakers CC-BY-SA

Cada vez que pulsemos sobre el signo mas se añadirá una cláusula "else if" a la principal y podemos añadir tantas como necesitemos. Esta cláusula nos va a permitir establecer una nueva condición. En la imagen siguiente vemos añadidas dos para el caso de "if ... else". En la imagen observamos el código equivalente y las distintas condiciones que se pueden establecer.



Clausula if...else con dos else if Imagen Federico Coca Notas sobre ESP32 STEAMakers CC-BY-SA

El signo menos (-) que aparece en la imagen sirve para eliminar la correspondiente cláusula "else if".

En la TdR STEAM

La placa TdR STEAM dispone de dos pulsadores denominados SW1 y SW2 y conectados a los pines digitales D2 y D7 respectivamente.

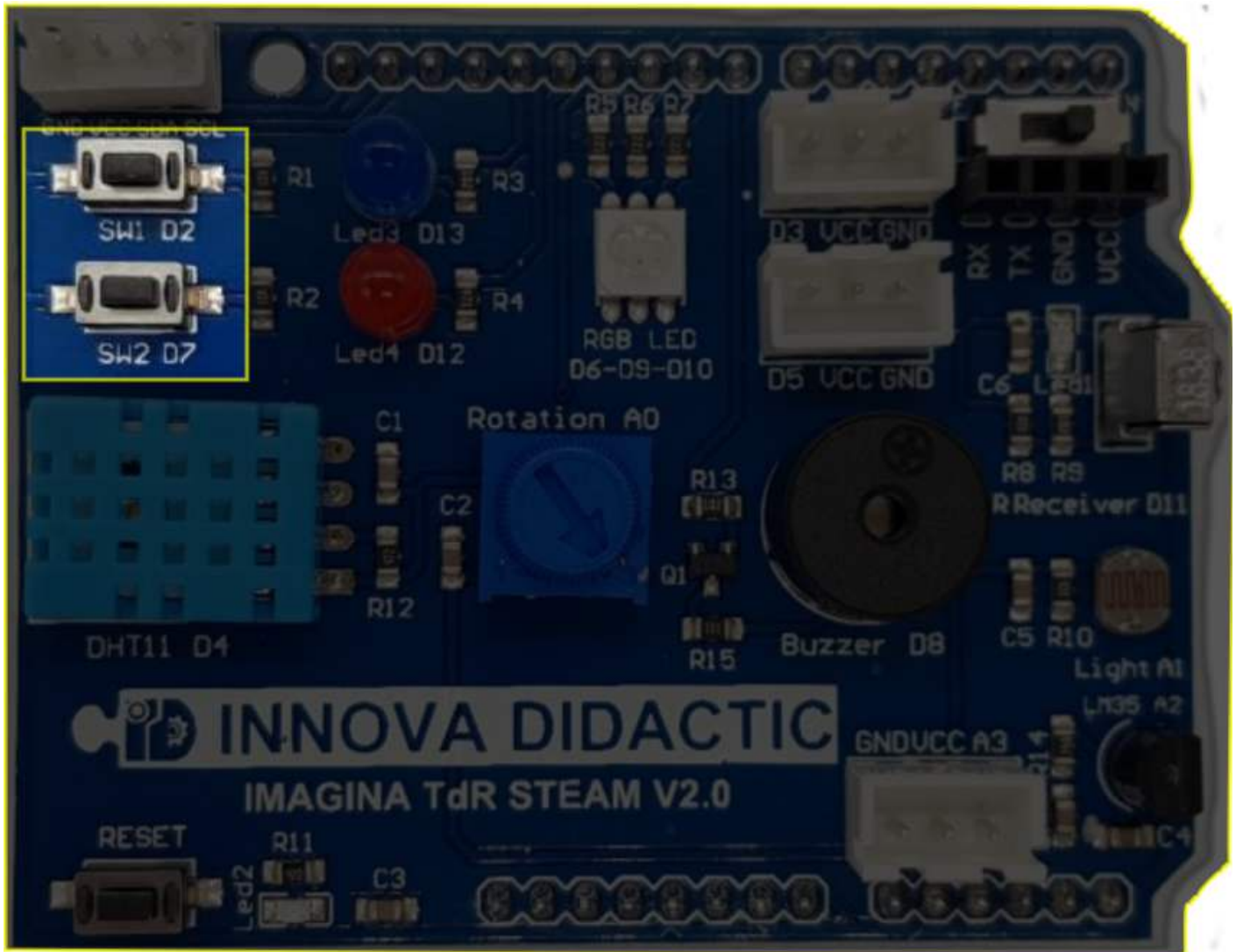


Imagen Federico Coca Notas sobre ESP32 STEAMakers CC-BY-SA

Programando la actividad

La aplicación de un pulsador para hacer algo requiere saber si este está pulsado o no lo está y para ello vamos a necesitar de las sentencias condicionales que hemos visto anteriormente.

Vamos a hacer un programa en el que preguntemos si el pulsador SW1 (D2) está o no pulsado y si lo está que se encienda el LED rojo (D12) y si no lo pulsamos que permanezca apagado. La solución al reto la tenemos disponible en [Actividad-06](#). A modo de comentario se ha colocado otra forma de realizar la misma programación que podemos probar simplemente arrastrando el bloque que hay en bucle fuera del mismo y colocando el otro en su lugar.

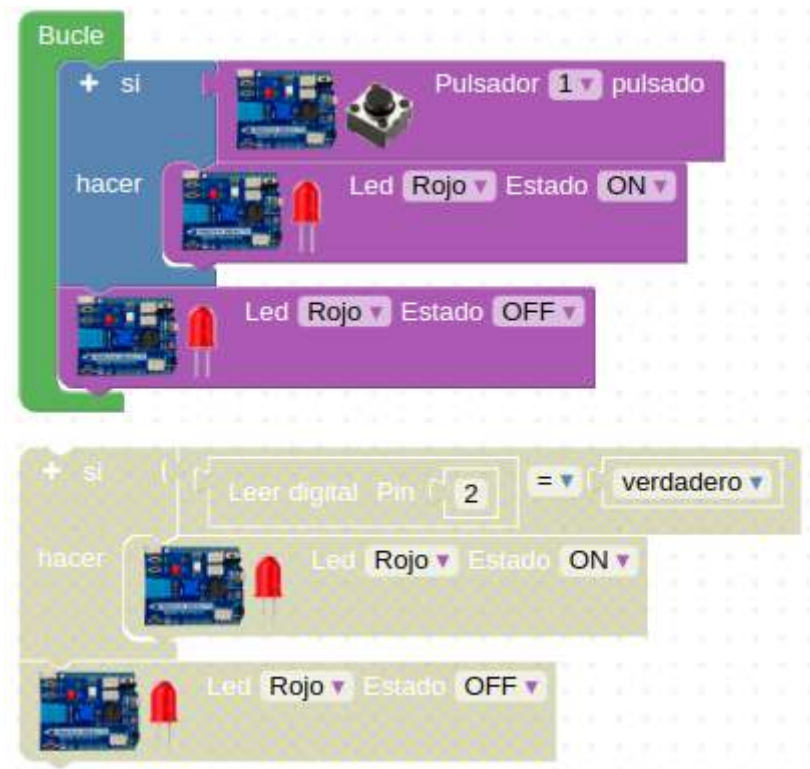


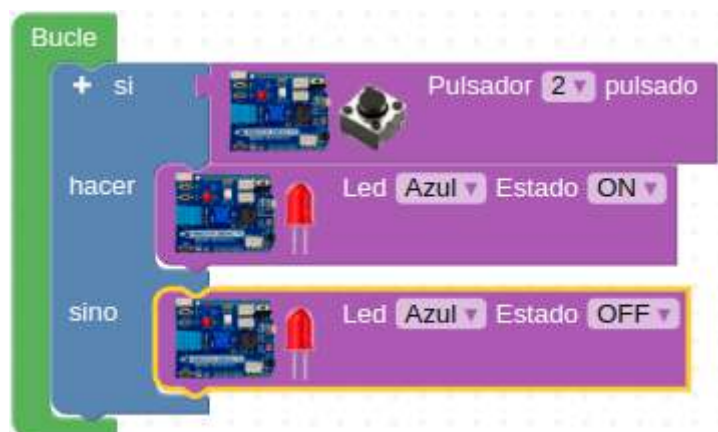
Imagen Federico Coca Notas sobre ESP32 STEAMakers CC-BY-SA

Ampliación sobre programación de la actividad

Ya se ha explicado el funcionamiento del signo mas (+) de la parte superior izquierda tanto del condicional "if" como la del "if ... else". Cada vez que pulsemos sobre el signo mas se añadirá una cláusula "else if" a la principal y podemos añadir tantas como necesitemos. Esta cláusula nos va a permitir establecer una nueva condición. El signo menos (-) que aparece en la imagen sirve para eliminar la correspondiente cláusula "else if".

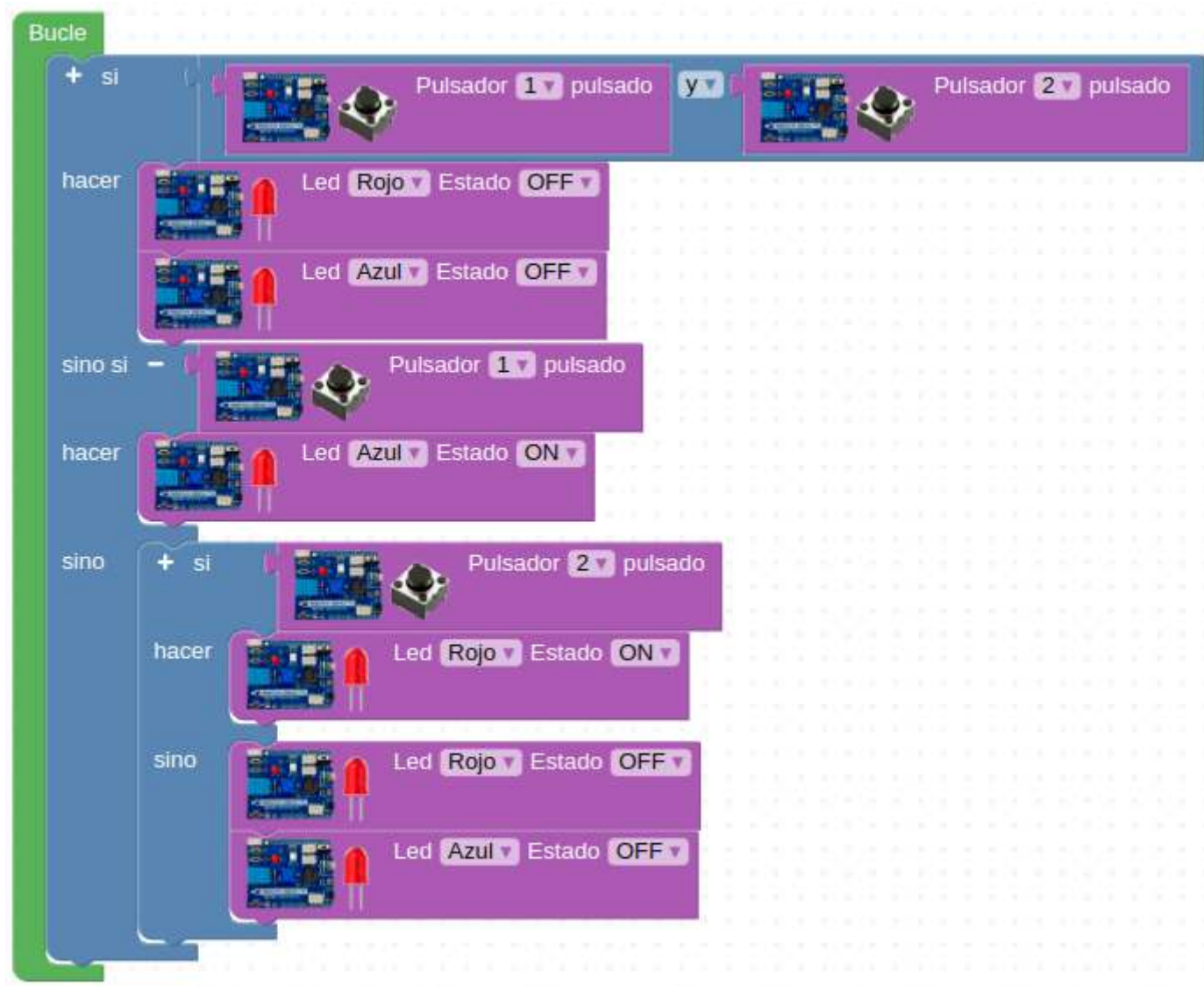
Vamos a hacer un programa similar a la Actividad-06 para hacer que si pulsamos SW2 se encienda el LED azul y si no está pulsado permanecerá apagado. La solución la tenemos disponible en

[Actividad-06-ampliación-1.](#)



Actividad-06-ampliacion-1 Imagen Federico Coca Notas sobre ESP32 STEAMakers CC-BY-SA

El siguiente programa hará que se encienda el LED azul al pulsar SW1 y el LED rojo al pulsar SW2 se encienda el rojo permaneciendo apagados si no están pulsados. Observe la primera condición AND (Y) que impide que si pulsamos ambos pulsadores al mismo tiempo se enciendan los LEDs. La solución la tenemos disponible en [Actividad-06-ampliación-2](#).



Actividad-06-ampliacion-12 Imagen Federico Coca Notas sobre ESP32 STEAMakers CC-BY-SA

Retos de ampliación

A6.R1. Resolver el reto inicial de forma que el funcionamiento sea al contrario, es decir, que el LED rojo esté siempre encendido y al pulsar SW1 se apague.

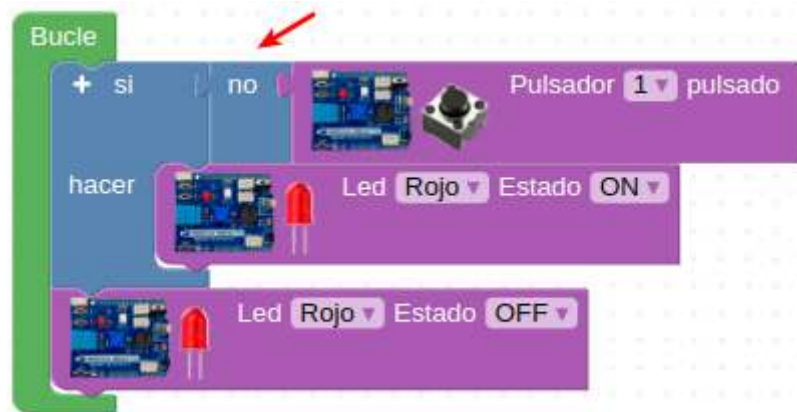
A6.R2. Hacer un programa que al pulsar SW1 se encienda el LED azul y que al pulsar SW2 se apague. Como ampliación se sugiere modificar el programa para que encienda y apague los dos LEDs a un tiempo.

A6.R3. Hacer un programa que emita, usando SW1 como si fuese un telégrafo, el código Morse universal de solicitud de socorro, SOS.

A6.R4. Hacer un programa que al pulsar SW1 encienda el LED azul y que este permanezca encendido hasta que no pulsemos SW1 dos veces mas (3 pulsaciones en total), en cuyo caso se apagará.

Solución A6.R1

La solución está en utilizar el operador NOT. El Programa es el de la imagen siguiente:



Reto 1 de la actividad 6 *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Solución A6.R2

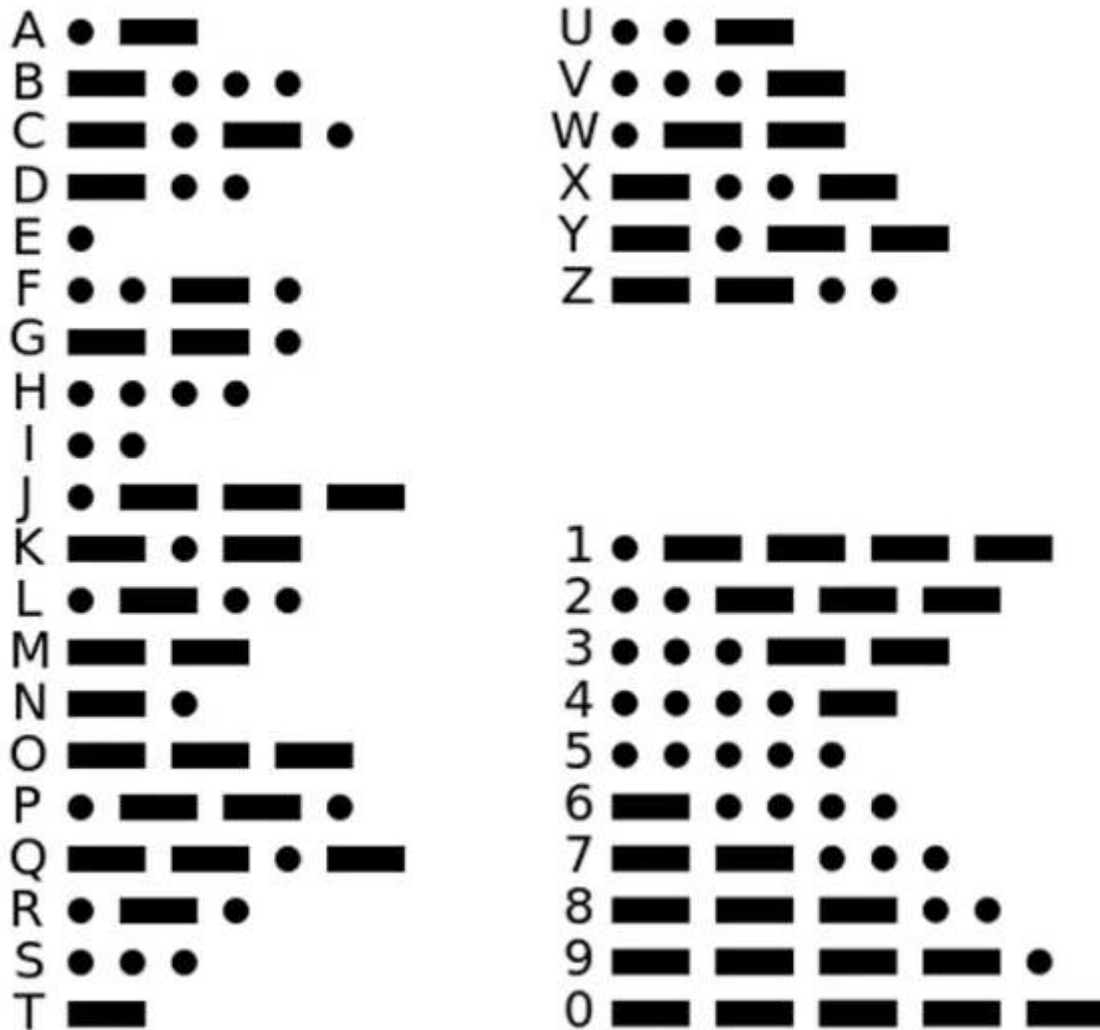
La solución en esta ocasión está en utilizar el operador NOT y el operador AND. El Programa es el de la imagen siguiente:



Reto 2 de la actividad 6 *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Solución A6.R3

El alfabeto Morse lo vemos en la imagen siguiente, donde podemos observar que la S son tres puntos o pulsaciones cortas y la O son tres rayas o pulsaciones largas. Sin entrar en mas detalles daremos la solución considerando SOS como una palabra (es lo universalmente adoptado) de forma que no haremos pausas entre letras.



Alfabeto Morse *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Solución A6.R3

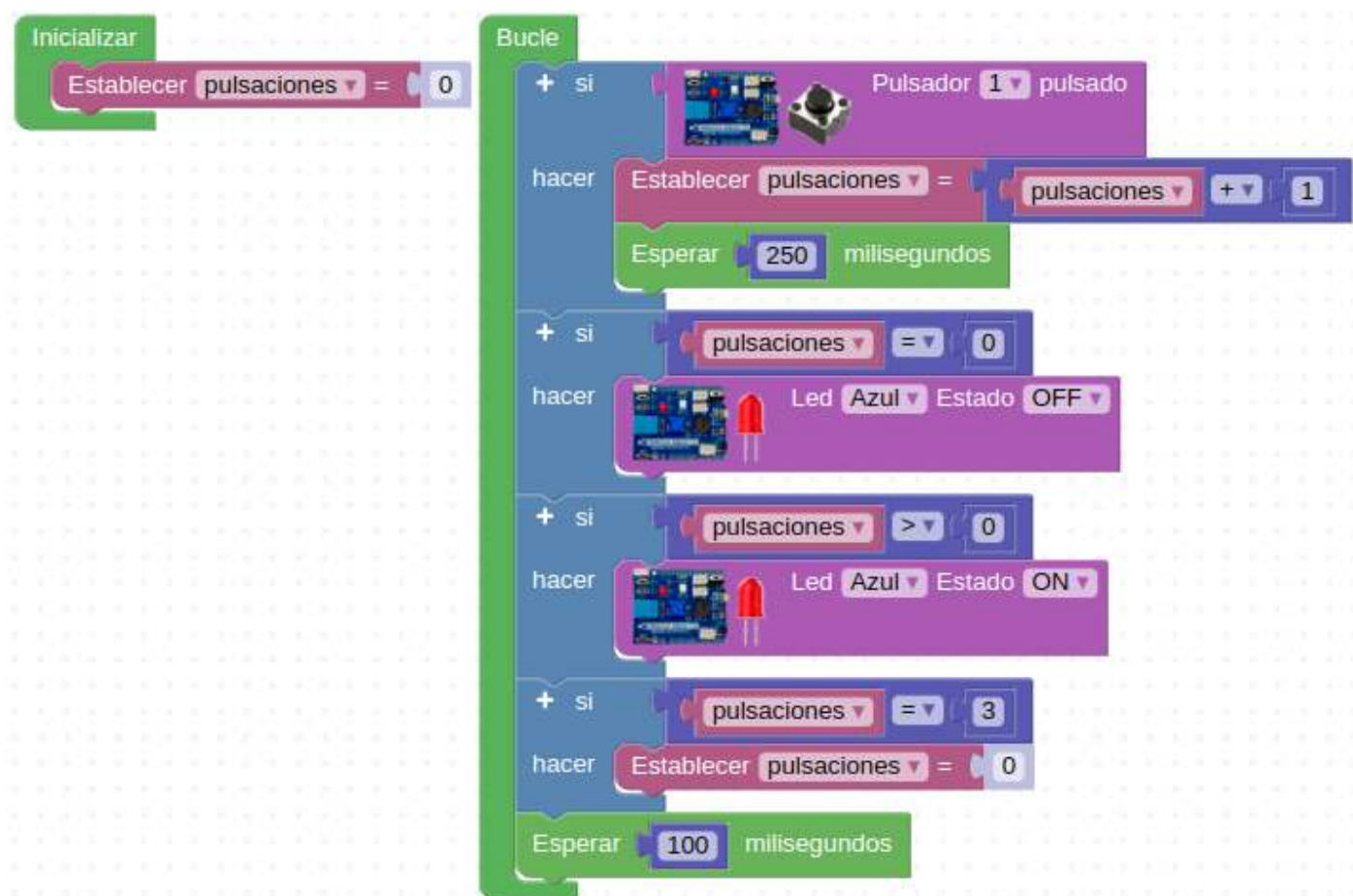
El Programa es el de la imagen siguiente:



Reto 3 de la actividad 6 Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Solución A6.R4

Definimos una variable y la utilizamos para resolver el programa mediante un contador. El Programa es el de la imagen siguiente:



Reto 4 de la actividad 6 Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Prácticas con sensores analógicos

Prácticas con sensores analógicos

Actividad-07. El potenciómetro y Plotter

Página extraída de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Enunciado

Introducir el uso de las entradas analógicas para visualizar los cambios que hagamos con el potenciómetro en el terminal serie o consola serie. Cuando hablamos de lecturas analógicas suele ser muy útil el uso del mapeo, concepto que también vamos a introducir y que veremos en la segunda parte del reto.

Teoría

Antes de nada ya hemos indicado que el potenciómetro va conectado a un pin analógico y es en este momento cuando vamos a establecer la diferencia entre los conceptos de analógico y de digital.

Una clasificación de los circuitos electrónicos es dividirlos en dos grandes categorías: digitales y analógicos.

- La electrónica digital utiliza magnitudes con dos valores discretos conocidos como 0 - 1, alto - bajo, on - off, etc, y que se corresponden con la presencia o no de tensión en un determinado punto.
- La electrónica analógica emplea magnitudes con valores continuos. En concreto en las placas En la ESP32 Plus STEAMakers, las entradas analógicas pueden tener 2^{12} valores (12 bits de resolución = 4.096 valores), es decir, valores comprendidos entre 0 y 4.095. Estos valores se pueden expresar de forma numérica o de forma porcentual, correspondiendo el 0 a un 0% y el 4.095 al 100%.

El potenciómetro

En la imagen siguiente vemos el aspecto de un modelo concreto de potenciómetro así como los símbolos habituales del mismo. La numeración indica lo siguiente:

Terminales 1 y 3 son los contactos unidos a los extremos de la resistencia fija o resistencia total del potenciómetro.

Terminal 2 es el contacto que va unido al cursor o parte móvil que se desliza sobre la resistencia fija haciendo que la resistencia entre un terminal y el cursor varíe en función de la posición de este.



[*Imagen Federico Coca Notas sobre ESP32 STEAMakers CC-*](#)

BY-SA

La posición del cursor se determina de forma mecánica y son adecuados para usarlos como elementos de control de tensión (conexión en serie) o de corriente (conexión en paralelo). Los potenciómetros del tipo que estamos viendo (existen de otros muchos tipos) tienen un funcionamiento en forma de rotación con un ángulo de unos 270 grados entre los puntos mas extremos.

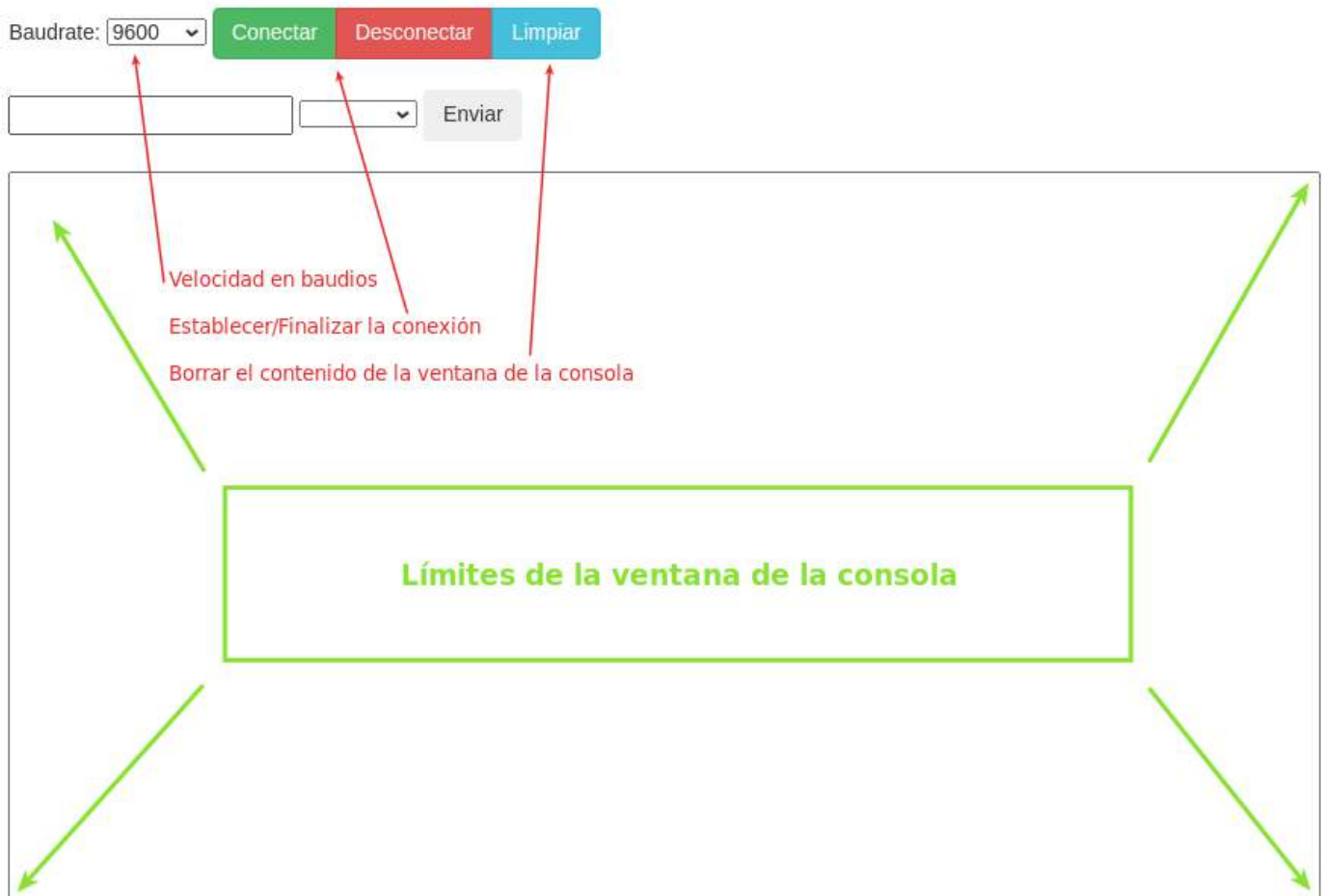
El terminal o consola serie

Sirve para visualizar en un ordenador los datos recibidos a través del puerto serie y en realidad es una aplicación que controla las comunicaciones bidireccionales a través de la UART integrada en el microcontrolador.

Es muy importante tener siempre presente que el puerto serie es el que se utiliza para "subir" el firmware o programa a la placa, tarea que se realiza a través de una aplicación pregrabada en el microcontrolador y que se denomina "bootloader".

En ArduinoBlocks es posible usar la consola serie solamente si tenemos instalada y en funcionamiento la aplicación ArduinoBlocks-Connector que es la encargada de establecer las comunicaciones locales de nuestra placa con las remotas de la aplicación. En la imagen siguiente vemos el aspecto de la consola.

ArduinoBlocks :: Consola serie



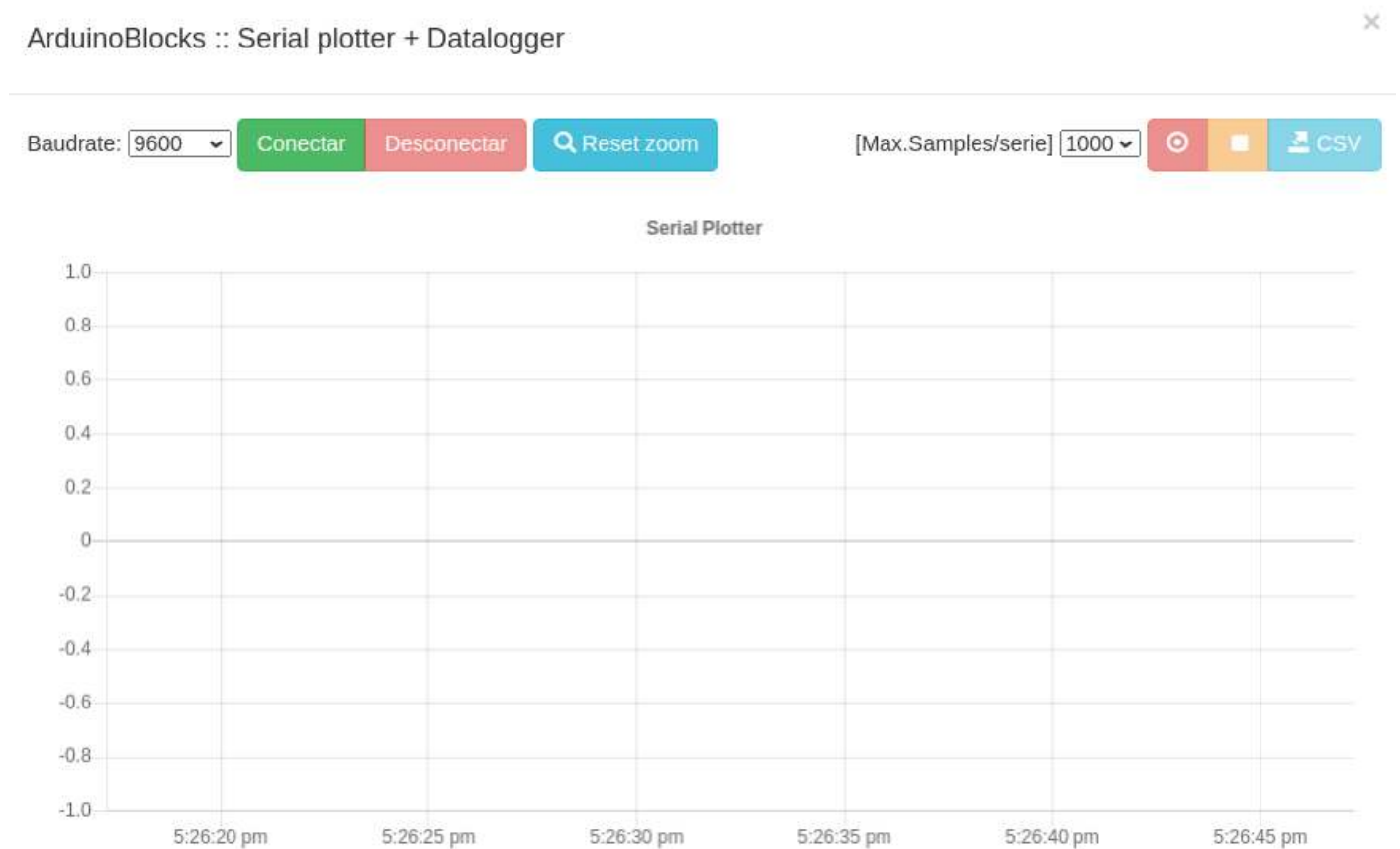
Aspecto de la Consola en AB [Imagen Federico Coca Notas sobre ESP32 STEAMakers](#) CC-BY-SA

A la hora de establecer una conexión serie los dos extremos que intervienen en la conexión (placa ESP32 STEAMakers y ordenador) deben establecer el mismo valor en la velocidad de la conexión. Por defecto esta velocidad es de 115200 baudios o bits por segundo en el bloque inicializar y otras velocidades comunmente utilizadas son: 4800, 9600, 19200, 38400, 57600. Es por lo tanto imprescindible incluir en el bloque "Inicializar" el bloque "Iniciar" y establecer la velocidad de comunicación.

Serial Plotter - Datalogger

Es otra funcionalidad relacionada con la comunicación serie que nos permite visualizar información en forma de gráfica en tiempo real. Además el "Serial Plotter" implementa un sencillo datalogger con el que podemos ir grabando los datos para exportarlos posteriormente. En ArduinoBlocks existen bloques que nos permiten trabajar con el serial plotter. El serial plotter + datalogger se activa haciendo clic en la flecha a la derecha de Consola y tiene el aspecto que vemos en la

imagen siguiente:



Serial Plotter + Datalogger [Imagen Federico Coca Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Mapeo

Hemos indicado que las entradas analógicas trabajan con 12 bits, o sea valores comprendidos entre 0 y 4095 (2^{12}) y también que las salidas digitales trabajan con 8 bits, o sea valores entre 0 y 255 (28), por lo que si queremos combinar en nuestro programa entradas analógicas con salidas digitales debemos realizar un ajuste de escala en los datos. A este ajuste se le conoce como "mapear" y es un bloque disponible en el menú Matemáticas con el aspecto de la imagen siguiente:



[Imagen Federico Coca Notas sobre ESP32 STEAMakers](#) CC-BY-SA

En la TdR STEAM

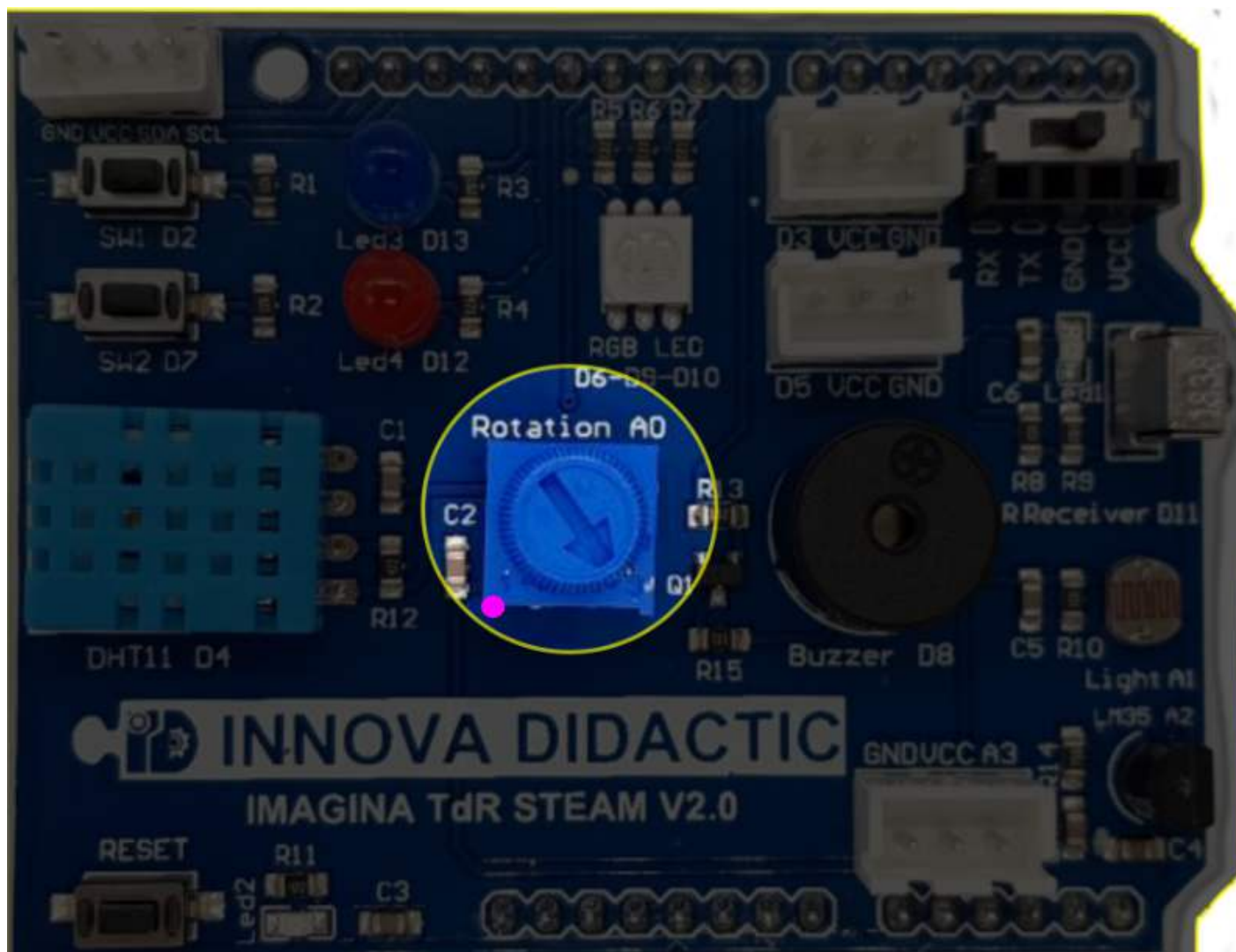


Imagen Federico Coca Notas sobre ESP32 STEAMakers CC-BY-SA

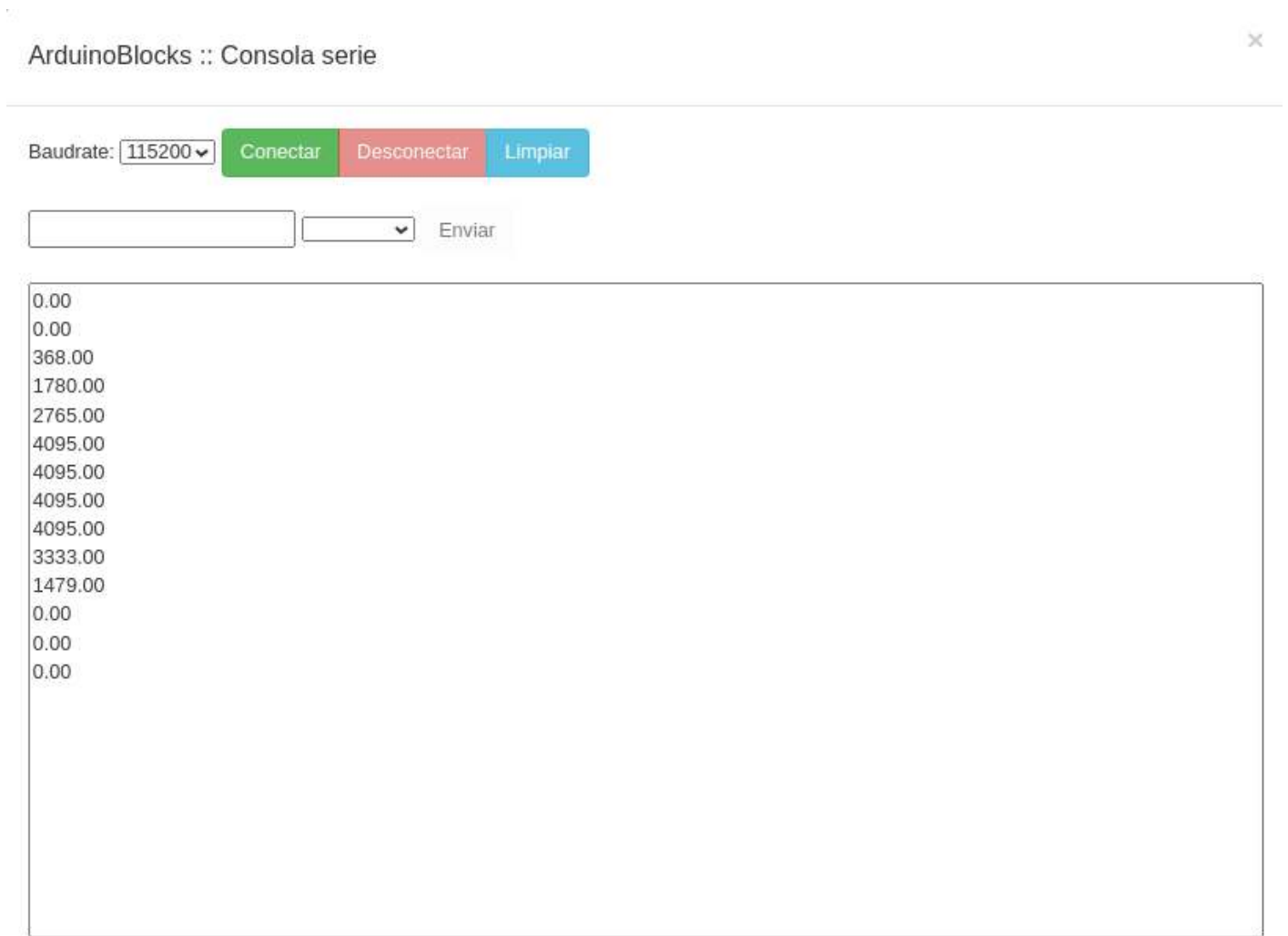
Programando la actividad

Parte 1. Vamos a guardar los datos leídos del potenciómetro en una variable y mostrarlos a través de la consola serie. La solución la tenemos disponible en [Actividad-07: Parte 1](#) que es el programa que vemos en la imagen siguiente:



[Imagen Federico Coca Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Si cargamos el programa en nuestra placa y posteriormente activamos la consola y conectamos ArduinoBlocks con nuestro ordenador podemos ver un resultado similar al de la imagen siguiente que se corresponde con variaciones de extremo a extremo del potenciómetro.



Aspecto de la consola [Imagen Federico Coca Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Para entender el significado del check "salto de linea" del bloque "Enviar" podemos desactivarlo y volver a hacer el mismo proceso.

¡MUY IMPORTANTE! cuando terminemos de manipular el potenciómetro que el mismo esté totalmente girado a la posición izquierda (hacia el reset), ya que comparte la conexión A0 (GPIO02) con el sistema de grabación del programa. Si no está en esa posición se producirá un error en el envío del siguiente programa.

Parte 2. Vamos a modificar el programa de la parte 1 del reto para mapear los datos antes de enviarlos a la consola serie. La solución la tenemos disponible en [Actividad-07: Parte 2](#) que es el programa que vemos en la imagen siguiente:



Parte 2 de la actividad 7 [Imagen Federico Coca Notas sobre ESP32 STEAMakers](#) CC-BY-SA

El resultado ahora lo vemos en la imagen siguiente:



Aspecto de la consola [Imagen Federico Coca Notas sobre ESP32 STEAMakers](#) CC-BY-SA

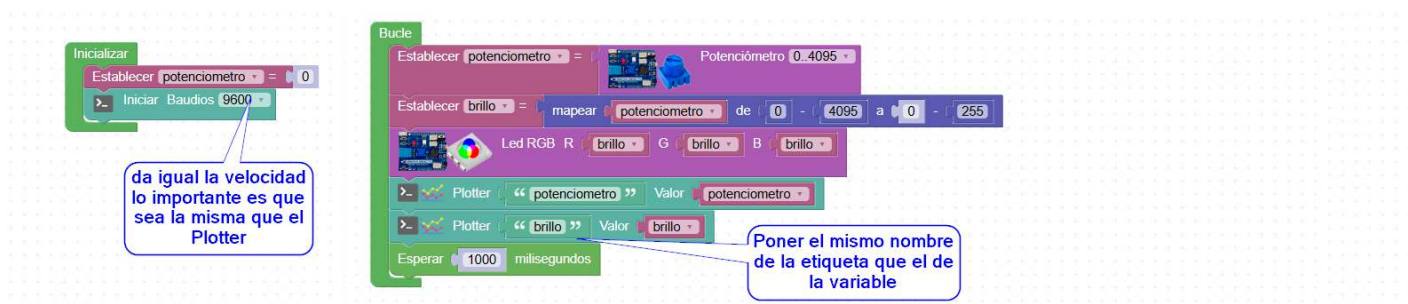
ENVIO DE DATOS SERIE PERO EN VEZ DE CONSOLA POR PLOTTER

(Autor Javier Quintana)

Ahora vamos a cambiar el "Enviar" serie por "Plotter"

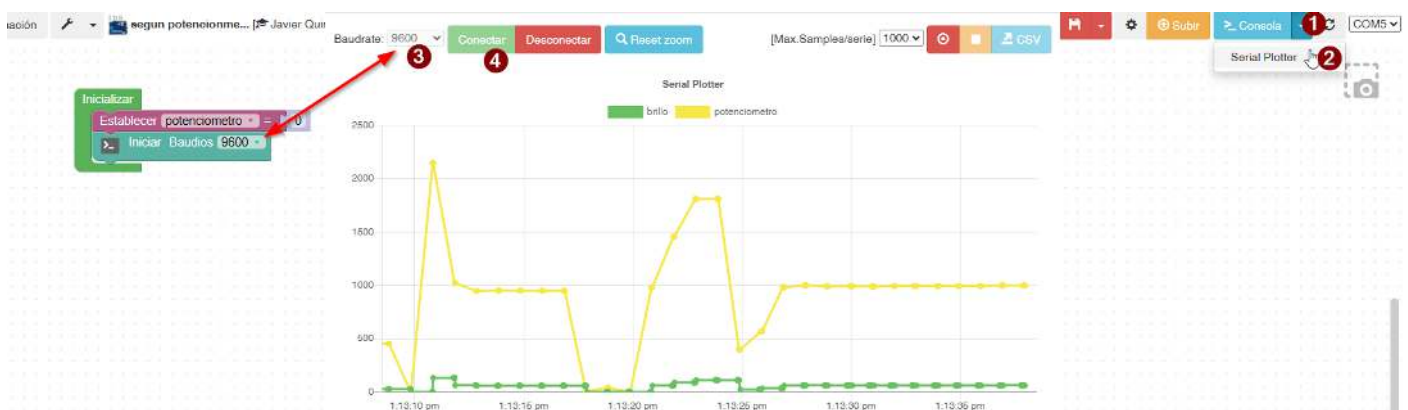


Y además vamos a mapear el valor del potenciómetro de 0-4095 a 0-255 y lo almacenaremos en una variable "brillo" que nos servirá para que el RGB se ilumine más o menos:



Es importante poner un retardo (en este caso 1000 mseg) si no lo ponemos, saturamos el puerto serie

Abrimos el plotter y jugamos:



¡MUY IMPORTANTE! cuando terminemos de manipular el potenciómetro que el mismo esté totalmente girado a la posición izquierda (hacia el reset), ya que comparte la conexión A0 (GPIO02) con el sistema de grabación del programa.
Si no está en esa posición se producirá un error en el envío del siguiente programa.

Retos de ampliación

- **A7.R1.** Utilización del bloque "crear texto con ..." del menú "Texto". Utilizaremos cualquiera de los programas vistos en la actividad y configuraremos el nuevo bloque para que en cada línea nos muestre el mensaje "El valor del potenciómetro es: ", a continuación nos muestre el valor y tras el mismo el símbolo % porque configuraremos la lectura en porcentaje. Dejaremos transcurrir un tiempo de 3s entre cada muestra de salida por consola.
- **A7.R2.** Control del LED RGB con el potenciómetro. Vamos a dividir el rango total en 8 partes y asignarle a cada una de ellas uno de los colores RGB según vemos en la tabla siguiente:

Color	Rango	R	G	B
Rojo	0 a 512	255	0	0
Verde	513 a 1023	0	255	0
Azul	1024 a 1536	0	0	255
Amarillo	1537 a 2048	255	255	0
Cian	2049 a 2560	0	255	255
Magenta	2561 a 3072	255	0	255
Blanco	3073 a 3584	255	255	255
Naranja	3584 a 4096	255	127	0

- **A7.R3.** Repetir el reto 2 de esta actividad pero ahora además mostrando por consola el valor del potenciómetro.

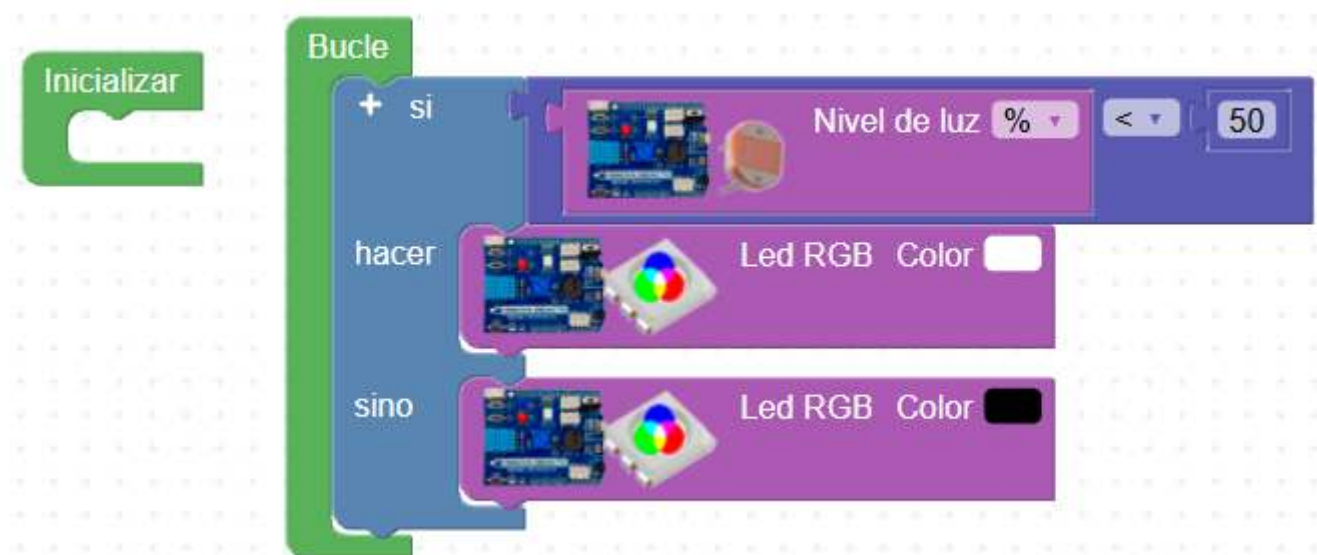
Prácticas con sensores analógicos

Actividad-08. La fotorresistencia o LDR

Interruptor Crepuscular

Para ver cómo funciona, qué mejor ejemplo que hacer un interruptor crepuscular, es decir "Que se encienda la luz cuando se haga de noche"

Para ello ponemos el nivel de luz en % y definimos qué es día y qué es noche con un valor umbral, en este ejemplo hemos puesto 50%



<https://www.youtube.com/embed/1yza0ayy9wg>

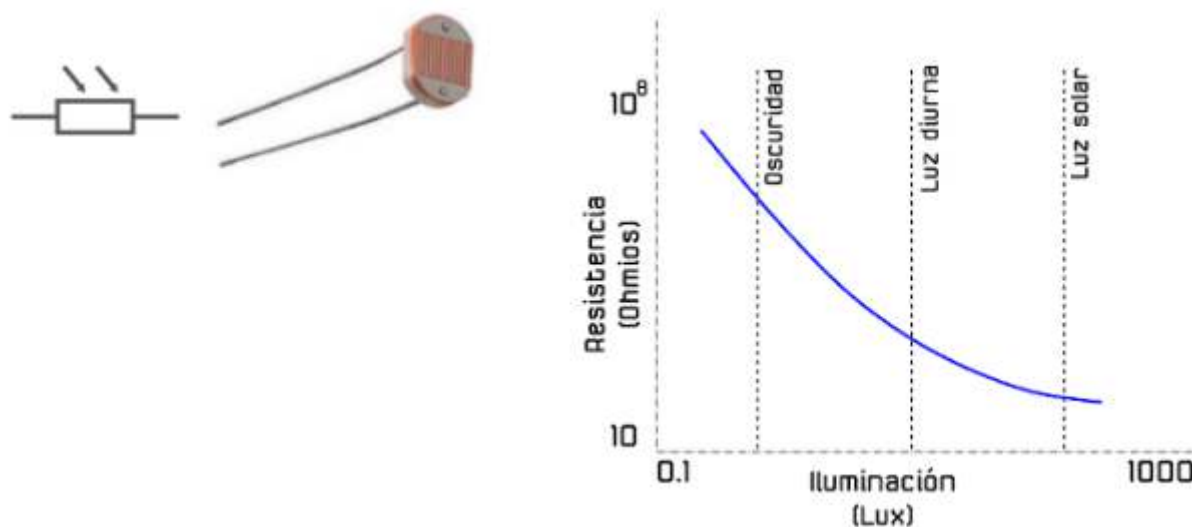
Página extraída de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Enunciado

Utilizaremos la consola serie para mostrar los valores de luz detectados por la resistencia LDR o fotocélula.

Teoría

Una fotorresistencia o fotorresistor es un componente electrónico cuya resistencia disminuye de forma exponencial con el aumento de la intensidad de luz incidente. Las siglas LDR vienen de su nombre en inglés, que es Light Dependent Resistor. En la imagen siguiente tenemos el símbolo, el aspecto real de una LDR y su curva característica de variación de resistencia con la iluminación.



Símbolo y aspecto de la LDR y curva característica *Imagen Federico Coca* [Notas sobre ESP32](#)

[STEAMakers](#) CC-BY-SA

En la TdR STEAM

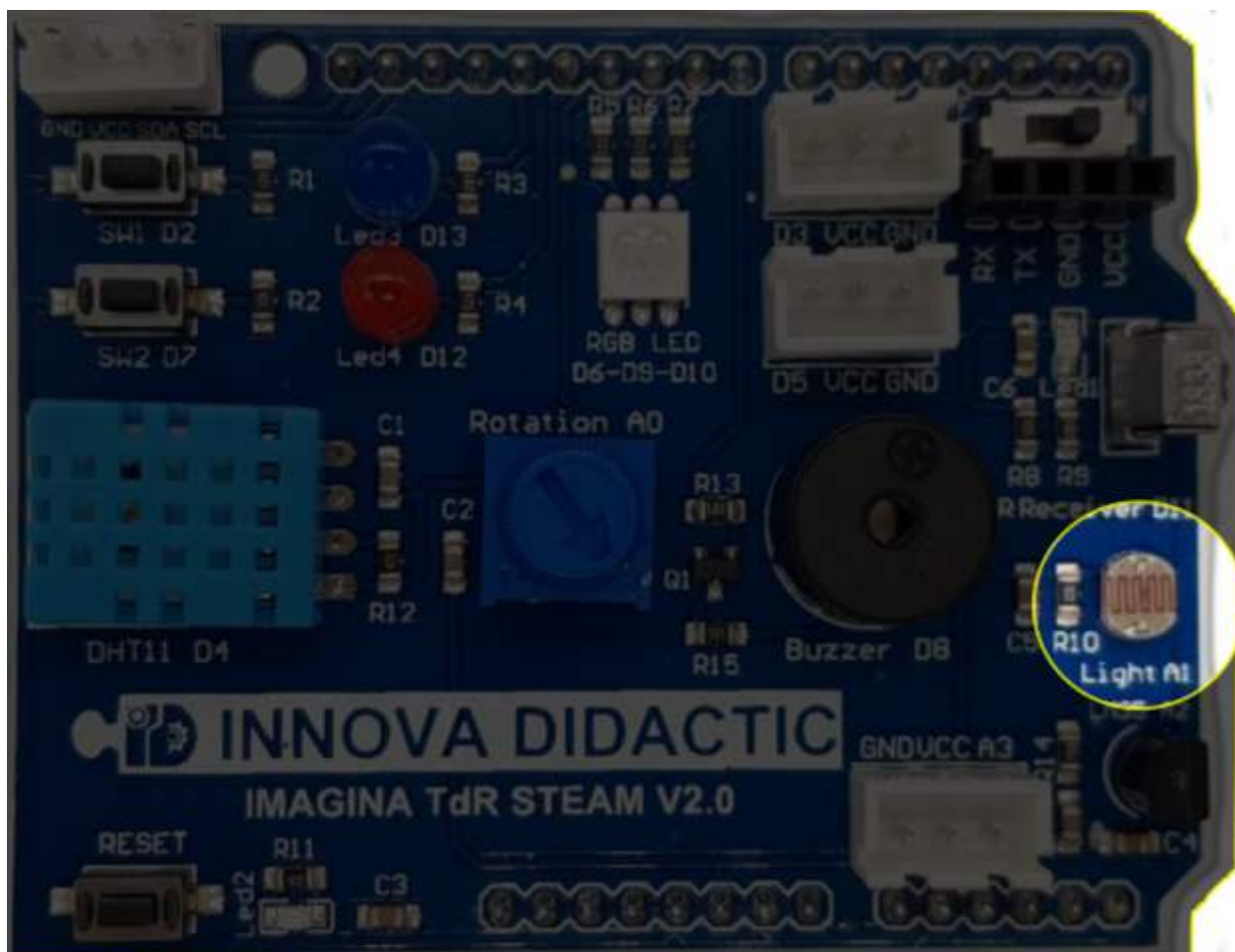


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Programando la actividad

Dado que en el menú TDR STEAM de ArduinoBlocks tenemos disponible un bloque que nos devuelve el nivel de luz en porcentaje o de forma numérica, resolver la actividad es sumamente sencillo. La solución la tenemos disponible en [ESP32-SM-Actividad-08](#) que es el programa que vemos en la imagen siguiente:



Actividad-08 Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Esto nos generará algo como lo de la imagen siguiente en la consola:

ArduinoBlocks :: Consola serie ✕

Baudrate: 115200 Conectar Desconectar Limpiar

▼ Enviar

Nivel de luz: 4095.00
Nivel de luz: 4095.00
Nivel de luz: 4095.00
Nivel de luz: 3921.00
Nivel de luz: 4095.00
Nivel de luz: 1607.00
Nivel de luz: 1586.00
Nivel de luz: 1763.00
Nivel de luz: 1419.00
Nivel de luz: 1327.00
Nivel de luz: 1335.00
Nivel de luz: 4095.00
Nivel de luz: 4095.00
Nivel de luz: 4095.00
Nivel de luz: 4095.00

Consola que produce la actividad 8 *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Opción Plotter gráfico

Es más visual y dinámico



ArduinoBlocks :: Serial plotter + Datalogger (BETA)



Extraído de la página 167 Manual Steam Maker ESP32

- [Descarga desde libros.catedu..es Manual Actividades ESP32 SteaMakers 2022 Junio.pdf](#)

- [Descarga desde el drive del auto tutorial Manual de Actividades ESP32 SteamMaker](#)
[2022_Junio](#)

Retos de ampliación

A8.R1. Detectar el nivel de iluminación de la sala y condiciones de iluminación en las que estemos utilizando la LDR y enviar el resultado a la consola.

A8.R2. Programar un interruptor crepuscular utilizando la LDR y uno de los LEDs para simular el farol. El nivel de luz mínimo permitido antes de encender el farol dependerá del resultado obtenido en la reto 1, de forma que con un valor menor o igual se encienda el LED y con un valor mayor permanezca apagado.

A8.R3. Hacer el interruptor crepuscular utilizando como farol en LED RGB que se encenderá en color blanco.

Prácticas con sensores analógicos

Actividad-09. El sensor de temperatura LM35D

Página extraída de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Enunciado

Utilizaremos este sensor de temperatura para medir la temperatura de la habitación en la que estemos resolviendo el reto mostrando el resultado en la consola.

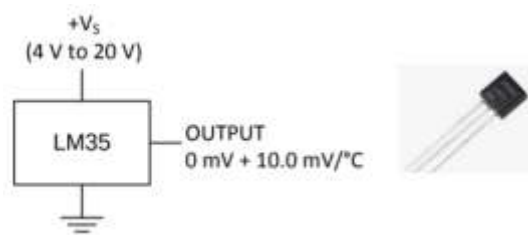
Teoría

LM35

El LM35 es un circuito integrado de medida de temperatura de precisión con una tensión de salida lineal y proporcional a la temperatura en grados centígrados. Algunas de las características que se pueden extraer de la hoja de datos del LM35 son:

- Directamente calibrado en grados Celsius (Centígrados)
- Factor de escala lineal de $+10\text{-mV}/^{\circ}\text{C}$
- Resolución asegurada de 0.5°C
- Rango de medida de -55°C a 150°C

En la imagen siguiente vemos su representación en circuito y el aspecto físico que tiene en uno de sus encapsulados mas usual.



Representación esquemática y aspecto *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-

BY-SA

Bloques de tiempo

Para esta actividad vamos a explicar los bloques de tiempo que se implementan en ArduinoBlocks. Las funciones de tiempo o retardo nos permiten realizar pausas y obtener información sobre el tiempo transcurrido dentro del microcontrolador.

1.- **Bloques esperar.** Realizan una pausa del tiempo que establezcamos hasta seguir con la ejecución del siguiente bloque. Se corresponden con las instrucciones `delay(1000)` y `delayMicroseconds(1000)`. Tenemos disponibles los dos de la imagen siguiente:

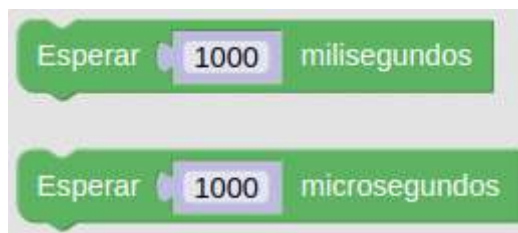


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

2.- **Tiempo transcurrido.** Obtiene un valor con el tiempo transcurrido desde el inicio o reset del microcontrolador de la placa ESP32 STEAMakers. Se trata de las funciones `millis()` y `micros()`. El valor puede ser en milisegundos o microsegundos, tal y como vemos en la imagen siguiente:

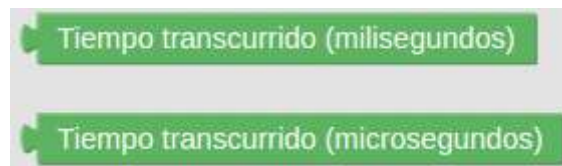


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

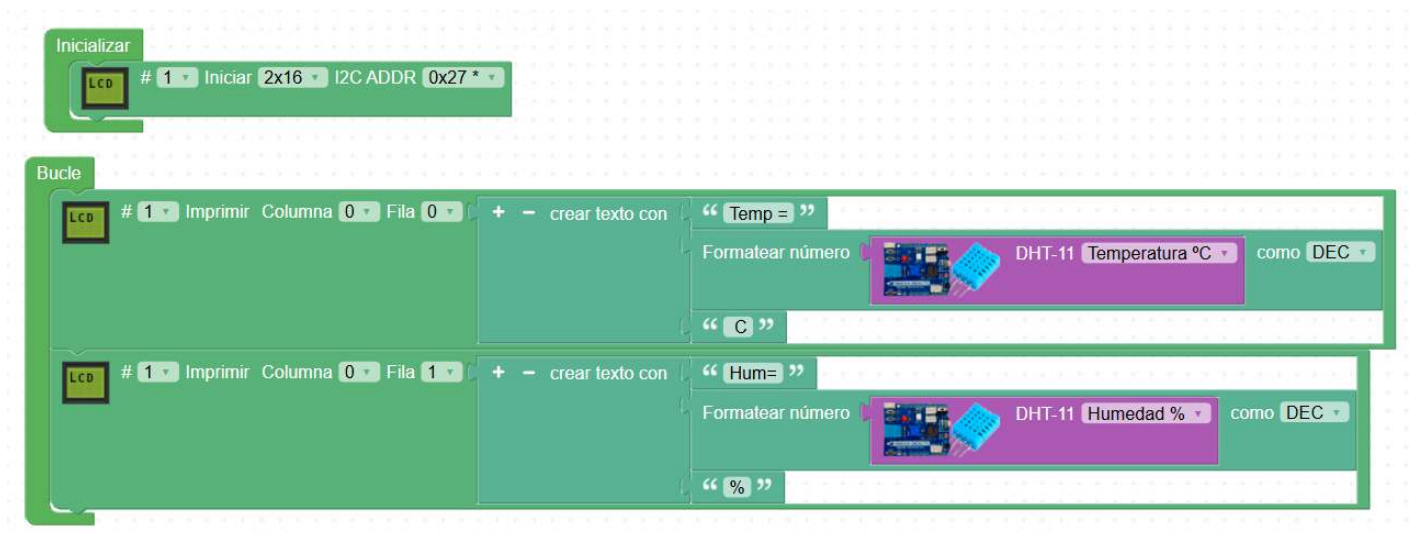
Prácticas con sensores analógicos

Actividad-10. El sensor de temperatura y humedad DHT-11

Visualización humedad y temperatura por el display LCD

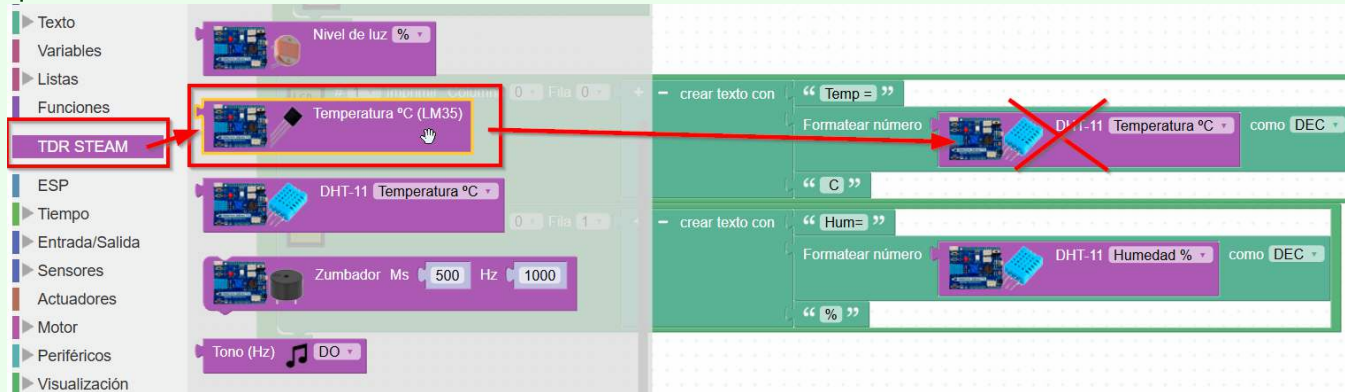
El clásico es visualizar por el LCD la temperatura y la humedad instantánea.

- Más adelante veremos el LCD y por qué hay que inicializarlo con la dirección 0x27
- Fíjate que hay que formatear el valor que devuelve la función DHT-11 como decimal, la explicación [lo vimos aquí en sensores DHT11](#)



Haz un pequeño experimento:

Sustituye la instrucción de temperatura DHT11 por el LM35 ¿Sale el mismo valor? ¿Cual crees que es el verdadero?



Convierte el ESP32 EN ESTACIÓN METEROLÓGICA

En la página 158 del Manual actividades ESP32 tienes un ejemplo de grabar la temperatura y la humedad en un fichero texto en una tarjeta microsim (la placa ESP32 incorpora un lector)

- [Descarga desde libros.catedu..es Manual Actividades ESP32 SteaMakers 2022 Junio.pdf](#)
- [Descarga desde el drive del auto tutorial Manual de Actividades ESP32 SteamMaker 2022 Junio](#)

Página extraída de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Enunciado

Realizar un programa básico de medida de temperatura y humedad relativa de la habitación en la que estamos trabajando con nuestra TdR STEAM.

Teoría

DHTxx

El DHT11 es un modelo de sensor que permiten realizar la medición simultánea de temperatura y humedad. Dispone de un procesador interno que es el encargado de realizar la medición entregando la información mediante una señal digital.

Se presenta en un encapsulado plástico típico de color azul. Sus principales características son:

- Rango de temperatura: 0 a 50°C
- Precisión de la medida de temperatura: $\pm 2^\circ\text{C}$
- Rango de humedad: 20 a 80%
- Precisión en la medida de humedad: $\pm 5\%$.
- Frecuencia de muestreo: 1 muestra por segundo (1 Hz)

El DHT11 es un sensor bastante limitado que podemos usar con fines de formación, pruebas, o en proyectos que realmente no requieran una medición precisa.

Si necesitamos mayor precisión y rango podemos recurrir al DHT22 que es de la misma familia y lo único que cambia es sus características y el precio. Sus características son:

- Rango de temperatura: -40 a 125°C
- Precisión de la medida de temperatura: $\pm 0.5^\circ\text{C}$
- Rango de humedad: 0 a 100%
- Precisión en la medida de humedad: ± 2 a 5%
- Frecuencia de muestreo: 2 muestras por segundo (2 Hz)

En la imagen siguiente vemos el aspecto de ambos sensores:

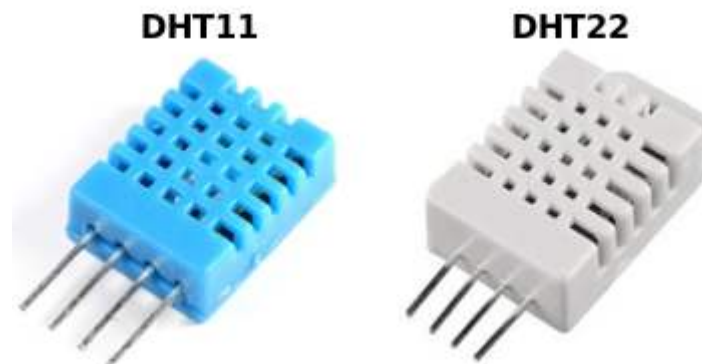


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

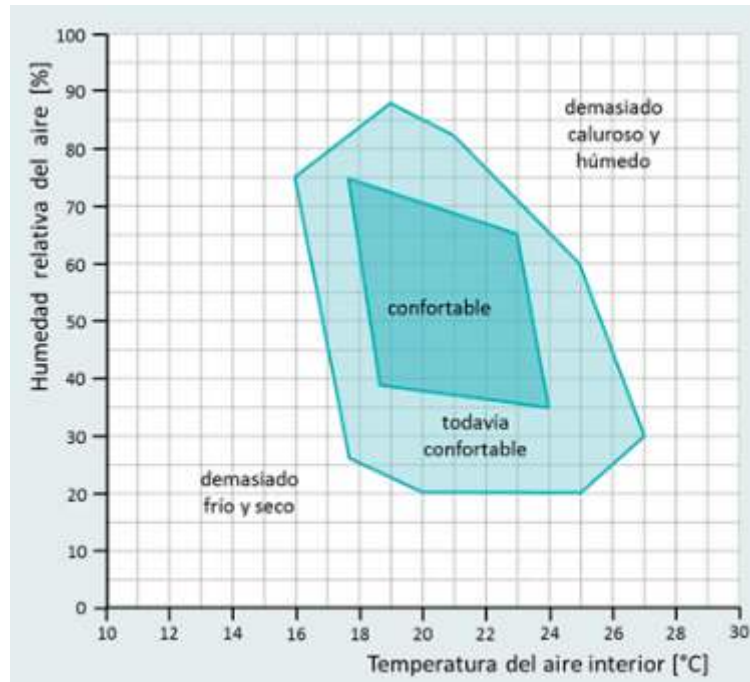
Estado de confort

En la web [ARQUITECTURA & ENERGÍA](#) podemos encontrar un artículo donde se nos explica con bastante profundidad el tema del confort térmico.

Puede definirse confort térmico, o más propiamente comodidad higrotérmica, como la ausencia de malestar térmico. En fisiología, se dice que hay confort higrotérmico cuando no tienen que intervenir los mecanismos termorreguladores del cuerpo para una actividad sedentaria y con una indumentaria ligera. Esta situación puede registrarse mediante índices que no deben ser

sobrepasados para que no se pongan en funcionamiento los sistemas termorreguladores (metabolismo, sudoración y otros).

En la imagen siguiente vemos los valores de temperatura y humedad que delimitan las zonas de confortabilidad.



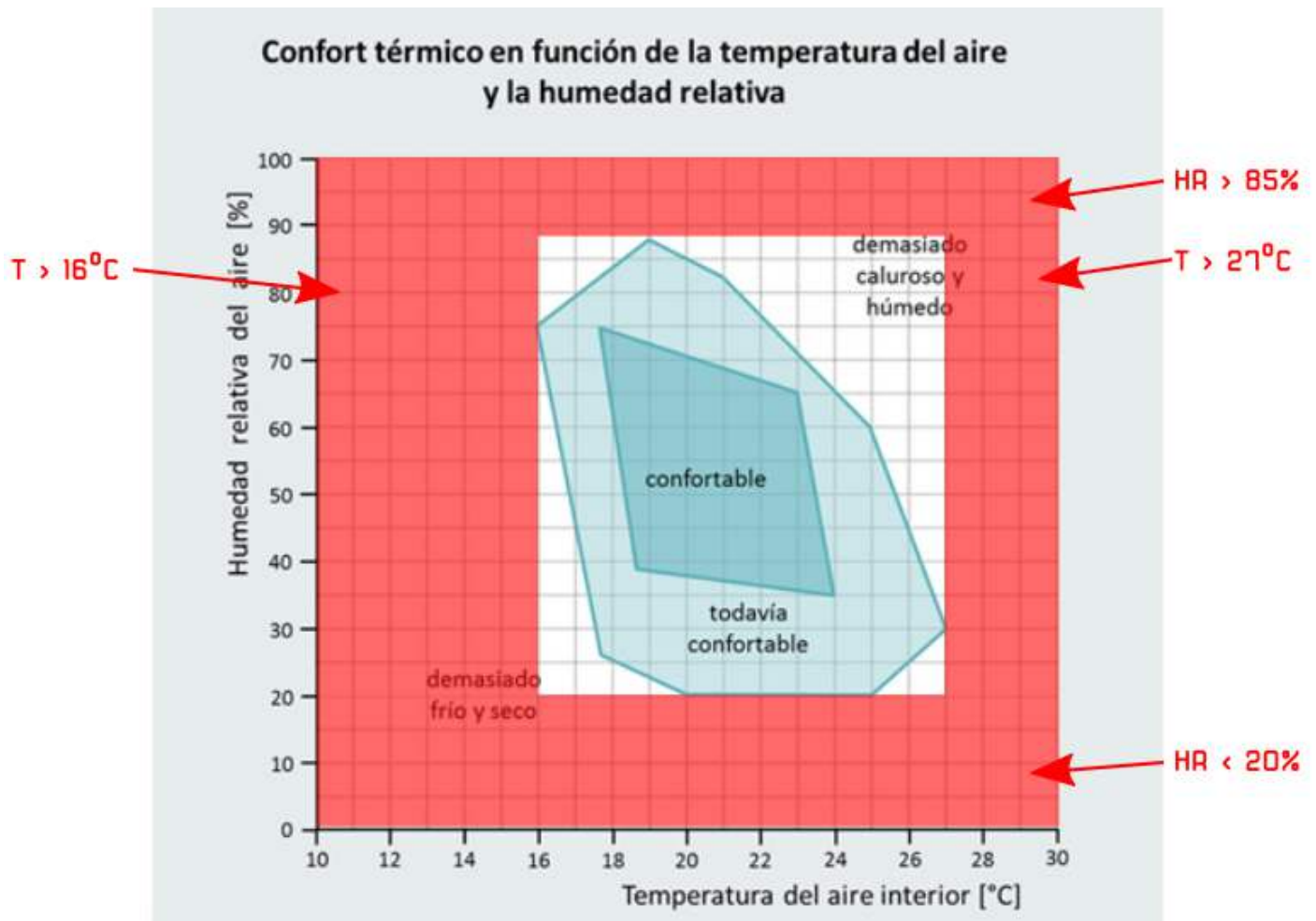
Confort térmico en función de temperatura y humedad Imagen Federico Coca [Notas sobre ESP32](#)

[STEAMakers](#) CC-BY-SA

Sobre el gráfico vamos a delimitar zonas de temperatura y humedad para establecer su color. Por motivos de simplicidad lo vamos a hacer delimitando zonas rectangulares, pero comprobamos que no cometemos grandes errores y para nuestro propósito nos sirve.

1.- Zona Roja: en la imagen siguiente tenemos delimitadas las zonas:

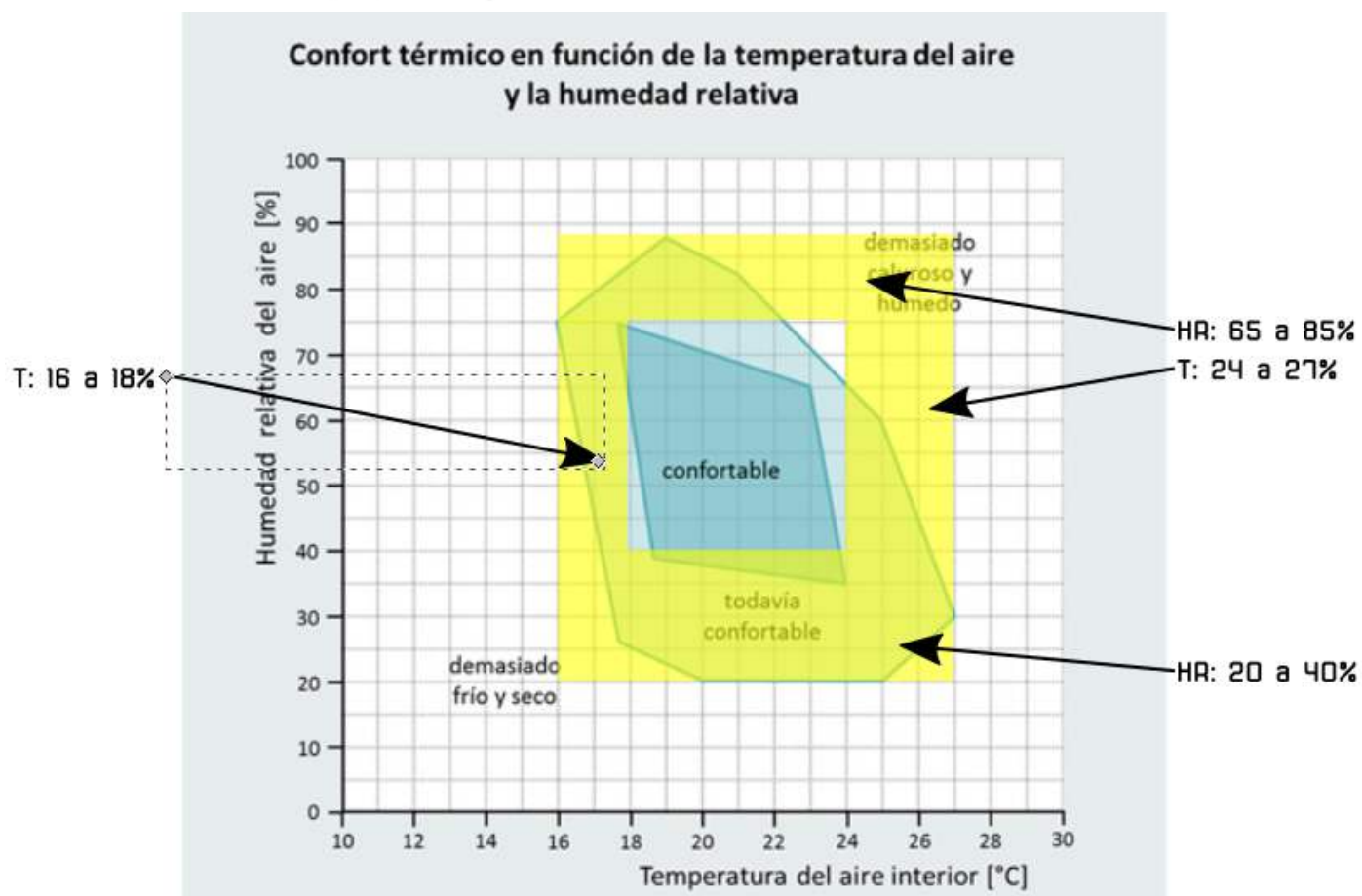
- Humedad Relativa: superior al 85% e inferior al 20%
- Temperatura: superior a 27°C e inferior a 16°C



Delimitación color rojo zona de confort *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

2.- Zona Amarilla: en la imagen siguiente tenemos delimitadas las zonas:

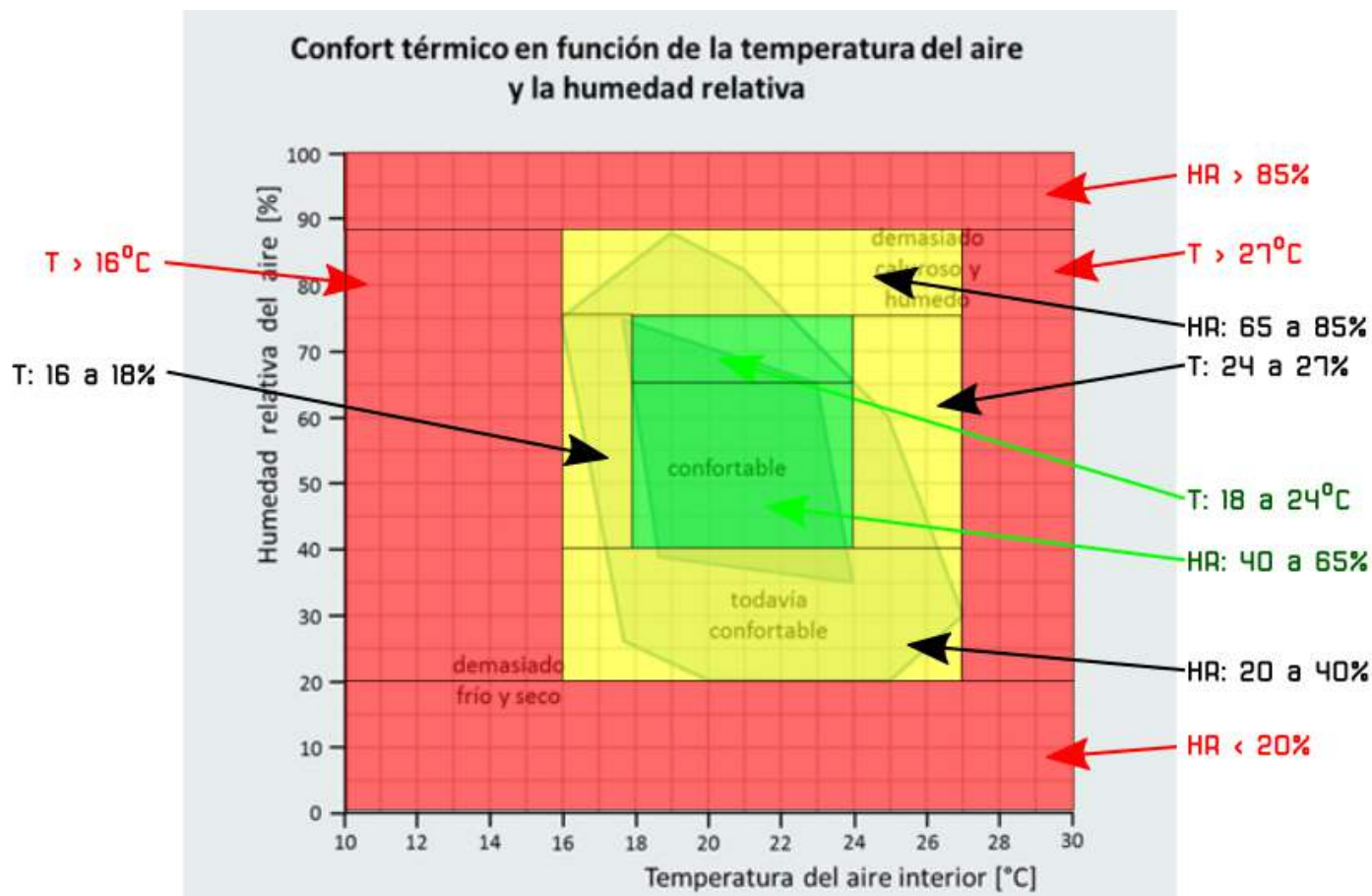
- Humedad Relativa: entre el 20% y el 40% y entre el 65% y el 85%
- Temperatura: entre 16°C y 18°C y entre 24°C y 27°C



Delimitación color amarillo zona de confort *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#)
CC-BY-SA

3.- Zona Verde, rojo y amarillo: en la imagen siguiente tenemos delimitadas todas las zonas, correspondiendo a la verde los siguientes datos:

- Humedad Relativa: entre el 40% y el 65%
- Temperatura: entre 18°C y 24°C



Delimitación colores zona de confort Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Con este [enlace al archivo colores-A10.svg](#) puedes descargarte el archivo vectorial, editarlo con [Inkscape](#) y ver como se han realizado estos gráficos.

En la TdR STEAM

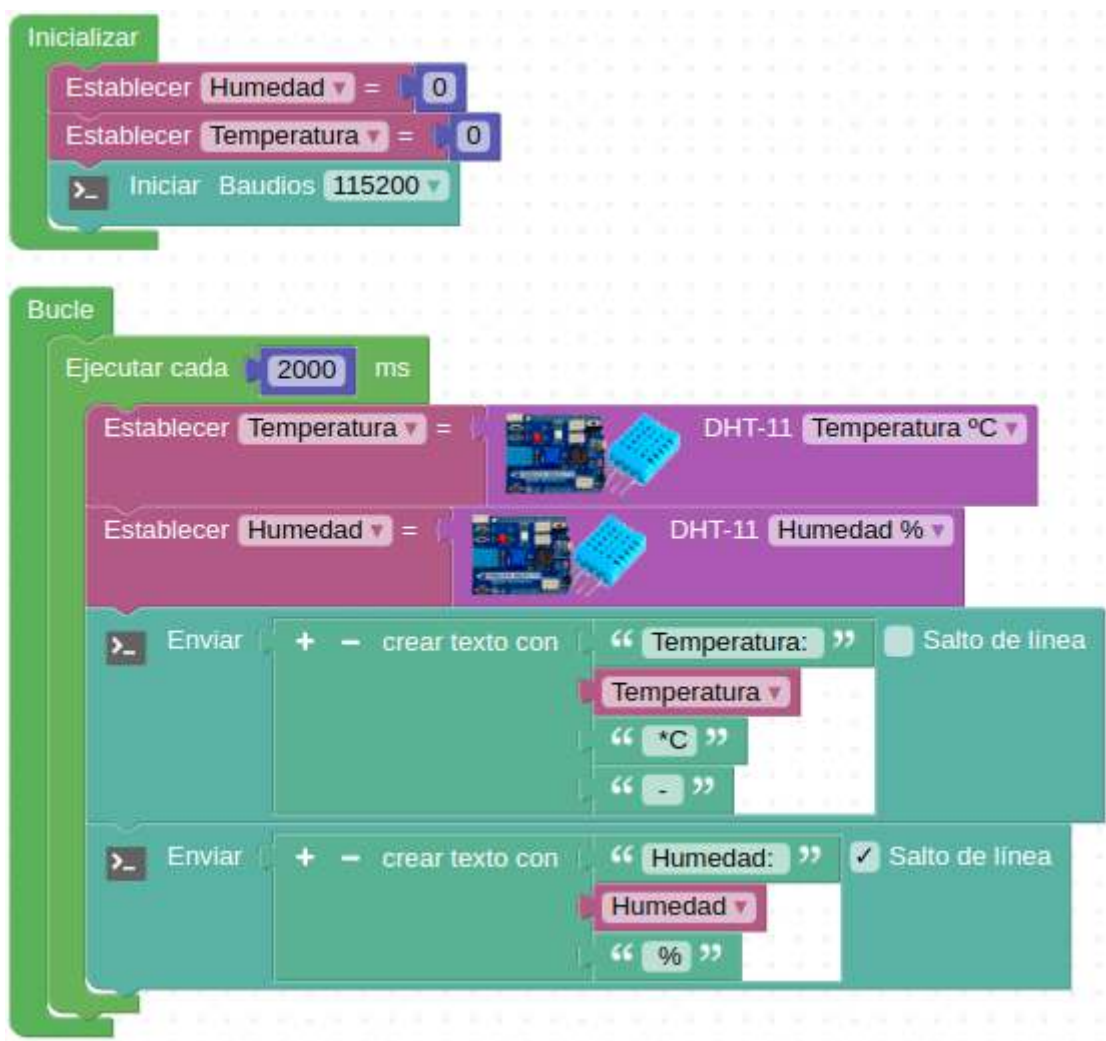


Imagen Federico Coca [Notas sobre](#)

[ESP32 STEAMakers](#) CC-BY-SA

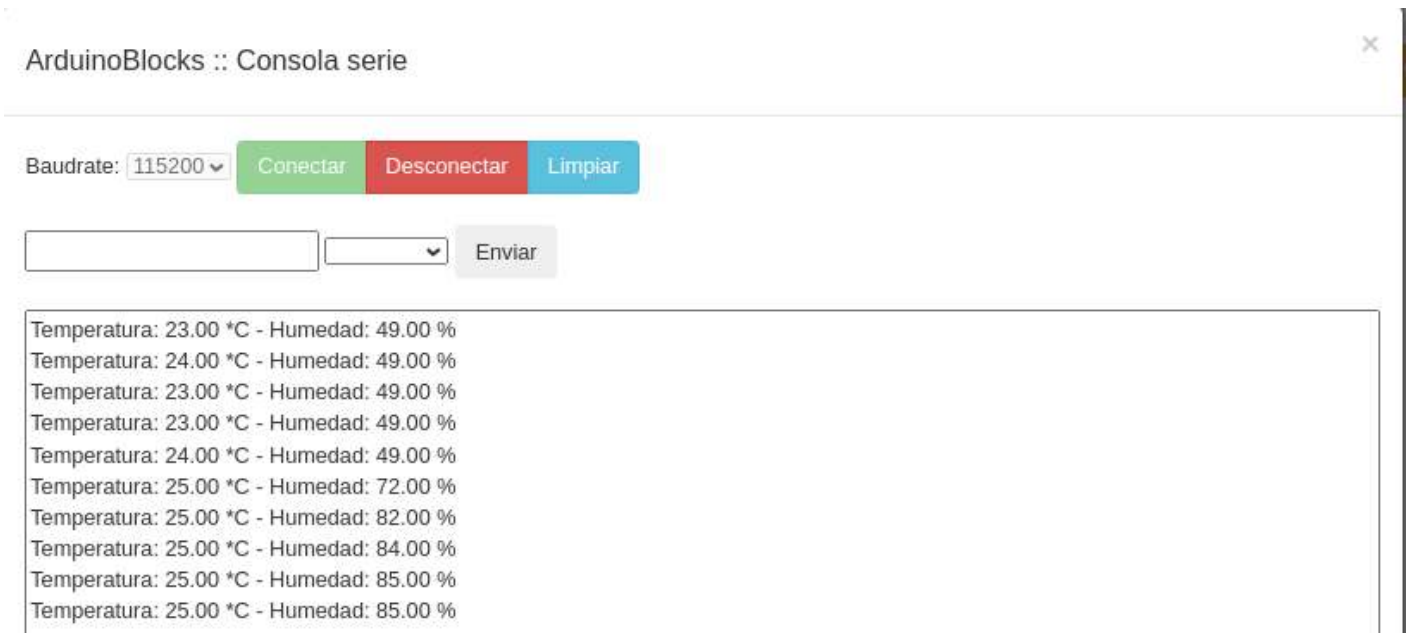
Programando la actividad

De nuevo realizaremos la programación como en las actividades anteriores, utilizando el monitor serie para mostrar los datos. El programa [ESP32-SM-Actividad-10](#) lo vemos en la imagen siguiente:



Actividad-10 Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Esto nos generará algo como lo de la imagen siguiente en la consola:



Consola que produce la actividad 10 *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Retos de ampliación

A10.R1. Realizar un programa que nos muestre el estado de confort según las explicaciones vista y la idea de un semáforo que utilice el LED RGB para componer esos colores rojo, verde y amarillo.

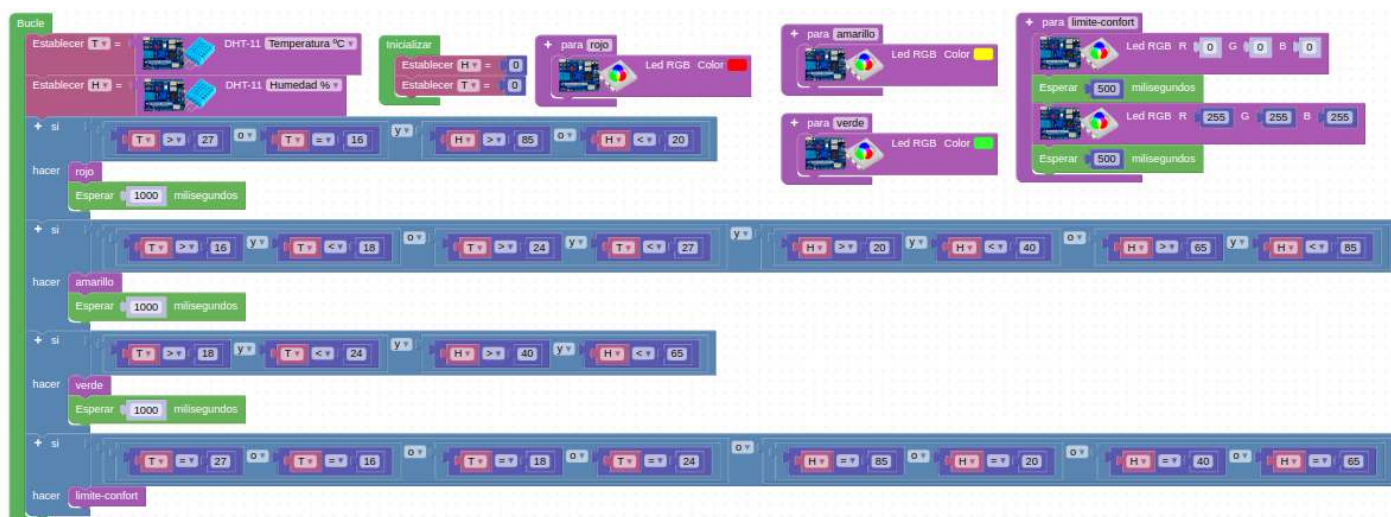
Para resolver la actividad vamos a necesitar varios bloques del menú "Lógica" y especialmente combinando funciones AND y OR múltiples. Si necesitamos, por ejemplo, aumentar el número de operadores AND dentro de un bloque simplemente tenemos que combinarlo como vemos en la imagen siguiente, donde se han combinado cuatro bloques AND.



Combinación de 4 bloques AND *Imagen*

Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

El Programa es el de la imagen siguiente y está disponible en [ESP32-SM-Reto-1-A10](#).



Reto 1 de la actividad 10 *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

A10.R2. Idear un método para probar de forma completa la funcionalidad del programa anterior, aunque se requiera modificarlo. También se pide explicar la misión de la función "limite-confort" creada en el reto 1 de la actividad 10.

Prácticas con sensores analógicos

Actividad-11. Emisor y receptor de infrarrojos

Página extraída de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Enunciado

Enviaremos códigos desde el control remoto por infrarrojos que serán recibidos por el LED de infrarrojos para actuar según el código recibido.

Teoría

¿Qué son los infrarrojos?

Son una clase de radiación electromagnética con una longitud de onda que resulta superior a la longitud de onda de la luz visible, siendo su frecuencia superior a las microondas. Dentro del espectro electromagnético, la radiación infrarroja se encuentra comprendida entre el espectro de luz visible y las microondas. Tiene longitudes de onda mayores o más largas que el rojo. En la imagen siguiente, obtenida del blog de Mercedes González Mas vemos caracterizados los infrarrojos dentro del espectro.

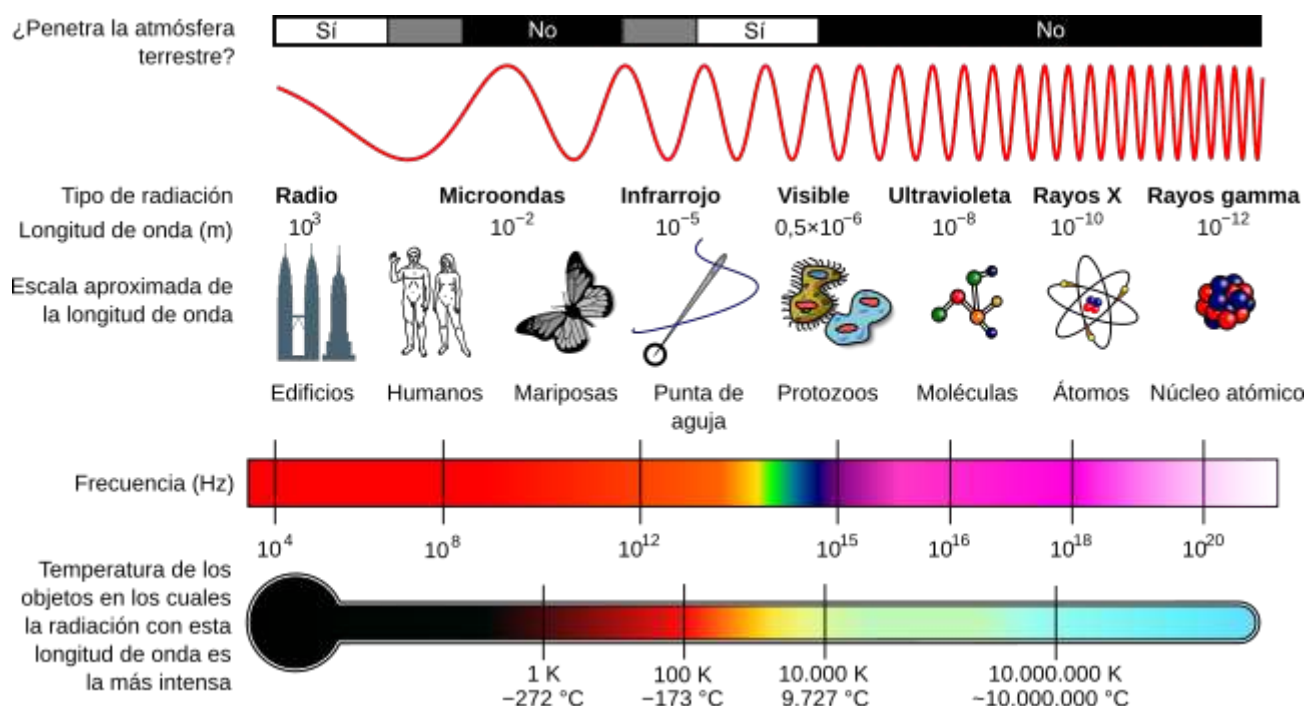


Espectro electromagnético Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Como podemos observar en la imagen, los rayos infrarrojos son clasificados, de acuerdo a su longitud de onda, del siguiente modo:

- infrarrojo cercano, con longitud de onda entre $0.7\ \mu\text{m}$ y $1.1\ \mu\text{m}$, es la parte del espectro infrarrojo que ese encuentra más próximo a la luz visible.
- infrarrojo medio, con longitud de onda entre $1,1\ \mu\text{m}$ y $15\ \mu\text{m}$.
- infrarrojo lejano o región más cercana a las microondas, con longitud de onda entre $15\ \mu\text{m}$ y $100\ \mu\text{m}$

En la imagen siguiente, obtenida de [Wikipedia](#), sobre espectro electromagnético podemos ver más información del tema.



Espectro electromagnetico [Crates. Original version in English by Inductiveload](#), Public domain, via Wikimedia Commons

Todos los cuerpos emiten una cierta cantidad de radiación, y aunque esta resulta invisible para el ojo humano, existen dispositivos electrónicos capaces de "verla" por estar diseñados para ello.

Receptor de infrarrojos

Uno de los receptores más universal utilizado en placas tipo Arduino es el receptor de infrarrojos universal TL1838, VS1838B o simplemente 1835 de 38KHz, que es el que lleva montado la placa

TdR STEAM y cuyo aspecto podemos ver en la imagen siguiente:



Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

En el [datasheet TL1838](#) del dispositivo tenemos toda la información sobre el mismo destacando las siguientes características:

- Voltaje de funcionamiento: 2.7V a 5.5V
- Frecuencia: 37.9KHz
- Ángulo de recepción: 90°
- Rango de recepción: 18m

El dispositivo genera una señal de salida que sirve para controles remotos universales y utiliza la codificación NEC. El receptor de infrarrojos permite codificar los protocolos de señales de pulsos infrarrojos utilizados por los mandos a distancia. Los protocolos detectados son los siguientes: RC5, RC6, NEC, SONY, PANASONIC, JVC, SAMSUNG, WHYNTER, AIWA, LG, SANYO, MITSUBISHI y DENON. Es decir, detectaría cualquier señal emitida por cualquiera de esos mandos.

Emisor de infrarrojos

En nuestro caso como emisor de infrarrojos vamos a utilizar el control remoto de Keyestudio que vemos en la imagen siguiente:



El mini control remoto tiene 17 teclas de función y tiene las siguientes especificaciones:

- Batería: pilas de botón CR2025
- Distancia de transmisión: hasta 8 m
- Ángulo efectivo: 60°

El control remoto, o mando a distancia, por IR funciona emitiendo trenes de pulsos de luz infrarroja. Diferentes señales corresponden a botones diferentes. La señal infrarroja transmite el código correspondiente al botón del mando a distancia pulsado al dispositivo en forma de una serie de impulsos de luz infrarroja. El receptor recibe la serie de impulsos de infrarrojos y los pasa a un procesador que decodifica y activará una determinada función del dispositivo. En el reto y las actividades del mismo obtendremos estos códigos. En ArduinoBlocks se han asignado los siguientes nombres a las teclas:



Nombre teclas control remoto en ArduinoBlocks *Imagen Federico Coca* [Notas sobre ESP32](#)
[STEAMakers](#) CC-BY-SA

TRUCO En las prácticas con Infrarrojos, cuando las cosas no van, es muy útil saber si el problema es que el emisor de infrarrojos funciona o no funciona para descartar otros problemas.

El truco es simple, y sirve **para cualquier mando**: Como las cámaras de los móviles **si** detectan el Infrarrojo no visible, utiliza la cámara de tu móvil hacia el led del emisor del infrarrojos y aprieta cualquier botón, si se enciende el led (visto desde la cámara de tu móvil) el emisor **funciona**.

Bloques en ArduinoBlocks

El sensor receptor de infrarrojos permite obtener la cadena de texto con el código en formato hexadecimal correspondiente al tren de pulsos de IR generado al pulsar una determinada tecla. El bloque es el que vemos en la imagen siguiente:



Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

El valor devuelto por el bloque de recepción será una cadena de texto con valor vacío en caso de no detectar ningún código. Al devolver el bloque una cadena de texto debemos recordar que lo tenemos que almacenar en una variable de tipo texto.

Si utilizamos mandos genéricos RC5 como el modelo de Keyestudio, podemos usar el bloque de la imagen siguiente para comparar los códigos recibidos y así identificar fácilmente cada tecla.



Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

En la TdR STEAM

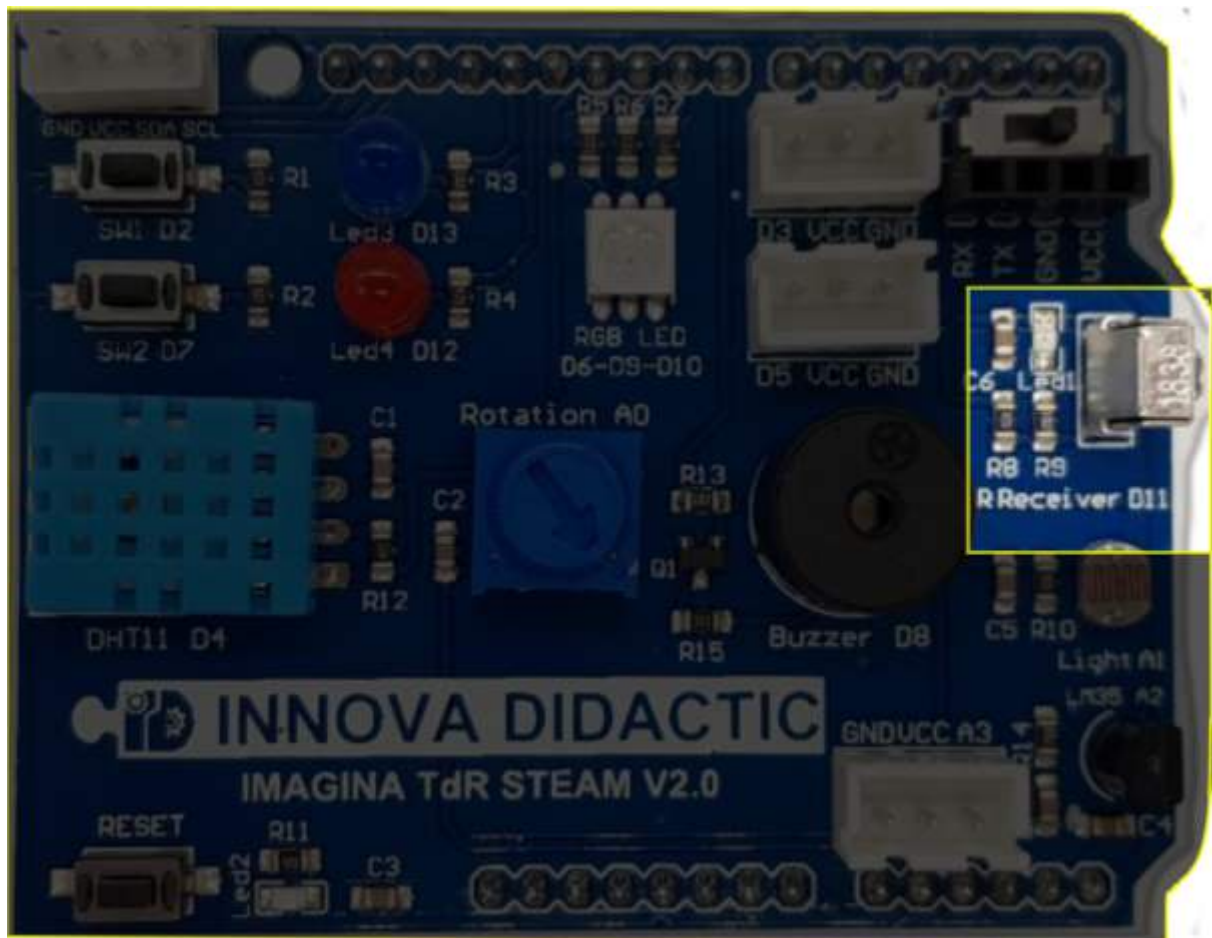
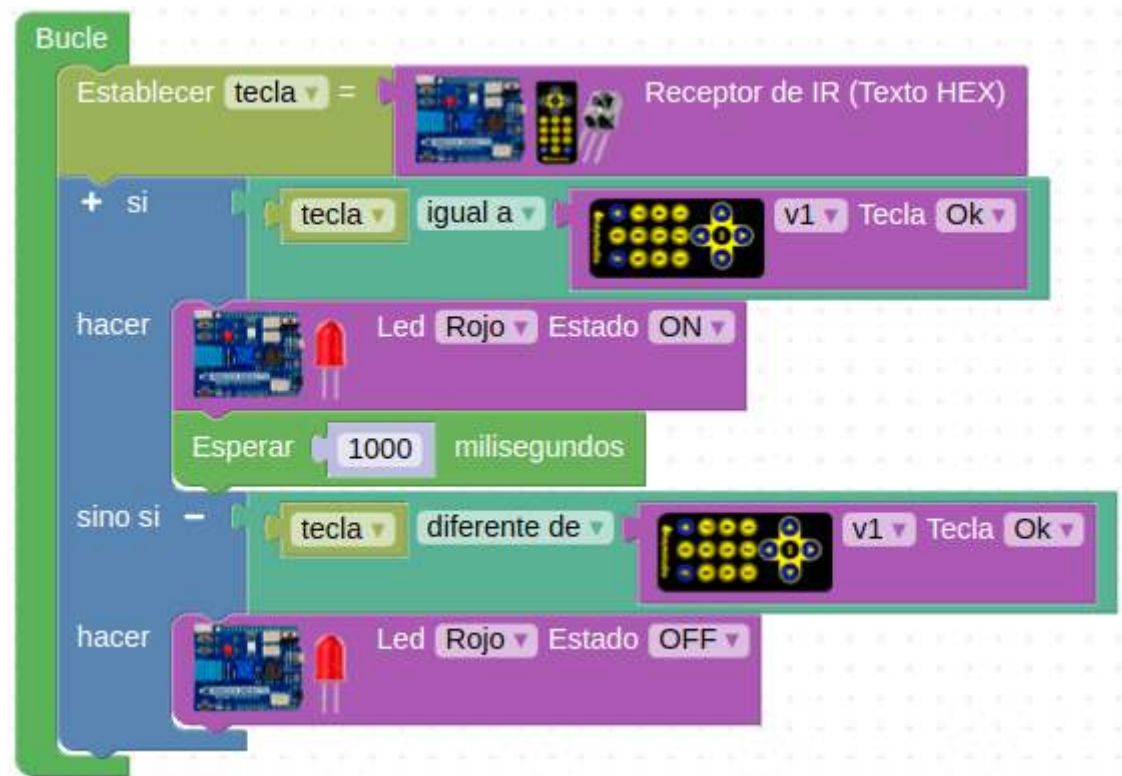


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Programando la actividad

Vamos a realizar un programa en el que si pulsamos la tecla Ok del control remoto se nos encienda el diodo LED rojo. La solución la tenemos disponible en [ESP32-SM-Actividad-11](#) que es el programa que vemos en la imagen siguiente:



Solución Actividad 11 Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Sabemos que el diodo IR está recibiendo un código porque junto al mismo hay un diodo LED rojo que parpadea.

Retos de ampliación

- **A11.R1.** Realizar un programa que encienda el LED RGB en los colores establecidos a continuación y según la tecla flecha pulsada.
 - Flecha arriba = Rojo
 - Flecha izquierda = Verde
 - Flecha derecha = Amarillo
 - Flecha abajo = azul
- **A11.R2.** Realizar un programa que nos muestre por consola el código hexadecimal correspondiente a cada una de las teclas pulsadas junto a un texto descriptivo indicador de la tecla pulsada en cada caso, es decir, que mantenga una estructura del tipo: El CODIGO se corresponde con la tecla TECLA pulsada.

Prácticas con otros elementos del kit

Prácticas con otros elementos del kit

Actividad 12 Puerto de expansión I2C. La LCD de 2x16

Página extraída de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Enunciado

Utilizaremos las comunicaciones I2C para mostrar textos en la pantalla LCD I2C incluida en el kit.

Teoría

Las comunicaciones I2C

Dado que la placa TdR STEAM dispone de un puerto de expansión I2C (del inglés Inter-Integrated Circuit = Circuito inter-integrado) vamos a explicar un poco en que consiste este sistema de conexionado.

Bus conocido por las siglas I2C, IIC o I²C, es un bus serie de datos desarrollado en 1982 por Philips Semiconductors (hoy NXP Semiconductors, parte de Qualcomm). Se utiliza principalmente internamente para la comunicación entre diferentes partes de un circuito, por ejemplo, entre un controlador y circuitos periféricos integrados. Posteriormente fue adoptado progresivamente por otros fabricantes hasta convertirse en un estándar del mercado con miles de circuitos integrados de diferentes fabricantes.

I2C también se denomina TWI (Two Wired Interface) únicamente por motivos de licencia, denominación introducida por Atmel. No obstante, la patente caducó en 2006, por lo que actualmente no hay restricción sobre el uso del término I2C.

El bus I2C requiere únicamente dos cables o líneas de señal para su funcionamiento, uno para la señal de reloj (SCL, Serial Clock) y otro para el envío de datos (SDA, Serial Data). Ambas líneas precisan resistencias de pull-up hacia Vcc. Cualquier dispositivo conectado a estas líneas es de drenador o colector abierto (Open Collector), lo cual en combinación con las resistencias pull-up, crea un circuito Wired-AND. En la imagen siguiente vemos el diagrama básico de conexionado del bus con algunos ejemplos de dispositivos. La señal de reloj siempre es generada por el circuito que actúa como Master.

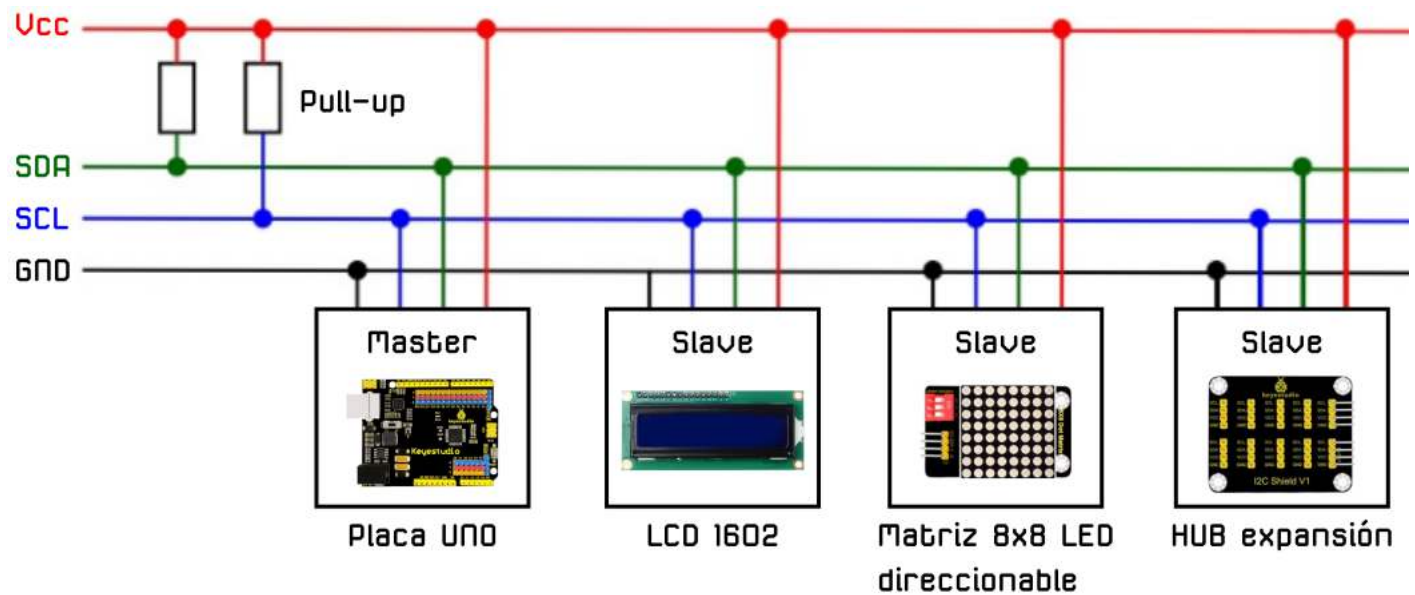


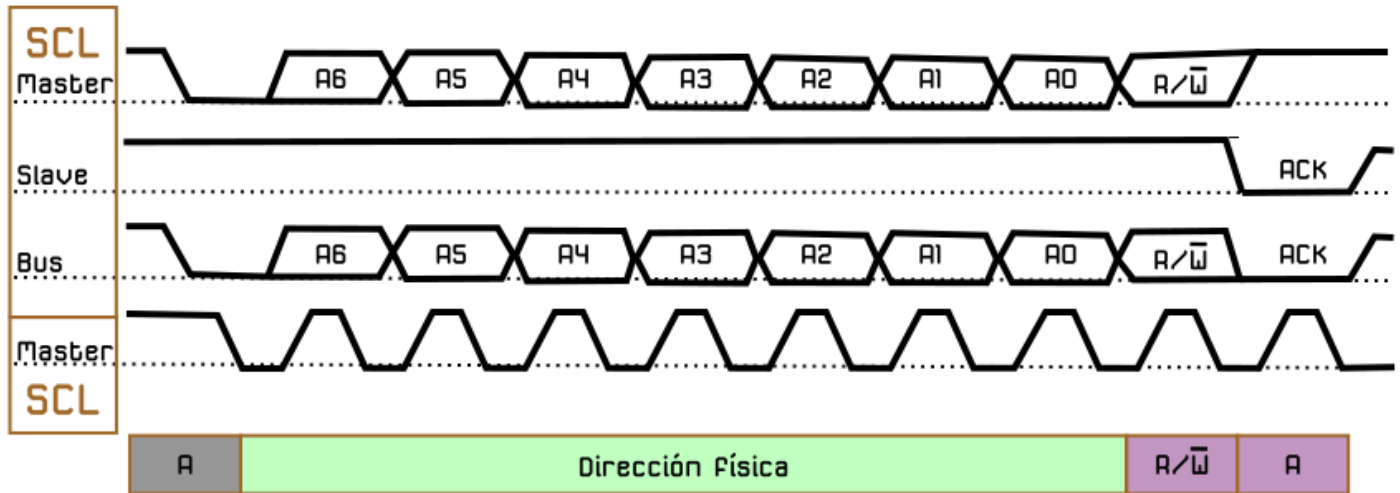
Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Para ser reconocido en el bus, cada dispositivo dispone de una dirección física, que se emplea para acceder a cada uno de ellos de forma individual. Esta dirección puede ser fijada por hardware, en cuyo caso se pueden modificar los últimos 3 bits mediante “jumpers” o microinterruptores (ejemplo de la matriz de 8x8), o por software.

En general, cada dispositivo conectado al bus debe tener una dirección única. Si tenemos varios dispositivos similares tendremos que cambiar la dirección o, en caso de no ser posible, implementar un bus secundario.

El bus I2C tiene una arquitectura de tipo master-slave, lo que indica que el master es el encargado de controlar al resto de dispositivos tipo slave con los que se comunica y que se comunican con el, teniendo siempre el master prioridad absoluta. El dispositivo master es el que inicia la comunicación con los slaves. Los slaves no pueden iniciar la comunicación (el master tiene que preguntarles), ni hablar entre sí directamente.

El bus I2C debe ser por lo tanto síncrono, es decir debe existir una señal de reloj que controle las comunicaciones. Es el master el que proporciona la señal de reloj, que mantiene sincronizados a todos los dispositivos del bus. De esta forma, se elimina la necesidad de que cada dispositivo tenga su propio reloj, de tener que acordar una velocidad de transmisión y mecanismos para mantener la transmisión sincronizada como en UART o SPI. En la imagen vemos un cronograma ejemplo del funcionamiento del sistema.



Cronograma trabajo bus I2C *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

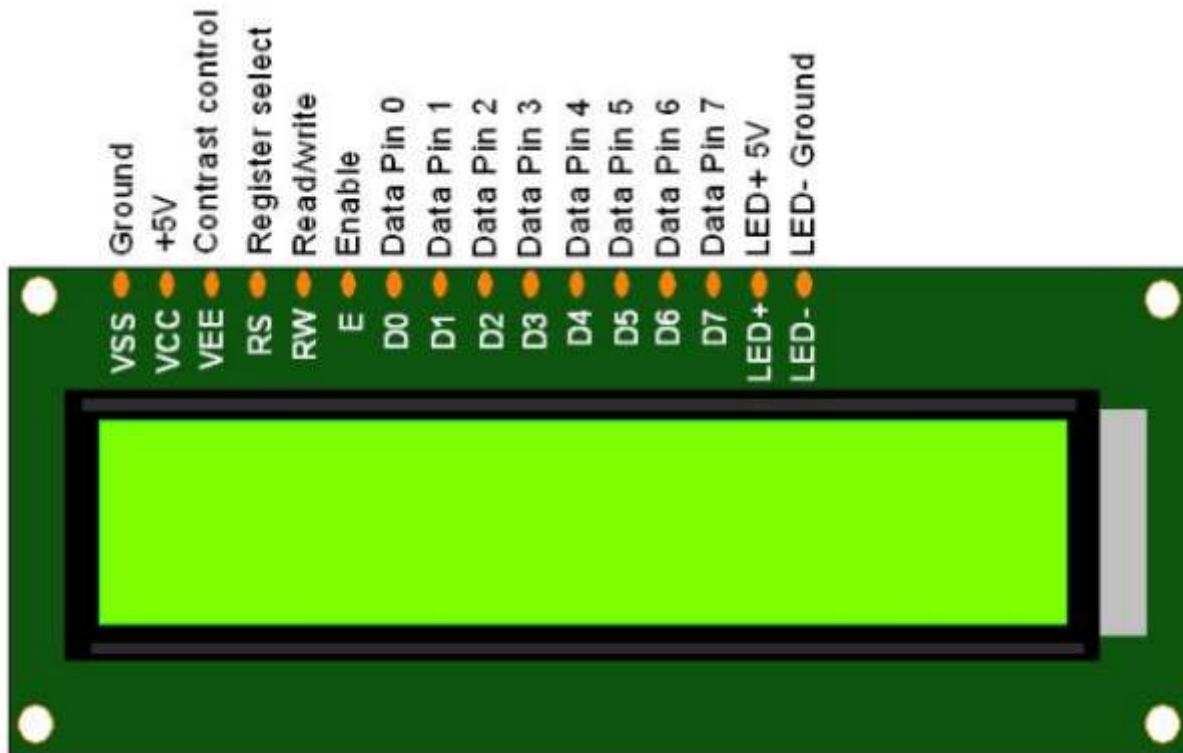
El protocolo de comunicación I2C sigue la siguiente secuencia:

- Primero, el master genera la señal de reloj del bus (SCL).
- Se inicia la comunicación por orden del master al establecer la condición de START, que se produce cuando SDA pasa de uno a cero y se mantiene en cero durante un tiempo.
- El master direcciona a los slaves.
- El master indica si se va a leer o escribir.
- El slave direccionado responde con una señal de conformidad ACK (acknowledge).
- Se transmite los datos byte a byte desde SDA al receptor. Por cada pulso desde SCL se transmite un bit de información.
- El destinatario de la información responde con una señal de conformidad ACK.
- Se acaba la comunicación cuando el master establece la condición de STOP, que se produce cuando SDA, por orden del master pasa de cero a uno y se mantiene en uno durante un tiempo.

Son muchos los dispositivos I2C que se pueden direccionar por este bus I2C, siendo lo más común en los dispositivos para I2C que utilicen direcciones de 7 bits, aunque existen dispositivos de 10 bits, pero es un caso raro. Una dirección de 7 bits implica que se pueden poner hasta 128 (2⁷) dispositivos sobre un bus I2C. Hemos visto que las direcciones son de 8 bits y esto es porque el bit extra de los 7 de la dirección lo emplea el master para informar al slave si va a leer o escribir. Si el bit de lectura/escritura es cero, el dispositivo master está escribiendo en el slave. Si el bit es 1 el master está leyendo desde el slave. La dirección de 7 bit se coloca en los 7 bits más significativos del byte y el bit de lectura/escritura es el bit menos significativo.

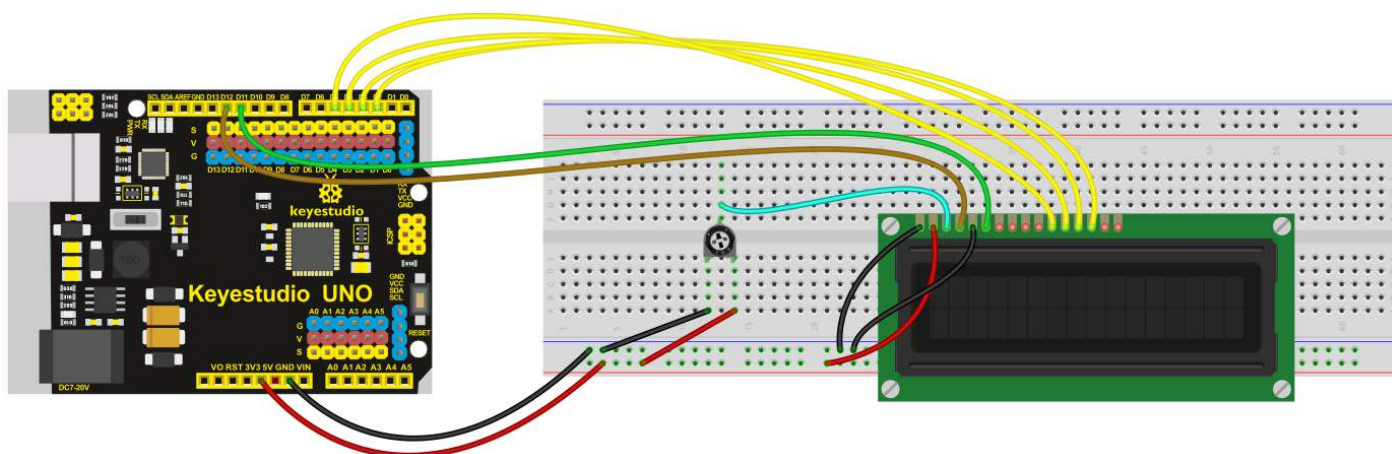
La LCD 1602

Una pantalla LCD (del inglés, Liquid Cristal Display) de 2 líneas de 16 caracteres tiene el aspecto y la distribución de pines que vemos en la imagen siguiente.



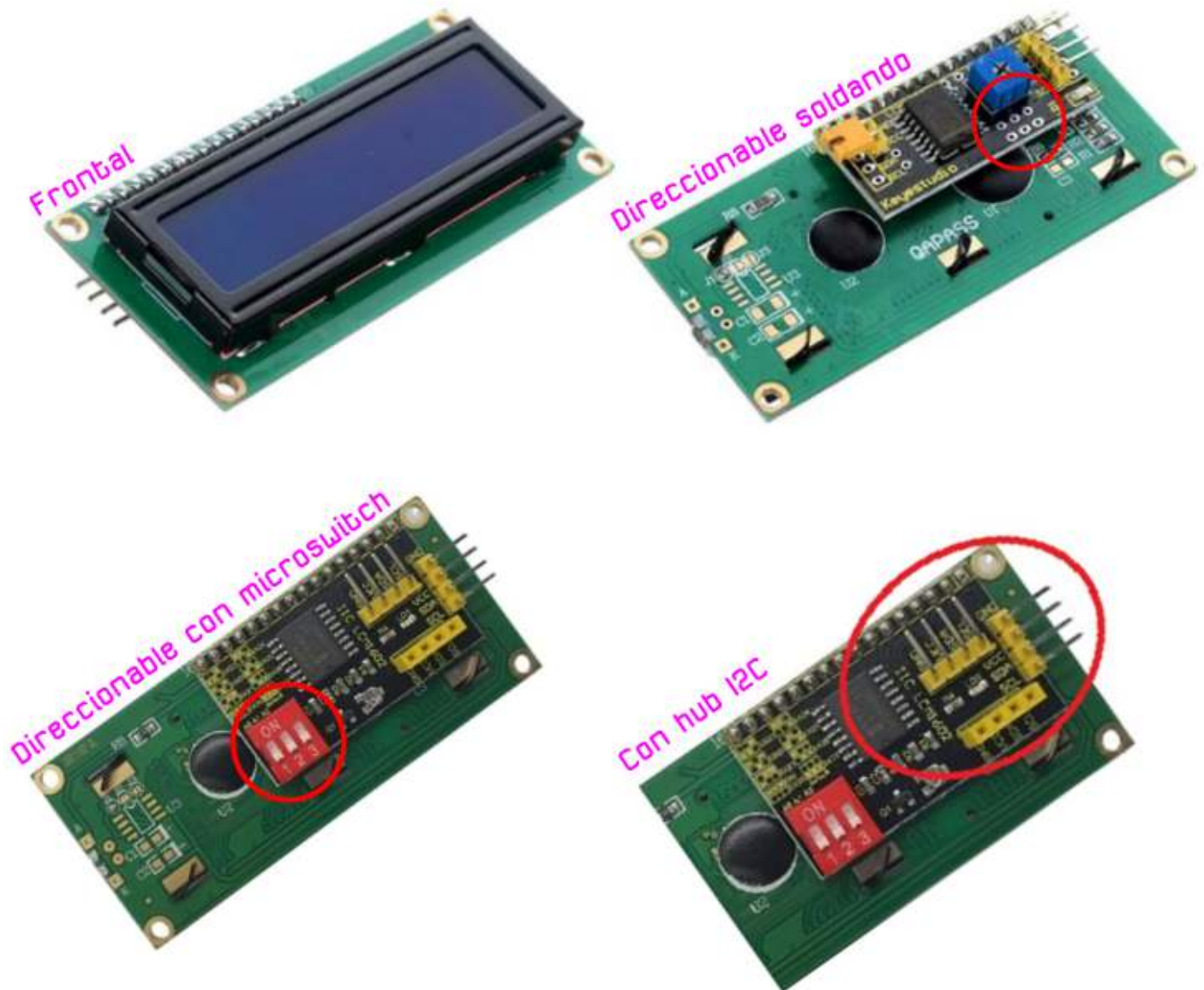
LCD 2x16 Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Es evidente que deberíamos utilizar bastantes patillas de nuestra placa tipo UNO (es el formato que tiene la ESP32 STEAMakers) para su control. En la imagen siguiente se muestra el conexionado mínimo necesario en una pantalla de este tipo: 4 bits para datos y dos señales de control En (Enable) y Rs (Register select). La conexión RW la ponemos a GND. Además se debe añadir una resistencia ajustable o un potenciómetro para regular el contraste de la pantalla.



Conexionado mínimo LCD 2x16 Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Es preferible utilizar alguna de las que tienen el aspecto que vemos en la imagen siguiente:



LCD 2x16 con módulo I2C *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

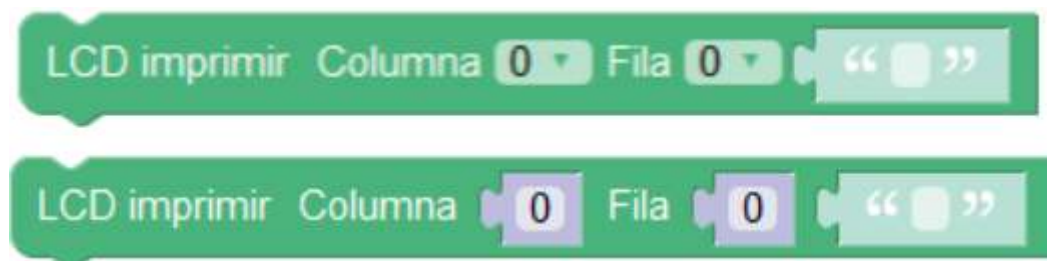
En realidad el conjunto que vemos en la imagen anterior no es mas que una LCD 2x16 a la que se ha incorporado un módulo adaptador de LCD a I2C. Este modulo está especialmente diseñado para poder soldarlo directamente a la LCD y convertirla así en un dispositivo I2C que además ya lleva incorporado el potenciómetro de ajuste de contraste en alguno de los casos.

Este tipo de pantalla requiere cuatro cables para establecer las conexiones, dos cables SDA (datos) y SCL (reloj) para el bus de comunicaciones I2C y los dos cables de alimentación VCC y GND.

La dirección I2C por defecto de este tipo de módulos puede ser 0x3F o en otros casos 0x27, e incluso hay modelos en los que se puede cambiar. Para un correcto control es muy importante identificar correctamente la dirección I2C de nuestro modulo pues de otra forma nuestro programa

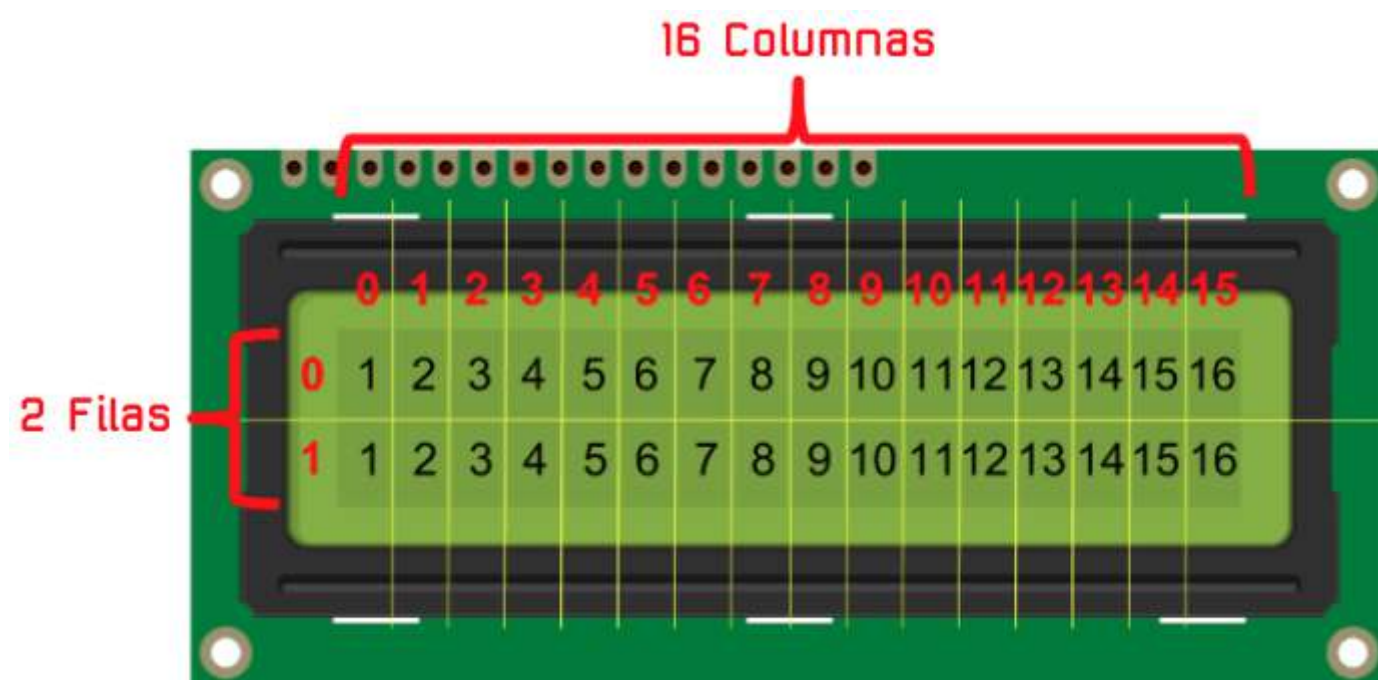
no funcionará correctamente.

Una LCD 1602 I2C es muy sencilla de controlar a partir de los bloques que nos proporciona ArduinoBlocks. En la imagen siguiente ponemos como ejemplo los bloques para imprimir un texto o variable en un par fila-columna determinado.



Bloques para imprimir en un par fila-columna *Imagen Federico Coca* [Notas sobre ESP32](#)
[STEAMakers](#) CC-BY-SA

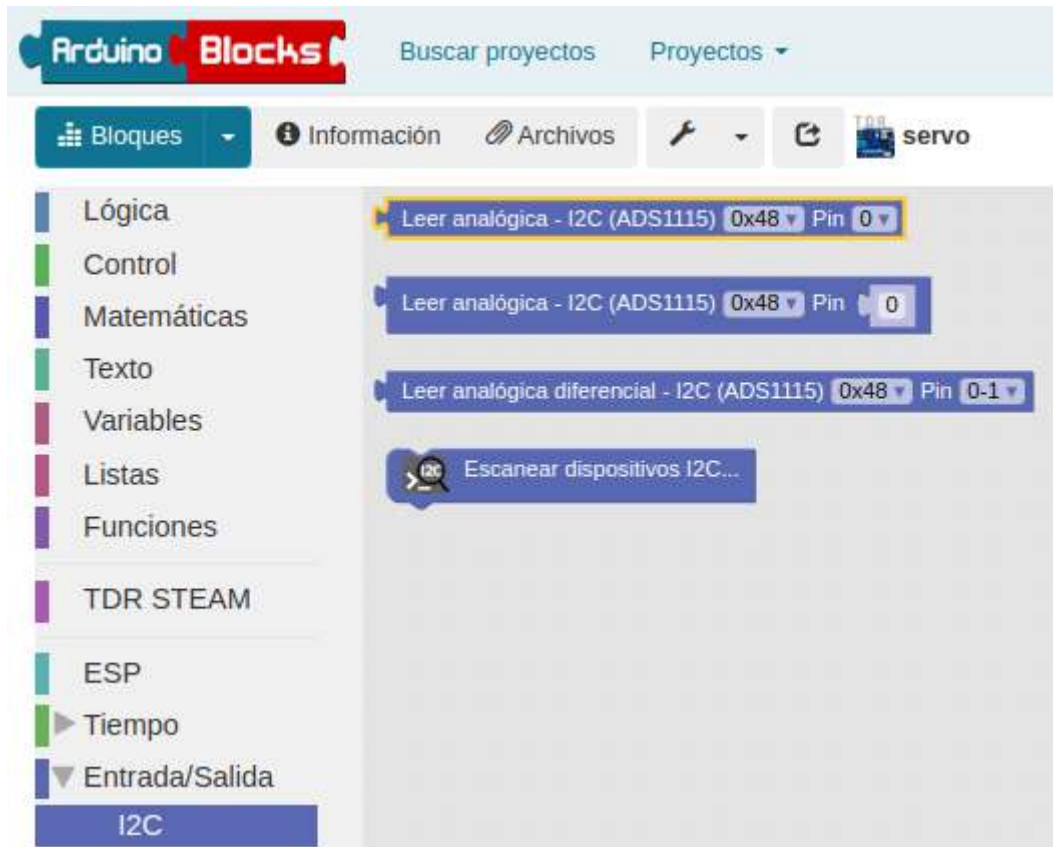
En la siguiente figura se muestra el sistema de distribución de filas y columnas.



Sistema de coordenadas en una LCD 1602 *Imagen Federico Coca* [Notas sobre ESP32](#) [STEAMakers](#)
 CC-BY-SA

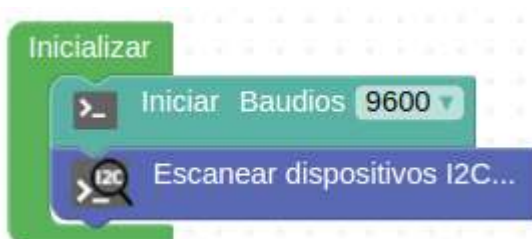
Escanear dispositivos I2C

Si no conocemos la dirección específica de nuestro módulo podemos utilizar un pequeño programa que llamaremos [Escaner-I2C](#) y que se encargará de identificar la dirección I2C y todos los dispositivos I2C conectados a nuestra placa. Debemos crear un proyecto, en esta ocasión, de tipo "Arduino UNO" para tener disponible el menú I2C que nos de acceso al bloque "Escanear dispositivos I2C..." tal y como vemos en la imagen siguiente:



Menu I2C Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Un sencillo programa como el de la imagen siguiente nos permite ver los dispositivos I2C conectados y su dirección física asociada. El programa está disponible como [Escaner-I2C](#).



Programa Escaner-I2C Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Si conectamos la consola el resultado de tener la placa TdR STEAM con una LCD I2C conectada debe ser similar al de la imagen siguiente:

ArduinoBlocks :: Consola serie

Baudrate: 9600

I2C device: 0x27  **Dirección física**

I2C devices found: 1

Consola para el programa Escaner-I2C *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

IMPORTANTE En el caso de la placa ESP32 STEAMakers no ocurre como en las tipo Arduino, que cuando conectamos la consola se efectúa un reset automático de la placa por lo que es muy posible que el programa de escaneo nos muestre una consola sin resultado alguno o en blanco. Simplemente, con la consola ya conectada, **hacemos un reset físico en la placa** y el problema quedará solucionado.

En la figura siguiente tenemos el resultado que se muestra con el programa anterior cargado, conectar la consola y hacer reset. En este caso vemos los cinco dispositivos conectados a través de un hub y la placa TdR STEAM.



Consola para el programa Escaner-I2C con 5 dispositivos *Imagen Federico Coca* [Notas sobre ESP32](#)
[STEAMakers](#) CC-BY-SA

Otra opción para hacer un escáner I2C que no necesita el reset es el programa que vemos en la figura siguiente y que lleva al bucle la orden de escaneo que se va a ejecutar cada 10 segundos, aunque este tiempo puede ser otro cualquiera.



Programa Escaner-I2C en el bucle con 5 dispositivos *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

En la figura siguiente vemos el resultado en la consola y como coincide con el anterior.



Consola para el programa Escaner-I2C en el bucle con 5 dispositivos

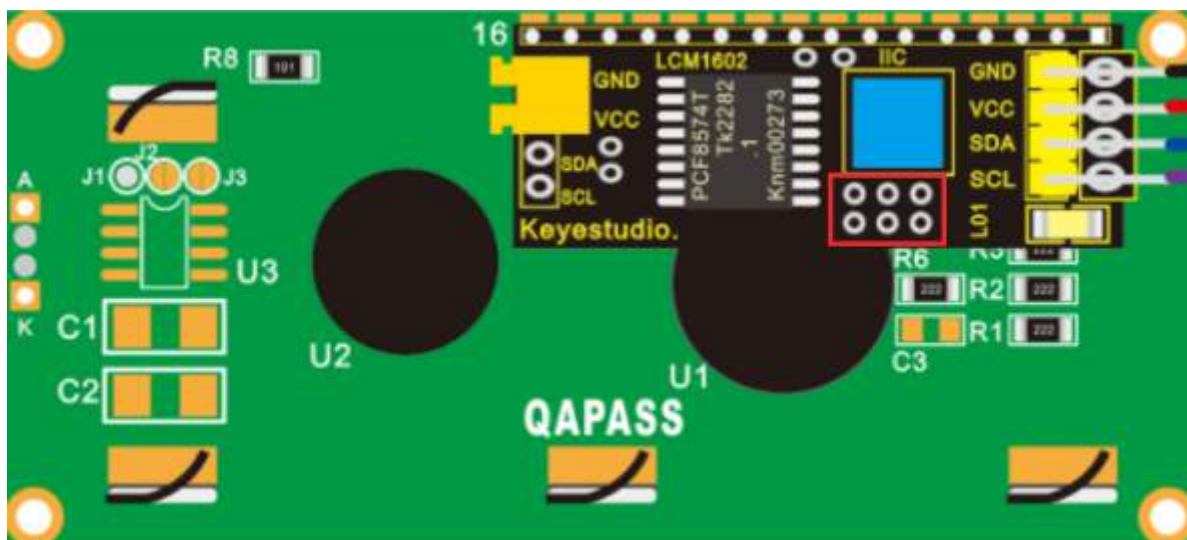
Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Cambiar la dirección física del dispositivo I2C

Esta tarea nos va a resultar especialmente útil si disponemos de LCDs I2C con una dirección fijada de fábrica y queremos conectar varias de ellas en nuestro proyecto.

LCD SIN micro interruptores

La parte posterior de la LCD 1602 de Keystudio tiene el aspecto de la imagen siguiente. Nos fijaremos especialmente en los tres grupos de agujeros enmarcados en rojo. Aunque en este caso no vienen nombrados los vamos a denominar A0, A1 y A2 de izquierda a derecha.

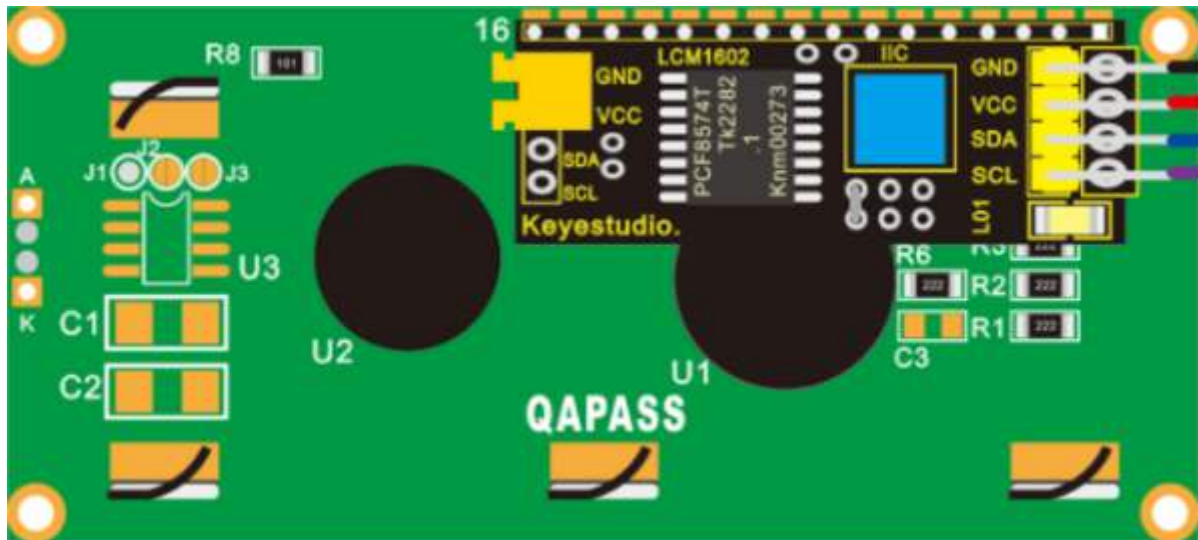


Pads configuración dirección física en la parte posterior LCD I2C Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

La dirección por defecto de fábrica en este caso es la 0x27, pero se puede cambiar alterando la situación de conexionado de estos agujeros entre si según la tabla siguiente:

A2	A1	A0	Dirección
0	0	0	0x27
0	0	1	0x26
0	1	0	0x25
0	1	1	0x24
1	0	0	0x23
1	0	1	0x22
1	1	0	0x21
1	1	1	0x20

Para establecer los unos de la tabla anterior basta con cortocircuitar los dos pads correspondientes. En la imagen siguiente se ha establecido la dirección física como 0x26.



Dirección 0x26 Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

LCD CON micro interruptores

La parte posterior de la LCD 1602 de Keyestudio con micro interruptores para cambiar su dirección física tiene el aspecto de la imagen siguiente. Nos fijaremos especialmente en el recuadro azul donde están los microinterruptores que permiten cambiar el estado del bit y encima de los mismo está la información de la dirección física que se asigna a cada combinación. La tabla es exactamente la misma que en el caso anterior.



Configuración dirección física en la parte posterior LCD I2C con microinterruptores

Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

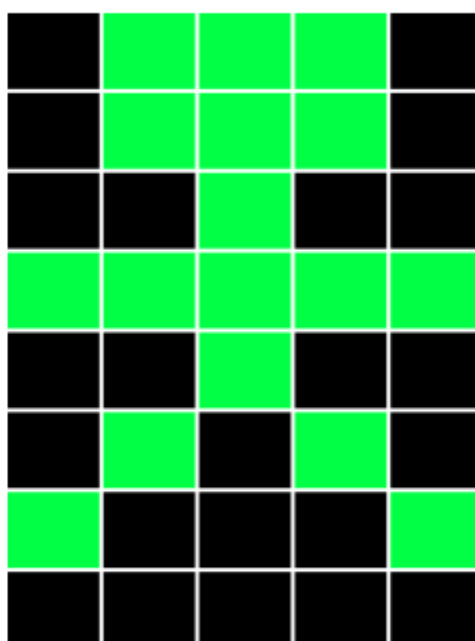
Definición de símbolos en la LCD

Dentro de los bloques del menú Visualización -> Pantalla LCD está el de "definir símbolo", que permite definir uno de los 8 símbolos personalizables que puede almacenar el micro de la pantalla LCD. El símbolo se define por un mapa de bits (unos y ceros indicando cada píxel del símbolo). Los

símbolos tienen una resolución de 5x8 píxeles (blanco o negro).

En ArduinoBlocks disponemos de una herramienta que nos ayuda a definir nuestros propios símbolos y podemos acceder a ella desde herramientas o haciendo clic derecho sobre el bloque, desplegándose en cualquier caso un editor muy sencillo de usar y que vemos con un ejemplo en la imagen en la siguiente:

LCD -Symbol editor



Clear

Fill

Copy data:

B01110,B01110,B00100,B11111,B00100,B01010,B10001,B00000

Ejemplo de símbolo creado con el editor *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

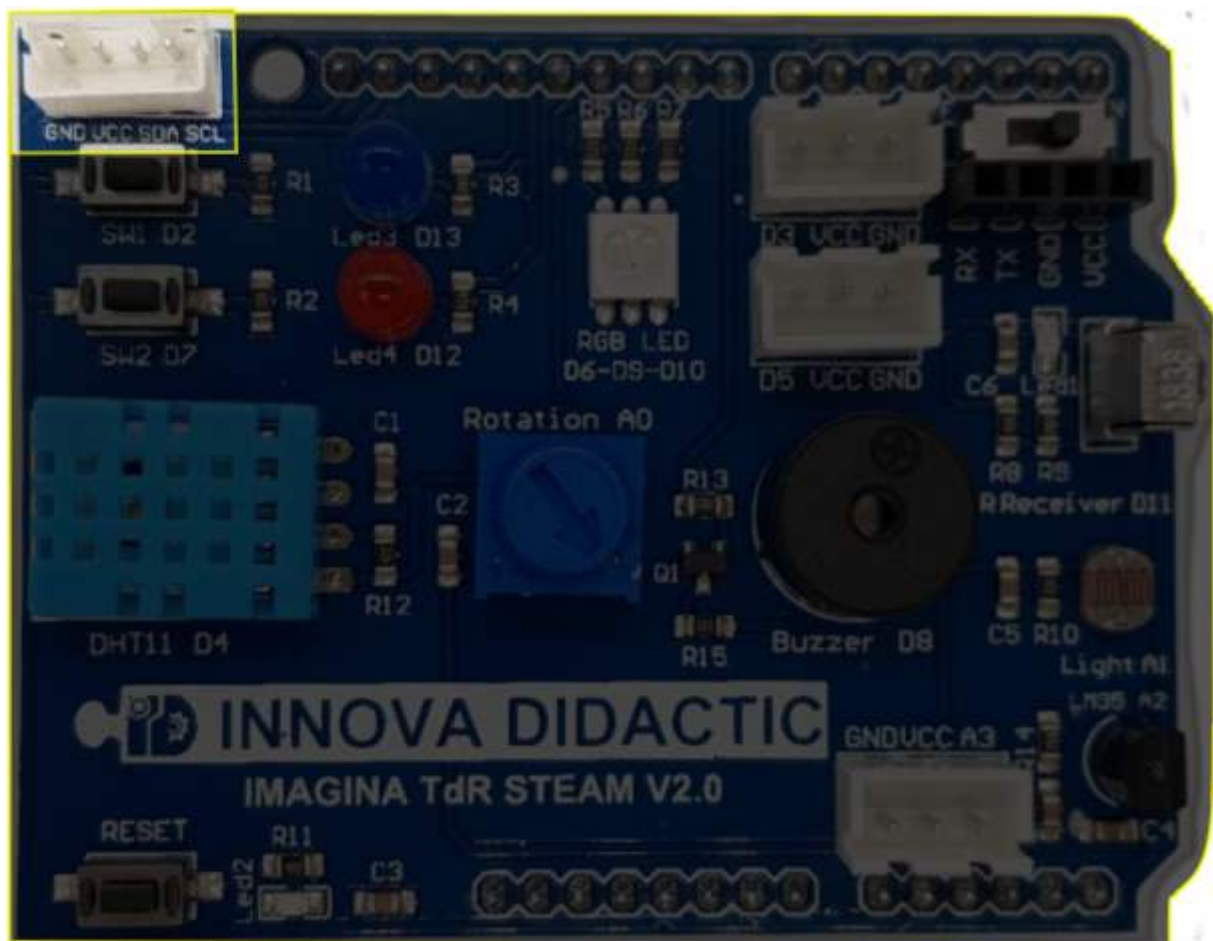
Para tener el símbolo disponible simplemente copiamos la cadena generada en el lugar correspondiente del bloque.

En la TdR STEAM

En la placa TdR STEAM tenemos el conector I2C con los mismo pines de conexión que la LCD y debemos respetar el orden de conexionado indicado en la siguiente tabla:

TdR STEAM	LCD 1602	Color cable
GND	GND	Negro
Vcc	Vcc	Rojo
SDA	SDA	Amarillo
SCL	SCL	Blanco

La ubicación del conector I2C en la TdR STEAM lo vemos en la imagen siguiente:



Conector I2C en la TdR STEAM Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Recordemos que en el caso de la LCD con micro interruptores o LCD direccionable dispone de un pequeño hub I2C que permite conectar dos dispositivos I2C mas. El término direccionable no me parece una denominación muy correcta dado que el otro tipo también es direccionable, aunque tengamos que soldar.

Programando la actividad



Como primer ejemplo de uso vamos a mostrar una determinada información en la pantalla LCD, en concreto vamos a mostrar una serie de seis textos en ambas filas que se irán alternando cada tres segundos en la pantalla. El programa de la imagen siguiente lo tenemos disponible en [ESP32-SM-Actividad-inicial](#).



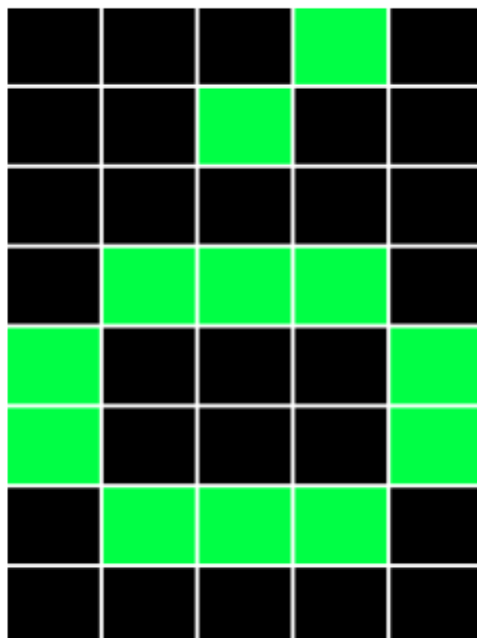
Actividad inicial *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Retos de ampliación

LCD.R1. Corregir el error ortográfico en la palabra "Robotica" de la actividad anterior. Para ello tenemos que definir nuestro propio símbolo de "ó" (o acentuada).

Definimos el símbolo requerido tal y como vemos en la imagen siguiente:

LCD -Symbol editor



Clear

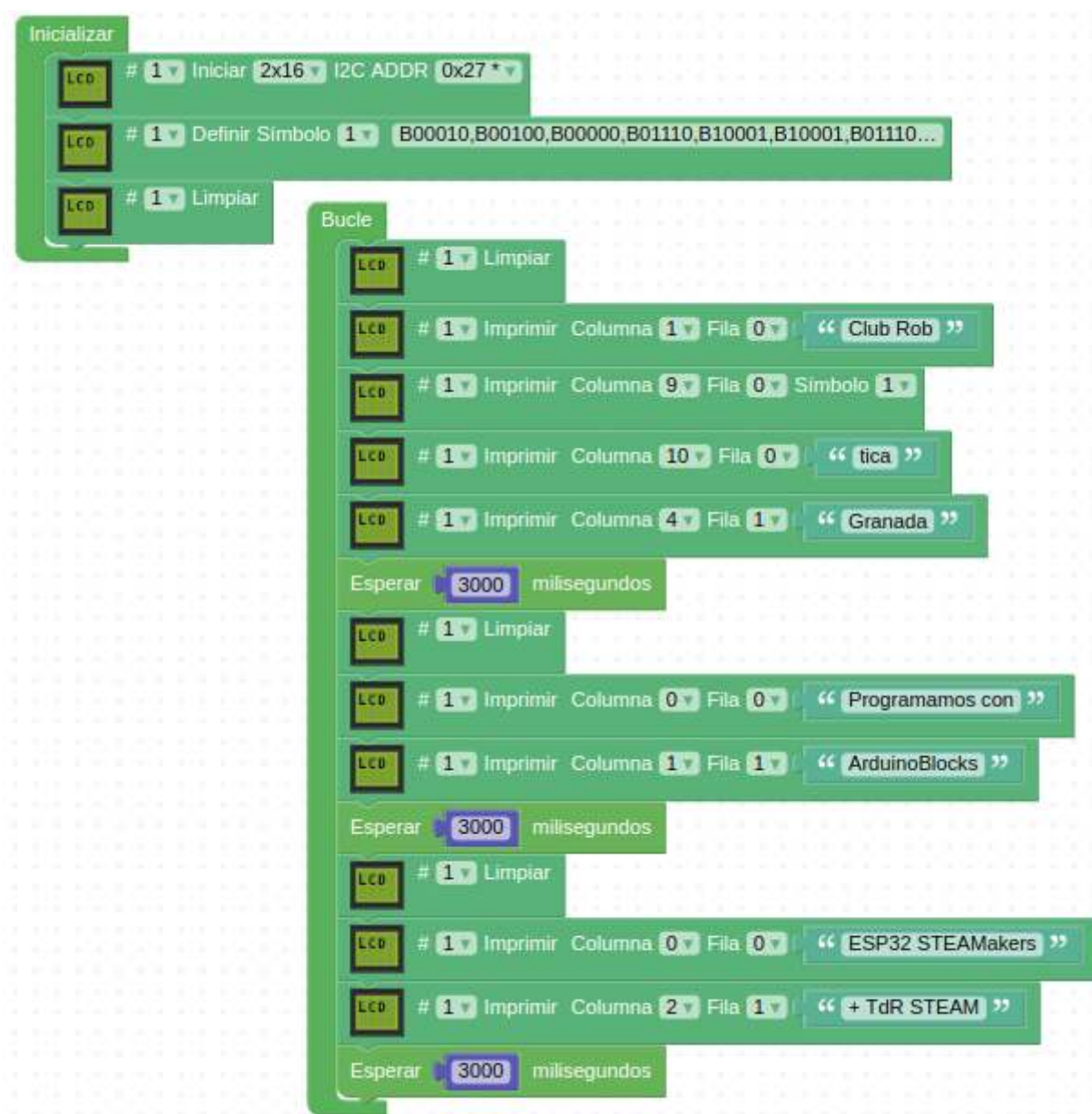
Fill

Copy data:

B00010,B00100,B00000,B01110,B10001,B10001,B01110,B00000

Símbolo 'o' acentuada *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

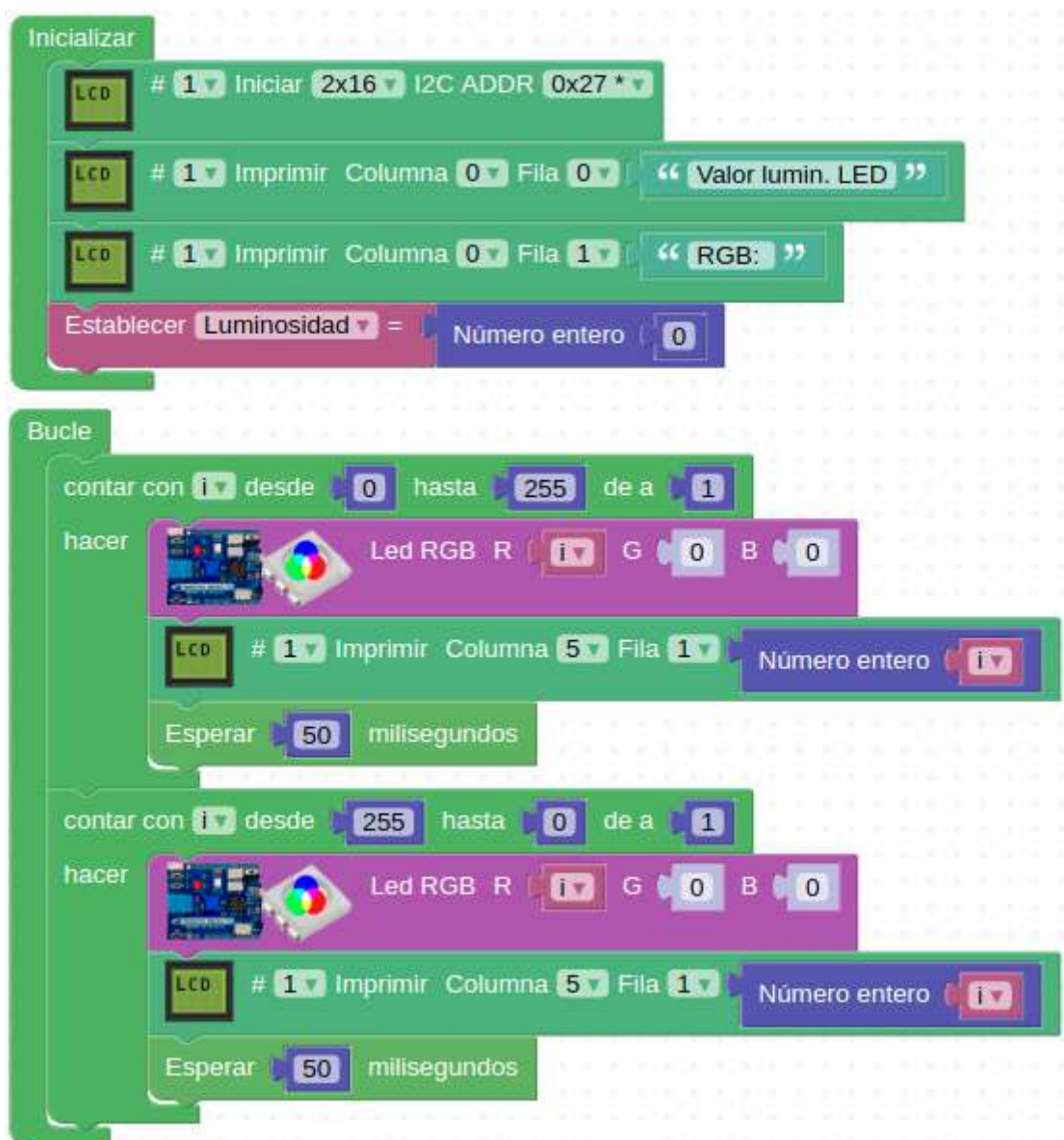
El programa resuelto es el de la figura siguiente:



Solución al reto 1 de la actividad LCD *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

LCD.R2. Realizar un control de iluminación del LED RGB mediante PWM y mostrar por pantalla el valor de la iluminación en cada instante. Se realizará el control desde el mínimo de iluminación a máximo y cuando se alcance el máximo bajamos otra vez a mínimo.

El programa resuelto es el de la figura siguiente:



Solución al reto 2 de la actividad LCD *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Se ha resuelto con un contador ascendente y otro descendente sobre el LED rojo. Se pueden probar diferentes opciones de LEDs o incluso combinación de ellas. También se anima al lector a comprobar que ocurre si quitamos la opción "Número entero" y ponemos directamente la variable del bucle y buscar la explicación.

Prácticas con otros elementos del kit

Actividad 13 Sensor de sonido o micrófono

Actividad 13 Visualización del nivel sonoro

Vamos a hacer este programa:



[vernivelsonoro.abp](#)

<https://www.youtube.com/embed/u3ipwFAjf9Y>

Página extraída de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Actividad 13 bis Enunciado

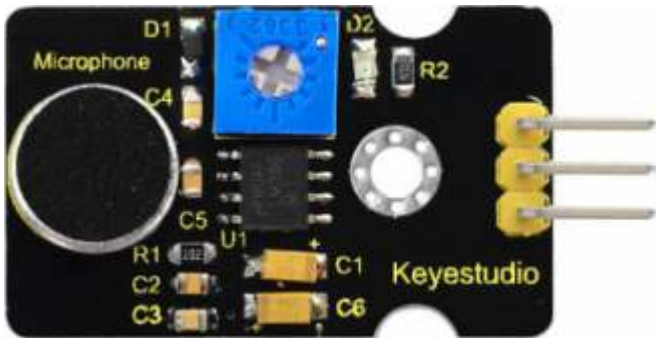
Realizaremos pruebas básicas de detección de nivel de sonido añadiendo esta opción externa al pin analógico A3 disponible en el conector correspondiente.

Teoría

Un micrófono es un transductor (dispositivo que convierte energía de una forma a otra) que convierte la energía sonora en señales eléctricas. Micrófonos hay una amplia diversidad tanto en formas como tamaños. Dependiendo de la aplicación, un micrófono puede utilizar diferentes tecnologías para convertir sonidos en señales eléctricas.

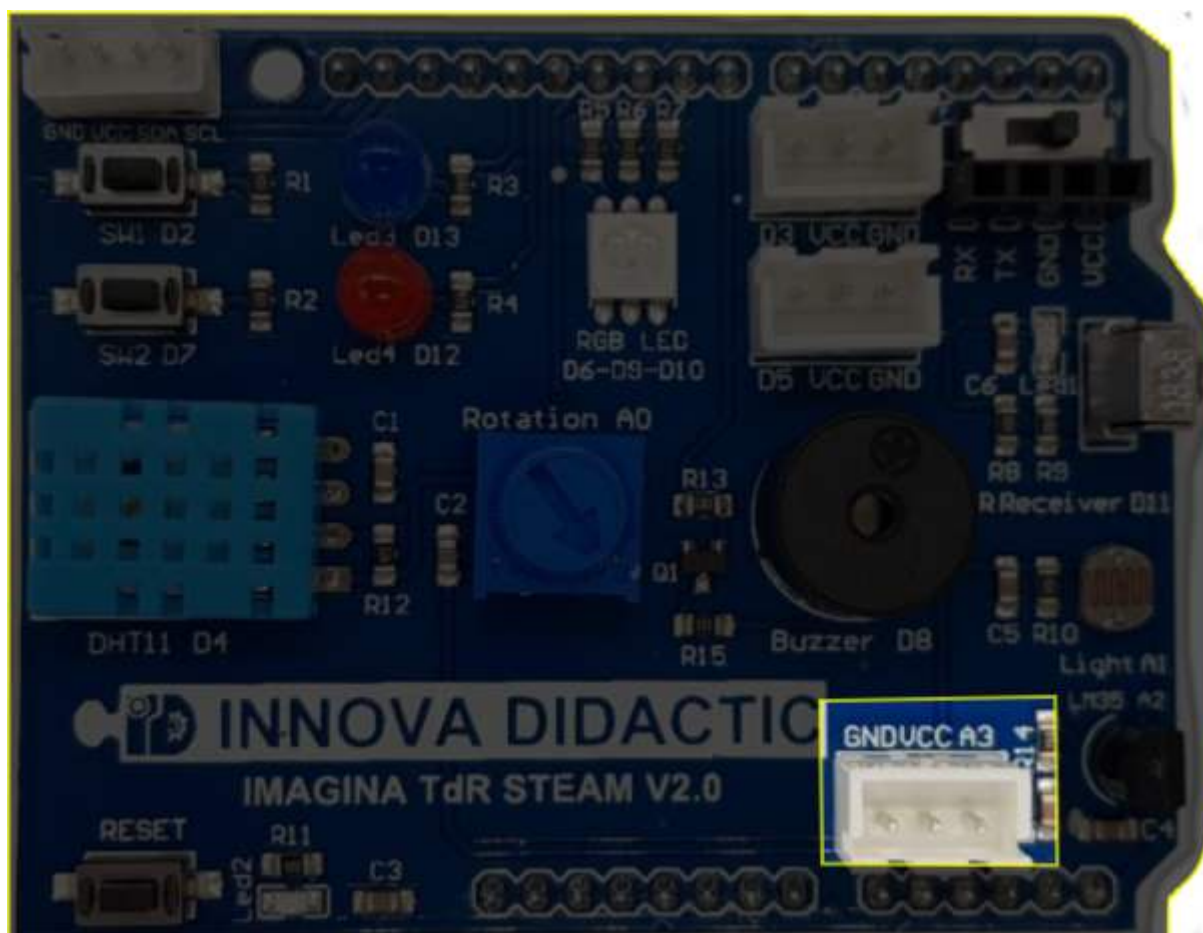
Para el caso de aplicaciones con placas tipo Arduino suelen usarse sensores basados en el micrófono de condensador electret que es un condensador de placas paralelas y trabaja como una capacitancia variable. Se forma con una placa fija (placa trasera) y una movable (diafragma) con una pequeña separación entre ellas. Cuando el sonido golpea al diafragma este se mueve cambiando así la capacitancia entre las placas.

El sensor microfónico de keyestudio se utiliza normalmente para detectar el nivel de ruido en el ambiente. El pin 5 del mismo es una salida analógica, es decir, una tensión de salida en tiempo real del micrófono. El sensor viene con un potenciómetro que permite ajustar la ganancia de la señal dentro de un determinado rango. Su aspecto es el de la imagen siguiente:



Sensor de sonido con micrófono KS0035 con potenciómetro *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

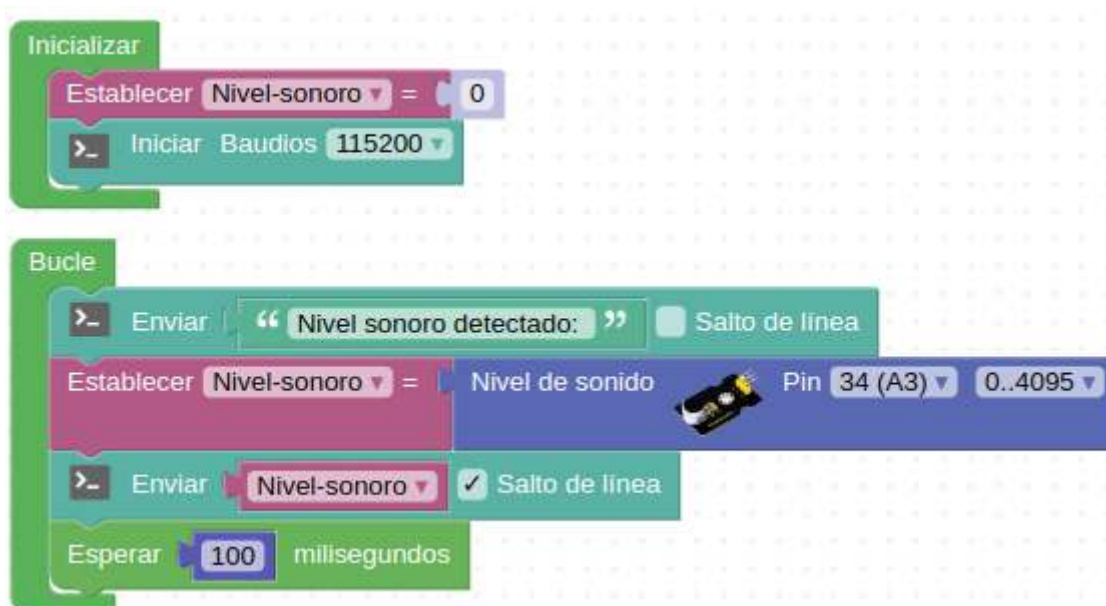
En la TdR STEAM



El conector micrófono externo en A3 Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Programando la actividad

Como programa sencillo inicial vamos simplemente a enviar a la consola el nivel de sonido que detecte nuestro micrófono. El programa de la imagen siguiente esta disponible como [ESP32-SM-micro-inicial](#).



Actividad inicial con el micro *Imagen Federico Coca* [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Retos de ampliación

Micro.R1. Modificar el programa de la actividad inicial para que muestre los datos por la LCD.

Micro.R2. Partiendo del programa del reto 2 de la actividad LCD (LCD.R2) modificarlo para que muestre los datos del micrófono tanto al final de la cuenta ascendente como de la descendente.

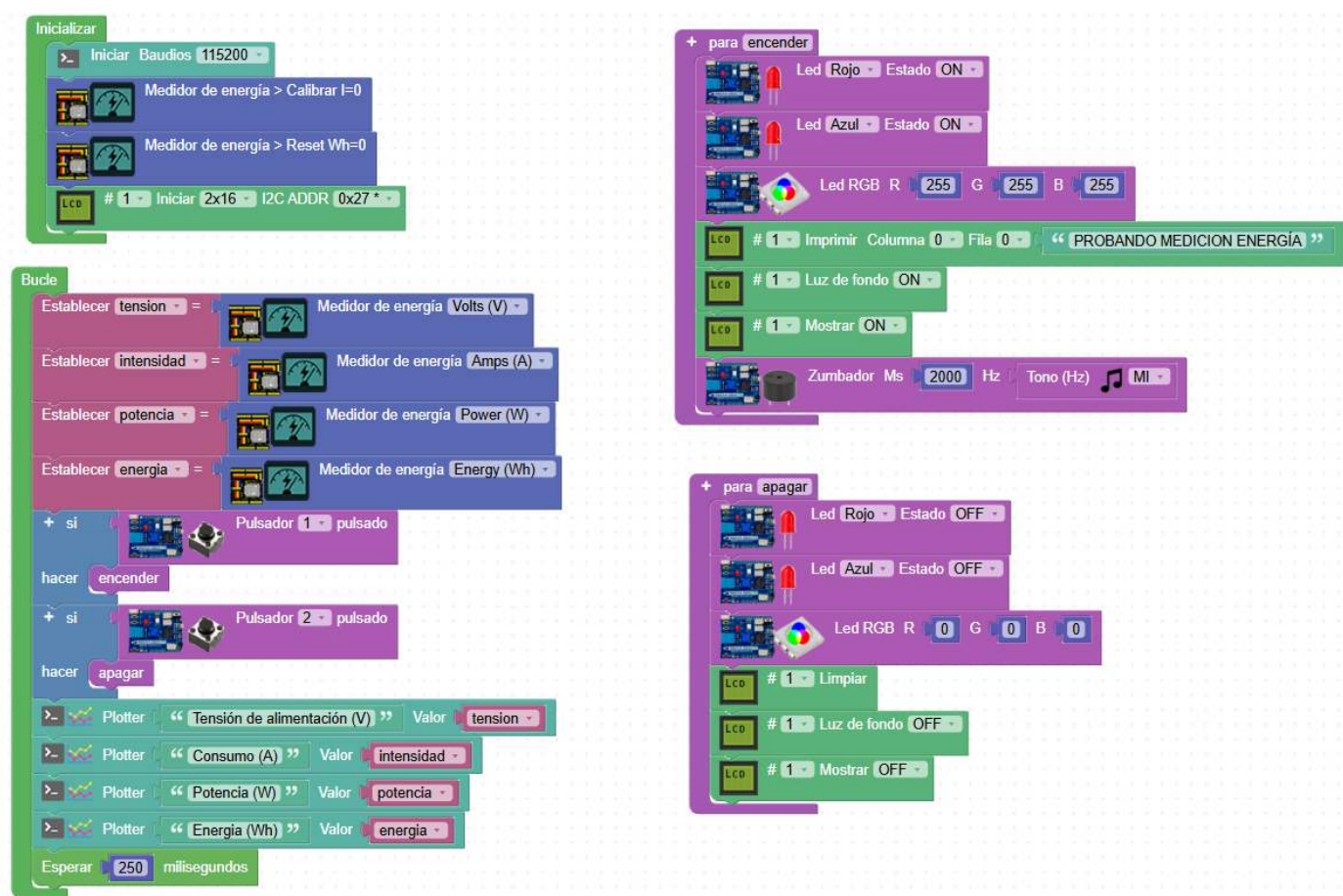
Prácticas con otros elementos del kit

Actividad 14 Sensores internos de ESP32

Ya vimos en [SENSORES ESP32](#) que esta placa tiene internamente unos sensores que podemos utilizar

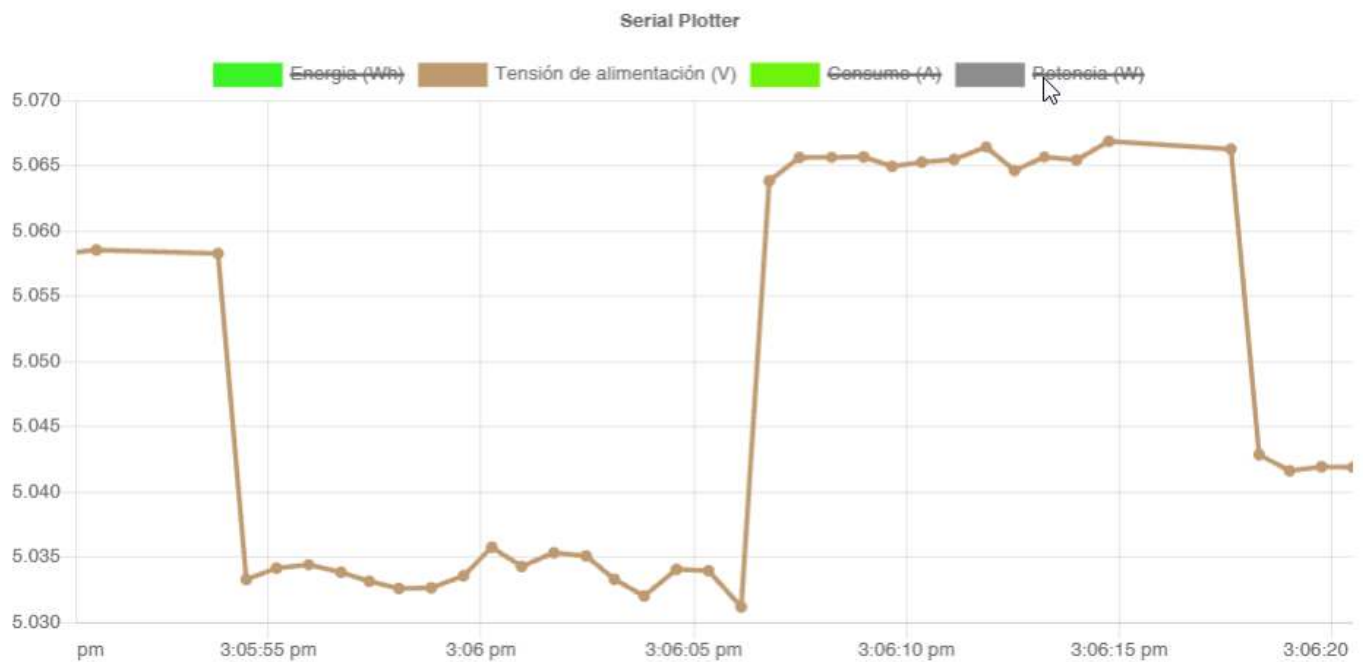
Nuestra propuesta para experimentar es con los sensores de potencia. Vamos a ver este experimento consistente en encender y apagar diferentes elementos de TDR STEAM Imagina y ver las variaciones:

experimentoenergia.abp



Se puede observar las variaciones de tensión cuando se apaga y se enciende

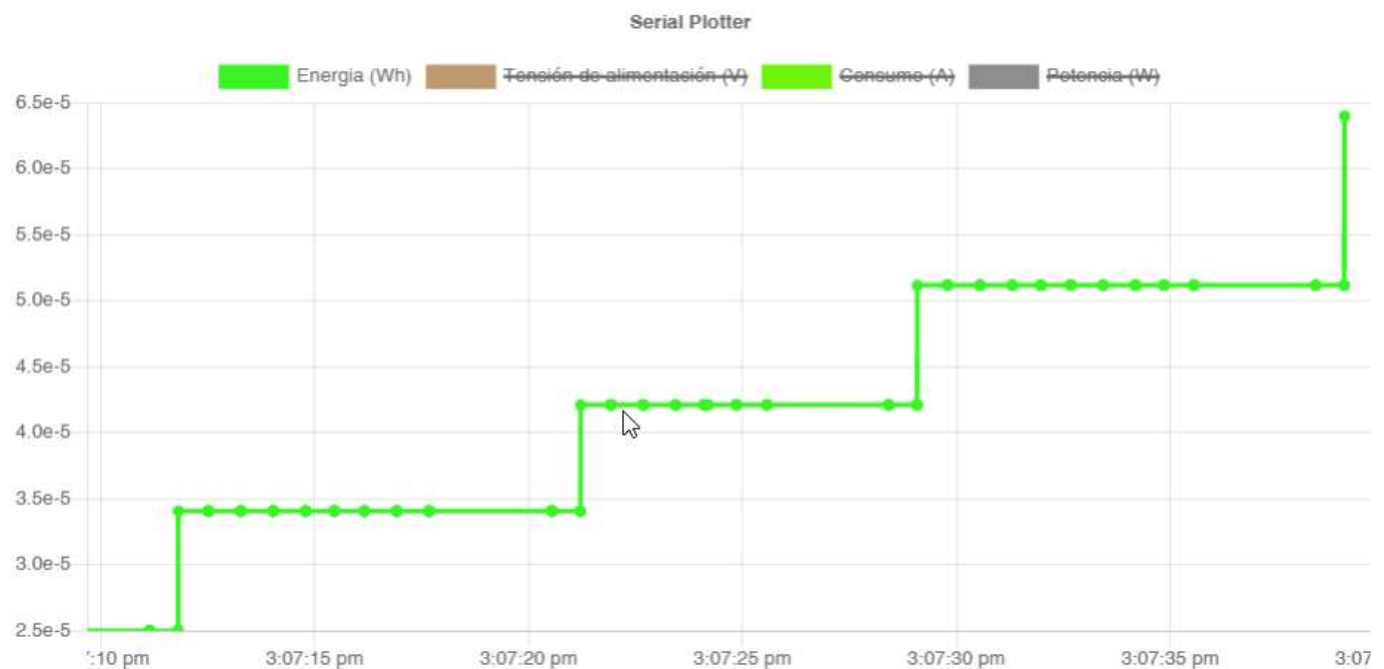
baudrate: 115200 Conectar Desconectar Reset zoom [Max.Samples/serie] 1000 Target Stop CSV



Y también se puede ver como el consumo va a aumentando cada vez que encendemos y apagamos

Serial plotter + Datalogger

baudrate: 115200 Conectar Desconectar Reset zoom [Max.Samples/serie] 1000 Target Stop CSV



¿No tendría que subir linealmente? Pues por lo visto el consumo en los cambios es mucho mayor y los sensores lo detectan y en los estados estacionarios o los sensores no lo detectan

Página extraída de Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

A1. Leer valor de tensión de alimentación

Un programa para verificar la tensión de alimentación de la placa ESP32 Plus STEAMakers lo vemos en la figura siguiente:



Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Medir la tensión de alimentación

Que nos arroja un resultado en la consola como el que vemos en la figura siguiente:

ArduinoBlocks :: Consola serie

Baudrate:

VCC = 4.96 V
VCC = 4.96 V

Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

A.2. Graficar todos los parámetros del 'Medidor de energía'

El programa [SP-medidor-energia](#) de la figura siguiente realiza la captura en variables de los cuatro parámetros disponibles y los envía al Serial Plotter. Para modificar el valor de los mismos se han creado las funciones encender y apagar que encienden y apagan los diodos LEDs de la TdR STEAM y de la tira de 8 LEDs **(NO PROPORCIONADO POR CATEDU)**

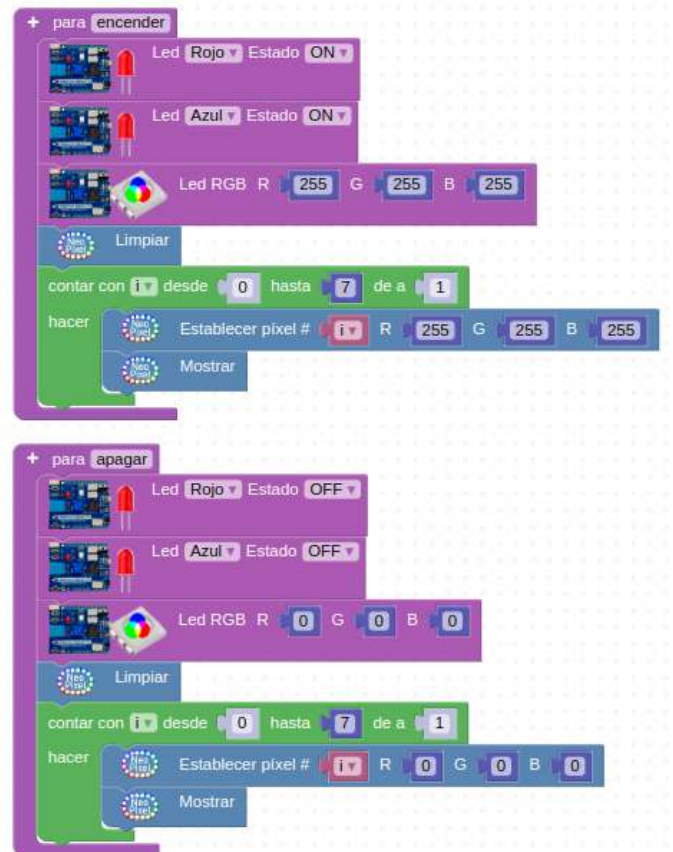
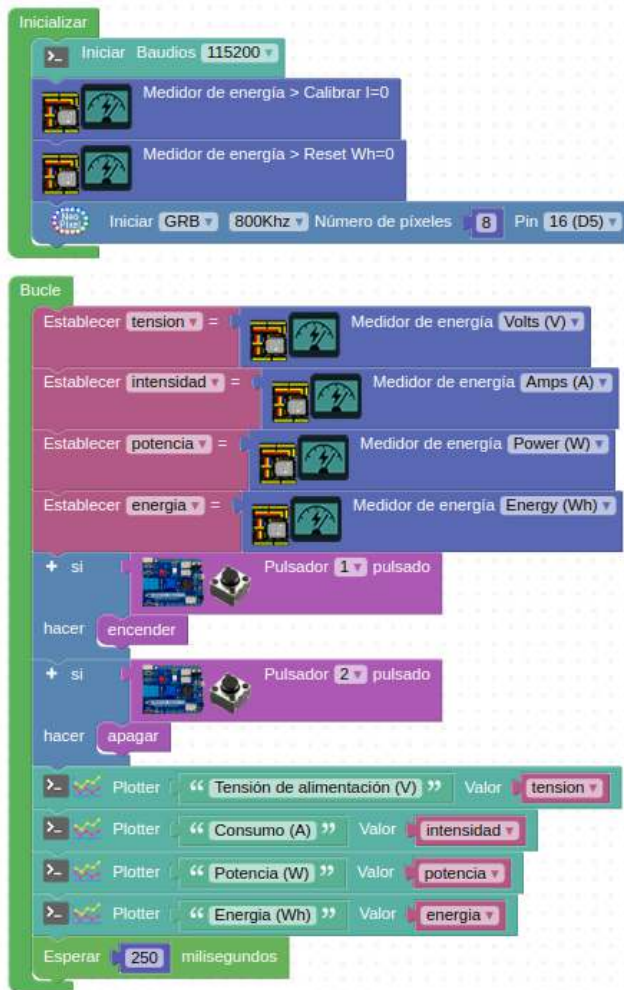


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Programa Serial Plotter

En la figura siguiente vemos el resultado de forma gráfica.

ArduinoBlocks :: Serial plotter + Datalogger



Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

A.3. Sensores internos

Un programa como el de la figura siguiente nos va a mostrar por consola los valores leídos por los sensores internos.

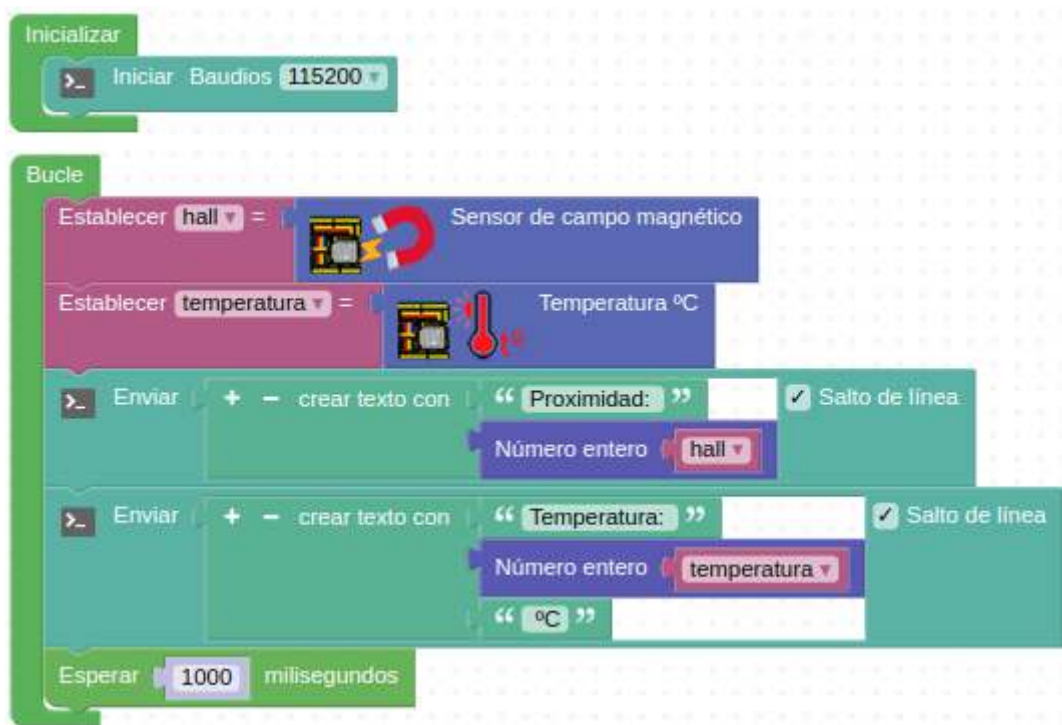


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Sensores internos

El resultado en consola lo vemos en la figura siguiente:

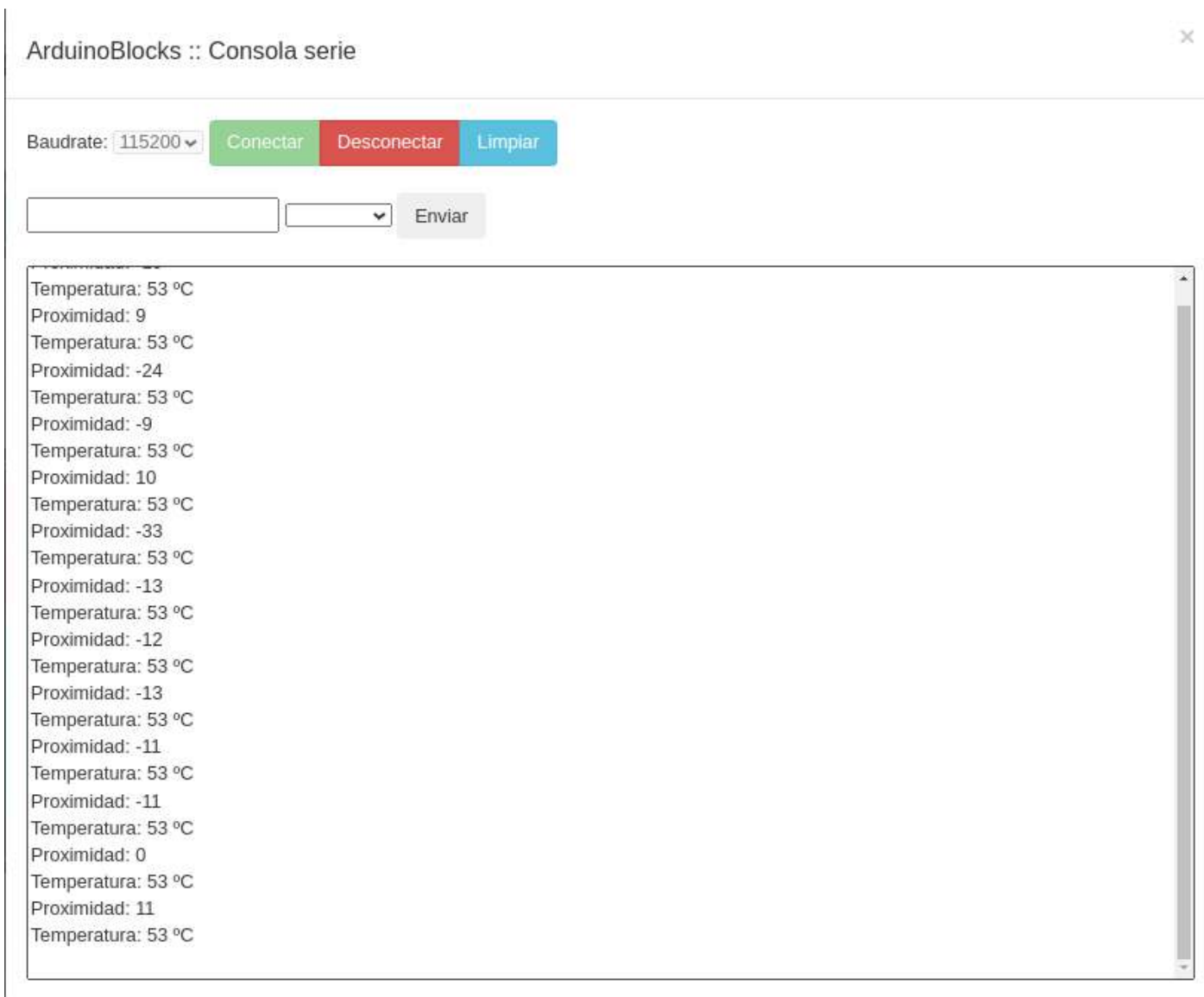


Imagen Federico Coca [Notas sobre ESP32 STEAMakers](#) CC-BY-SA

Retos

- R1. Realizar la actividad A2 mostrando los datos por consola.
- R2. Realizar la actividad A2 mostrando los datos en la LCD.
- R3. Hacer una captura de datos en formato CSV de alguno de los parámetros del 'Medidor de energía'.

Prácticas con otros elementos del kit

Avanzado: Multitarea, interrupciones, memoria Flash y tarjeta SD

Multitarea

Las placas convencionales y ordenadores antiguos que sólo tienen un núcleo en el microprocesador, sólo pueden hacer una tarea. Se puede hacer por software que el microprocesador realice varias tareas simplemente dando tiempo a cada una de ellas e ir saltando de una a otra, es decir, no es una programación en paralelo, y si el micro no es potente, acaba por saturarse.

ESP32 Plus Steammaker tiene un microprocesador pero tiene dos núcleos. Tiene capacidad para hacer algunas tareas en paralelo.

Como estos conceptos pertenecen a un perfil más avanzado, se escapa de los propósitos de este curso. Si se quiere trabajar con ellos recomendamos el siguiente [Descarga desde el drive del autro tutorial Manual de Actividades ESP32 SteamMaker 2022 Junio](#)

Aquí están las páginas y los retos para más información:

7.17	Reto A17. Multitarea.	177
7.17.1	Reto A17.1. Multitarea I.	182
7.17.2	Reto A17.2. Multitarea II.	183
7.17.3	Reto A17.3. Multitarea III.	184
7.17.4	Reto A17.4. Multitarea IV.	185
7.17.5	Reto A17.5. Multitarea V.	188

Interrupciones

Una interrupción es un evento (por ejemplo pulsar un pulsador) y hace que el micro deje lo que estaba haciendo y salte a realizar una tarea concreta. En la placa TDR STEAM hay dos pulsadores, en D3 y en D5 que pueden realizar interrupciones al ESP32.

Aquí están los retos y las páginas en

- [Descarga desde libros.catedu.es Manual Actividades ESP32 SteaMakers 2022_Junio.pdf](#)
- [Descarga desde el drive del autro tutorial Manual de Actividades ESP32 SteamMaker 2022_Junio](#)

7.18	Reto A18. Interrupciones.	189
7.18.1	Reto A18.1. Control de interrupciones.	190

Memoria Flash

El ESP32 Plus STEMaker tiene una memoria Flash que puede mantener los datos incluso después de apagar la placa. Su capacidad es muy limitada y no se almacenan eternamente. [Descarga desde el drive del autro tutorial Manual de Actividades ESP32 SteamMaker 2022_Junio](#)

7.19	Reto A19. Memoria FLASH/EEPROM.	192
7.19.1	A19.1. Lectura/escritura en la memoria EEPROM I.	195
7.19.2	A19.2. Lectura/escritura en la memoria EEPROM II.	198

Tarjeta SD

El ESP32 Plus STEAMaker tiene un zócalo para insertar tarjetas microSD. Tiene que estar formateada a FAT32 y admite hasta 2 Teras con archivos máximos de 2G.

Aquí están los retos y las páginas en [Descarga desde el drive del autro tutorial Manual de Actividades ESP32 SteamMaker 2022_Junio](#)

7.13	Reto A13. Tarjeta SD.	155
7.13.1	Reto A13.1. Almacenar datos: Datalogger.	157
7.13.2	Reto A13.2. Leer datos almacenados.	159
7.13.3	Reto A13.3. Escribir y leer datos.	160

Bluetooth

Bluetooth

Un poco de teoría Bluetooth

ONDAS

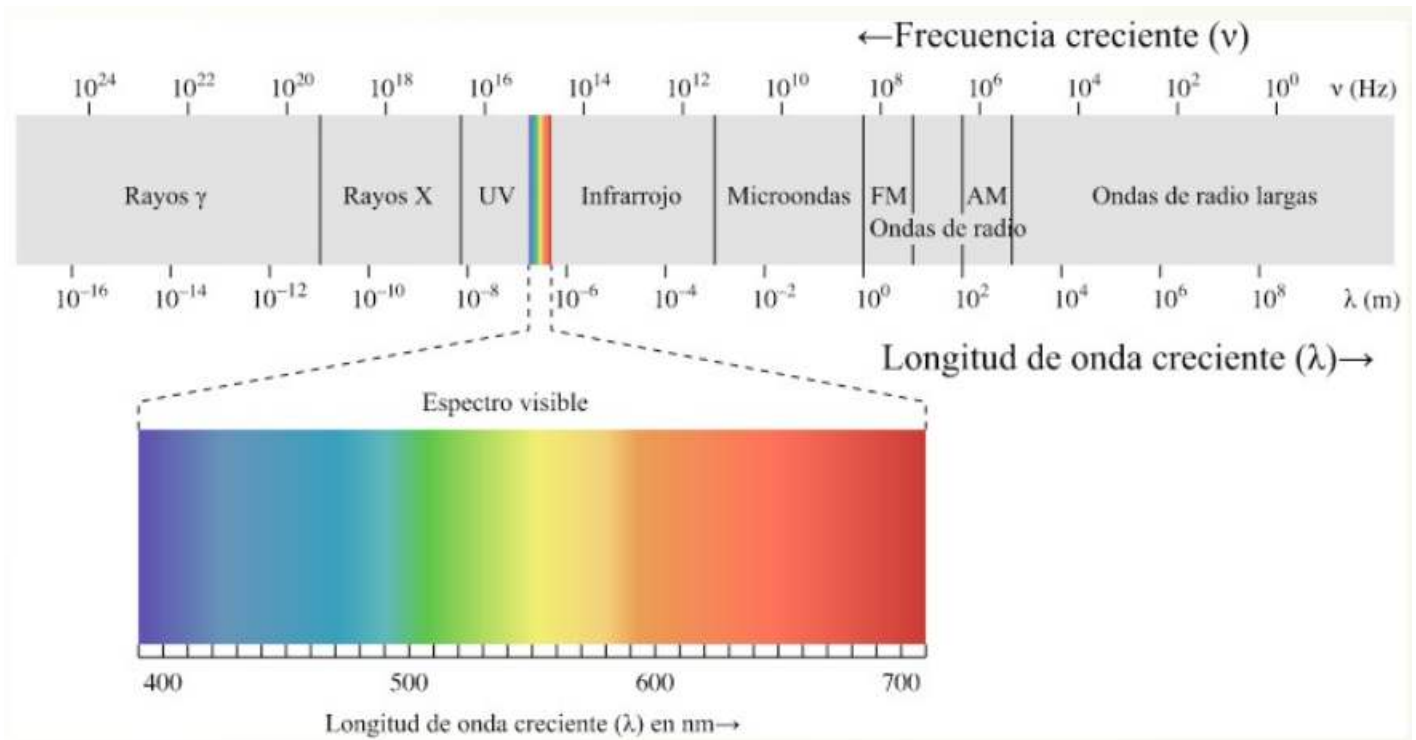
Una onda es una señal que se propaga por un medio. Por ejemplo el sonido, que es una onda mecánica que viaja usando el aire o cualquier otro material. Pero en el caso de las señales eléctricas pueden ser enviadas por el cable o a través del vacío (no necesitan un medio para transmitirse).

Dependen de 3 parámetros principalmente:

- **Amplitud:** altura máxima de la onda. Hablando de sonido representaría el volumen. Si nos referimos a una onda eléctrica estaríamos representando normalmente el voltaje.
- **Longitud de onda λ :** distancia entre el primer y último punto de un ciclo de la onda (que normalmente se repite en el tiempo).
- **Frecuencia f :** Número de veces que la onda repite su ciclo en 1 segundo (se mide en hertzios).
- **Periodo T** es simplemente es la inversa de la frecuencia. $T=1/f$

La relación entre ellas es muy fácil pues las ondas electromagnéticas viajan a la velocidad de la luz c y si velocidad es espacio/tiempo luego $c = \lambda/T$ luego **$c = \lambda * f$**

Dentro del espectro electromagnético encontramos diferentes tipos de señales dependiendo de las características de su onda.



TRANSMISIÓN INALÁMBRICA: BLUETOOTH.

- Hoy en día, este grupo está formado por miles de empresas y se utiliza no sólo para teléfonos sino para cientos de dispositivos.
- Bluetooth es una red inalámbrica de corto alcance pensada para conectar pares de dispositivos y crear una pequeña red punto a punto, (sólo 2 dispositivos).
- Utiliza una parte del espectro electromagnético llamado “**Banda ISM**”, reservado para fines no comerciales de la industria, área científica y medicina. Dentro de esta banda también se encuentran todas las redes WIFI que usamos a diario. En concreto funcionan a 2,4GHz. (Un G son 10^9) luego entre FM y Microondas.

¿Sabías que?

Su curioso nombre viene de un antiguo rey Noruego y Danés, y su símbolo, de las antiguas ruinas que representan ese mismo nombre.

Hay 3 clases de bluetooth que nos indican la máxima potencia a la que emiten y por tanto la distancia máxima que podrán alcanzar:



CLASE	POTENCIA	DISTANCIA
Clase 1	100 mW	100 m
Clase 2	2,5 mW	10 m
Clase 3	1 mW	1 m

También es muy importante la velocidad a la que pueden enviarse los datos con este protocolo:

Versión	Velocidad
1.2	1 Mbps
2	3 Mbps
3	24 Mbps
4	24 Mbps

Mbps : Mega Bits por segundo. MBps: Mega Bytes por segundo.

kb = 1.024 b M = 1.024 k G = 1.024 M

¿Te atreves a calcularlo ?

¿Cuántos ciclos por segundo tendrán las ondas que están en la **Banda ISM**? ¿Cuál es el periodo de esas ondas?

Solución

a) $f = 2.4\text{G}$

b) $\lambda = c/f = 12.5\text{cm}$ o sea, las antenas tendrían que ser de esta longitud. Hay muchos trucos para reducirla, una de ellas es la forma de serpiente que puedes ver en el HC-06

¿Te atreves a calcularlo...?

¿A qué distancia y cuanto tiempo tardarían en enviarse los siguientes archivos por Bluetooth?

1. Un vídeo de 7Mb usando versión 2 clase 2
2. Una imagen de 2.5Mb usando versión 3 clase 1
3. Un archivo de texto de 240KB usando versión 1.2 clase 1

Solución

1) $7\text{Mb} / 3\text{Mbps} = 2.3 \text{ seg.}$

2) $2.5\text{Mb} / 24\text{Mbps} = 0.1 \text{ seg.}$

3) $240 \text{ kB } 8\text{b/B} = 1.920 \text{ kb}$ $1.920 \text{ kb} / 1.024 = 1.875 \text{ Mb}$ $1.875\text{Mb} / 1\text{Mbps} = 1.875 \text{ seg.}$

¿Bluetooth clásico o Bluetooth Low Energy = BLE?

Es un protocolo similar al clásico Bluetooth pero diseñado a consumir menos potencia manteniendo funcionalidad. Su popularidad ha crecido en multitud de dispositivos

En robótica, el clásico device que utiliza BLE es la **Micro:bit**. Aunque la Micro:bit no tiene Wifi integrada, posee una radiofrecuencia que podemos configurar para Bluetooth (hay que elegir, o utilizar sus comandos de Radio o utilizar comandos de Bluetooth)

Por eso a la hora de elegir la APP tienes que tener en cuenta:

- Si acepta Bluetooth clásico o BLE
- Que la APP acepte leer datos desde el robot como enviar

Nosotros hemos elegido uno sencillo que cumple las dos condiciones (hay muchas APPs) [Serial Bluetooth Terminal](#)

Serial Bluetooth Terminal

Kai Morich

Compras en la aplicación

Terminal para los dispositivos conectados en serie con Bluetooth Classic / LE



4,6★

3,31 mil reseñas

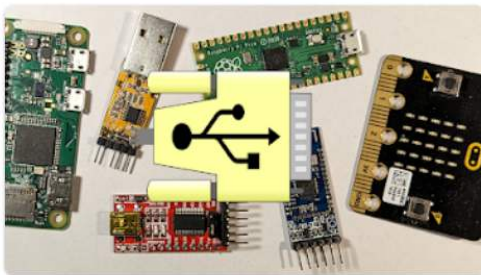
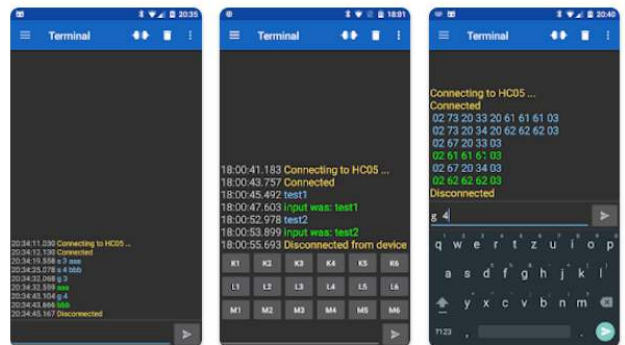
1M+

Descargas



PEGI 3

Descargar



Bluetooth

APP Serial Bluetooth Terminal

DESCARGA LA APP

Esta APP es muy sencilla y la puedes descargar [aquí](#). Tiene las siguientes ventajas :

- **Enviar / Recibir** mensajes
- Permitir conexiones tanto
 - **BLUETOOTH CLÁSICO** por ejemplo HC06 de Arduino, Echidna, ESP32 ...
 - **BLUETOOTH LE** (Low emission) por ejemplo para la MICRO:BIT

Serial Bluetooth Terminal

Kai Morich
Compras en la aplicación

4,6★
3,28 mil reseñas ⓘ

1 M+
Descargas

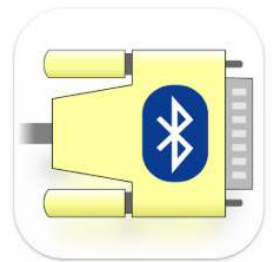
3
PEGI 3 ⓘ

Descargar

Compartir

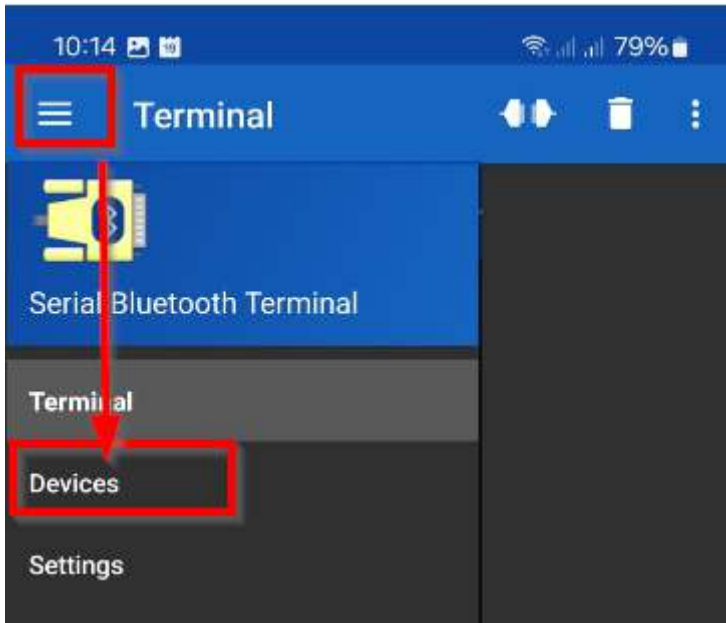
Añadir a la lista de deseos

Esta aplicación está disponible para tu dispositivo



EMPAREJAR DISPOSITIVOS

Si no esta emparejado con el móvil NO TE PUEDES CONECTAR, para ello entramos en Devices :



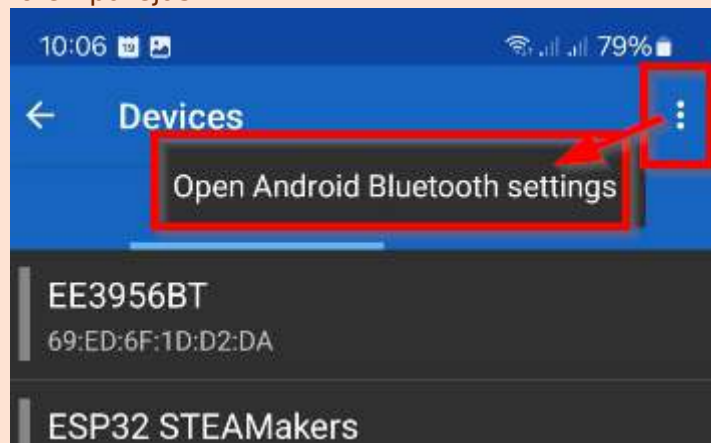
- **Microbit**: Entramos en **Devices** y en **Bluetooth LE** y nos conectamos a la Micro:bit
- **HC06** para Arduino Echidna **ESP32**.... igual pero en **Bluetooth clásico**

Aquí puedes ver dos capturas de dispositivos en Bluetooth clásico y Bluetooth BLE

LOS QUE ESTAN EN **VERDE** SON LOS QUE TIENES EMPAREJADOS Y PUEDES CONECTARTE

BBC micro:bit D3:32:4F:38:73:52 (paired)	69:ED:6F:1D:D2:DA
CC41-A 00:15:83:00:BE:69	ESP32 STEAMakers 88:13:BF:DC:97:BE
	ESP32-javier 94:B5:55:28:FF:46
	HC-06 00:23:09:01:2D:B3
	JBL Flip Essential 2 F8:5C:7E:53:E2:81

¿Y si no aparece o no esta emparejado? Entrás en el diálogo de Android de Bluetooth y lo emparejas



CONECTARTE

Una vez seleccionado el dispositivo emparejado ya puedes conectarte :

1. Menú
2. Entrás en Terminal
3. Enchufe
4. Sale conectado, ya estas preparado para enviar y recibir



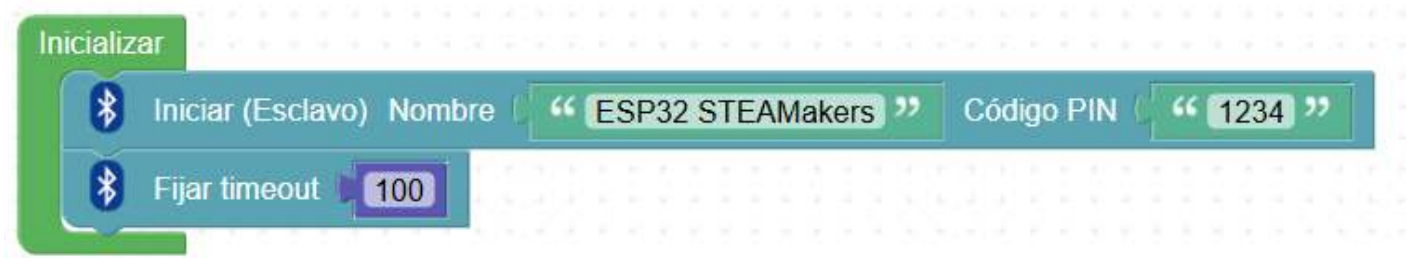
Bluetooth

Actividad 15 APP->ESP32APP Serial Bluetooth Terminal

Preliminar

A diferencia del [Arduino que necesita un módulo exterior el HC06](#), el **ESP32 tiene el Bluetooth integrado**

Para inicializar el Bluetooth, STEAMAKERBLOCKS tiene instrucciones específicas, lo primero es definir un nombre a la señal Bluetooth para que los otros dispositivos (tu móvil) lo encuentre, por ejemplo en la figura ESP32 STEAMakers



El bloque **Fijar timeout** significa que fijamos el tiempo en la transmisión de datos por bluetooth. Por defecto está fijado en 1000 ms pero podemos reducirlo a 100 ms como en la figura para que funcione más fluido.

Enunciado

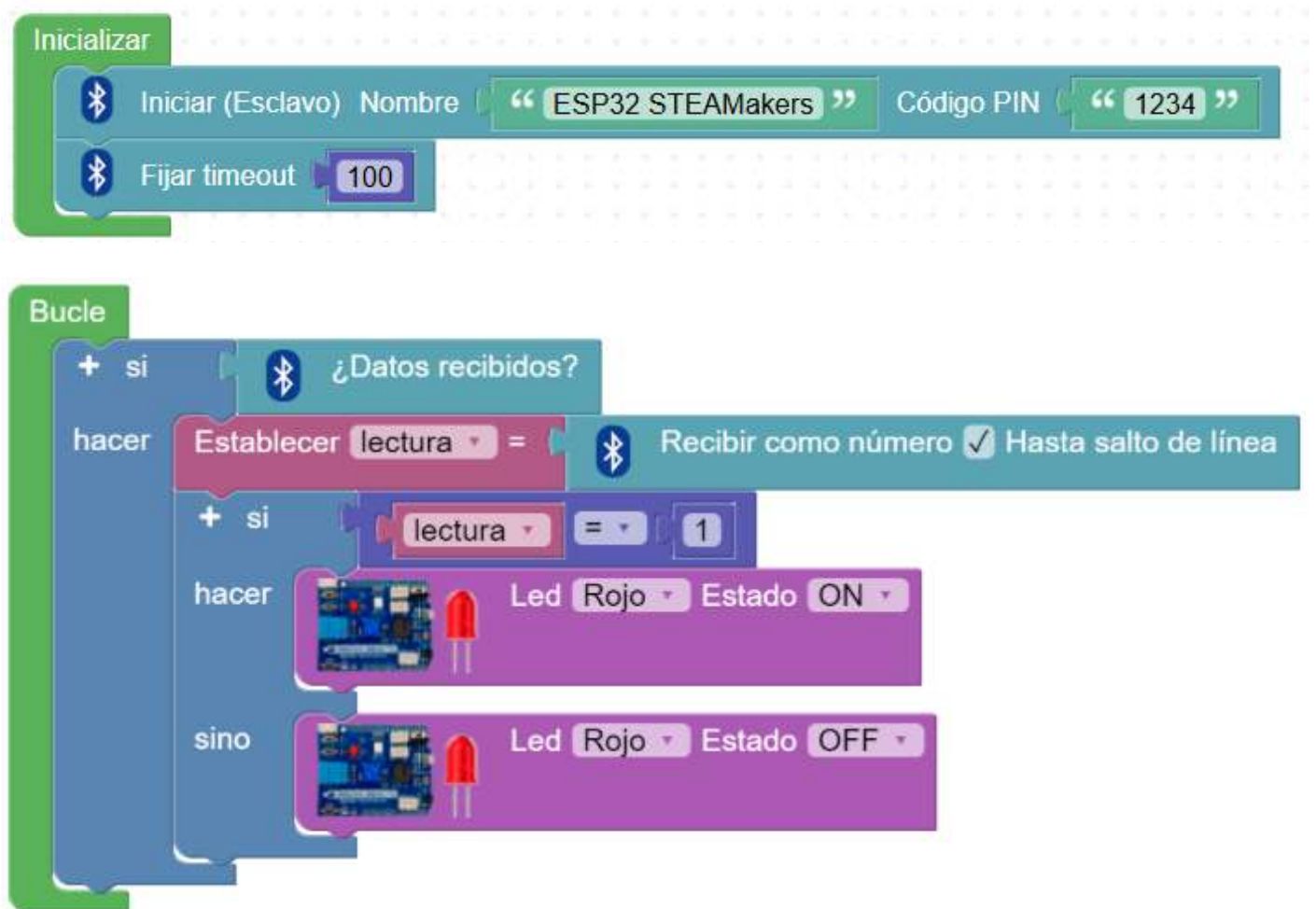
Si recibe un código, que encienda un led. Por ejemplo:

- si recibe un 1 que se encienda el led rojo.
- si recibe otro código, apaga el led rojo

Este enunciado parecido [a este programa en Arduino con Bluetooth](#)

Programa

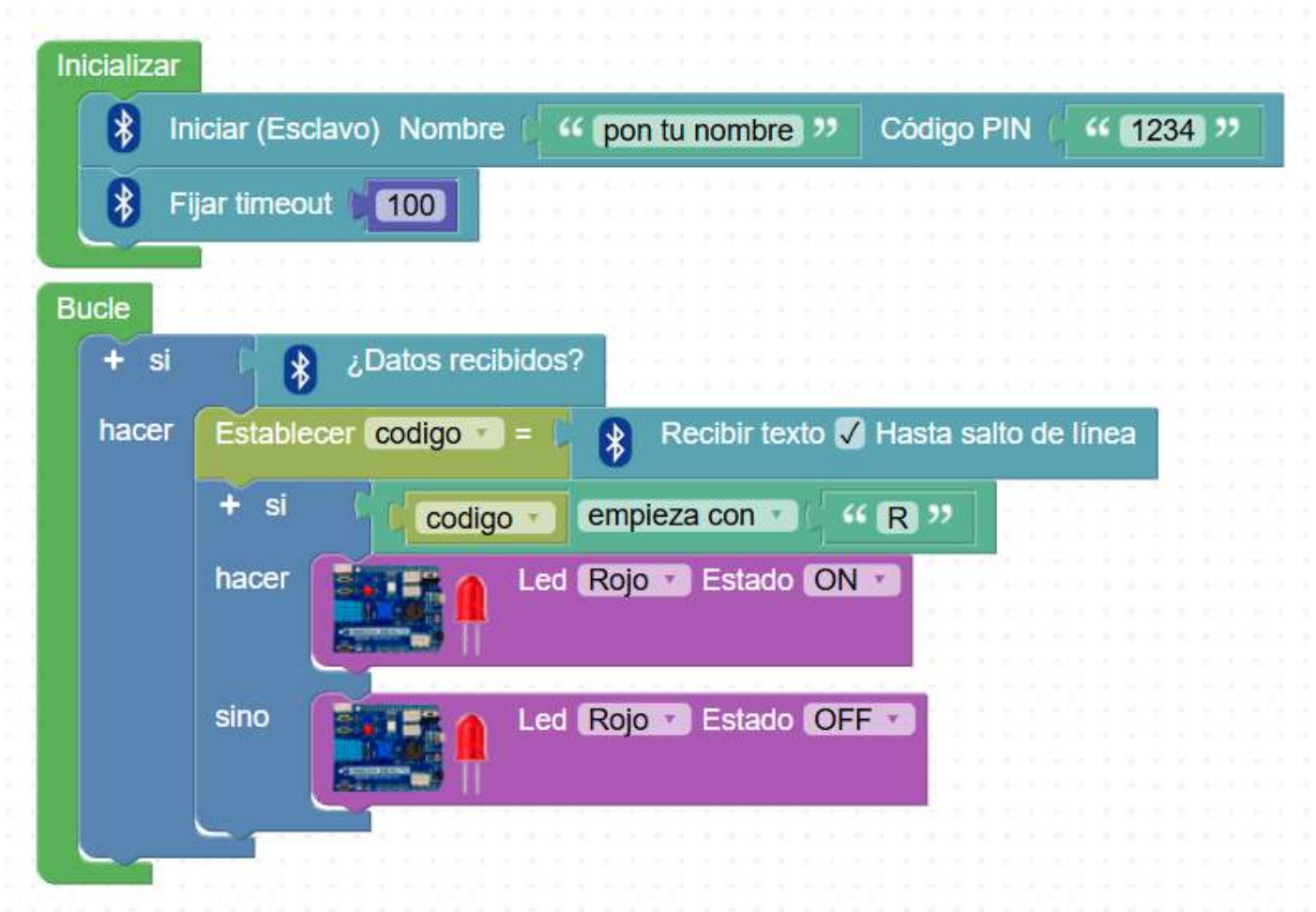
El programa es sencillo: <http://www.arduinoblocks.com/web/project/1013787>



<https://www.youtube.com/embed/GaB3B9wSA8s>

Opción con texto

Podemos poner en vez de una variable numérica un texto, pero el Bluetooth añade un carácter al final que no siempre se lee bien por lo que recomendamos usar en la condicional "empieza con" en vez de "="



con la palabra ROJO se enciende pero también con cualquiera que empiece con R

Bluetooth

ACTIVIDAD 15 con otra APP: Arduino Bluetooth Control

La APP Arduino Bluetooth Control

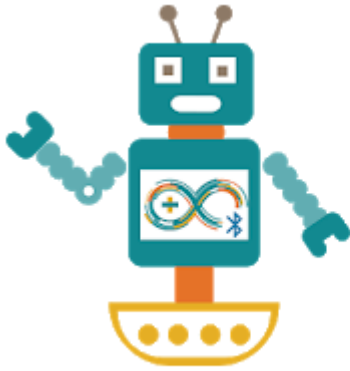
Esta APP es para comunicación PLACA->APP es decir, el Bluetooth será como receptor de datos nuestra placa electrónica (slave) y el móvil (master) como emisor de datos.

Esta APP permite enviar datos pero de distintas formas



Arduino Bluetooth Control

Esta APP es muy completa y configurable, [aquí para descargarla de Google Play](#).



Arduino Bluetooth Control

broxcode Herramientas

★★★★★ 172

PEGI 3

Contiene anuncios

Esta aplicación es compatible con todos tus dispositivos.

Instalada

El código de programa que tenemos que cargar en la placa se basa en escuchar de forma continua el puerto serie. **Cuando llegue el dato, se ejecutará la acción que le indiquemos.** ¡¡así de sencillo !!

<https://giphy.com/embed/xTilzoyw4Yh3mRM5DG>

Vincular móvil

Hay que vincular nuestro móvil y nuestra APP de Android con el Arduino, para ello sigue [este sencillo tutorial](#):

<https://docs.google.com/presentation/d/e/2PACX-1vT0vG1z61MuZXKmdiw4ga7z15FIQfeussqDNYzMauJSZUU2G2NIL7M-JjXb4PFT4YTigj9Yal8PzHmR/embed?start=false&loop=false&delayms=3000>

Recuerda que para que funcione el Bluetooth tienes que poner el interruptor alejado del borde.

Ver <https://libros.catedu.es/books/esp32-en-el-aula/page/bluetooth-en-el-esp32>

El resultado

<https://www.youtube.com/embed/e3y4DSwCQBQ>

Reto

¿Te atreves a hacerlo con voz? Por ejemplo si dices AZUL que encienda el led AZUL si dices ROJO que encienda el ROJO y si dices APAGAR que los apague

Bluetooth

Actividad 16 APP<->ESP32

Enunciado y programa

STEAMAKERBLOCKS

Enunciado

Ahora vamos a realizar la comunicación bidireccional. El propósito es que si recibe un código, que encienda y apague los leds, pero además vamos añadir comunicación hacia la APP del móvil, que nos envíe información de los sensores. Un ejemplo de enunciado sería:

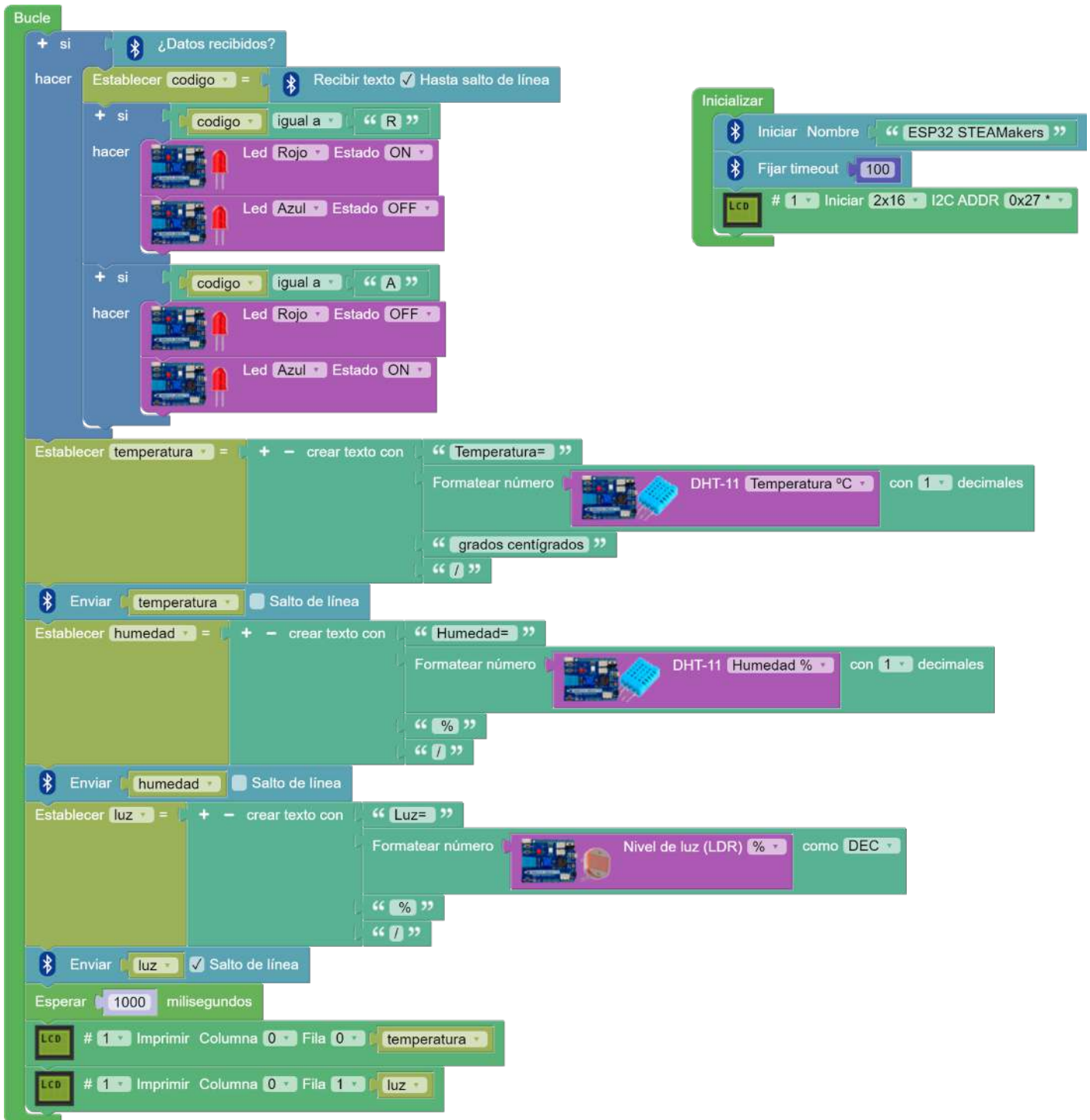
- APP->ES`32
 - si recibe un R que se encienda el led rojo y apague el led azul.
 - si recibe un A que se encienda el led azul y apague el led rojo.
- APP<-ESP32:
 - Que envíe a la APP el valor de la temperatura del DHT11.
 - Que envíe a la APP el valor de la humedad del DHT11.
 - Que envíe a la APP el valor de la luz del LDR.

Además lo vamos a complicar un poco más: que visualice los valores de temperatura y luz en el display LCD para que se vea que corresponden a los que muestra en la APP (no se muestra el de la humedad pues no cabe en el LCD).

Este enunciado parecido al programa en Arduino <https://libros.catedu.es/books/arduinoblocks-en-el-aula/page/app-tdr-steam-arduinoblocks>

Programa

El programa ya es más complejo: <http://www.arduinoblocks.com/web/project/1050441>



Bluetooth

Actividad 16 con la APP Serial Bluetooth Terminal

<https://www.youtube.com/embed/c89tMSy4V6s>

Bluetooth

Actividad 16 con APP INVENTOR

La APP en el móvil

La comunicación con la placa en este caso no es tan sencilla como en el caso anterior, no nos vale las APPs que hay en Google Play que envían un código y en paz, ahora tenemos que tener una APP que si reciben un código que hagan algo. Esto ya es **exclusivo** por lo tanto **hay que programar una APP que cumpla los requisitos. En nuestro caso lo hemos hecho en APP INVENTOR**

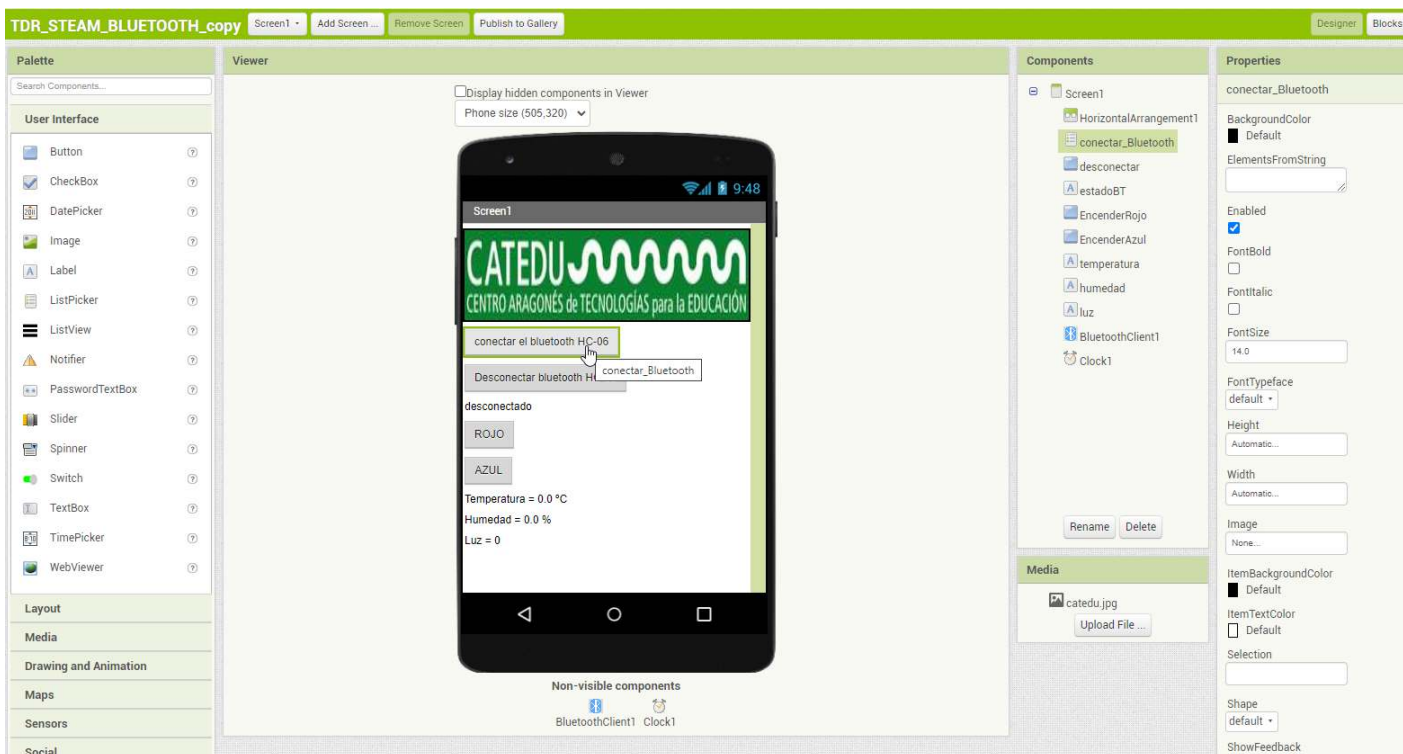
<https://appinventor.mit.edu/>

NO ES EL OBJETIVO DE ESTE CURSO FORMARTE EN APP INVENTOR

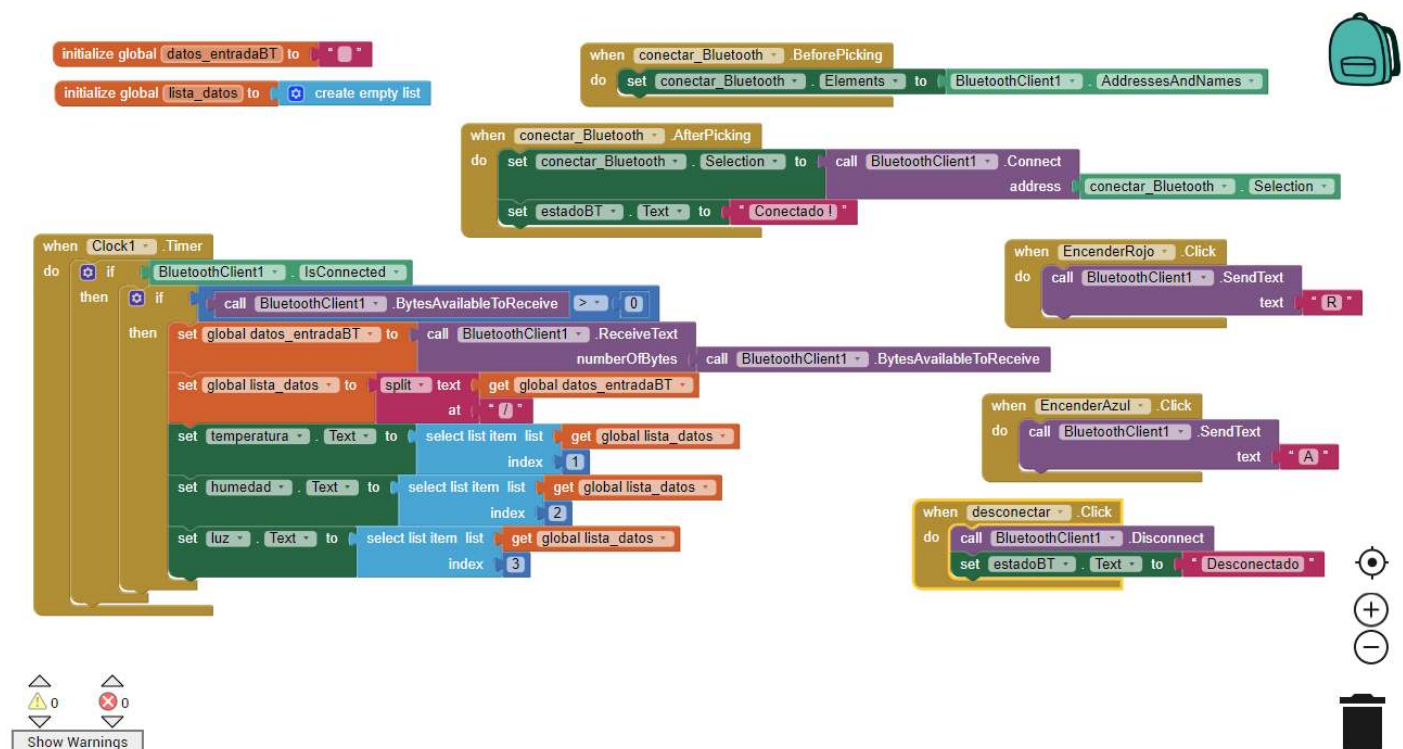
Pero si no conoces el programa, te recomendamos que vayas a estos contenidos que de una manera muy fácil aprenderás a crear programas con APP INVENTOR y exportarlos al móvil

<https://libros.catedu.es/books/app-inventor>

En la parte de DESIGNER



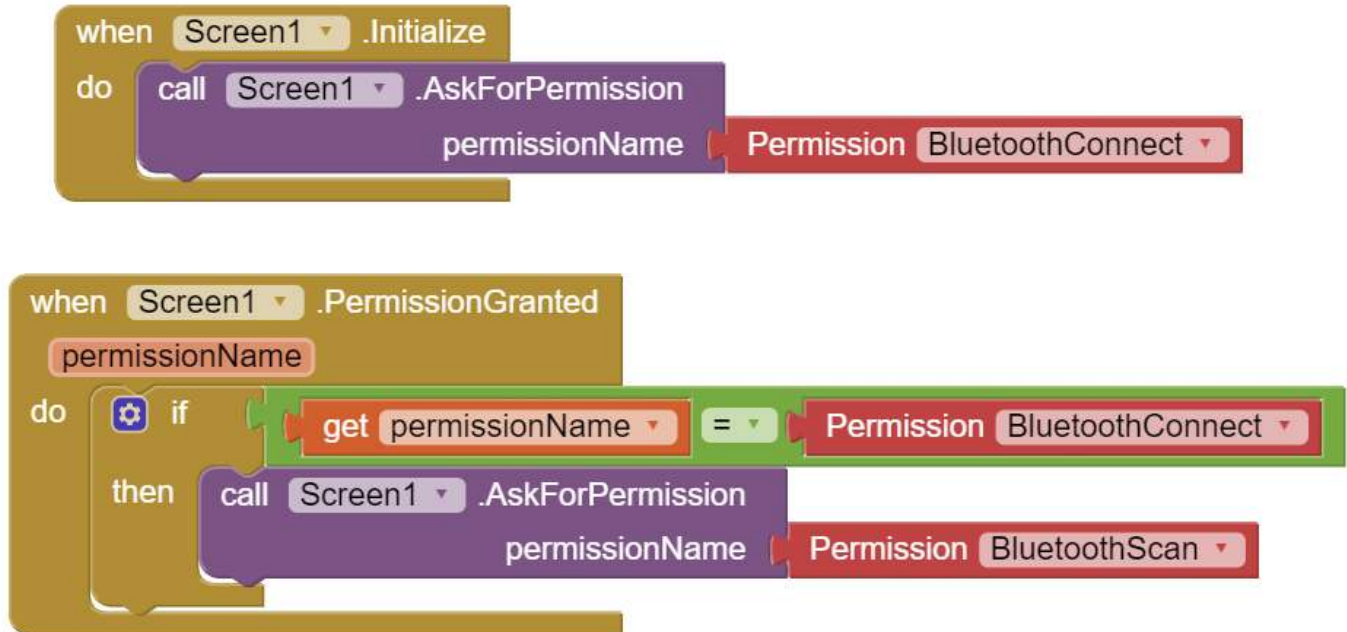
En la parte de Blocks



ATENCIÓN: A partir de Android 12 hay que dar permisos a la aplicación para que pueda visualizar los dispositivos bluetooth disponibles, para ver la explicación

<https://www.youtube.com/watch?v=MNLzS-MOXAg>

Hay que añadir este código:



Puedes descargarte la APP [en este enlace](#) pero tienes que permitir en el móvil instalar APPs desconocidas.

Otra forma es en este código QR utilizando la APP de Google Play [MIT AI2 Companion](#) y poner el código *sxbbnv*

Aquí tienes los pasos de su realización

https://docs.google.com/presentation/d/e/2PACX-1vS3LG1KfIZ_hr17TpJ3kOsPfX9E4Cq85Q3tfL0VTGExQzr_QpOFsVHYvbS0oUTlwXS_kuiSYftWGthX/embed?start=false&loop=false&delayms=3000

El programa lo tienes aquí <https://gallery.appinventor.mit.edu/?galleryid=e4780fa4-3252-4d73-b846-b2a0935c6d01>

La APP a tu móvil

Tienes dos opciones

- **EN VIVO CONNECT - AI COMPANION** esta opción es la más rápida, y realmente lo simula a través de la APP INVENTOR.
 - Tienes que tener instalada la APP MIT AI2 COMPANION
 - Se le pasa el código de tu APP a la APP
- **OTRAS OPCIONES**
 - Ver <https://appinventor.mit.edu/explore/ai2/setup>

OPCIÓN EN VIVO AI COMPANION

Instalas la [APP MIT AI2 COMPANION](#)

MIT AI2 Companion

MIT App Inventor

Desarrolla tus propias aplicaciones Android usando MIT App Inventor 2!



1.9★

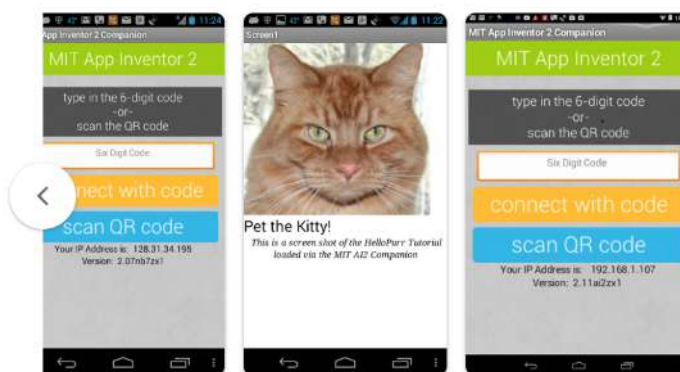
28 mil reseñas

5 M+

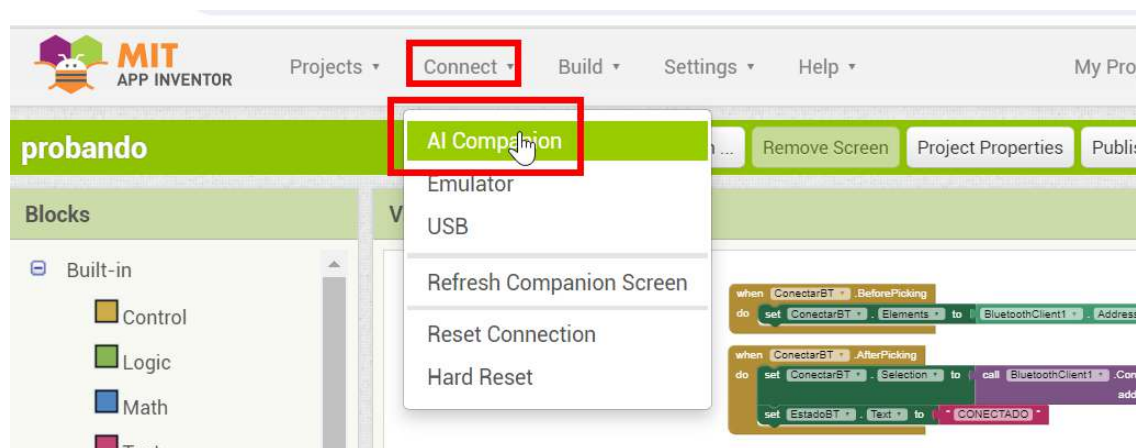
Descargas

PEGI 3

Instalar en más dispositivos



En APP INVENTOR



Y sale un código y un QR asociado al código



Launch the MIT AI2 Companion on your device and then scan the barcode or type in the code to connect for live testing of your app.
[Need help finding the Companion App?](#)



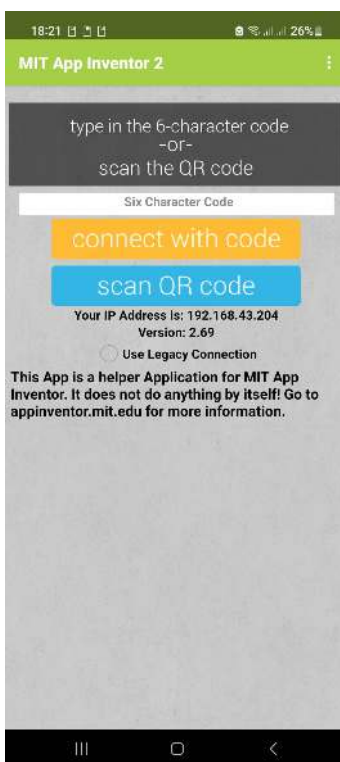
Your code
is:

000000

Note: You are on a secure connection, legacy mode on the Companion will not work [More Information](#)

Cancel

Abrimos la [APP MIT AI2 COMPANION](#) y metemos el código anterior (o lo escaneamos con el QR)



En APP INVENTOR verás que sale una barra de progreso enviando tu APP a tu móvil. Cuando termina automáticamente lo ejecuta.

El resultado

<https://www.youtube.com/embed/fAnqXJSY4tY>

Que es IoT

El **Internet de las cosas** (Internet of Thing IoT) describe objetos físicos —o grupos de estos— con sensores, capacidad de procesamiento, software y otras tecnologías que se conectan e intercambian datos con otros dispositivos y sistemas a través de internet u otras redes de comunicación. El Internet de las cosas se ha considerado un término erróneo porque los dispositivos no necesitan estar conectados a la Internet pública. Sólo necesitan estar conectadas a una red y ser direccionables individualmente

[Fuente Wikipedia IoT Internet de las cosas CC-BY-SA](#)



[De Drawed by Wilgengebroed on FlickrTranslated by Prades97 CC BY-SA 3.0](#)

Estamos hablando de dispositivos que se conectan a internet de forma desatendida, por vía hardware (o mejor dicho firmware) a diferencia de un ordenador, tablet o móvil, donde tienes que configurar por software el dispositivo y hay un diálogo entre usuario y dispositivo sobre el uso de Internet (el software solicita tal página web, tales datos etc por voluntad del usuario o por diálogo con el usuario) Aquí los dispositivos están ya configurados de los datos que se comunican. Es decir "conectar y olvidar".

Piensa en la diferencia entre un enchufe inteligente y un ordenador, el primero es lo que se considera dentro de IoT

Desventajas: El acceso a Internet de dispositivos caseros puede generar problemas a nivel mundial:

- [el caso Mirai](#)
- [aspiradores que nos espían](#)

IoT en los cursos de Aularagón

- **Blynk:** lo que nos gusta de esta herramienta es que es casi "instantánea" o "síncrona". Esto es imprescindible con ciertos robots como el **Rover Marciano con Arduino**. Necesitamos que "gire" para evitar un obstáculo, no podemos esperar !!! Veremos con **BLYNK** un protocolo que entre el dispositivo electrónico (nuestro robot) y nosotros (en ordenador, en una APP en el móvil) la comunicación es instantánea, gracias a un servidor que hará de intermedio, que puede ser local (BLYNK LEGACY) o en Internet (BLYNK IoT).
 - **Blynk legacy** es la que se va a trabajar en
 - [Rover Marciano con Arduino](#)
 - [Arduinoblocks en el aula](#)
 - [ESP32 en el aula](#)
 - **Blynk IoT** es la que se va a trabajar con
 - [En ESP32 en el aula](#)
 - [En Smart Home ESP32](#)
- **ThinkSpeak y SmartioSpace**
 - [Smart Agriculture Kit para Micro:bit](#)
- **MQTT** El emisor envía datos, se almacenan en un servidor, y cuando puede, lo vuelca al cliente. Cliente y emisor pueden ser el dispositivo electrónico y nosotros o viceversa. Veremos que esto es lo que hace el protocolo **MQTT** y está tremendamente extendido por lo barato y fácil que es. Hace que los servidores no estén tan ocupados, por lo tanto hay varios proveedores que ofrecen este servicio gratuitamente. Hay robots como los que tienen la placa **TDR STEAM IMAGINA** que envía datos de temperatura, humedad, .. y pueden recibir datos pero no precisan de esta exigencia instantánea como un rover.
 - [ESP32 EN EL AULA](#)
 - [En Smart Home ESP32](#)
- **TELEGRAM**
 - [ESP32 EN EL AULA](#)
 - [En Smart Home ESP32](#)



- **Arduino cloud IoT**
 - [Arduino Alvik](#)
- **Cyberpi y mBot2**
 - [IoT con Cyberpi](#)

IoT-Wifi-Telegram

IoT-Wifi-Telegram

Crear bot en Telegram



BotFather 

bot



BotFather is the one bot to rule them all. Use it to create new bot accounts and manage your existing bots.

Description

@BotFather

Username



Entramos en nuestro Telegram y chateamos con el creador de los bots: **@BotFather** y nos saldrá esta pantalla:

BotFather 

bot

What can this bot do?

BotFather is the one bot to rule them all. Use it to create new bot accounts and manage your existing bots.

About Telegram bots:

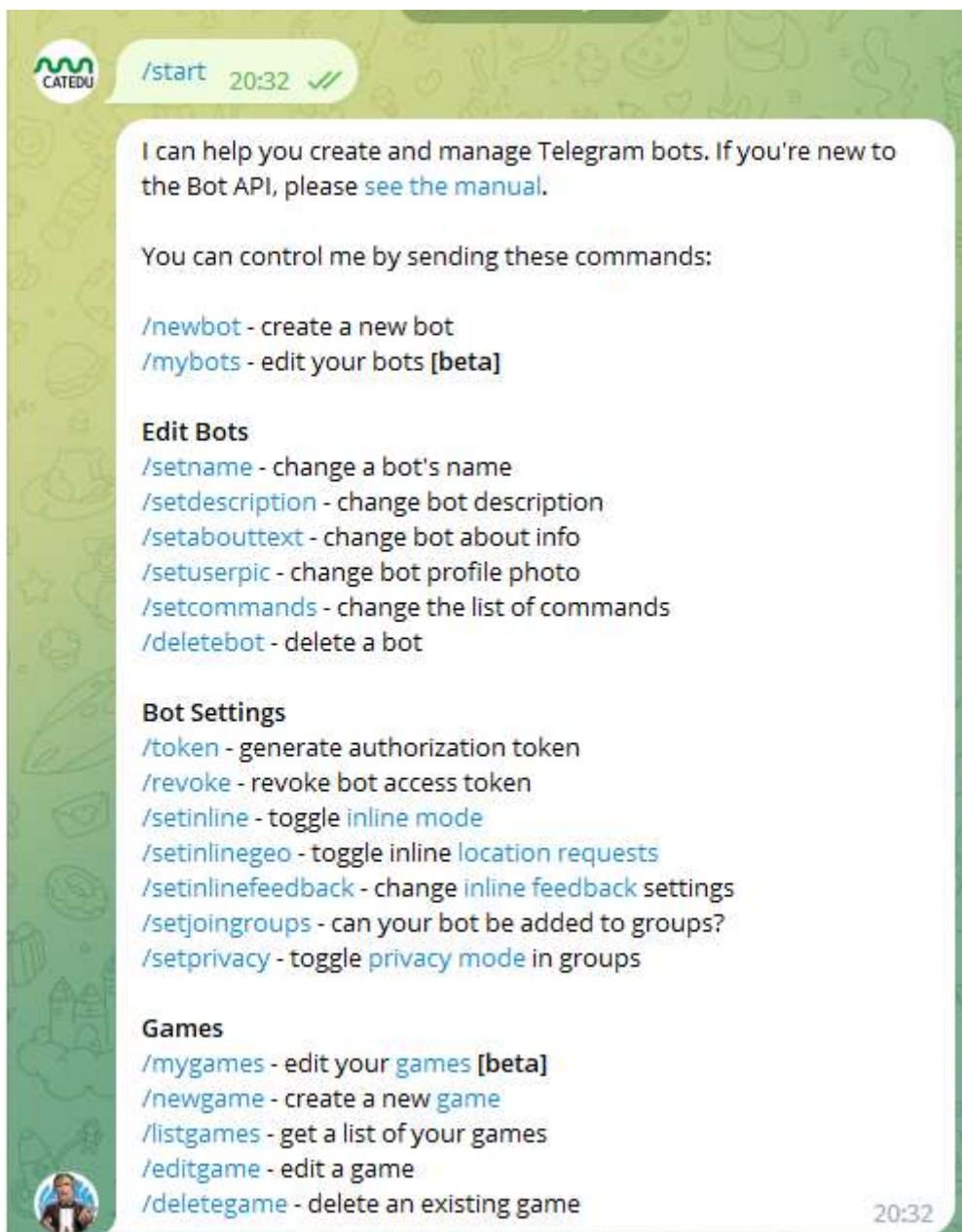
<https://core.telegram.org/bots>

Bot API manual:

<https://core.telegram.org/bots/api>

Contact [@BotSupport](#) if you have questions about the Bot API.

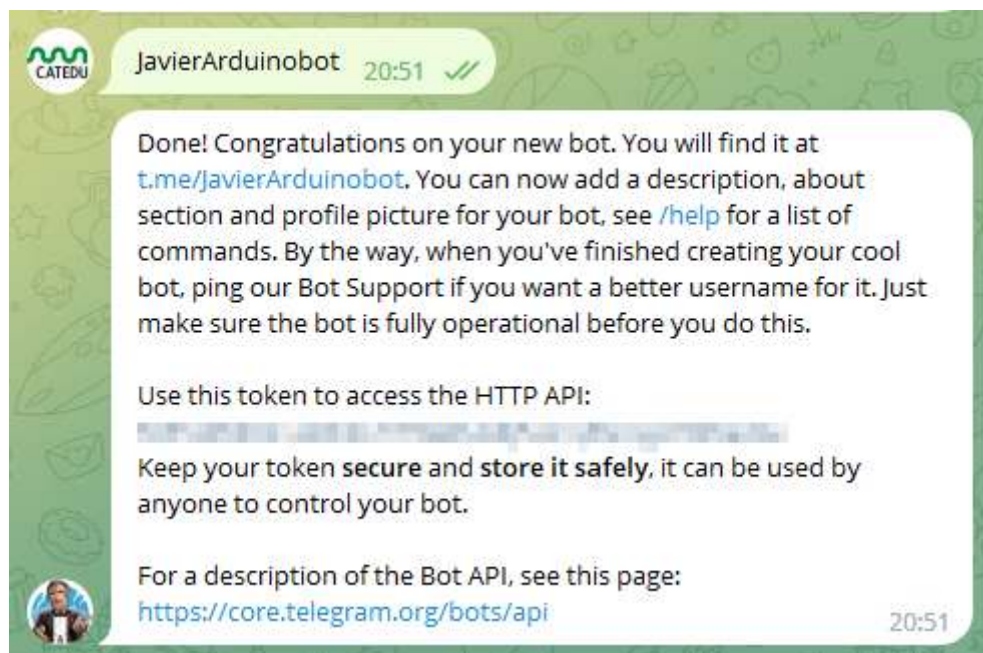
Si tecleamos **/start** nos sale las diferentes opciones



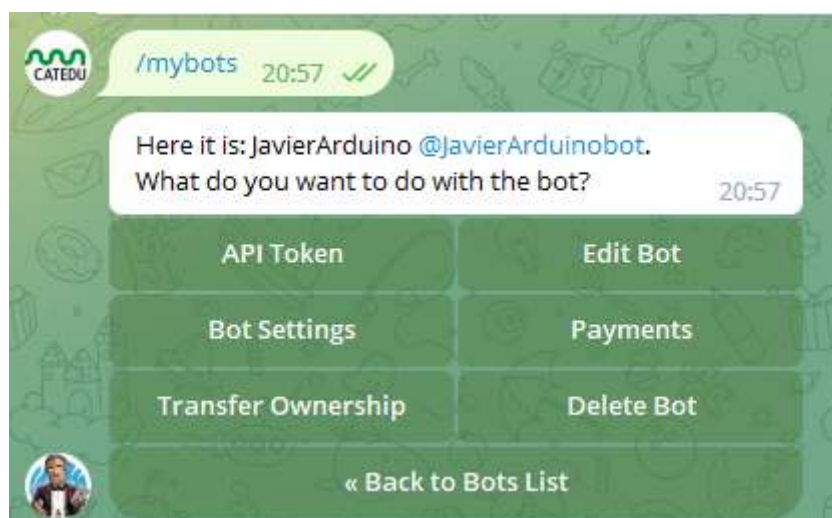
Para crear un nuevo bot, tecleamos **/newbot** y nos preguntará el nombre del bot

Por cierto, el nombre tiene que acabar con las letras **bot**,

NOS PROPORCIONARÁ EL **TOKEN** DEL ROBOT, QUE TOMAREMOS NOTA



Si tecleamos **/mybots** nos sale los diferentes bots creados y al pulsar en uno de ellos nos salen sus opciones



IoT-Wifi-Telegram

Encontrar tu ID en Telegram

Buscar tu ID : chat privado

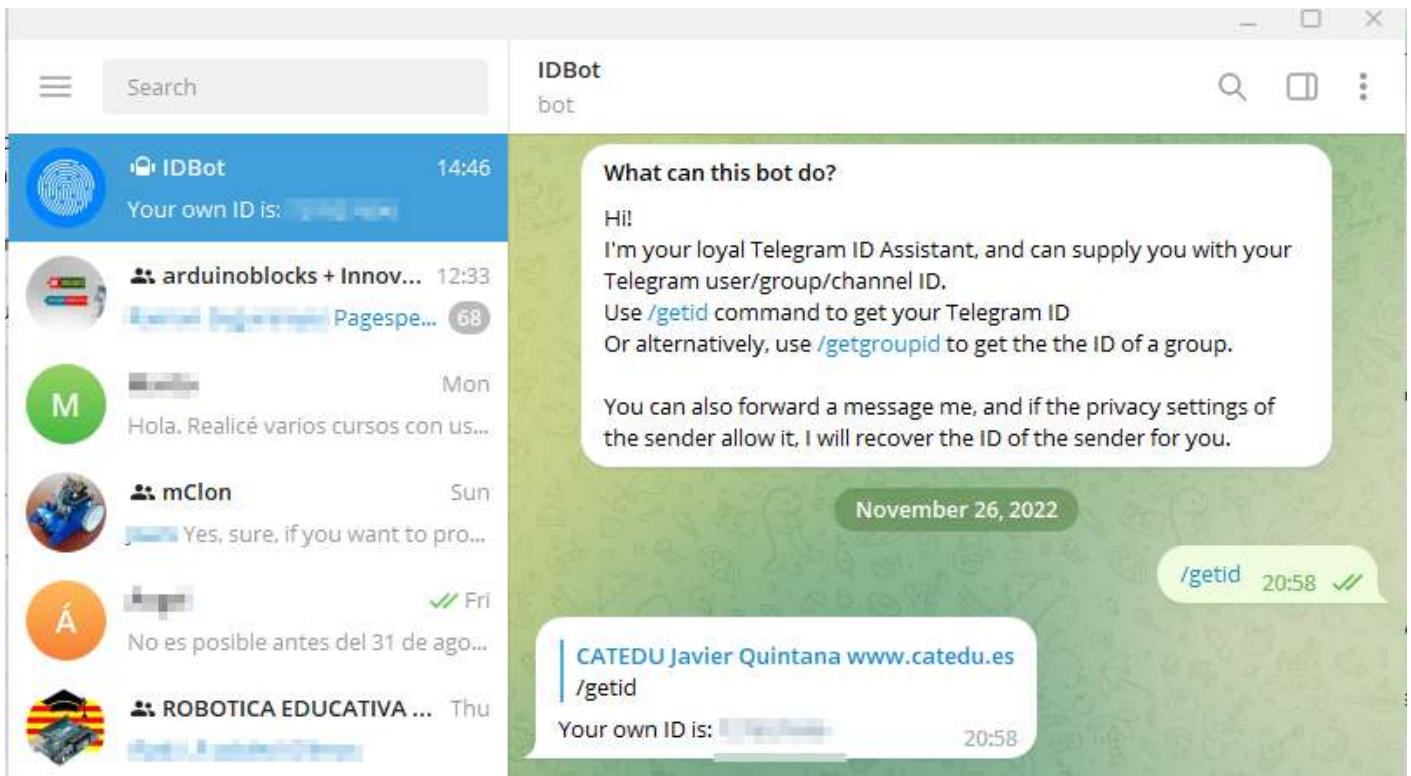
En este caso para que mi Bot me envíe mensajes a mi usuario de Telegram directamente busco mi ID.

Vamos a chatear con **@myidbot**



y le preguntamos por nuestro identificador con **/getid**

TOMAMOS NOTA DE NUESTRO IDENTIFICADOR ID



Buscar ID de un grupo

En este caso tendríamos que añadir a **@myidbot** al grupo y ejecutar el comando en el chat del grupo **/getgroupid** saldrá un identificador **negativo**

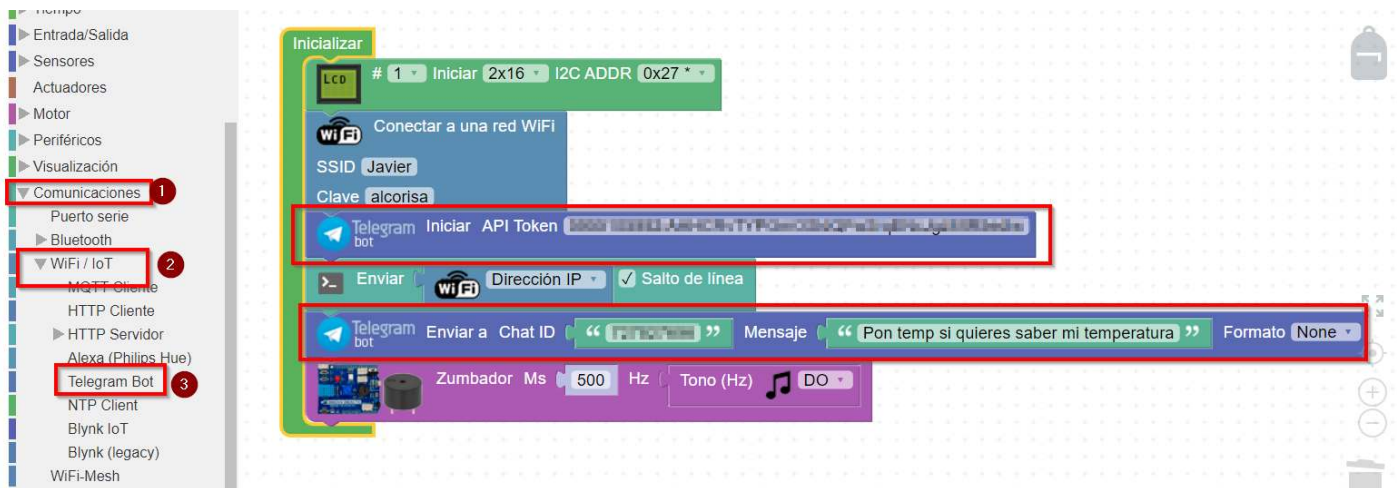
Una vez conseguido el ID podemos eliminar **@myidbot** del grupo

IoT-Wifi-Telegram

Código para enviar y recibir

Bucle Inicio

1. Lo primero es conectar con la wifi
2. Y lo segundo conectar con el **Bot** que hemos creado, gracias al **TOKEN** con la instrucción **INICIAR API TOKEN**
3. Enviamos al **ID** un mensaje de bienvenida con las instrucción **Enviar a Chat ID Mensaje ... Formato**



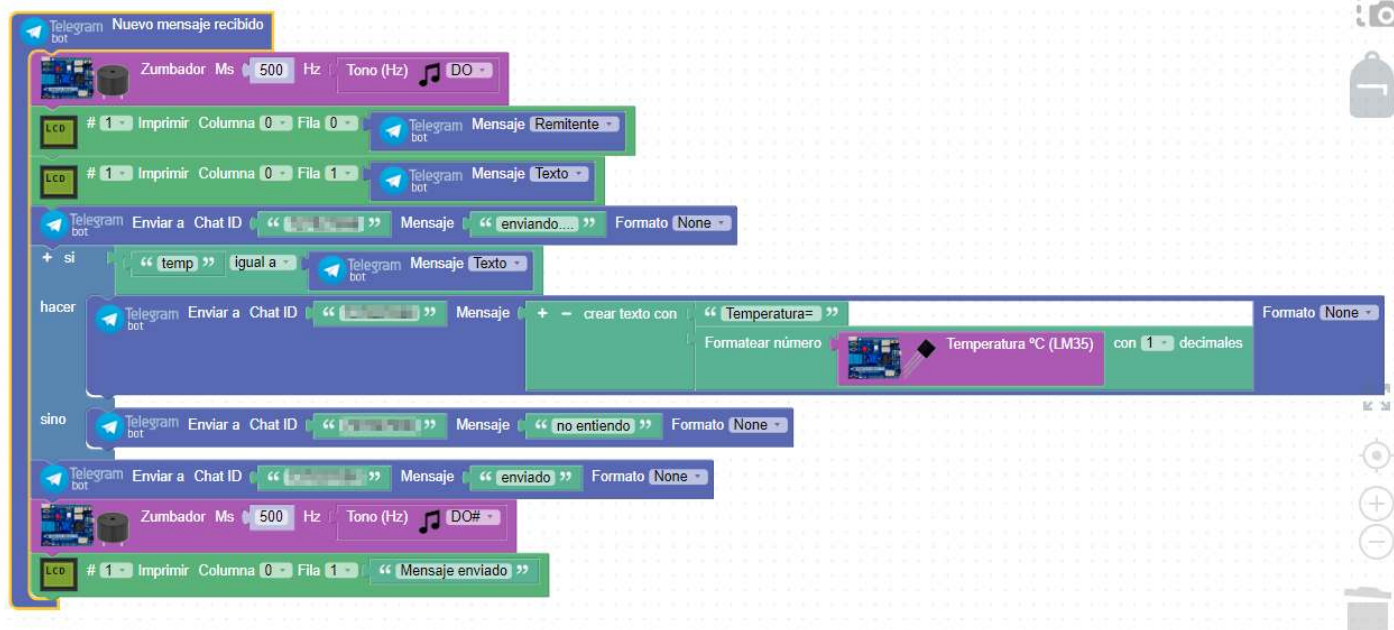
Hemos añadido algunos elementos de control, como el LCD y un zumbador

ATENCIÓN si en el mensaje a enviar pones caracteres no standard, puede ser que no funcione. Por ejemplo a mi no me ha funcionado enviar °C

Enviar y recibir

En el evento **TELEGRAM - NUEVO MENSAJE RECIBIDO** se puede utilizar el comando **TELEGRAM MENSAJE** en el siguiente programa se puede ver como en la pantalla del LCD aparecerá quien es el remitente y qué mensaje

Luego envía la temperatura si el usuario de Telegram envía la palabra *temp*



IoT-Wifi-Telegram

Actividad 17 Telegram

Este es el resultado



<https://youtube.com/shorts/Uaan2VYEHcM?feature=share>

<https://www.youtube.com/embed/Uaan2VYEHcM>

Puedes probarlo en t.me/JavierArduinobot

Amplíemos con avisos de alarma

Puedes hacer que ante un evento, te envíe un mensaje por ejemplo el pulsador 1



El resultado es:

<https://www.youtube.com/embed/-UopsDIMKsc>

Imagina que el pulsador está conectado a una ventana, entonces por Telegram te puede avisar si ha entrado un ladrón

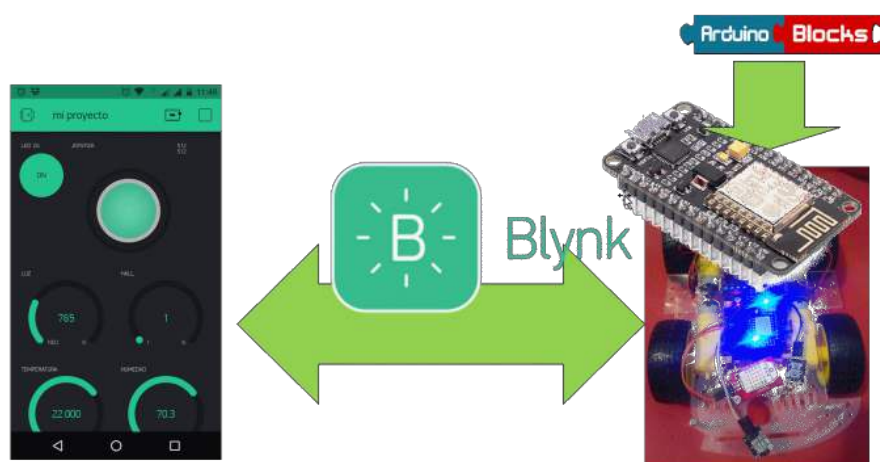
RETO AVISO TEMPERATURA ELEVADA Suponte que tienes vacunas en una nevera, y quieres se te avise de forma remota que se supera un valor determinado para tomar medidas antes de que se estropeen. Haz un programa que te envíe un aviso por Telegram si la temperatura alcanza un determinado valor.

IoT-Wifi-Blynk IoT

IoT-Wifi-Blynk IoT

¿Qué es Blynk.io?

Blynk nació como software libre como **Blynk legacy** y en Catedu aprovechamos este servicio para realizar [Rober Marciano con Arduino](#). La idea era sencilla: Programabas en **STEAMAKERBLOCKS** y el robot se comunicaba con el servidor **Blynk legacy** y con escritorio **Dashboard** via web (**Panel Web**) o APP BLYNK ponías **Gadgets** y controlabas el robot:



En junio 2022 Blynk legacy deja de dar servicio y pasa a [Blynk.io](#), una plataforma de pago pero que tiene **Gadgets** gratuitos suficientes para nivel educativo

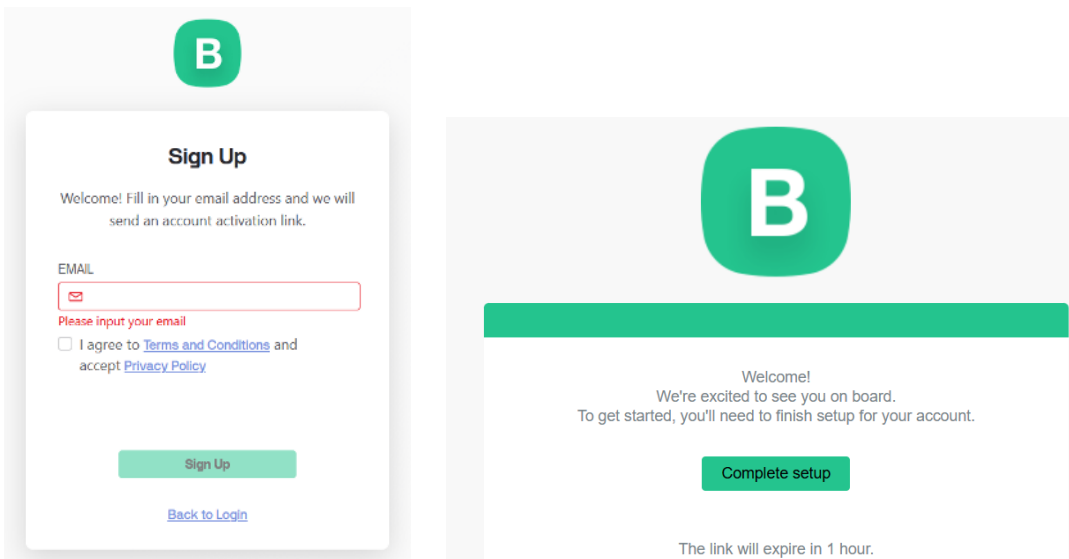
Sabías que...? Blynk legacy al ser software libre te lo puedes descargar y montar tu propio servidor Blynk legacy con lo que tendrías recursos y gadgets ilimitados. Esto lo trataremos en un capítulo aparte.

Registro en Blynk.io

Entramos en la página <https://blynk.io/> de y nos registramos en LOG IN



Nos pide un email. Cuando lo ponemos, nos llega este mensaje que tenemos que dar al botón "Complete setup". Nos saldrá un diálogo para elegir la contraseña, una pequeña encuesta



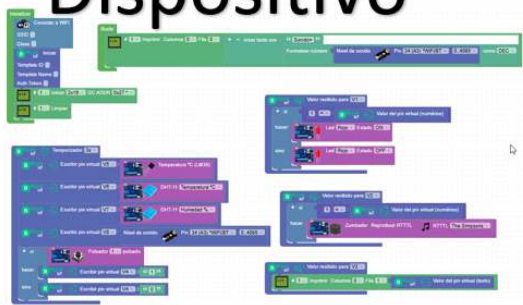
CONCEPTOS CLAROS

- En BLYNK se crea el **Dispositivo o Device** ESP32 que contendrá el programa realizado en **STEAMAKERBLOCKS** y en el programa se insertan estos datos que nos proporciona BLYNK :
 - ID DE LA PLANTILLA
 - NOMBRE DE LA PLANTILLA
 - TOKEN
- Se definen en BLYNK las variables de flujo, **flujo de datos o Datastream** que son **variables virtuales** V1 V2, tú en tu programa de STEAMAKERBLOCKS almacenas o recoges la información
- Y finalmente nuestro objetivo: un **Panel Web o Dashboard** que es un panel donde controlas online tu ESP32 y cada **gadget** que pones esa asociado a una variable virtual



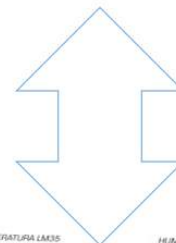
Device

Dispositivo



LED ROJO	V1	Entero
SIMPSON	V2	Entero
LCD	V3	Cadena
PULSADOR	V4	Entero
TEMPERATURA LM35	V5	Entero
TEMPERATURA DHT11	V6	Entero
HUMEDAD	V7	Entero
SONIDO	V8	Entero

Flujo de datos
Datastreams



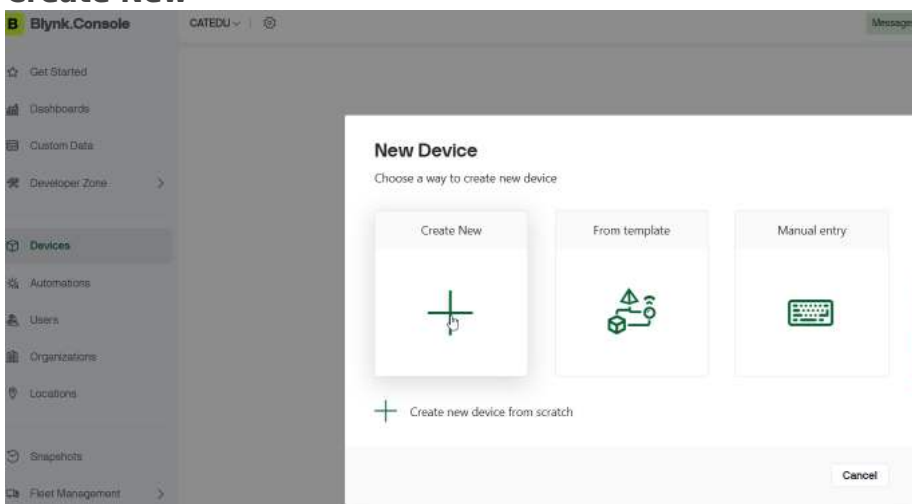
IoT-Wifi-Blynk IoT

Crear DASHBOARD en Blynk.io

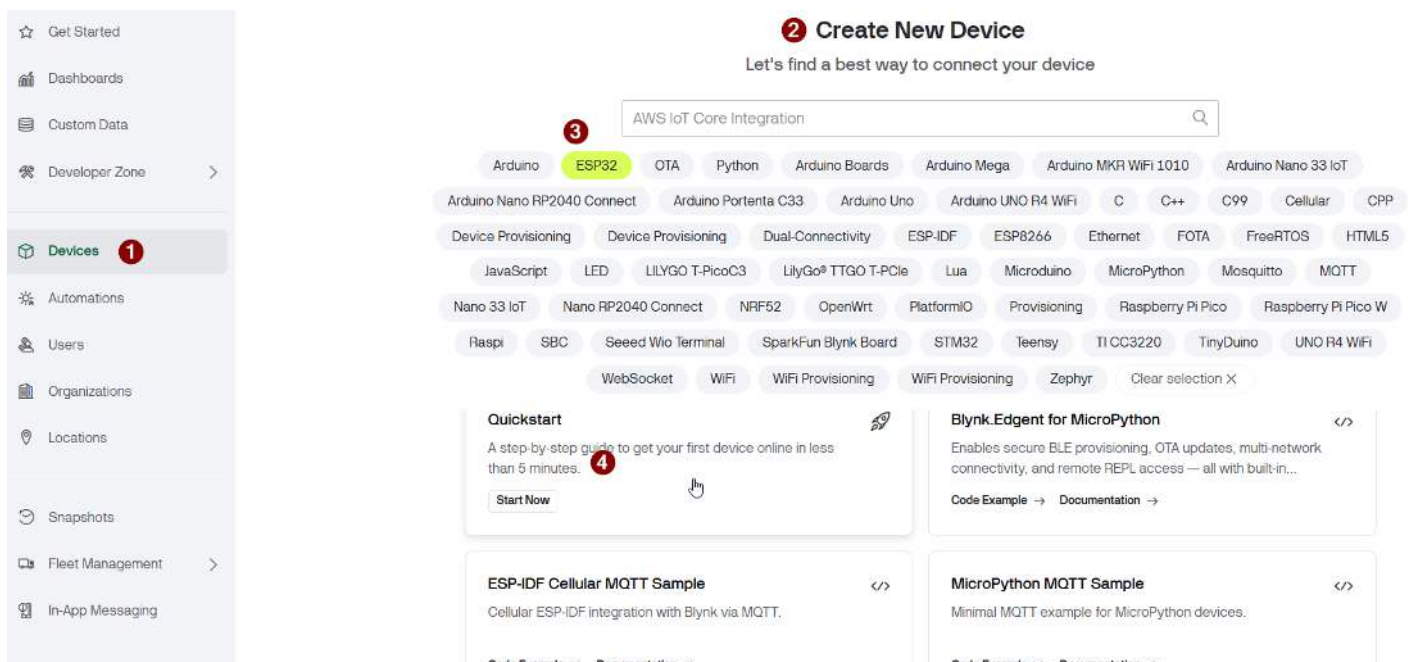
Vamos a crear un DASHBOARD o PANEL DE CONTROL para controlar nuestro ESP32.

AÑADIR DEVICE

1. Vamos a **Devices**
2. **Create New**



3. Buscamos **ESP32**
4. Elegimos **Quick start**



LED ROJO	V1	entero		0	1
SIMPSON	V2	entero		0	1
LCD	V3	CADENA			
PULSADOR	V4	entero		0	1
TEMPERATURA LM35	V5	entero	°C	0	100
TEMPERATURA DHT11	V6	entero	°C	0	100
HUMEDAD	V7	entero	%	0	100
SONIDO	V8	entero		0	4095

Ves añadiendo las variables que quieras visualizar o gobernar, dándoles las unidades, valores min, max correspondientes

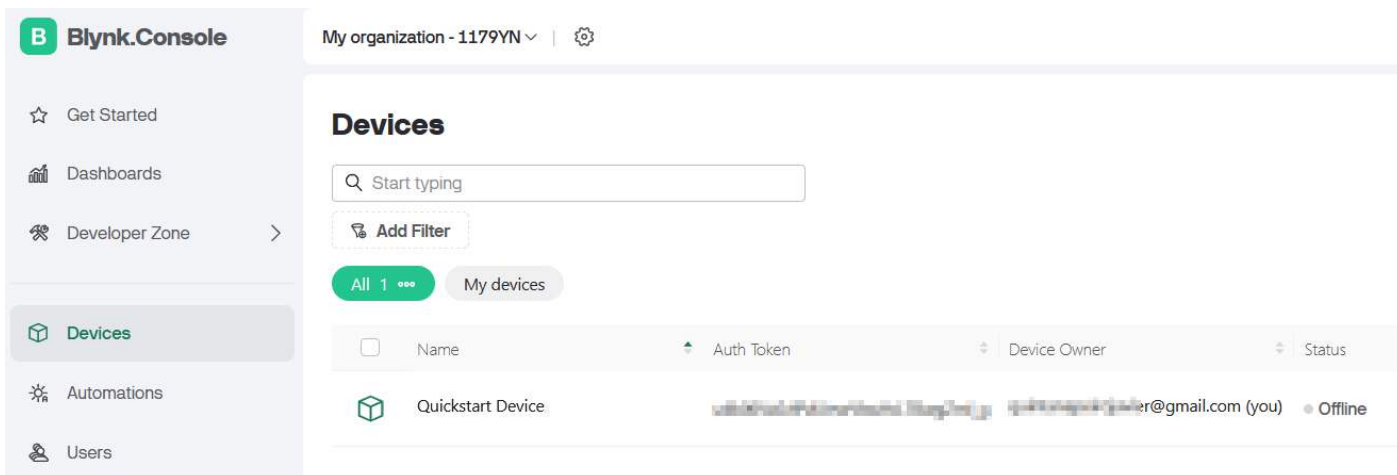
No utilices ni el potenciómetro ni el sensor de luz, tal y como vimos en Sensores y actuadores de la placa Imagina TDR STEAM, cuando hay comunicación Wifi, están inutilizados.

The screenshot shows the Blynk Console interface. On the left sidebar, the 'Zona de desarrollador' (Developer Zone) is highlighted with a red box. In the main area, the 'Flujos De Datos' (Data Flows) section is active, displaying a table of data flows. A red box highlights the 'Nuevo Flujo De Datos' (New Data Flow) button in the top right corner. A red arrow points from the 'Flujos de datos' section to the 'Nuevo Flujo De Datos' button.

ID	Nombre	Pin	Color	Tipo de dato	Unidades	Escala	Min	Max	Decimales	Valor predeterminado	Tipo de automatización	Condición	Acciones
8	LED ROJO	V1	Red	Entero		False	0	1	--	0	Interruptor	On	
7	SIMPSON	V2	Yellow	Entero		False	0	1	--	0	Interruptor	On	
4	LCD	V3	Green	Cadena		False			--	TEXTO EN LCD	Color	On	
6	PULSADOR	V4	Blue	Entero		False	0	1	--	0	Interruptor	On	
1	TEMPERATURA LM35	V5	Orange	Entero	°C	False	0	100	--	0	Interruptor	On	
3	TEMPERATURA DHT11	V6	Orange	Entero	°C	False	0	100	--	0	Interruptor	On	
2	HUMEDAD	V7	Blue	Entero	%	False	0	100	--	0	Interruptor	On	
5	SONIDO	V8	Blue	Entero		False	0	4095	--	0	Interruptor	On	

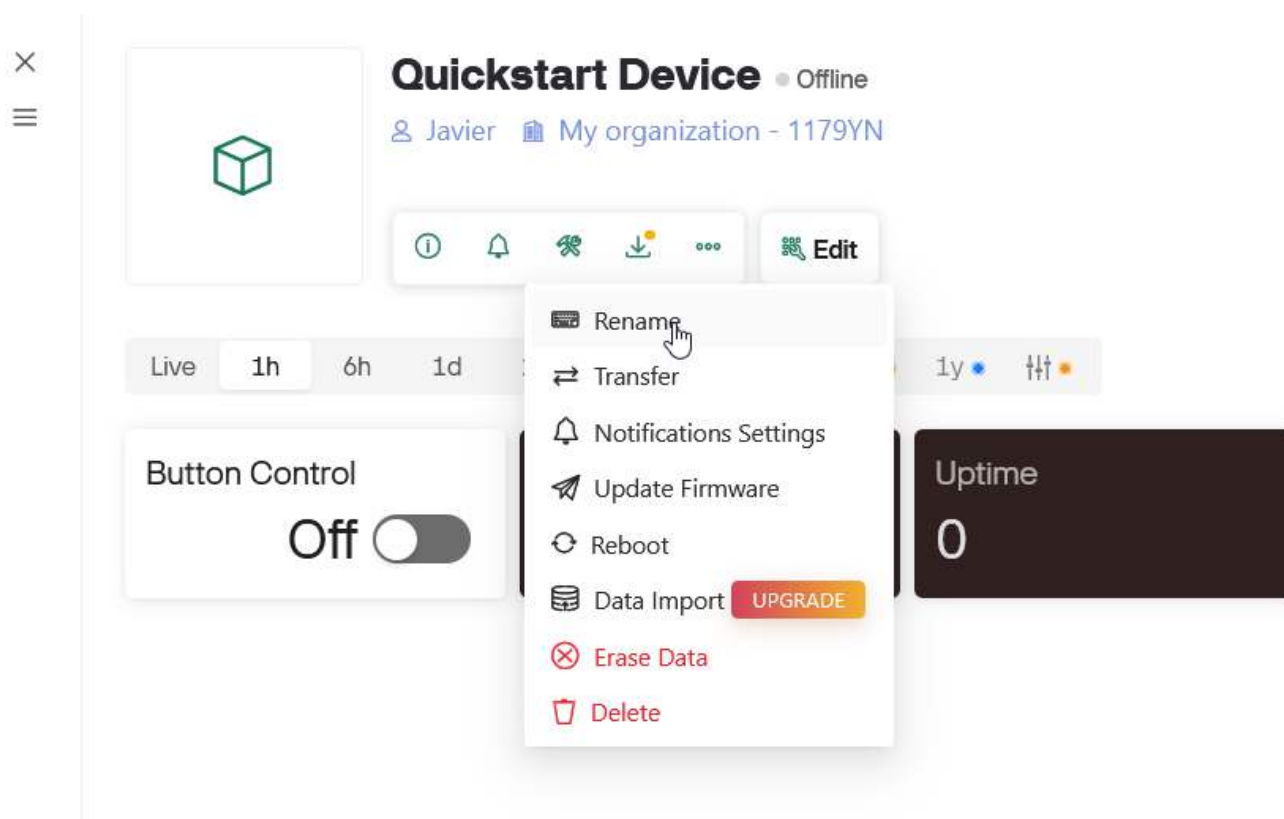
TEMPLATE PANEL WEB

Una vez creado el **Device**, le damos dos clicks

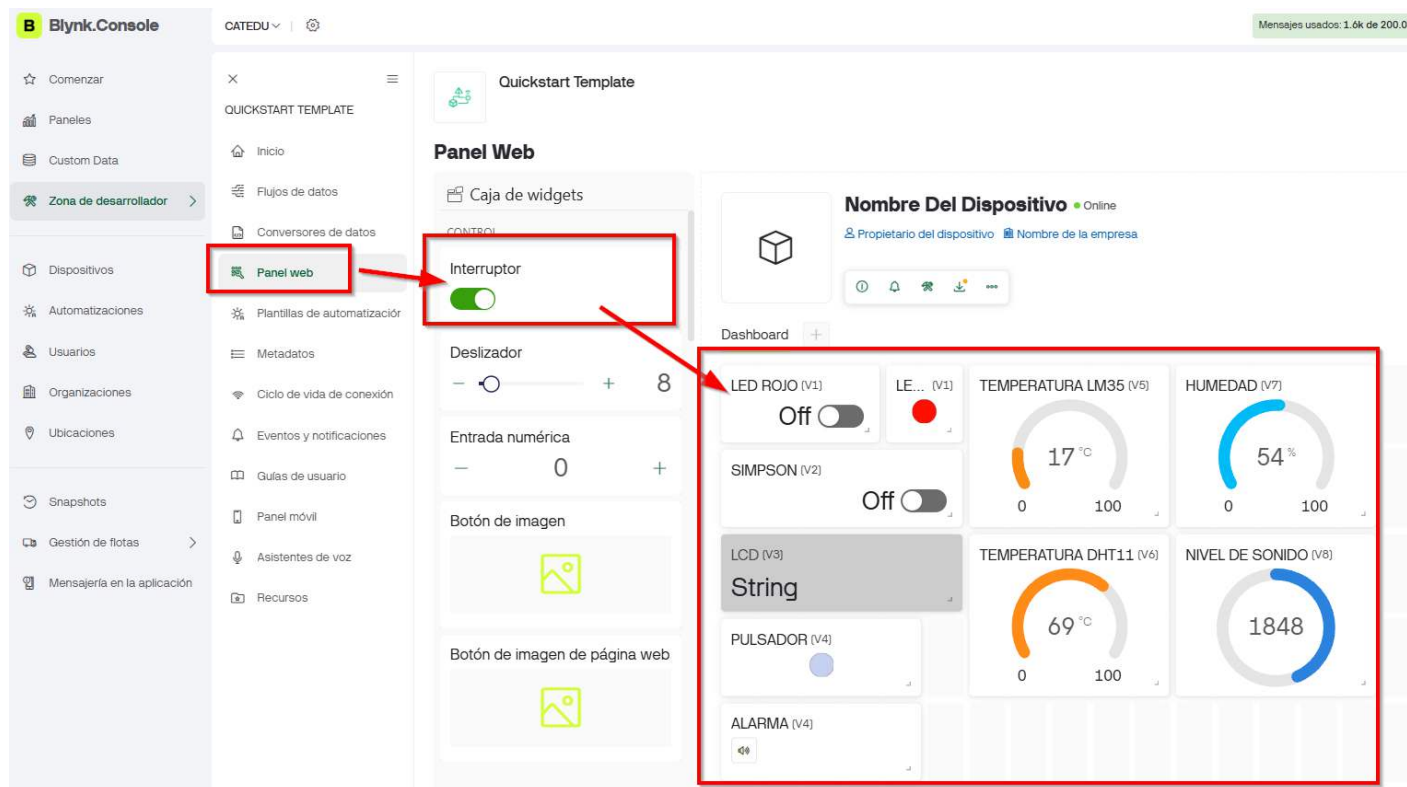


Si le damos dos clicks sale el **Template**

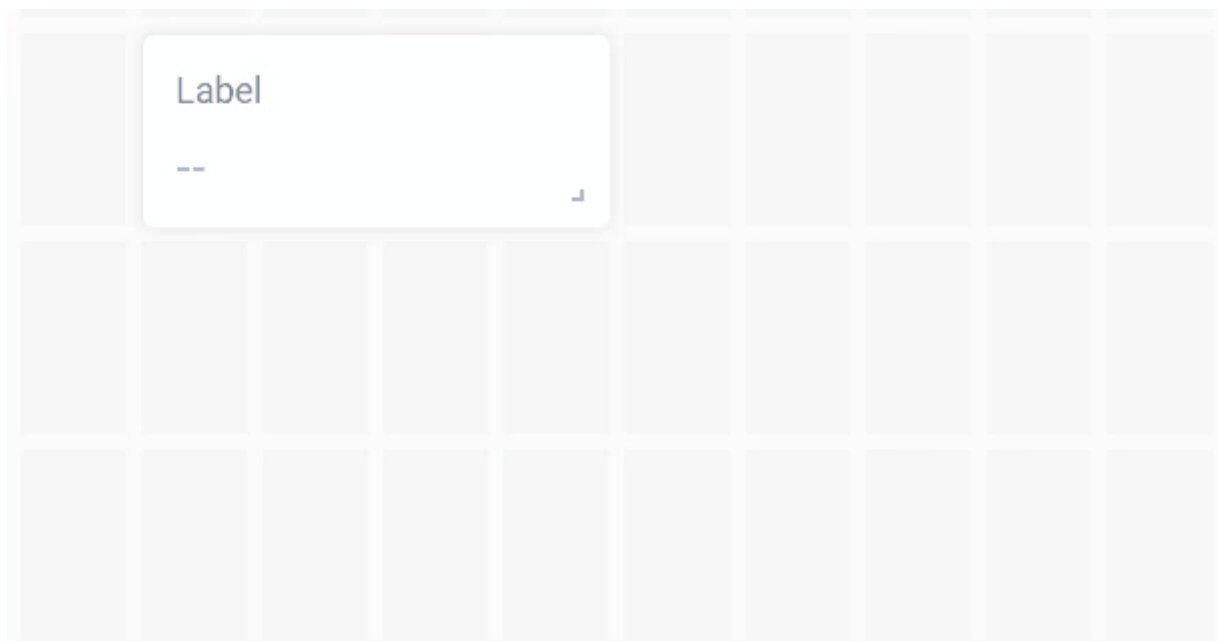
- En los tres puntos horizontales le podemos cambiar el nombre
- En el Edit podemos poner nuestros elementos



Vamos a ir añadiendo **WIDGETS ASOCIADOS A ESOS DATASTREAMS** para crear tu panel de control, algunos son de pago ☐



Los puedes redimensionar, borrar, etc... En el botón de la rueda dentada vas asociando el Widget con el pin virtual V1, V2, ... V8



Fuente <https://blynk.io/>

APP BLYNK IOT

Hay una APP sencilla, que al loguearse YA APARECE EL DEVICE lo único que hay que hacer es el TEMPLATE asociado, y es muy sencillo

Blynk IoT

Blynk Technologies Inc.
Compras en la aplicación

Aplicación personalizada a velocidad sin código



Instalar en más dispositivos

B Blynk
We make Internet of Things simple



Podemos visualizar y crear las plantillas en el móvil, para ello sigue las instrucciones

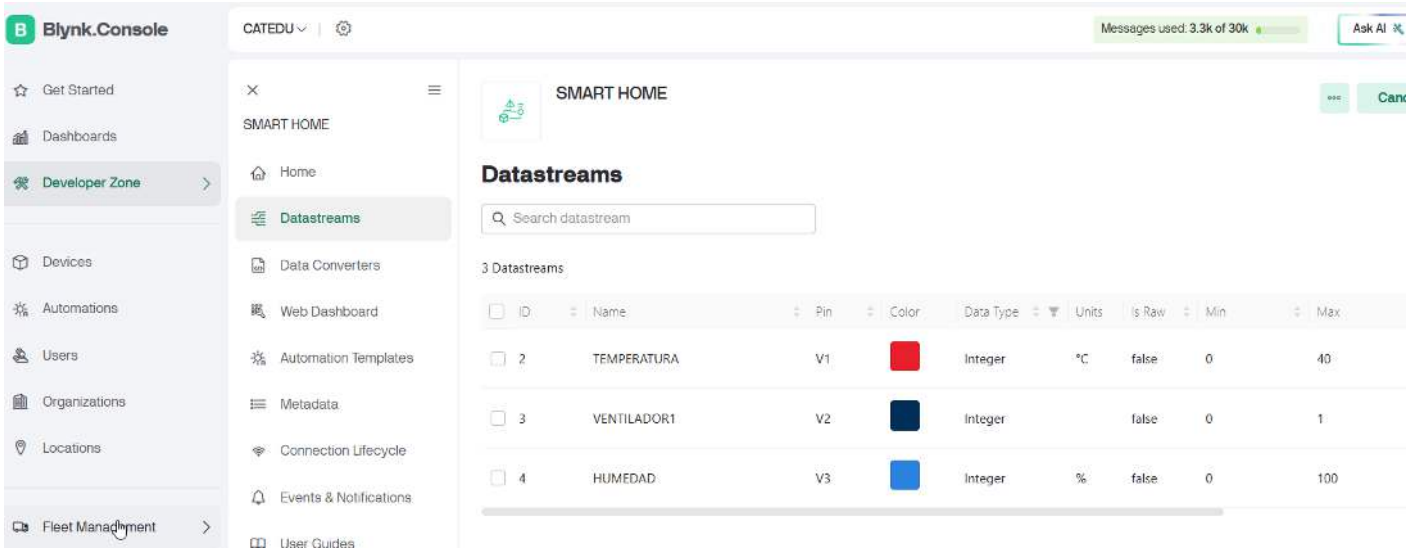
CASO ESP32 SMART HOME



ESP32 SMARTHOME he puesto de pines virtuales

- **TEMPERATURA** asociado al pin virtual **V1** entero unidades °C mínimo 0 máximo 100 (por ejemplo)

- VENTILADOR asociado al pin virtual V2
- HUMEDAD asociado al pin virtual V3



SMART HOME

Datastreams

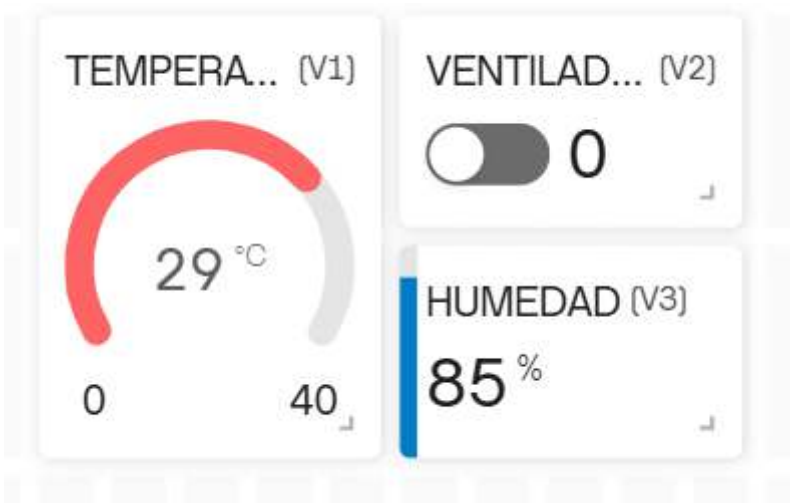
Search datastream

3 Datastreams

ID	Name	Pin	Color	Data Type	Units	Is Raw	Min	Max
2	TEMPERATURA	V1	Red	Integer	°C	false	0	40
3	VENTILADOR1	V2	Dark Blue	Integer		false	0	1
4	HUMEDAD	V3	Blue	Integer	%	false	0	100

Podemos poner multitud de widgets, vamos a simplificar con

- ESP32 al Template: Temperatura y humedad
- Template al ESP32 un botón para encender y apagar el ventilador



TEMPERA... (V1)

29 °C

0 40

VENTILAD... (V2)

0

HUMEDAD (V3)

85 %

IoT-Wifi-Blynk IoT

Crear programa en STEAMAKERBLOCKS Actividad18 Blynk IoT

Programa Steamakerblocks: Al inicializar...

Tiene que cargar los siguientes datos de la Wifi y de la conexión con Blynk.io

- El nombre de la red wifi y su contraseña para conectarse
- El **ID de la plantilla** creada
- El nombre del **Device**
- El **token** para dar acceso a esa plantilla y device

Si no te acuerdas de los tres últimos, mira más abajo cómo puedes localizarlos

De paso hemos inicializado el LDC para que nos muestre datos



A programar de Dashboard ->ESP32.

Vamos a hacer que si se pulsa en el dashboard el interruptor asociado al pin virtual V1 que se encienda el led rojo



Luego si se pulsa en el dashboard el pin 2 que suene los simpsons ☐☐

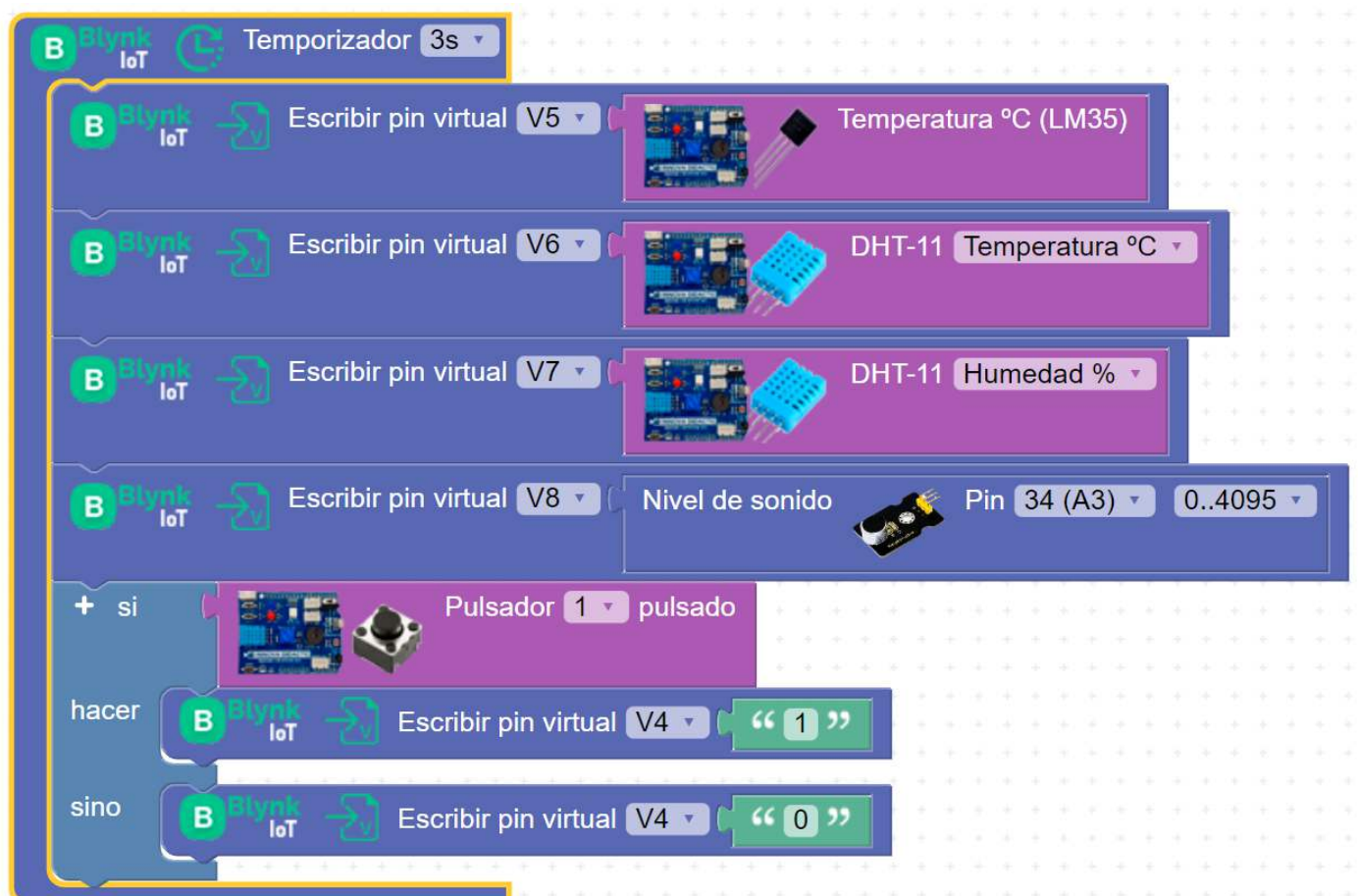


Luego si se introduce texto en V3 que en la pantalla LCD lo diga

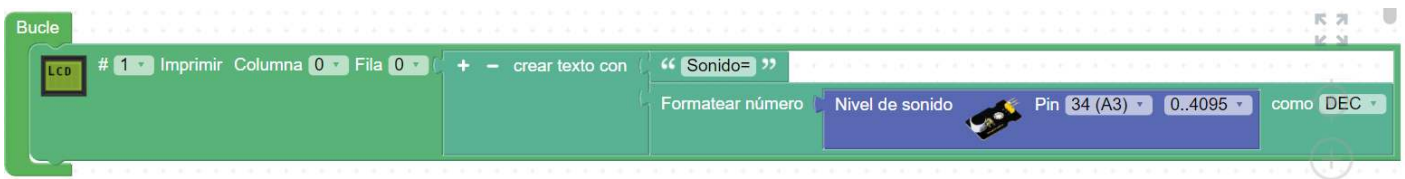


A programar de ESP32 ->Dashboard.

Vamos a crear un programa que vaya recogiendo los valores de ESP32 cada 3 segundos (para no agobiar al servidor Blynk) y los vuelque al Dashboard



- V4 tiene los valores 0 o 1 según el valor del pulsador 1 de la placa TDR SteaMaker por lo tanto enciende el LED del dashboard de Blynk.io si se pulsa.
- V5 capta la temperatura del LM35, da un valor más exacto que es que visualiza el DHT11
- V6 copia la temperatura del DHT11
- V7 tiene los datos de la humedad del DHT11
- V8 registra los datos del nivel de sonido del sensor micrófono incorporado y conectado en el A3
 - Para ver que realmente los valores de V8 no siguen fielmente a los reales, vamos a poner en el bucle esta instrucción para que visualice los valores de sonido registrados:



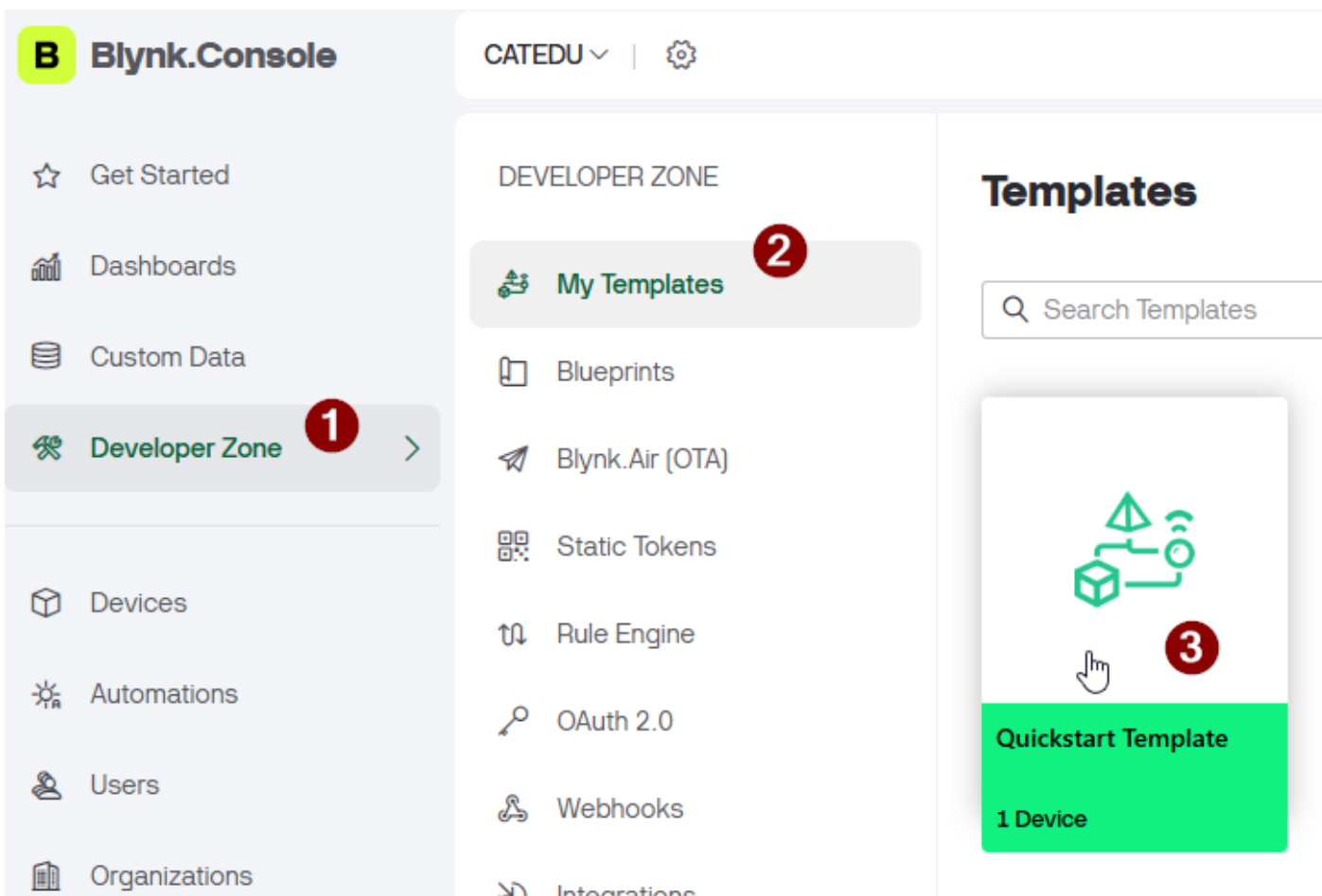
ESTE ES EL RESULTADO

<https://www.youtube.com/embed/nCK1QpBMfLI>

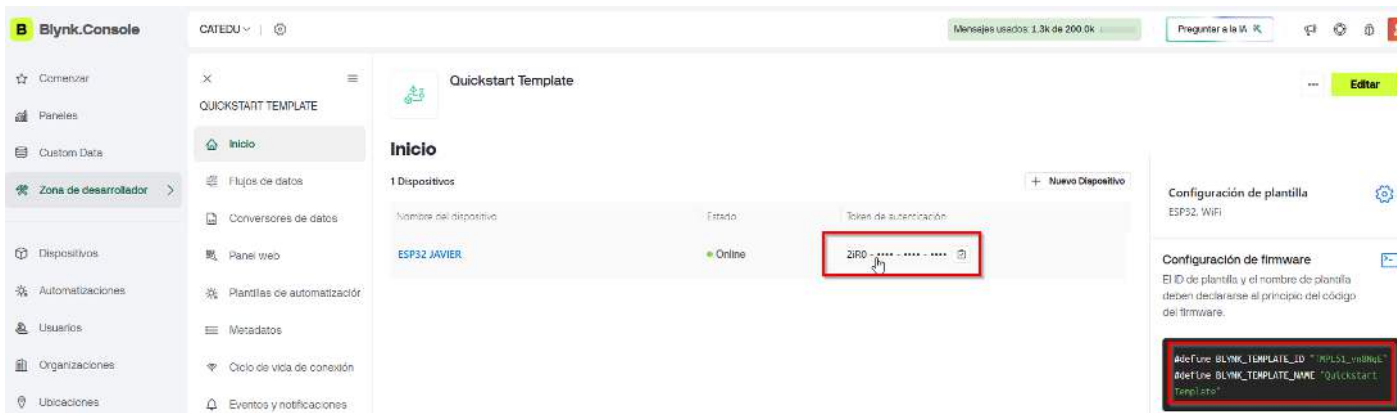
No me acuerdo de estas tres cosas: TOKEN -ID PLANTILLA - NAME PLANTILLA ¿Dónde puedo encontrarlo?

Primero Entrás en [Blynk.io](https://blynk.io) y Te logueas **Log in** y luego :

1. Entrás en **Developer Zone/Zona de desarrollador**
2. **My Templates/Mis plantillas**
3. Haz dos clicks en la **plantilla**



- Con dos clics en **Authtoken** ya se queda copiado en el portapapeles para que lo pegues en tu programa **STEAMAKERBLOCKS**
- Más a la derecha hay una ventana Firmware configuration/Configuración de firmware donde puedes encontrar el ID de la plantilla y su nombre: Copiar y pegar el trozo de código, pegarlo en un editor y copiar sólo el **ID** y el **NAME** para pegarlo en tu código **STEAMAKERBLOCKS**



Avanzado: Blynk Legacy local

Avanzado: Blynk Legacy local

Raspberry: Qué hay que hacer para empezar

Lo primero que tenemos que conseguir es:

- Tener una Raspberry con el sistema operativo propio de la raspberry instalado, antes se llamaba Raspbian, ahora [Raspberry Pi OS](#)
- Comunicarnos con la Raspberry por comandos SSH
- Una vez hecho esto que es lo básico instalaremos el servidor Blynk



Ahora vamos a ver unas páginas para

- Instalar un **sistema operativo** en una Raspberry
- Una vez instalado hay que **conectarlo a la red wifi y ponerle una IP FIJA**
- Para **comunicarse vía comandos SSH**, aconsejamos hacerlo de forma remota, pues lo contrario implica tener siempre una pantalla, teclado y ratón conectado en la Raspberry

¿Qué Raspberry habría que comprar?

Aconsejamos una que tenga la Wi-fi incorporada, nosotros hemos hecho pruebas con la

Raspberry Pi 3B+ y con la Raspberry Pi4 y no encontramos diferencias de velocidad para estos propósitos de robótica, luego por la mitad de precio aconsejamos la Raspberry Pi 3B

Avanzado: Blynk Legacy local

Raspberry: Poner un Sistema Operativo

¿Es el único sistema operativo que se puede instalar?

No, podemos instalar otros sistemas operativos en nuestra Raspberry (evidentemente uno sólo cada vez, poner dos a la vez en la *rasp* sería forzarlo demasiado). Por ejemplo:

- Podemos instalar un [Linux-Ubuntu](#) ligero pero que se puede usar en una aula de informática ([ver libro del curso de Aularagon Monta tu aula de informática con Raspberry](#)).
- Si no podemos vivir sin Windows, hay un [Windows 10 especial](#) gratis (*sí has leído bien, los de Microsoft van entendiendo eso de si no puedes con ellos únete a ellos, pero no te hagas ilusiones, sólo vale para la Raspberry y no esperes una máquina que sustituya a tu PC va lentísimo*).
- Sistemas operativos para aplicaciones concretas:
 - Video juegos: [Recalbox](#).
 - Dómótica y control [Home Assitant](#)
- Nuestro sistema operativo de este curso (y el mejor que "rula en la Rasp") pues está adaptado a la arquitectura ARM de la Raspberry es **RASPBIAN** o actualmente lo llaman **RASPBERRY PI OS**

Hay dos opciones, elige la que quieras (nuestra preferida es la A):

Opción A Instalar el sistema operativo en la tarjeta sin NOOBS

Con esta opción no tienes que encender la Raspberry.

https://docs.google.com/presentation/d/e/2PACX-1vQoxDLLHMvB-mCQwm2en9cBgb1faamFG0YJliFDFuNrGH8TuH8U-4zCDg_K9CkM4gFI-Wy6TxBNLg9j/embed?start=false&loop=false&delayms=3000

Opción B Instalar la Raspbian en la tarjeta con NOOBS

Con esta opción SI que tienes que encender la Rasperry con pantalla, teclado y ratón

https://docs.google.com/presentation/d/e/2PACX-1vR4wwk8BwdSEgk-aTQ_xgvrQYJ-cr2QhN_35Q-mYJxYedhT5P-vF6UkDbSsHJ5I_zLS7IqBirdNfl4t/embed?start=false&loop=false&delayms=3000

Esto es válido para cualquier ordenador: Descargar la ISO de un Sistema Operativo, grabarlo en un pincho de forma autoarrancable, pero por curiosidad, no es tan fácil como en la RASPBERRY
Se añade esta presentación como curiosidad:

https://docs.google.com/presentation/d/e/2PACX-1vSvr4JxqRNkTwl0mcFJdfble5BtDpSGyLO4ucyAyk65f3zXsFa3zxlyFZiPcqRuv_2YEfrGY39SFVi0/embed?start=false&loop=false&delayms=3000

Avanzado: Blynk Legacy local

Raspberry: Cambiar usuario y contraseña

El usuario por defecto es **pi** y su contraseña es **raspberrypi** pero lo podemos cambiar perfectamente

Opción A remota con SSH

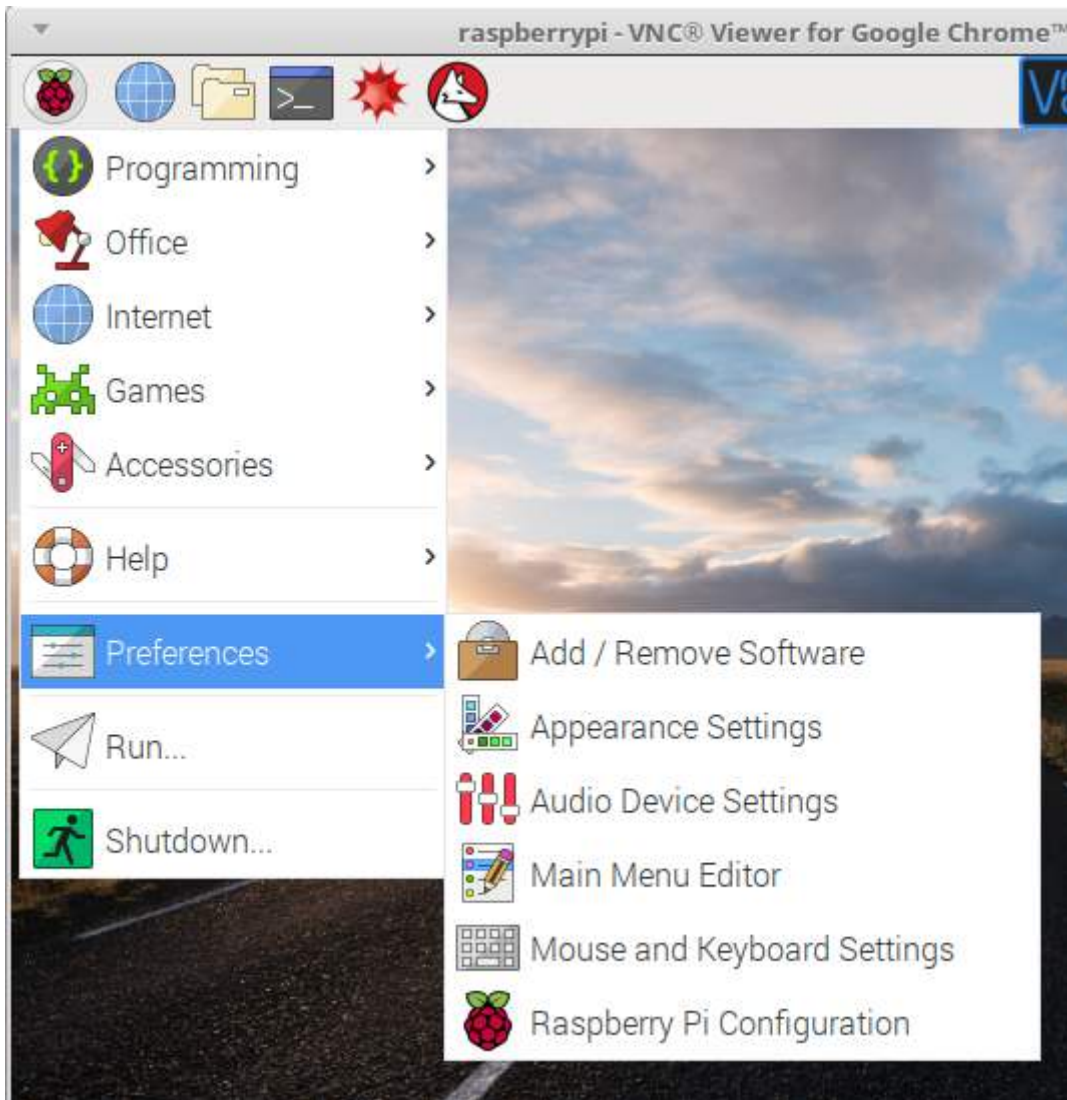
Entramos por SSH [como hemos aprendido](#), y tecleamos

```
sudo passwd pi
```

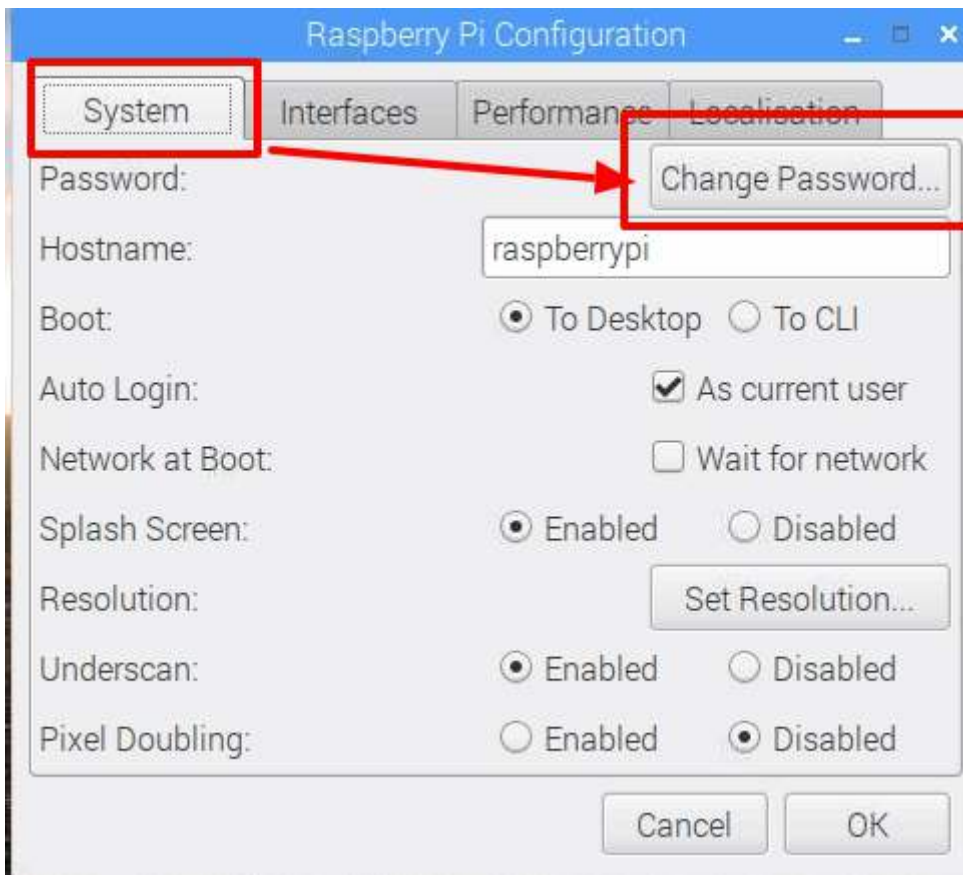
y nos pide enseguida la contraseña, la tenemos que insertar dos veces *no se puede ver*.

Opción B local por pantalla

Entramos por VNC o conectamos una pantalla, teclado y ratón a la raspberry y en **Preferencias-RaspberryPi configuration**



Y entramos en **System**



Avanzado: Blynk Legacy local

Raspberry: Conectarlo a la red

Opción A La Raspberry con pantalla, teclado y ratón

Esta es más fácil, pero tienes que encender la Raspberry con teclado, pantalla y ratón:

https://docs.google.com/presentation/d/e/2PACX-1vQShQenHWgn0_axlCmlKplX6kqQOHHzaIHhy7SxTtXjLOXZip40JO5B-UB7KRx6g1Z8M1LsUg2N8fi8/embed?start=false&loop=false&delayms=3000

Opción B La Raspberry sin pantalla, teclado y ratón

Con esta opción es **más difícil**, pero no tienes que encender la Raspberry para la configuración, sólo manipular los ficheros de la tarjeta micro SD.

<https://docs.google.com/presentation/d/e/2PACX-1vSuE389RRI67Vi1LGxySj5rWjhNMjV9t27FbO80BOqMqDjyKcH7hFZWdOXubwsuO8NtnwXtaTQfg4p/embed?start=false&loop=false&delayms=3000>

Texto que hay que poner en interfaces

```
auto wlan0  
iface wlan0 inet static
```

```
address 192.168.1.xxx
gateway 192.168.1.1
netmask 255.255.255.0
wpa-ssid el_nombre_tu_red_wifi
wpa-psk la_contraseña_de_tu_wifi
dns-nameservers 8.8.8.8 188.244.82.1
```

Texto que hay que poner en wpa_supplicant.conf

```
# /etc/wpa_supplicant/wpa_supplicant.conf
ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev
update_config=1
network={
    ssid="nombre de tu router o SSID"
    psk="tu contraseña del wi-fi"
    key_mgmt=WPA-PSK
}
```


Avanzado: Blynk Legacy local

Raspberry: Conexión SSH

Controlar Raspberry pi de forma remota y de forma textual es muy rápido y eficaz, sobre todo para la robótica con AlphaBot.

SSH (Secure Shell) según [Wikipedia](#) es el nombre de un protocolo y del programa que lo implementa, y sirve para acceder servidores privados a través de una puerta trasera. Te recomiendo visitar [esta página](#) de Luis Llamas.

Recuerda que lo que hemos hecho es:

- Hemos conectado la Raspberry a la Wifi
- Hemos activado el protocolo SSH y VNC en la raspberry
- Hemos fijado una IP y la sabemos para poder acceder a él.

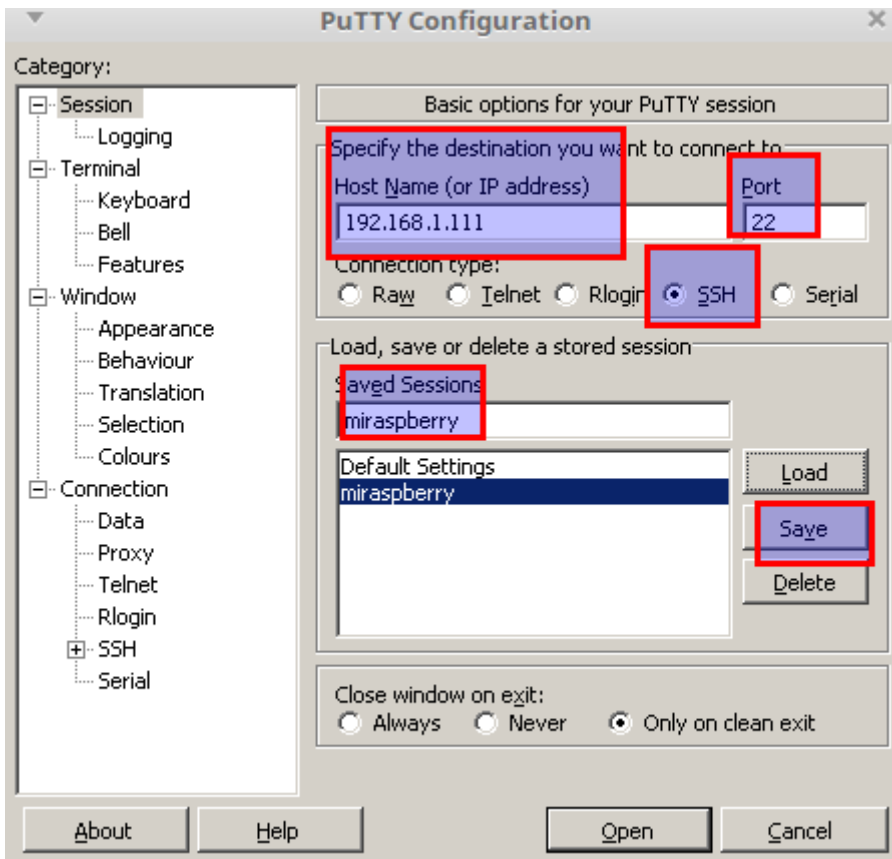
Ya podemos trabajar por SSH con la Raspberry, no nos hace falta que conectemos la raspberry a una pantalla, teclado, etc....

YA PODEMOS ENCENDER LA RASPBERRY si has elegido las opciones A de [3.-Raspbian](#) y [4.-Conectando](#) aún no habrás encendido la Raspberry.

Si trabajas con Windows

No lo tiene nativo, tenemos que instalar [Putty](#)

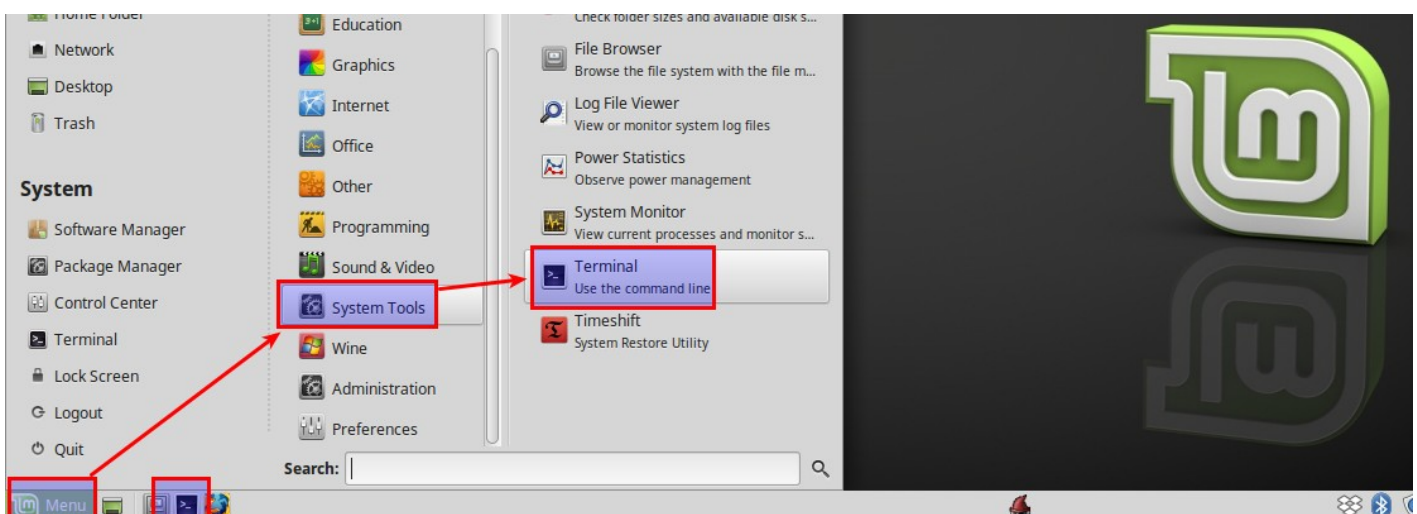
Y rellenamos los campos de la IP (la que hayas determinado, en la imagen es 192.168.1.111), puerto y el protocolo SSH, es recomendable grabar la sesión para tenerlo en futuras entradas.



Se abre una ventana pidiendo el usuario **pi** y contraseña por defecto **raspberry**

Si trabajas con Linux

Linux tiene de forma nativa el protocolo SSH, lo activamos en el icono en negro que es el **Terminal** :



Supongamos que la IP fija que le hemos asignado a la Raspberry la IP 192.168.1.131 Tecleamos

```
ssh pi@192.168.1.131
```

Te pide usuario y contraseña, por defecto es usuario pi y la contraseña *raspberry*, la tecleas **no la puedes ver**, te tiene que salir algo así en la pantalla:

Cuando te sale pi@raspberrypi: es que ya está preparado para recibir órdenes

```
cat@cat-Aspire-E5-571 ~ $ ssh pi@192.168.1.131
pi@192.168.1.131's password:
Linux raspberrypi 4.9.80-v7+ #1098 SMP Fri Mar 9 19:11:42 GMT 2018 armv7l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.

Last login: Fri Mar 23 18:03:09 2018 from 192.168.1.131
pi@raspberrypi:~ $
```

Nota por si te pasa

Si después de un intento fallido, o realizar la conexión con otra computadora.... te sale este error al intentar conectarte por ssh:

```
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ @ WARNING: REMOTE
HOST IDENTIFICATION HAS CHANGED! @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ IT IS POSSIBLE THAT
SOMEONE IS DOING SOMETHING NASTY! Someone could be eavesdropping on you right now
(man-in-the-middle attack)! It is also possible that the RSA host key has just been changed.
The fingerprint for the RSA key sent by the remote host is
Please contact your system administrator. Add correct host key in
/home/user/.ssh/known_hosts to get rid of this message. Offending key in
/home/user/.ssh/known_hosts:1 RSA host key for ras.mydomain.com has changed and you
have >requested strict checking. Host key verification failed.
```

Entonces antes de intentar conectarte por ssh, hay que obligar a la Raspberry que reinicie las claves de conexión de ssh, ejecuta esta orden :

Supongamos que la IP fija que le hemos asignado a la Raspberry en el [punto 4](#) es 192.168.1.131
Tecleamos

ssh-keygen -R 192.168.1.131

Avanzado: Blynk Legacy local

Raspberry: Apagar

Apagar de forma caliente (es decir simplemente quitando de la fuente de alimentación la corriente) daña al sistema operativo Raspbian dejando ficheros corruptos.

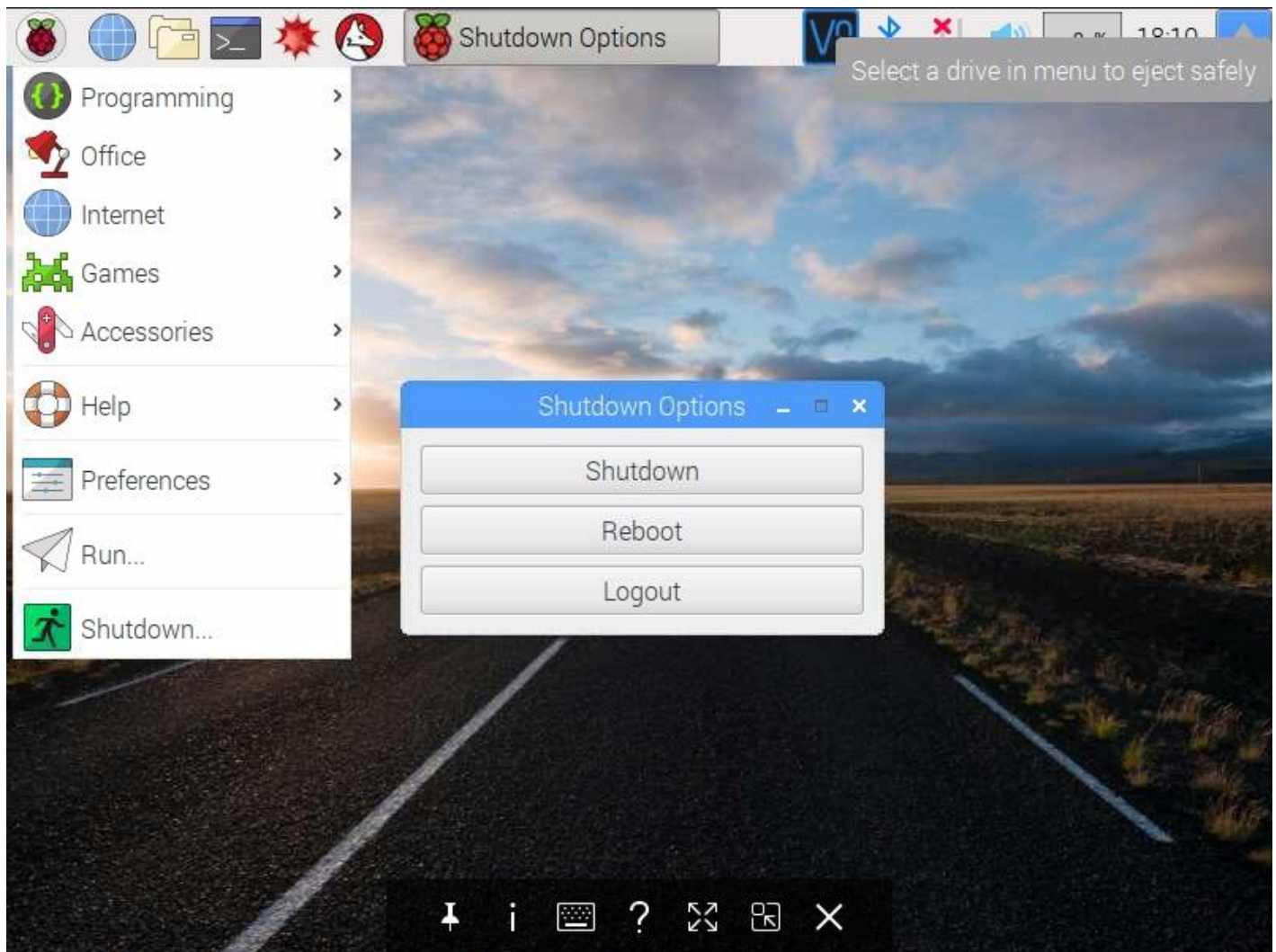
7.1 Opción de forma remota con SSH

Muy fácil, teclea

```
sudo poweroff
```

7.2 Opción local de forma gráfica

Igual que en otros sistemas operativos Windows o Linux tenemos que apagar desde el menú **Shutdown**



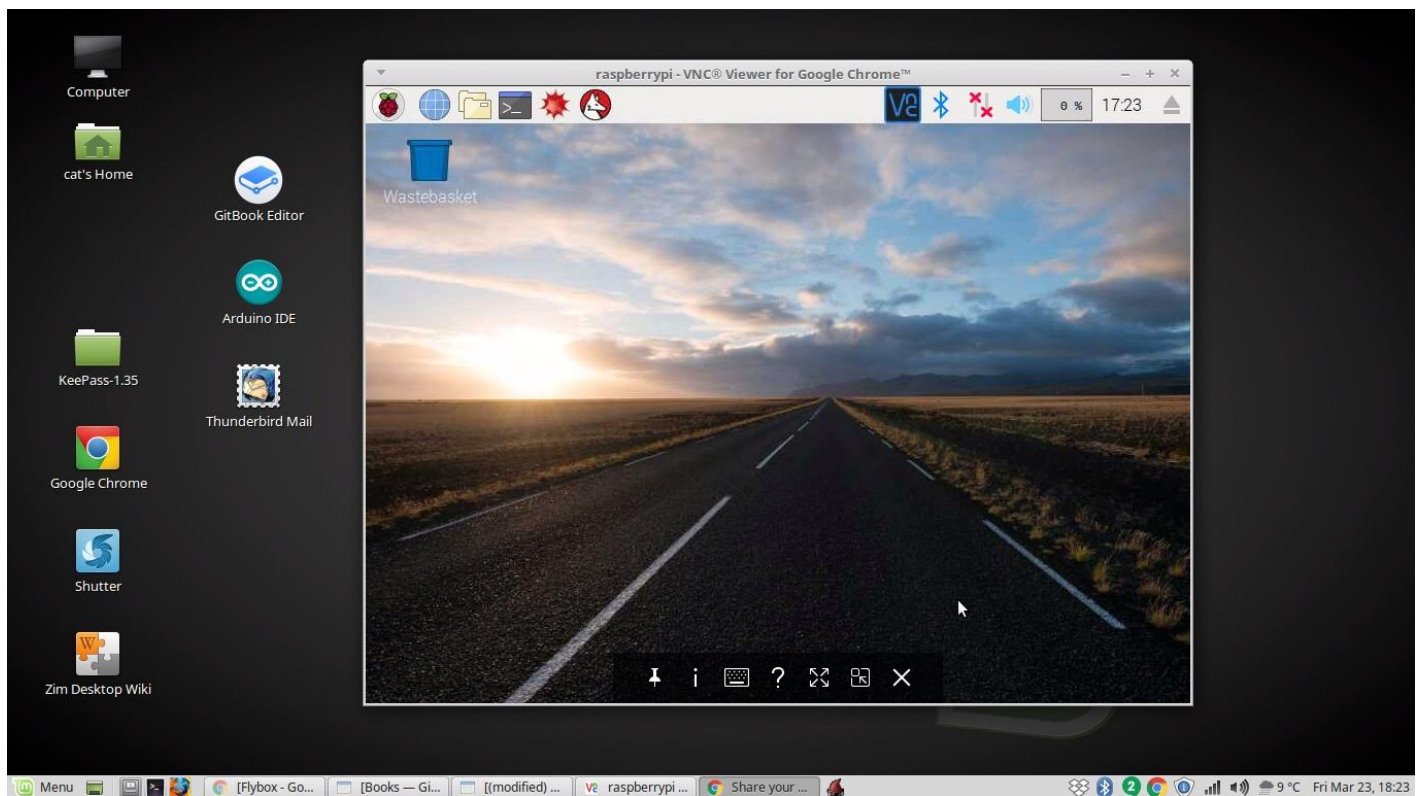
Avanzado: Blynk Legacy local

Raspberry: VNC

VNC es un programa que nos permite la conexión remota con los servidores pero de forma gráfica. Es el sistema elegido por RASPBIAN pues ya lo tiene incorporado, pero no activado.

Hay que tener claro que son **dos programas**: * **VNC SERVER** que es el programa que se ejecuta en la RASPBERRY y que aunque viene instalado, hay que activarlo. Es el programa que lanza el escritorio para que lo lea: * **VNC VIEWER** es el programa que tenemos que tener en nuestro ordenador para ver lo que lanza VNC SERVER.

En la imagen podemos ver un pantallazo de mi ordenador, el escritorio negro es mi escritorio y el de la carretera es el de la Raspberry:



Avanzado: Blynk Legacy local

Raspberry: Instalar Java 8

Una vez que tenemos la Raspberry funcionando y accesible por SSH o por VNC es el momento de instalar el servidor Blynk, primero hay que, para instalar Blynk, se necesita la **versión 8 de Java**

Seguimos las instrucciones de <https://github.com/Peterkn2001/blynk-server#blynk-server>

Entramos en la ventana de comandos SSH y ejecutamos las siguientes órdenes, si en algún momento nos pide confirmación [Y/n] es porque faltan descargar paquetes o confirmación de instalación, por lo tanto aceptamos.

Con esta orden busca las actualizaciones

sudo update

y ya ejecutamosque instala la versión 8 de java (curiosamente no funciona con la versión 64 bits de Raspberry OS sino con la de 32 bits)

sudo apt install openjdk-8-jdk

Por último una vez finalizado, comprobamos la versión que coge por defecto

java -version

Tiene que salir Openjdk version 1.8.0 etc...

Si no sale esa versión...

Suele pasar que tenga ya instalado la versión 11, tenemos que obligar a que sea la 8 por defecto, para ello ejecutamos

sudo update-alternatives --config java

```
pi@raspberrypi:~$ sudo update-alternatives --config java
Existen 2 opciones para la alternativa java (que provee /usr/bin/java).

  Selección    Ruta
-----
  0            /usr/lib/jvm/java-11-openjdk-armhf/bin/java      1111      modo a
automático
  1            /usr/lib/jvm/java-11-openjdk-armhf/bin/java      1111      modo m
anual
* 2            /usr/lib/jvm/java-8-openjdk-armhf/jre/bin/java    1081      modo m
anual

Pulse <Intro> para mantener el valor por omisión [*] o pulse un número de selecc
ión: █
```

Si seleccionamos el 2 nos lo pondrá como por defecto (sale con un (*). Podemos comprobar que ya nos sale la versión con la orden :

java -version

Pero esto no nos sirve, pues al reiniciar la Raspberry volverá a la versión 11.

Para ponerlo por defecto, edita el fichero *environment*

sudo nano /etc/environment

Y si en la orden **sudo update-alternatives --config java** salía que la carpeta donde está la versión 8 es */usr/lib/jvm/java-8-openjdk-armhf/jre/bin/java* luego tenemos que poner en el fichero *environment* :

JAVA_HOME="/usr/lib/jvm/java-8-openjdk-armhf/jre/bin/java"

Guardar, reiniciar

sudo reboot

Y comprueba

java -version

Y tiene que salir :

```
pi@raspberrypi:~/Blynk $ java -version
openjdk version "1.8.0_212"
OpenJDK Runtime Environment (build 1.8.0_212-8u212-b01-1+rpil-b01)
OpenJDK Client VM (build 25.212-b01, mixed mode)
```



Para más cosas

Por ejemplo, desinstalarlo, volver a la 11, etc.. visitar esta página :

<https://phoenixnap.com/kb/install-java-raspberry-pi>

Avanzado: Blynk Legacy local

Raspberry: Instalar y ejecutar el servidor Blynk local

1 Descarga Blynk

Crea una carpeta en tu directorio home, por ejemplo Blynk

mkdir Blynk

Y descarga Blynk :

```
wget "https://github.com/Peterkn2001/blynk-server/releases/download/v0.41.16/server-0.41.16-java8.jar"
```

El enlace <https://github.com/Peterkn2001/blynk-server/releases/download/v0.41.16/server-0.41.16-java8.jar> conviene actualizarlo, recomendamos visitar la página <https://github.com/Peterkn2001/blynk-server#blynk-server> y coger la última versión de aquí

GETTING STARTED

Blynk server

Blynk Server is an Open-Source [Netty](#) based Java server, responsible for forwarding messages between Blynk mobile application and various microcontroller boards and SBCs (i.e. Arduino, Raspberry Pi. etc).

Download latest server build [here](#).

release **v0.41.17** downloads **6.2k**

Requirements

- Java 8/11 required (OpenJDK, Oracle)
- Any OS that can run java
- At least 30 MB of RAM (could be less with tuning)
- Open ports 9443 (for app and hardware with ssl), 8080 (for hardware without ssl)

2 Configurar server.properties

Necesitamos crear un fichero de configuración para las diferentes opciones que queremos en nuestro servidor Blynk.

Entra en la carpeta creada Blynk y crea el fichero *server.properties*.

cd Blynk

sudo nano server.properties

A continuación se muestra un posible contenido de *server.properties*.

```
initial.energy=1000000
allow.reading.widget.without.active.app=false
user.message.quota.limit=100
logs.folder=./logs
user.dashboard.max.limit=100
lcd.strings.pool.size=6
server.ssl.key=./server_embedded.key
webhooks.response.size.limit=96
hardware.mqtt.port=8440
table.rows.pool.size=100
terminal.strings.pool.size=25
admin.email=admin@blynk.cc
admin.rootPath=/admin
user.widget.max.size.limit=20
listen.address=
blocking.processor.thread.pool.limit=6
stats.print.worker.period=60000
enable.db=false
force.port.80.for.csv=false
enable.raw.db.data.store=true
restore.host=blynk-cloud.com
csv.export.data.points.max=43200
restore=false
user.profile.max.size=256
allow.store.ip=true
```

```
allowed.administrator.ips=0.0.0.0/0,::/0
net.interface=eth
webhooks.frequency.user.quota.limit=1000
http.port=8080
web.request.max.size=524288
user.devices.limit=50
async.logger.ring.buffer.size=2048
user.tags.limit=100
server.ssl.key.pass=
admin.pass=admin
hard.socket.idle.timeout=10
product.name=Blynk
data.folder=/Path
map.strings.pool.size=25
profile.save.worker.period=60000
https.port=9443
log.level=info
server.ssl.cert=./server_embedded.crt
force.port.80.for.redirect=true
notifications.queue.limit=2000
notifications.frequency.user.quota.limit=5
server.host=192.168.137.1
app.ssl.port=8443
hardware.default.port=8442
hardware.ssl.port=8441
hardware.mqtt.port=8440
```

Para la explicación de cada línea, aconsejo consultar esta [página](#) en el apartado *Configuración avanzada del servidor local*

3 Ejecutar el servidor Blynk local

En la Raspberry por comandos SSH, entramos en la carpeta donde hemos creado el servidor Blynk

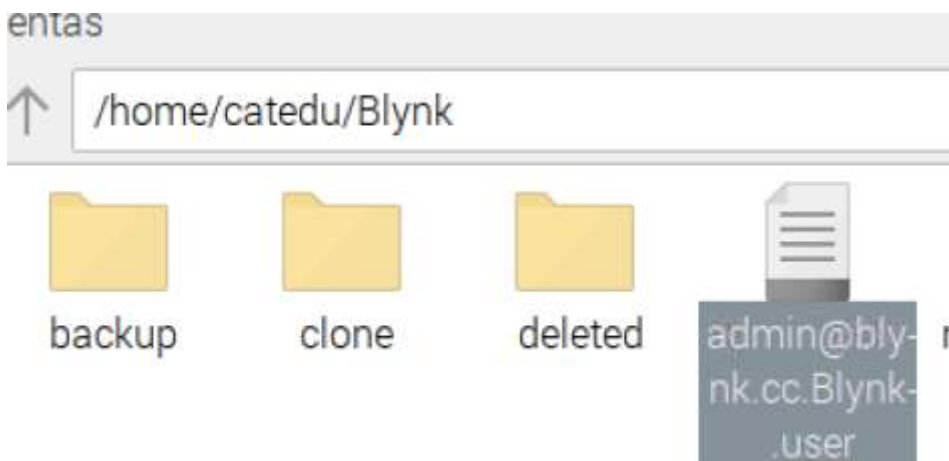
cd Blynk

Y ejecutamos el servidor Blynk instalado, pero que cargue la configuración de server.properties que en nuestro caso como el la Raspberry el usuario se llama catedu la carpeta es catedu:

```
java -jar server-0.41.16-java8.jar -dataFolder /home/catedu/Blynk -serverConfig  
/home/catedu/Blynk/server.properties
```

Curiosamente la primera vez que ejecutas esta instrucción te sale los datos del usuario

admin@blynk.cc y su contraseña sin encriptar que por defecto es admin. Si no has tomado nota, ejecutar la instrucción anterior no sirve de nada pues ya ha creado el fichero texto de este usuario.



La única forma de que te vuelva a mostrar la contraseña es borrar el fichero texto y ejecutar otra vez la orden `java -jar server....`

4 Que la orden de ejecución se haga automáticamente cada vez que se reinicie la Raspberry

Para no repetir estos dos comandos `cd Blynk` y `java -jar server-0.41.16-java8.jar -dataFolder /home/catedu/Blynk -serverConfig /home/catedu/Blynk/server.properties` cada vez que reiniciamos la Raspberry puedes generar un script para que lo ejecute automáticamente, puedes ver buenos tutoriales en Internet.

En internet puedes ver varios métodos:

Método1 es entrar en `/etc` y editar el fichero `rc.local` y añadir esta línea

```
## java -jar server-0.41.16-java8.jar -dataFolder /home/catedu/Blynk -serverConfig  
/home/catedu/Blynk/server.properties &
```

Método 2 usar el comando `crontab -e` y poner al final la siguiente línea `@reboot java java -jar server-0.41.16-java8.jar -dataFolder /home/catedu/Blynk -serverConfig`

/home/catedu/Blynk/server.properties

5 Probarlo

Si la IP de la Raspberry es 192.168.1.112 entonces entramos en:

<https://192.168.1.112:9443/admin>

Vale, ya estoy ¿y ahora qué?

Seguramente te saldrá la siguiente advertencia por el certificado SSL, dale a **Configuración avanzada** y luego a **Acceder a (la IP del servidor Blynk Legacy) sitio no seguro**





La conexión no es privada

Es posible que los atacantes estén intentando robar tu información de **192.168.43.111** (por ejemplo, contraseñas, mensajes o tarjetas de crédito). [Más información](#)

NET::ERR_CERT_AUTHORITY_INVALID



Para disfrutar del máximo nivel de seguridad en Chrome, [activa la protección mejorada](#).

Ocultar configuración avanzada

1

Volver para estar a salvo

Este servidor no ha podido probar que su dominio es **192.168.43.111**, el sistema operativo de tu ordenador no confía en su certificado de seguridad. Este problema puede deberse a una configuración incorrecta o a que un atacante haya interceptado la conexión.

[Acceder a 192.168.43.111 \(sitio no seguro\)](#)

2

Si quieres generar certificados SSL propios para que no salga la anterior pantalla consulta [aquí](#)

Tienes que entrar con el usuario y contraseña fijada en **server.properties** :

```
# Default admin name and password. Will be created on initial server start
admin.email=admin@blynk.cc
admin.pass=admin
```

Si quieres cambiar la contraseña, tienes que hacerlo como un usuario normal en la página de administración tal y como hemos visto en el capítulo [Entrando en el Blynk local: El panel](#)

6 Para saber más :

- <https://github.com/Peterkn2001/blynk-server#blynk-server>
- [Intalación de Blynk : How to Install a Blynk Local Server on Raspberry Pi](#)
- [Configuración de server.properties.](#)

7 Para saber más : Configurar mail.properties

Este aparatado ya comentamos que no lo aconsejamos, pues los alumnos no suelen tener email y la APP ya no permite crear usuarios con email, pero si queremos que envíe los tokens por email, hay que crear este fichero para que el servidor envíe por email los tokens de los proyectos

Entra en la carpeta creada Blynk y crea el fichero *mail.properties*.

cd Blynk

sudo nano mail.properties

A continuación se muestra una muestra del posible contenido de mail.properties :

```
mail.smtp.auth=true  
mail.smtp.starttls.enable=true  
mail.smtp.host=smtp.gmail.com  
mail.smtp.port=587  
mail.smtp.username=Your EMAIL ID  
mail.smtp.password=Password
```

Utilizando **Your EMAIL ID** y **Password** los datos de una cuenta de gmail tuya. En esa cuenta tienes que permitir accesos no seguros. Aquí se muestra dónde está en la pantalla de configuración de Gmail :



-  Inicio 1
-  Información personal
-  Datos y personalización
-  Seguridad 2
-  Contactos e información compartida
-  Pagos y suscripciones
-  Información general

[Gestionar dispositivos](#)[Gestionar i](#)

Acceso de aplicaciones poco seguras 3

Tu cuenta es vulnerable porque permites el acceso de aplicaciones y dispositivos que utilizan una tecnología de inicio de sesión poco segura. Para mantener tu cuenta protegida, Google desactivará automáticamente este ajuste si no se utiliza.

 Activado 4[Desactivar acceso \(opción recomendada\)](#)

Para saber más

- [Configuración mail](#)

Avanzado: Blynk Legacy local

En Windows Instalar Java 11

Entramos en la web de Oracle - Java - Technologies - JavaSE y descargamos e instalamos el servidor Java 11, si nuestro ordenador no es viejo, seguro que soportará la versión 64

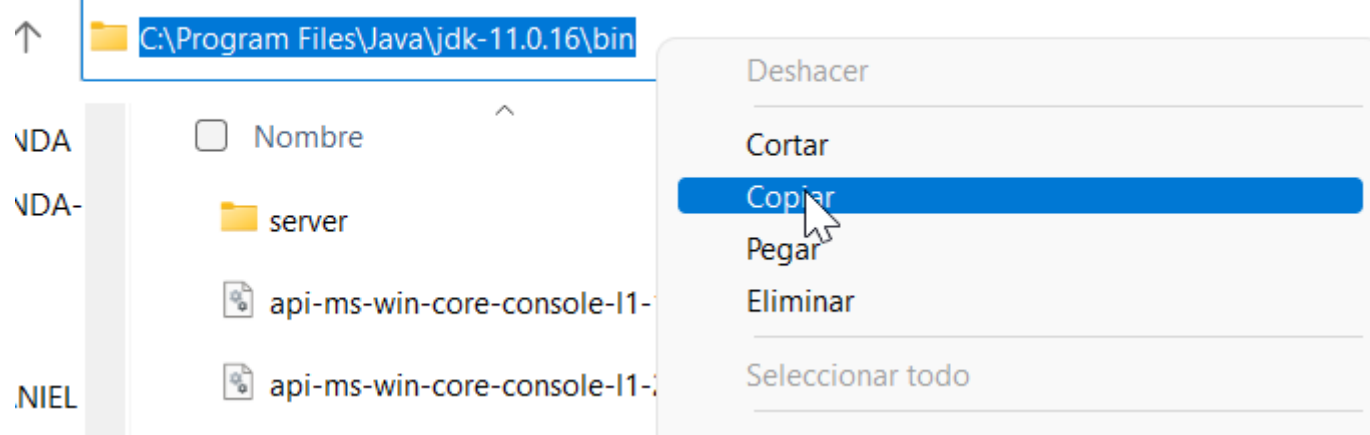
<https://www.oracle.com/es/java/technologies/javase/jdk11-archive-downloads.html>

ORACLE			Q	Ver cuentas	Comunicarse con
Productos Sectores Recursos Clientes Partners Desarrolladores Eventos					
macOS Arm 64 Compressed Archive	153.35 MB		jdk-11.0.16_macos-aarch64_bin.tar.gz		
macOS Arm 64 DMG Installer	152.83 MB		jdk-11.0.16_macos-aarch64_bin.dmg		
macOS x64 Compressed Archive	155.47 MB		jdk-11.0.16_macos-x64_bin.tar.gz		
macOS x64 DMG Installer	154.95 MB		jdk-11.0.16_macos-x64_bin.dmg		
Solaris SPARC Compressed Archive	184.75 MB		jdk-11.0.16_solaris-sparcv9_bin.tar.gz		
Windows x64 Installer	140.55 MB		jdk-11.0.16_windows-x64_bin.exe		
Windows x64 Compressed Archive	158.30 MB		jdk-11.0.16_windows-x64_bin.zip		

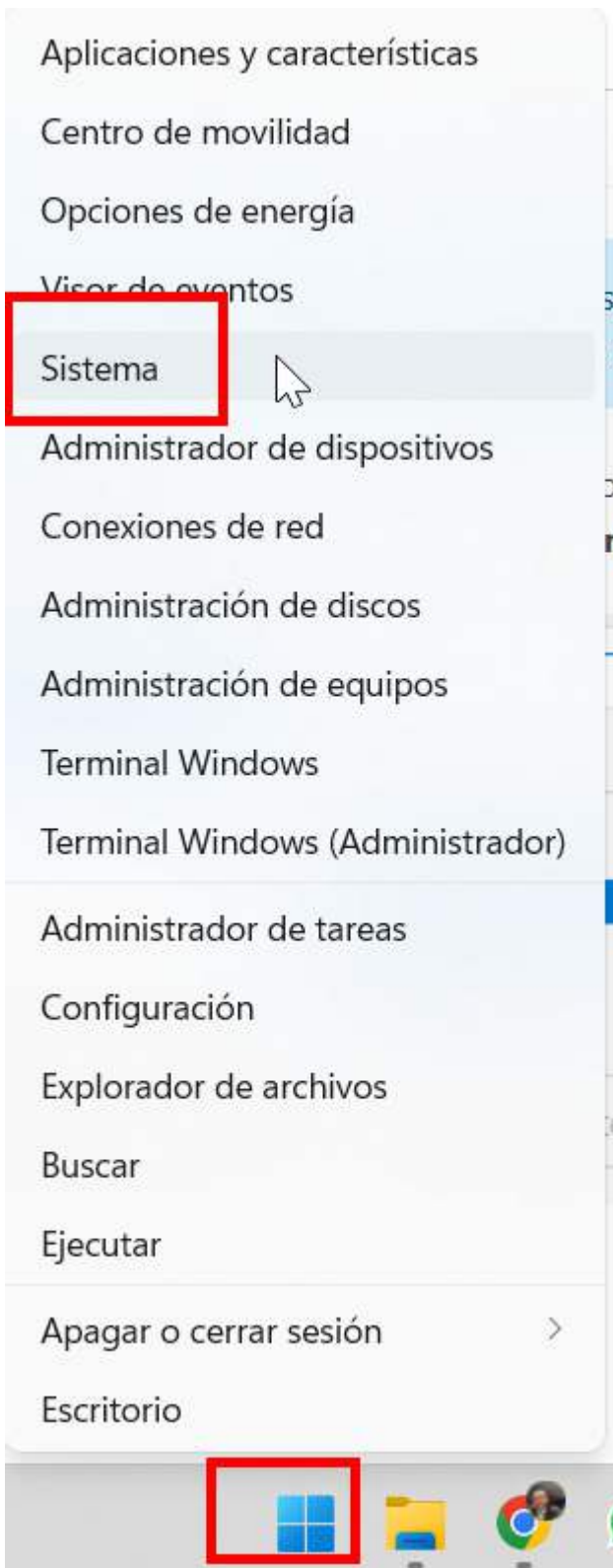
Seguramente para la descarga Oracle nos pide que nos registremos. No entendemos el por qué pero efectivamente es necesario proporcionar un email y a ese email envía un enlace para el registro, Una vez registrado ya permite la descarga

Instalamos el servidor Java descargado.

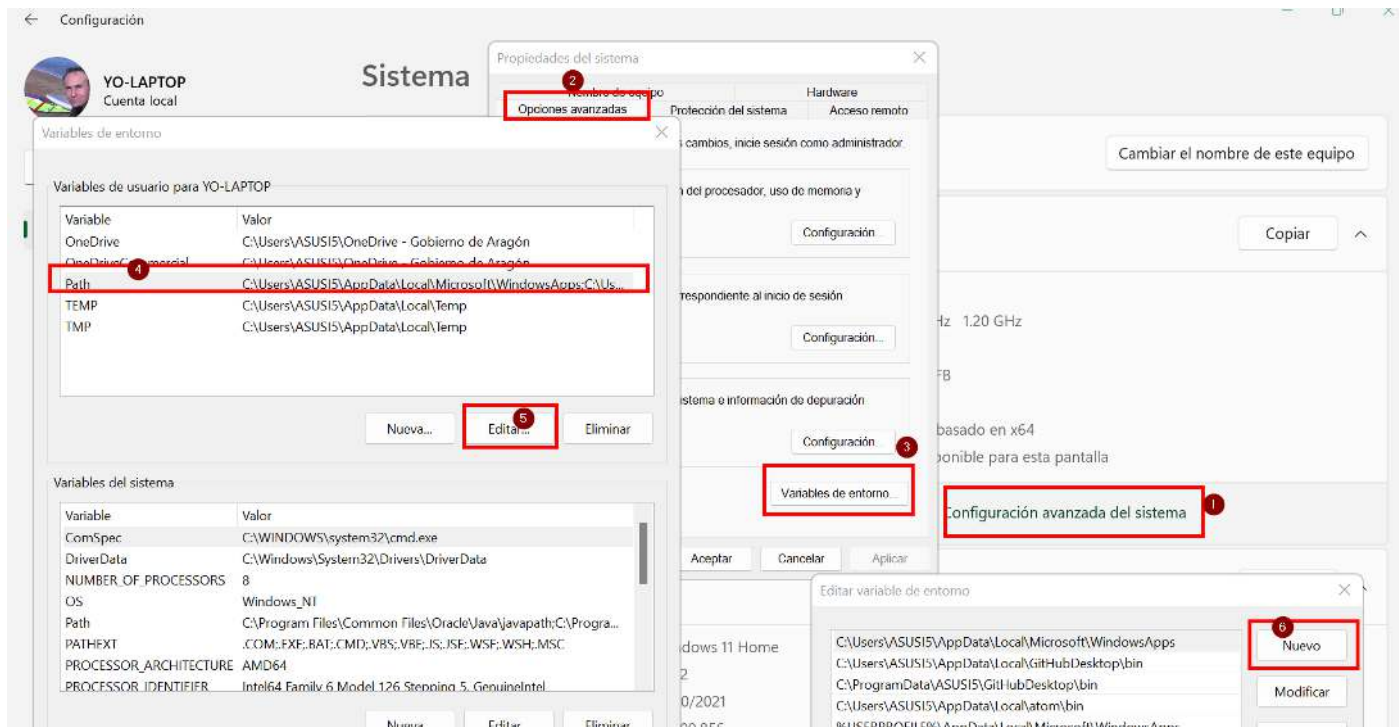
Para que Windows encuentre la ruta del Java instalado cuando ejecutemos comandos, le tenemos que decir en qué camino está la carpeta del java instalado está. Para ello vamos a **C -> Archivos de programa -> Java -> Jdk11...-> carpeta bin** y copiamos la ruta :



Nos vamos al sistema



Y configuración avanzada del sistema - variables de entorno- path - editar - nueva y pegamos la ruta que hemos copiado anteriormente



The image shows a Windows 11 desktop environment with several configuration windows open. The main window is 'Configuración' (Settings), specifically the 'Sistema' (System) section. Overlaid on this are three other windows: 'Propiedades del sistema' (System Properties), 'Variables de entorno' (Environment Variables), and 'Editar variable de entorno' (Edit Environment Variable). Numbered red boxes highlight specific elements:

- 1:** 'Configuración avanzada del sistema' (Advanced system settings) link in the System Properties window.
- 2:** 'Opciones avanzadas' (Advanced options) tab in the System Properties window.
- 3:** 'Variables de entorno' (Environment variables) link in the System Properties window.
- 4:** 'Path' variable in the 'Variables de usuario para YO-LAPTOP' (User variables for YO-LAPTOP) list.
- 5:** 'Editar...' (Edit) button at the bottom of the 'Variables de usuario' section.
- 6:** 'Nuevo' (New) button in the 'Editar variable de entorno' window.

The 'Variables de usuario para YO-LAPTOP' window shows the following table:

Variable	Valor
OneDrive	C:\Users\ASUSI5\OneDrive - Gobierno de Aragón
OneDriveCommercial	C:\Users\ASUSI5\OneDrive - Gobierno de Aragón
Path	C:\Users\ASUSI5\AppData\Local\Microsoft\WindowsApps;C:\Us...
TEMP	C:\Users\ASUSI5\AppData\Local\Temp
TMP	C:\Users\ASUSI5\AppData\Local\Temp

The 'Variables del sistema' (System variables) window shows the following table:

Variable	Valor
ComSpec	C:\WINDOWS\system32\cmd.exe
DriverData	C:\Windows\System32\Drivers\DriverData
NUMBER_OF_PROCESSORS	8
OS	Windows_NT
Path	C:\Program Files\Common Files\Oracle\Java\javapath;C:\Progra...
PATHEXT	.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC
PROCESSOR_ARCHITECTURE	AMD64
PROCESSOR_IDENTIFIER	Intel64 Family 6 Model 126 Stepping 5, GenuineIntel

The 'Editar variable de entorno' window shows the following table:

Variable	Valor
Path	C:\Users\ASUSI5\AppData\Local\Microsoft\WindowsApps
Path	C:\Users\ASUSI5\AppData\Local\GitHub\Desktop\bin
Path	C:\ProgramData\ASUSI5\GitHub\Desktop\bin
Path	C:\Users\ASUSI5\AppData\Local\atom\bin
Path	C:\Users\ASUSI5\AppData\Local\Microsoft\WindowsApps

Avanzado: Blynk Legacy local

En Windows Instalar Blynk local

Descargamos el servidor Blynk server desde

<https://github.com/blynkkk/blynk-server/releases>

o desde <https://drive.google.com/file/d/1nPZxn-9Q9q0ceUhCiZGd4WRC1c8pufEQ/view?usp=sharing>

y lo pegamos en una carpeta, por ejemplo en c:\Blynk_server

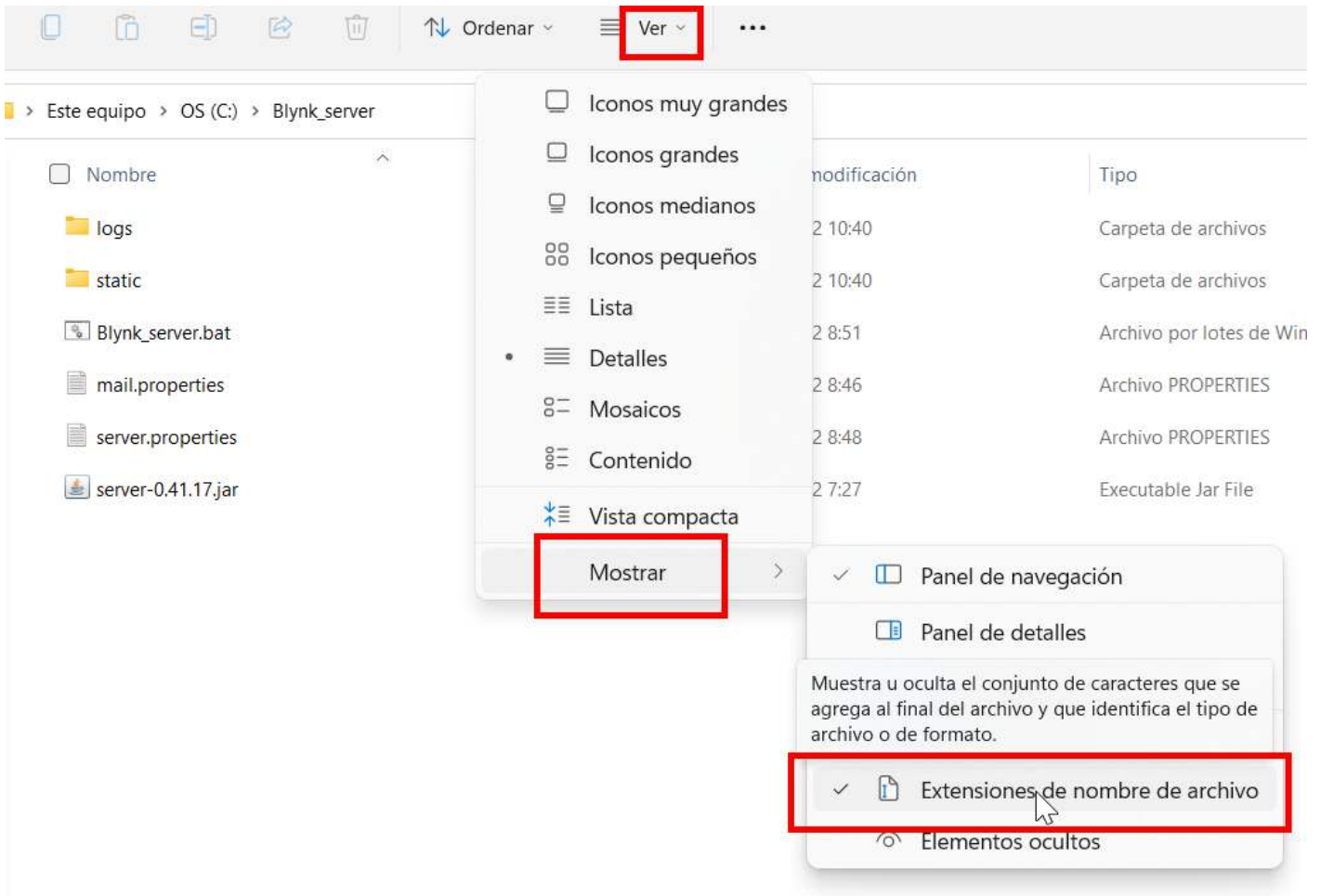
Creamos **mail.properties** y **server.properties** como ficheros nuevos, los editamos con un procesador de texto y pegamos el código que hemos explicado [aquí](#)

 mail.properties

 server.properties

  server-0.41.17.jar

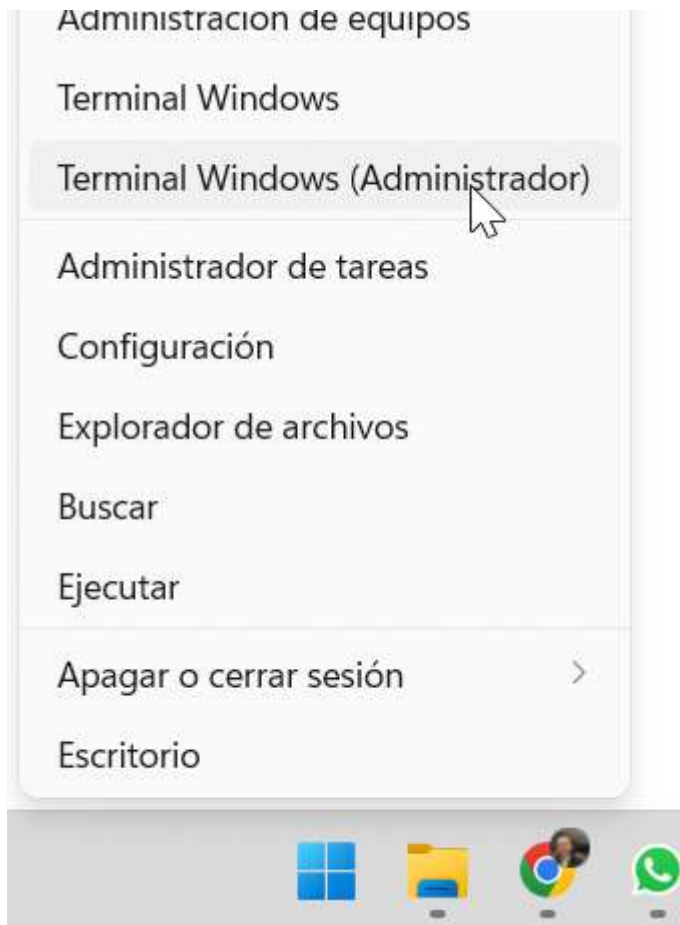
Ojo: Ten en cuenta que Windows le gusta colocar sus propias extensiones. Si editas un fichero con un procesador de textos, por ejemplo el bloc de notas, le añadirá .txt, si lo editas con Word le añadirá la extensión .docx, tienes que asegurarte que no se añada ninguna extensión, que los nombres de los ficheros sean **mail.properties** y **server.properties** y no otros. Ten activo la visualización de las extensiones para asegurarte que windows no añada las tuyas



Avanzado: Blynk Legacy local

En Windows Ejecutar Blynkserver

Entramos en comandos de windows como administrador



Y ejecutamos esta instrucción

java -jar server-0.41.17.jar -dataFolder /Path

de esta manera se crean las carpetas logs y static que nos dicen los errores que podemos encontrar

La primera pantalla nos dirá el nombre la url, el nombre de usuario y la contraseña para entrar en el servidor blynk



```
Administrador: Windows POW
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras.

PS C:\Users\ASUSI5> cd c:\Blynk_server
PS C:\Blynk_server> java -jar server-0.41.17.jar -dataFolder /Path

Blynk Server 0.41.18-SNAPSHOT successfully started.
All server output is stored in folder 'C:\Blynk_server\.\logs' file.
Your Admin url is https://192.168.137.1:9443/admin
Your Admin login email is admin@blynk.cc
Your Admin password is admin
```

Avanzado: Blynk Legacy local

En Linux: Insyalar Blynk Legacy local

Seguimos las instrucciones de <https://github.com/Peterkn2001/blynk-server#blynk-server>

Pero si tu centro tiene Vitalinux, ponte en contacto con nosotros soportecatedu@educa.aragon.es para instalar en un ordenador Vitalinux el servidor Blynk de forma local

Avanzado: Blynk Legacy local

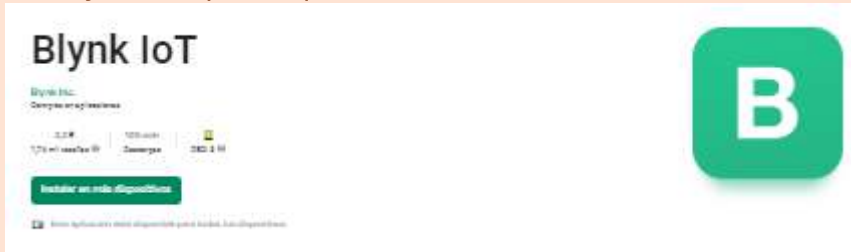
Blynk Legacy: La APP

La APP de **BLYNK LEGACY** no la encontramos en el buscador de Google Play de forma tradicional, pues está sin mantenimiento, tenemos varias opciones para descargar la APP.

OJO no confundir Blynk (legacy) que es el que vamos a trabajar



con Blynk IoT que **sí** que lo encuentras en el buscador de Google Play



OPCIÓN A Desde Google Play

En este enlace aún tiene la APP antigua, pero es de propiedad de Blynk Inc luego **no es de extrañar que desaparezca.**

<https://play.google.com/store/apps/details?id=cc.blynk>



21:56 AE

91%

Google Play

**Blynk (legacy)**

Blynk Inc.

In-app purchases

Uninstall

Open

What's new

Last updated 3 May 2022



Fixed issues with gps widget, home screen widgets on Android 11 and later

Rate this app

Tell others what you think

[Write a review](#)

Developer contact

**Join the beta**

Try new features before they're officially released and give your feedback to the developer. Certain data on your use of the app will be collected and shared with the developer to help improve the app.

[Join](#) [Learn more](#)

OPCION B Fuera de Google Play

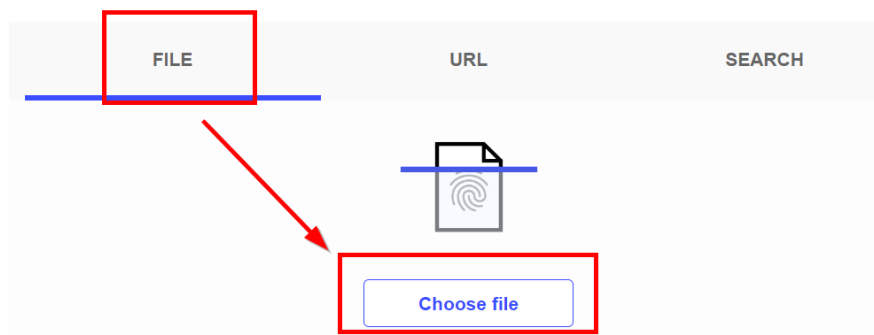
OPCIÓN B: ANTES DE LA INSTALACIÓN

No nos tenemos que fiar de descargar APK de repositorios fuera de Google Play, luego una vez descargado, pasamos el fichero por una página de Antivirus, por ejemplo

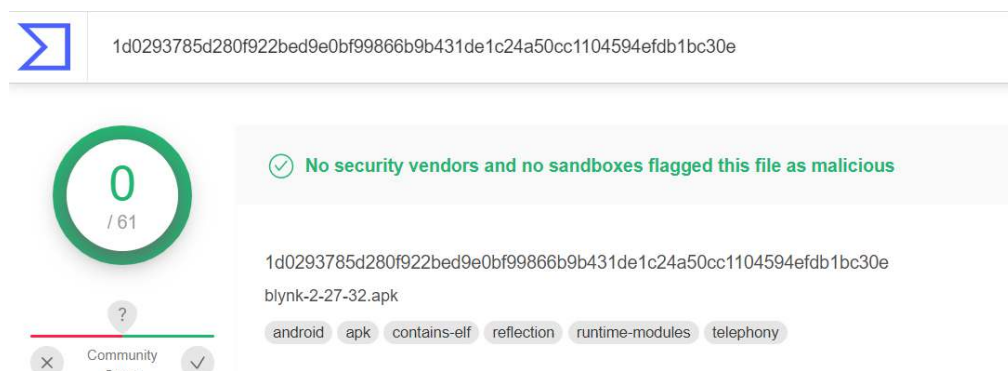
<https://www.virustotal.com/gui/home/upload>



Analyze suspicious files, domains, IPs and URLs to detect malware and other breaches, automatically share them with the security community



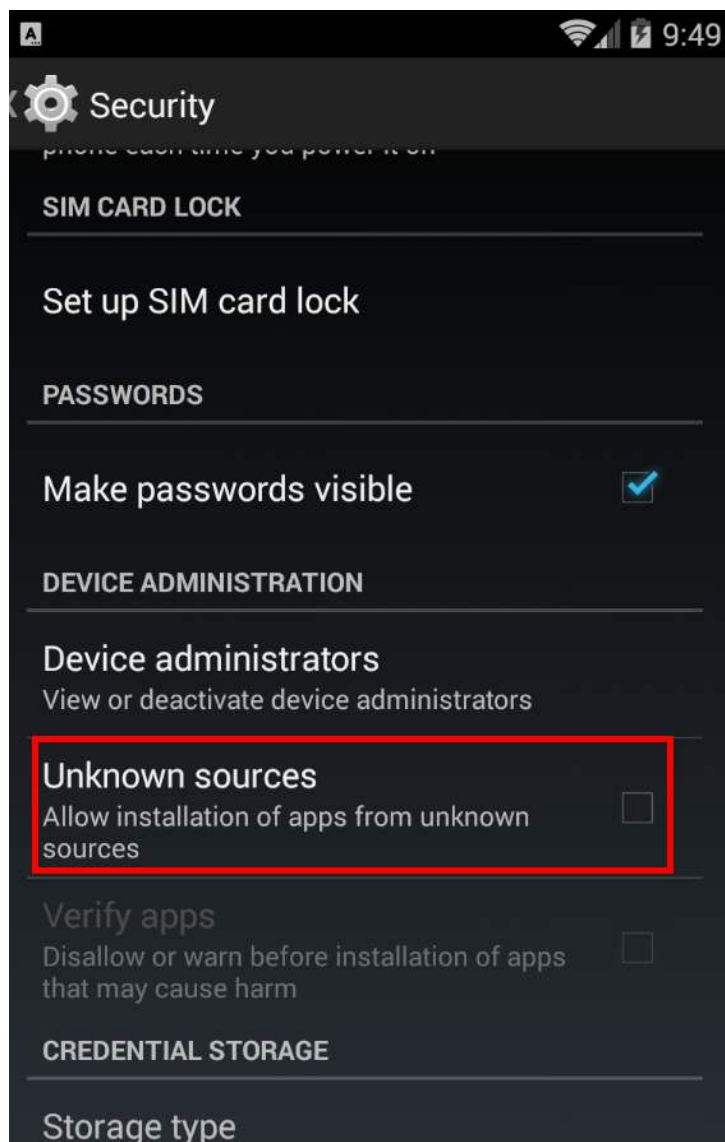
No nos tiene que dar ningún virus



Para instalarlo en tu móvil:

Primero, tenemos que llevar el fichero .apk a tu dispositivo, recomendamos usar email, por ejemplo enviándote a ti mismo un correo electrónico y lo consultas desde el móvil.

Segundo, tenemos que instalarlo y para ello hay que decirle que confiamos con apps instaladas fuera del repositorio oficial (Google Play), como depende las pantallas del modelo de tu móvil, puedes encontrar cualquier tutorial en internet cómo se hace pero prácticamente es llegar a la misma pantalla en seguridad/privacidad- permitir apps de orígenes desconocidos.



Tercero instalarlo

Cuarto, deshacer el segundo paso

OPCIÓN B DESCARGA

Puedes buscar en un navegador esta APP, encontraras varios sitios para descargarla, por ejemplo aquí:

- <https://blynk.uptodown.com/android/descargar>
- <https://blynk.en.uptodown.com/android/download/3921223>
- Github: <https://github.com/BlynkMobile>
- [Desde este repositorio nuestro](#)



Blynk Client

BlynkMobile

Follow

Since Blynk.io decided to shut down the Blynk app and local server, we decided to clone the mobile app and develop it with the community. Help us keep the legacy

👤 3 followers · 2 following

 <https://www.reddit.com/r/blynkmobileapp/>

 @MobileBlynk

Block or Report

Avanzado: Blynk Legacy local

Blynk Legacy: Crear cuentas

Antes se creaban cuentas en Blynk usando el correo electrónico: Cada usuario en la APP de Blynk podía crearse una cuenta, enviando un email. El servidor local Blynk puede enviar correos electrónicos si tiene configurado un fichero que se llama mail.properties y se envía el correo con las credenciales. No obstante esta opción de crear cuenta **ha desaparecido** de la APP (En CATEDU ya *barruntábamos* esto, que Blynk Inc al quitar crear nuevos usuarios es que pensaba quitar este servicio Blynk legacy).

No obstante, la creación de usuarios mediante emails era un problema pues menores de 16 años no pueden tener emails ni redes sociales, ni whatsapps (¿oigo risas?).

En el curso vas a utilizar los servidores de CATEDU luego tendrás asignado 12 cuentas de alumnos + un profesor

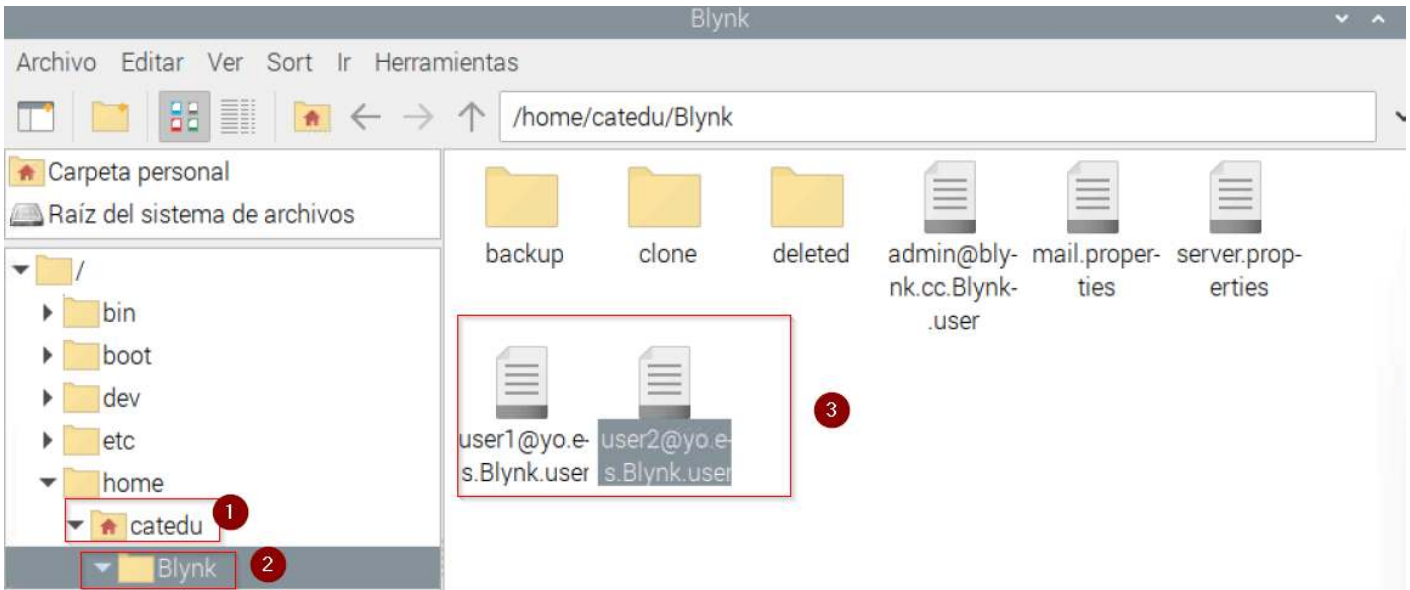
Recibirás un email con los nombres de usuario y las contraseñas

¿Y fuera del curso?

En este caso necesitarás tener un servidor Blynk legacy local que tal y como lo explicamos en los siguiente capítulos, montado en una Raspberry.

Entramos en la **carpeta de Blynk en nuestra Raspberry**, ya sea por VNC, o con una pantalla, teclado y ratón o por SSH y lo que hay que hacer es crear unos ficheros texto que se llamen **emailusuario.Blynk.user** los emails de los usuarios pueden ser inventados, por ejemplo en la figura puedes ver que en la raspberry hay dos usuarios user1@yo.es user2@yo.es luego los ficheros textos son **user1@yo.es.Blynk.user** y el otro es **user2@yo.es.Blynk.user**.

Entrando por VNC o pantalla, teclado y ratón:



Entrando por SSH

```
login as: catedu
catedu@192.168.245.80's password:
Linux raspberrypi 5.15.61-v7+ #1579 SMP Fri Aug 26 11:10:59 BST 2022 armv7l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Sep 13 15:14:43 2022
catedu@raspberrypi:~$ cd Blynk
catedu@raspberrypi:~/Blynk$ ls
admin@blynk.cc.Blynk.user  clone  mail.properties  user1@yo.es.Blynk.user
backup                    deleted  server.properties  user2@yo.es.Blynk.user
catedu@raspberrypi:~/Blynk$ sudo nano user1@yo.es.Blynk.user
GNU nano 5.4
{"name":"user1@yo.es","email":"user1@yo.es","appName":"Blynk","region":"local","ip
```

El contenido por ejemplo del fichero **user2@yo.es.Blynk.user** es:

```
“ { "name": "user2@yo.es", "email": "user2@yo.es",
    "appName": "Blynk", "region": "local", "ip": "ip_local_raspberry", "pass": "uffzlHwvej
    aeZZz2PzqmKuB22OACUDRLWvKEI7RCDQI=", "lastModifiedTs": 1663010486621,
    "lastLoggedIP": "192.168.1.73", "lastLoggedAt": 1635057921978, "profile": {}, "isFacebookUser": false, "isSuperAdmin": false, "energy": 99600, "id": "user2@yo.es-
```

Blynk"} }

Luego lo único que tienes que hacer para crear nuevos usuarios es **crear ficheros texto que se llamen userx@yo.es.Blynk.user y con contenido igual que el anterior pero cambiando el numero 2 por x** (si quieres respetar userx@yo.es pero puedes inventarte cualquier email ficticio)

¿Por qué puede ser que el contenido de un fichero pueda tener más información o menos que otros?

Porque en esos ficheros txt tiene la información de los proyectos creados por el usuario

¿Entonces si el fichero user1@yo.es.Blynk.user tiene toda la información del usuario y todas sus proyectos... si ese usuario quiere llevarse sus proyectos a otro servidor Blynk sólo tiene que copiar y pegar es fichero texto?

Respuesta : SI , así de sencillo, ese fichero texto (por lo tanto libre de virus) tiene toda la información.

¿En la APP no se almacena ningún proyecto?

Ninguno, todos los proyectos están almacenados en esos ficheros txt si no hay conexión con el servidor, en la APP no aparece NADA

Vale, ya sé crear los nombres de usuarios ¿Y las contraseñas?

En teoría están encriptadas en el fichero texto, como puedes ver, sale uffzIHwvejaeZZz2PzqmKuB22OACUDRLWvKEI7RCDQI= eso significa *alcorisa*

Pero tranquilo, luego veremos en el panel de control cómo se pueden poner las contraseñas sin encriptar

+información [aquí](#)

Avanzado: Blynk Legacy local

Blynk Legacy: El panel de control

Como entro en el panel de control

- **En el curso de CATEDU**
 - Habrás recibido un email con la URL para entrar en la administración web del Blynk Legacy, con usuario y contraseña
- **Fuera del curso de CATEDU usando tu servidor local**
 - Desde cualquier ordenador conectado en la misma red local que la raspberry, ejecutas en un navegador la siguiente dirección:
 - Si la IP de la Raspberry es 192.168.1.112 entonces entramos en:
 - <https://192.168.1.112:9443/admin>

Vale, ya estoy ¿y ahora qué?

Seguramente te saldrá la siguiente advertencia por el certificado SSL, dale a **Configuración avanzada** y luego a **Acceder a (la IP del servidor Blynk Legacy) sitio no seguro**



La conexión no es privada

Es posible que los atacantes estén intentando robar tu información de **192.168.43.111** (por ejemplo, contraseñas, mensajes o tarjetas de crédito). [Más información](#)

NET::ERR_CERT_AUTHORITY_INVALID



Para disfrutar del máximo nivel de seguridad en Chrome, [activa la protección mejorada](#).

Ocultar configuración avanzada

1

Volver para estar a salvo

Este servidor no ha podido probar que su dominio es **192.168.43.111**, el sistema operativo de tu ordenador no confía en su certificado de seguridad. Este problema puede deberse a una configuración incorrecta o a que un atacante haya interceptado la conexión.

Acceder a 192.168.43.111 (sitio no seguro)

2

Luego entramos con el usuario y contraseña que nos han proporcionado

Cambiando las contraseñas de los alumnos

Y podemos ver los usuarios creados con los ficheros textos mencionados anteriormente

Blynk Administration

-  Users
-  Stats >
-  Hardware Info >
-  Config

Users list

☐	Email
☐	user1@yo.es
☐	user2@yo.es
☐	admin@blynk.cc

Entrando en un usuario PODEMOS GENERARLE UNA CONTRASEÑA, por supuesto la pones normal, sin encriptar

También puedes borrar el usuario si no nos interesa este usuario.

Blynk Administration

-  Users
-  Stats >
-  Hardware Info >
-  Config

Edit user "user1@yo.es"

 List  Delete

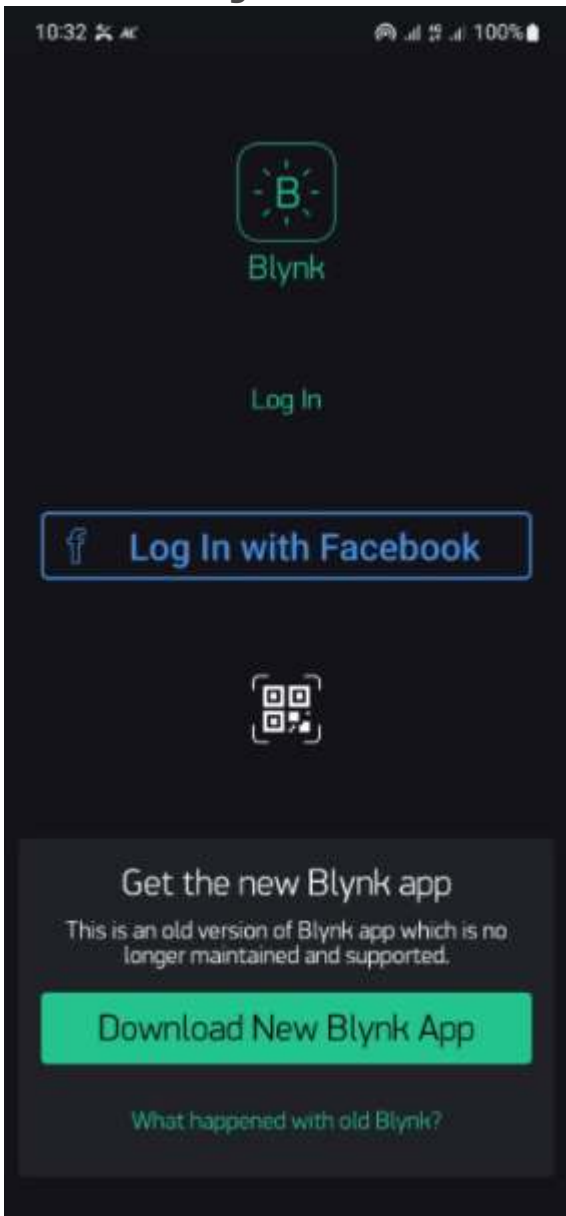
Email	user1@yo.es
Name	user1@yo.es
Pass	*****

Avanzado: Blynk Legacy local

Blynk Legacy: En la APP

Conectar con el nuevo usuario

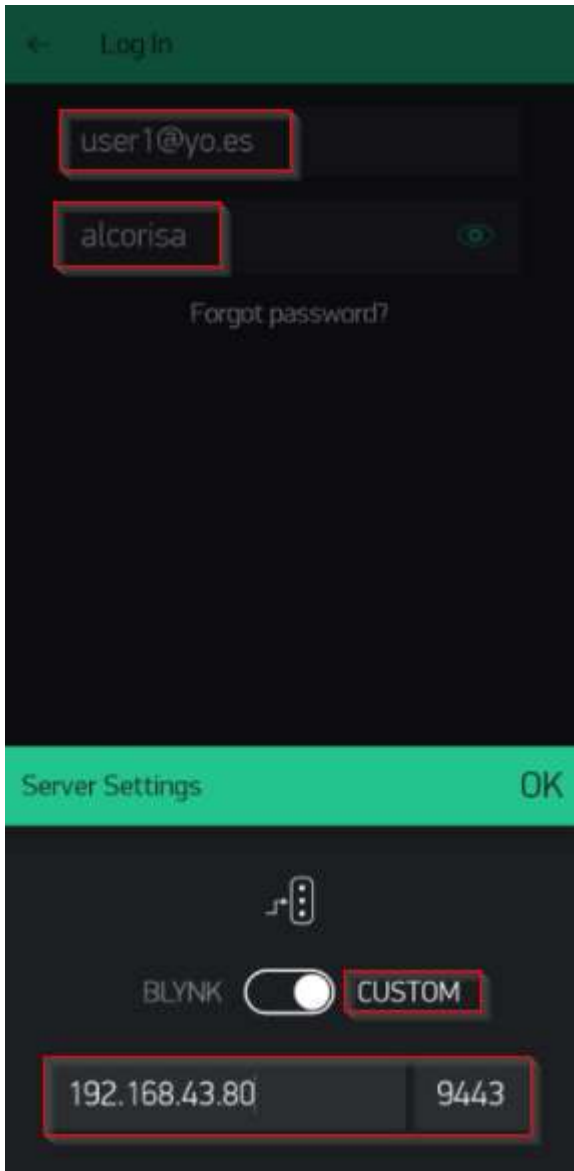
Entramos en la APP de Blynk que hemos visto anteriormente como descargarla e instalara y entramos en **Log In**



Y ponemos el nombre de usuario que hemos creado anteriormente, y la contraseña, la que hayamos definido en el panel de control

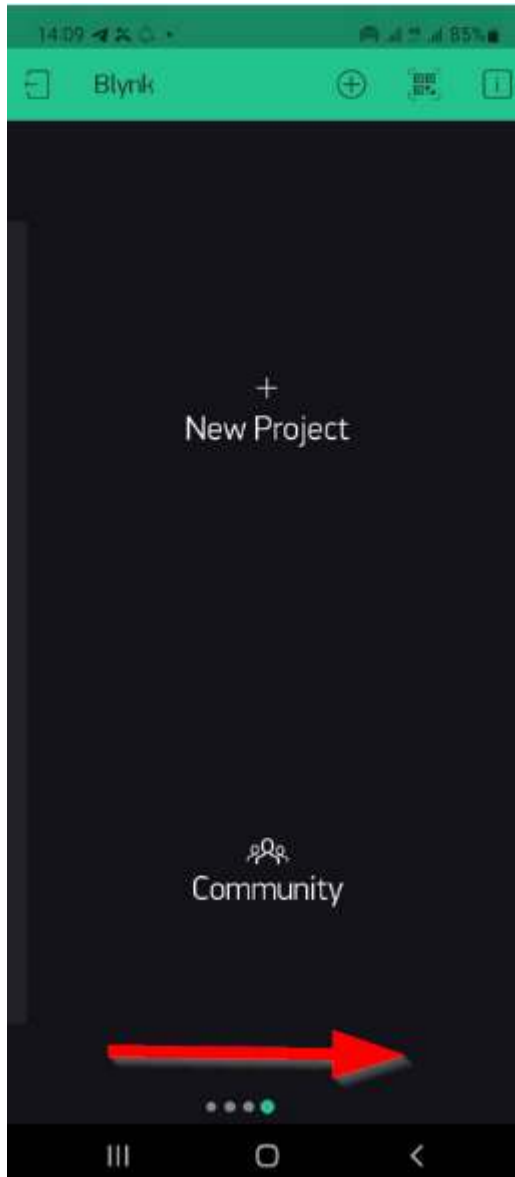
IMPORTANTE: Entra en **CUSTOM** y pon **LA DIRECCIÓN IP DEL SERVIDOR BLYNK LEGACY**

- Si es dentro del curso de CATEDU habrás recibido un email de las IPs y de los puertos que hay que poner en esta APP de Blynk Legacy
- Si es fuera del curso, la IP DE LA RASPBERRY, O SEA DEL SERVIDOR LOCAL BLYNK y puerto 9443

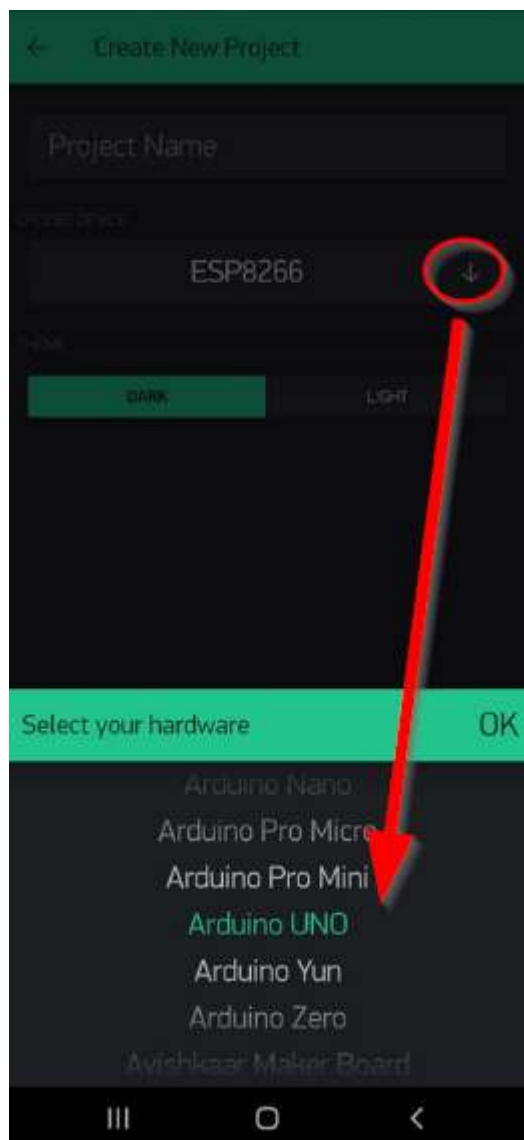


Crear un nuevo proyecto

Podemos ver los proyectos que hay ya almacenados, vamos a crear un nuevo proyecto:

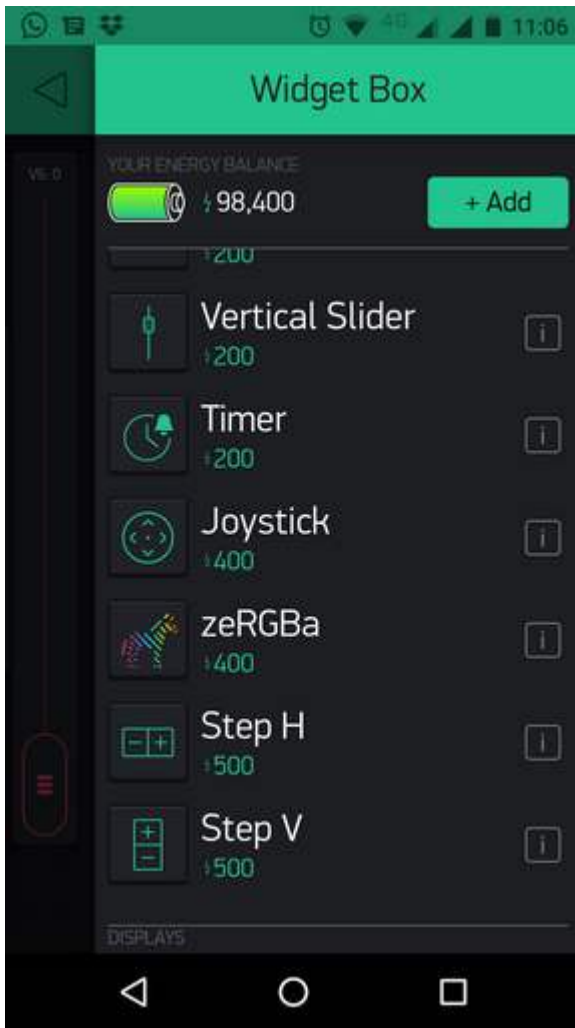


Nos preguntará por el **DEVICE**:



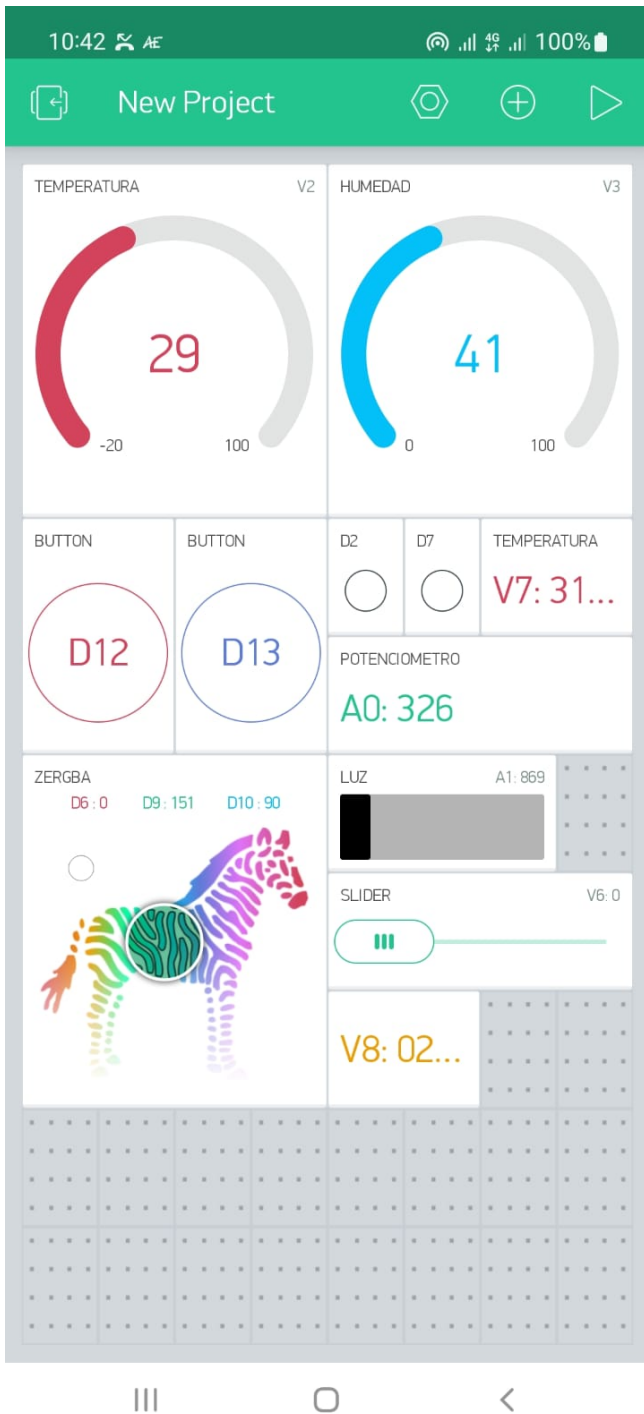
- En **Arduinoblocks en el Aula**, la placa es el TDR STEAM colocado en un Arduino con el ESP01 conectado, luego **DEVICE Arduino Uno CONNECTION TYPE Wifi**
- En **Rover con Arduino**, la placa es un NodeMCU v2, luego **DEVICE NodeMCU CONNECTION TYPE Wifi**
- En **ESP32 en el Aula**, la placa es un ESP32 con el Imagina TDR STEAM, luego **DEVICE ESP32 DevBoard CONNECTION TYPE Wifi**

Empieza un nuevo proyecto, tienes un montón de energía y widgets para hacerlo:



Crea un nuevo proyecto, y así se crea en el servidor Blynk local un **TOKEN** que ahora veremos qué es

Aquí tienes la captura de un proyecto o dashboard creado para el kit Imagina TDR Steam



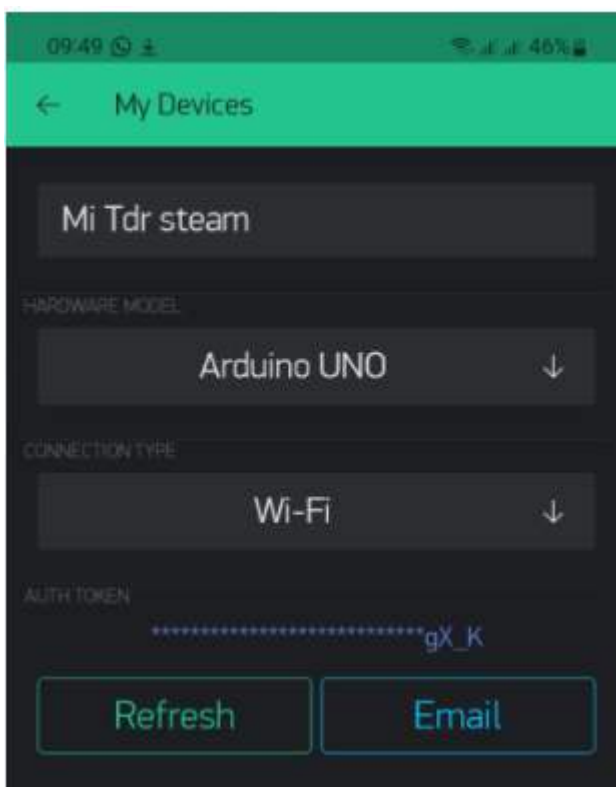
Avanzado: Blynk Legacy local

Blynk Legacy: Poner el TOKEN en Steamakerblocks

¿Cómo conseguir el TOKEN?

OPCION A: QUE LO HAGA EL ALUMNO En la APP

Es la opción más sencilla, entrar en Devices **pulsar en el Token y se copiará en el portapapeles** del móvil, luego ir a cualquier aplicación para poder enviarla al PC (email, whatsapp...) y pegarla en Arduinoblocks en la instrucción que luego veremos.



OPCION B: QUE LO HAGA EL PROFESOR En el panel de control

Entramos en el panel de control del servidor Blynk y podemos ver en los usuarios creados los tokens

Blynk Administration

 Users

 Stats >

 Hardware Info >

 Config

Users list

☐	Email
☐	user1@yo.es
☐	user2@yo.es
☐	admin@blynk.cc

Entrando en un usuario PODEMOS VER SUS PROYECTOS Y LOS TOKENS

 Add new widgets

Devices

Id

0

Name

mi proyecto

BoardTy

ESP8266  ▼

Token

JGnbIYNVLfbdgqNvHe3xK1LyJV0QDiE9

LastLoggedIn

192.168.1.131

Connect

WI_FI  ▼

 Add new devices



RECUERDA: UN PROYECTO UN TOKEN

¿Qué se hace con ese TOKEN? Se pone en ARDUINOBLOCKS:

Entramos en ArduinoBlocks y vamos al bloque de comunicaciones IoT - Blynk legacy

ATENCIÓN: ARDUINOBLOCKS CONNECTOR tiene que estar actualizado, al menos v5 (octubre 2022)

Tenemos que poner los datos de la **IP servidor BLYNK LOCAL**, y el **TOKEN** del proyecto

- **DENTRO DEL CURSO CATEDU** Habrás recibido un email con las IPs y los puertos a poner en Arduinobloks
- **SI ES EN TU SERVIDOR BLYNK LOCAL** Hay que poner la IP de la Raspberry o tu servidor local y **EL PUERTO 8080**

En el caso de que es un proyecto con **ARDUINO** o **TDR-STEAM- ARDUINO** con **ESP01** sale :



En el caso de sea un proyecto con **NodeMCU o ESP32** como en el curso del Rover con Arduino o IoT en el aula sale :

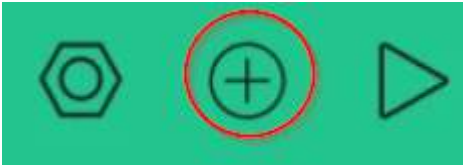


Avanzado: Blynk Legacy local

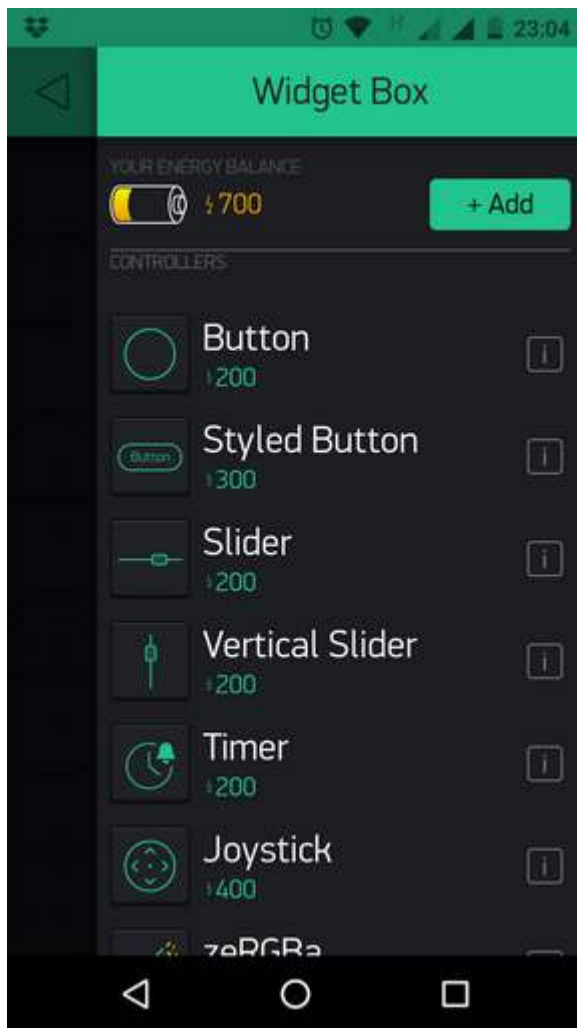
Blynk Legacy: 1programa: Encender los pines rojo y verde

Vamos a ver un primer programa de la APP al TDR STEAM

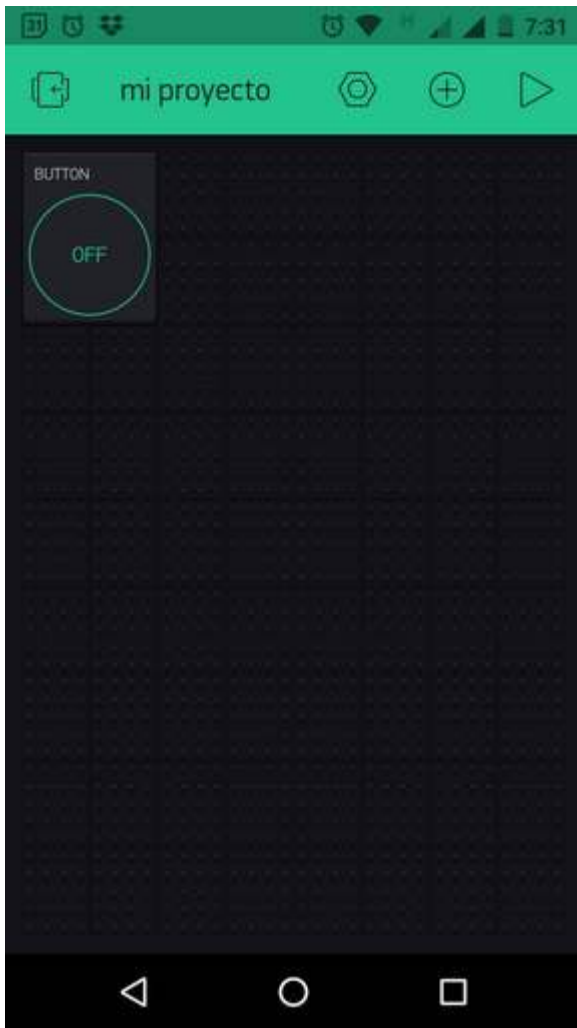
Una vez dentro del proyecto de la App aparece todo un panel si nada, añadimos controles
apretando al + que hay en la parte superior



Seleccionamos el botón



Nos aparece el botón



Pulsamos sobre él para entrar en sus propiedades, le decimos que vaya de 0 a 1 (es decir que cuando se apriete, que envíe un 1) que sea tipo **switch** (no pulsador, sino interruptor), le ponemos un color **rojo** para indicar que es el led rojo, y como el TDR-STEAM el led rojo lo tiene en el D12 le decimos que el **pin** es **DIGITAL** y **D12**

14:23  51%

← Button Settings 



Button

OUTPUT

D12 0  1

MODE

PUSH ☒ SWITCH

ON/OFF LABELS

OFF OFF ON ON

DESIGN

FONT SIZE T T T TEXT 

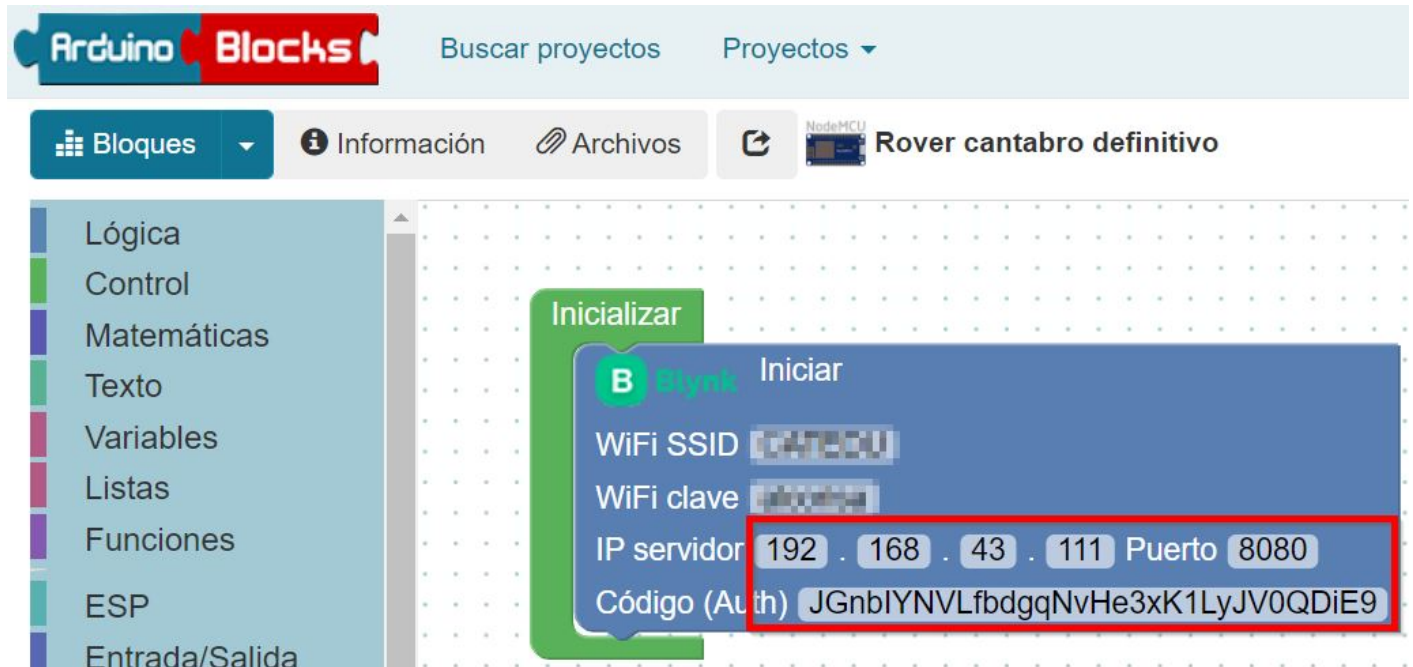
 Delete



Hacemos lo mismo con el **azul** pero **D13**

En ARDUINO BLOCKS

En Arduinoblocks dejamos el token tal y como hemos aprendido anteriormente



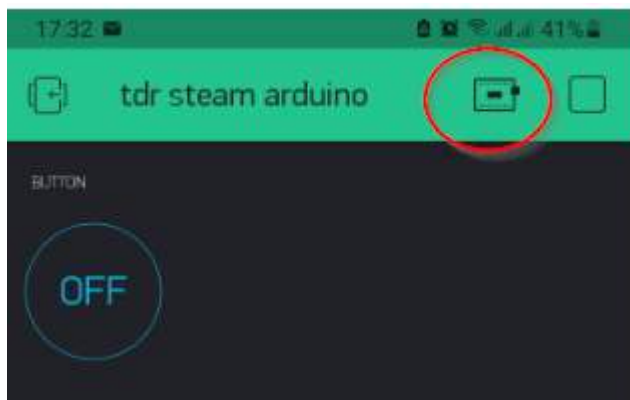
¿Nada más?



Nada más !! así de sencillo. Le damos a **subir** (teniendo el programa Arduinoblocks conector minimizado, eso lo podemos ver enseguida pues detecta en que COM está conectado, en la figura sale COM5)



Vamos a la APP de BLYNK, Pulsamos al botón de **play** ► de arriba arriba y vemos el estado NO TIENE QUE APARECER UN PUNTO ROJO si aparece es que nuestra placa no está conectada



Hay que tener algo de paciencia hasta que se comunica con la Raspberry, cuando no aparece el punto rojo, pulsamos en los botones y tienen que encenderse y apagarse correctamente los leds rojo y azul de nuestro TDR Steam

Para usuarios del servidor Blynk de CATEDU y la placa ARDUINO CON ESP32 WIFI

No sabemos por qué pero al darle al Play, el icono de la placa de arriba sale en rojo, como si no estuviera conectado, pero el proyecto funciona perfectamente. Si sabes por qué por favor ponte en contacto con nosotros <https://catedu.es/informacion/>



Financiado por el Ministerio de Educación y Formación Profesional y por la Unión Europea - NextGenerationEU

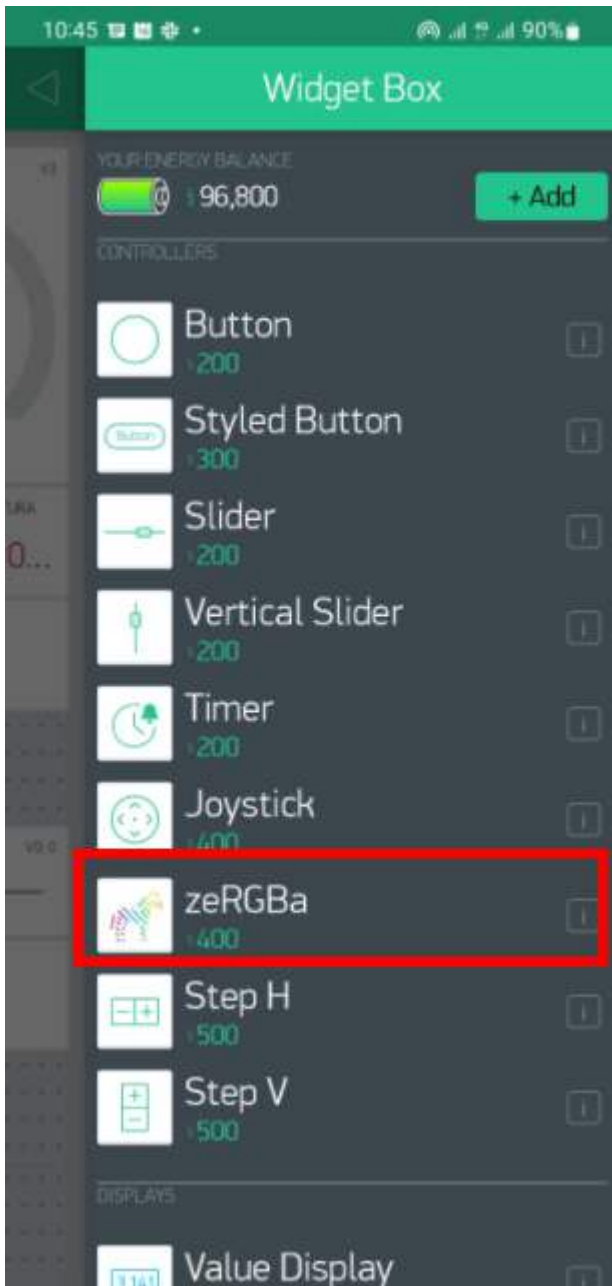


Avanzado: Blynk Legacy local

Blynk Legacy: 2 programa RGB y el potenciómetro

LED RGB

En la APP de Blynk ponemos el gadget "Cebra"



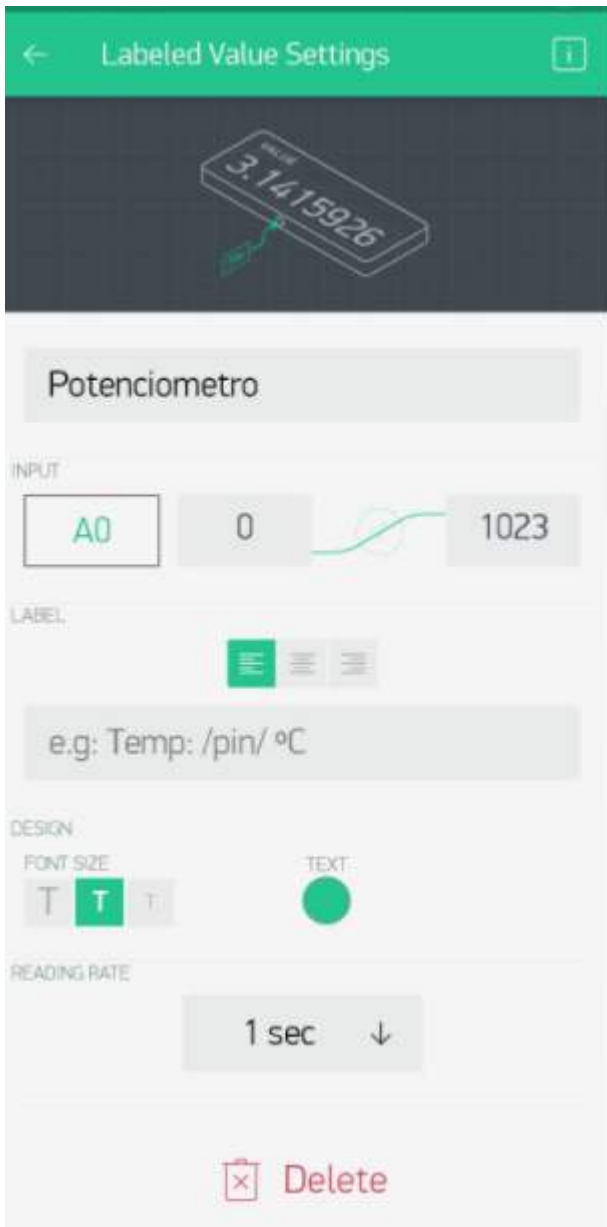
Y lo configuramos según sus pines. El led RGB en el TDR Steam está conectado a los siguientes pines

- Red D6
- Green D9
- Blue D10



Potenciómetro

El potenciómetro está conectado en A0 podemos insertar un gadget Label y lo configuramos como tal:

**¿En Arduinoblocks no hay que añadir nada?**

Nada !!! cuando son gadgets que leen directamente de los pines (digitales o analógicos) NO HACE FALTA MÁS CÓDIGO que el de conectar el servidor Blynk

Arduino Blocks Buscar proyectos Proyectos ▾

Bloques ▾ Información Archivos  Rover cantabro definitivo

- Lógica
- Control
- Matemáticas
- Texto
- Variables
- Listas
- Funciones
- ESP
- Entrada/Salida

Inicializar

 Blynk Iniciar

WiFi SSID CATEDU

WiFi clave 12345678

IP servidor 192 . 168 . 43 . 111 Puerto 8080

Código (Auth) JGnbIYNVLfbdgqNvHe3xK1LyJV0QDiE9

Financiado por el Ministerio de Educación y Formación Profesional y por la Unión Europea - NextGenerationEU

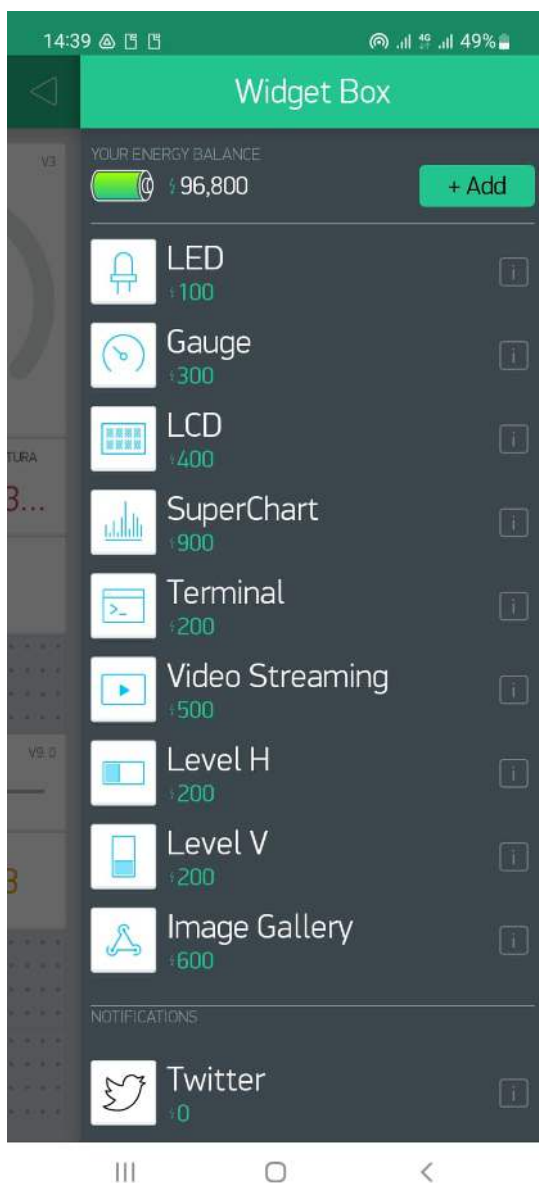


Avanzado: Blynk Legacy local

Blynk Legacy: 3 programa Medir la luz del LDR

Nuestro segundo programa será al revés del TDR Steam a la APP

Ahora añadimos otro control **level H**



Elegimos de PIN el analógico, el LDR está conectado al pin analógico A1 y como el LDR aumenta según la *oscuridad* vamos a poner que vaya de 1023 en formato negro.



Sin modificar nada del programa de ARDUINOBLOCKS vamos a darle al PLAY y ¡¡ Y FUNCIONA !!!

Financiado por el Ministerio de Educación y Formación Profesional y por la Unión Europea - NextGenerationEU



Avanzado: Blynk Legacy local

Blynk Legacy: 4 programa: Medir T y H con el DHT11

Este caso es distinto, pues

- En el **Rover marciano con Arduino** el DHT22 está conectado al pin D5 digital del NodeMCU
- En el **Arduinobloks en el aula**, el DHT11 de la placa Imagina TDR Steam está conectado al pin D4 digital del Arduino
- En el **IoT en el Aula** el DHT11 de la placa Imagina TDR Steam está conectado al pin D4 digital del ESP32

Pero los dos miden dos variables **Temperatura Y Humedad**

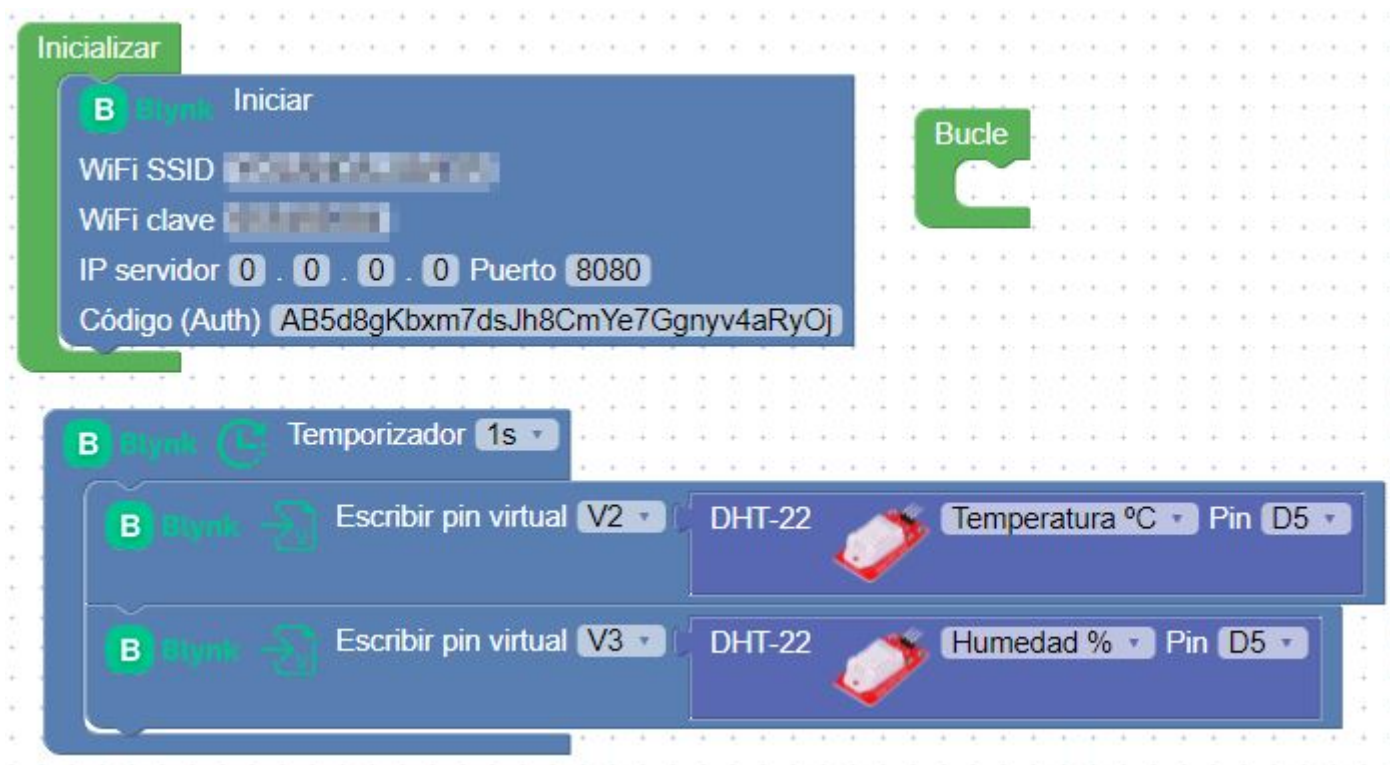
□□□□ ¿Cómo hacemos para medir dos variables en un sensor que está conectado a un sólo PIN? □□□□

Solución PINES VIRTUALES: si, has leído bien, vamos a utilizar **pines virtuales**.

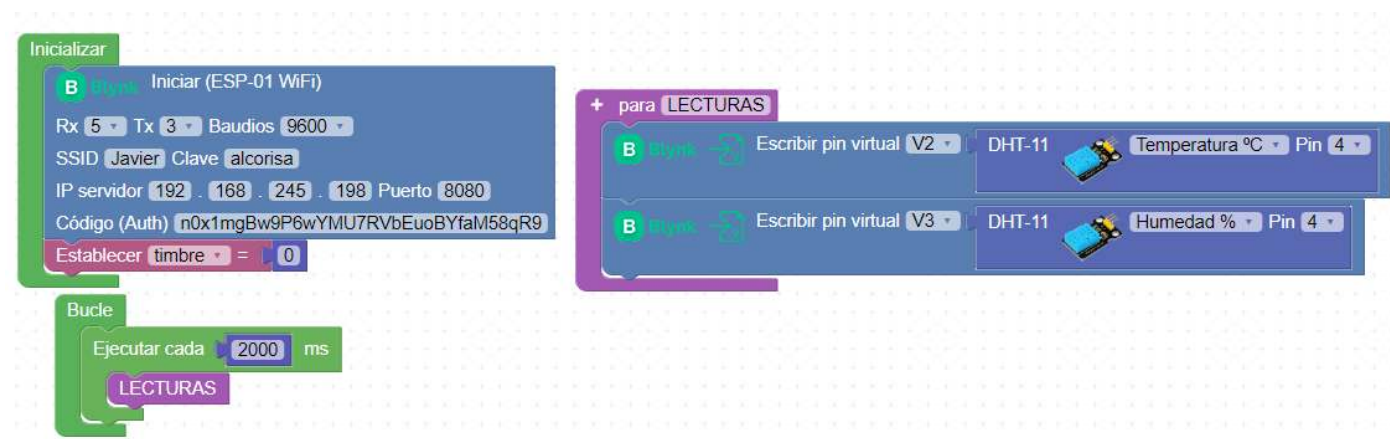
En ARDUINOBLOCKS

Vamos a ARDUINOBLOCKS y establecemos dos pines virtuales, uno para la temperatura y otro para la humedad, arbitrariamente he puesto V2 y V3 pero puede ser cualquiera.

- Para el kit de **ROVER MARCIANO CON ARDUINO** poner pin **D5**
 - Si el sensor es AZUL entonces DHT11 si el sensor es BLANCO entonces DHT22



- Para el kit que tenga la placa **TDR STEAM IMAGINA** es decir los cursos **ARDUINO BLOCKS EN EL AULA** y **ESP32 EN EL AULA** poner pin **D4**
 - Si el sensor es AZUL entonces DHT11 si el sensor es BLANCO entonces DHT22



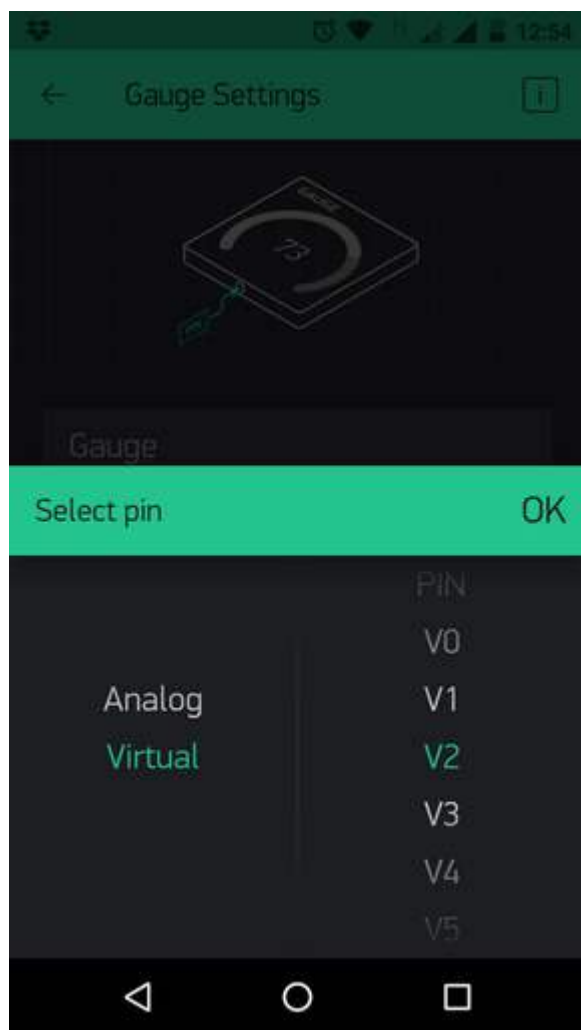
Fíjate que hay dos maneras de programar estos eventos:

- en el primero, utilizamos un bucle propio de Blynk que simplemente cada segundo lee el sensor y los almacenan los pines virtuales V2 y V3
- en el segundo no se ha utilizado ese bucle propio de Blynk sino simplemente dentro de Bucle hemos puesto otro bucle que se ejecuta cada 2 segundos, y dentro una funciona que se llama LECTURAS. dentro de lecturas esta la lectura de los pines virtuales V2 y V3

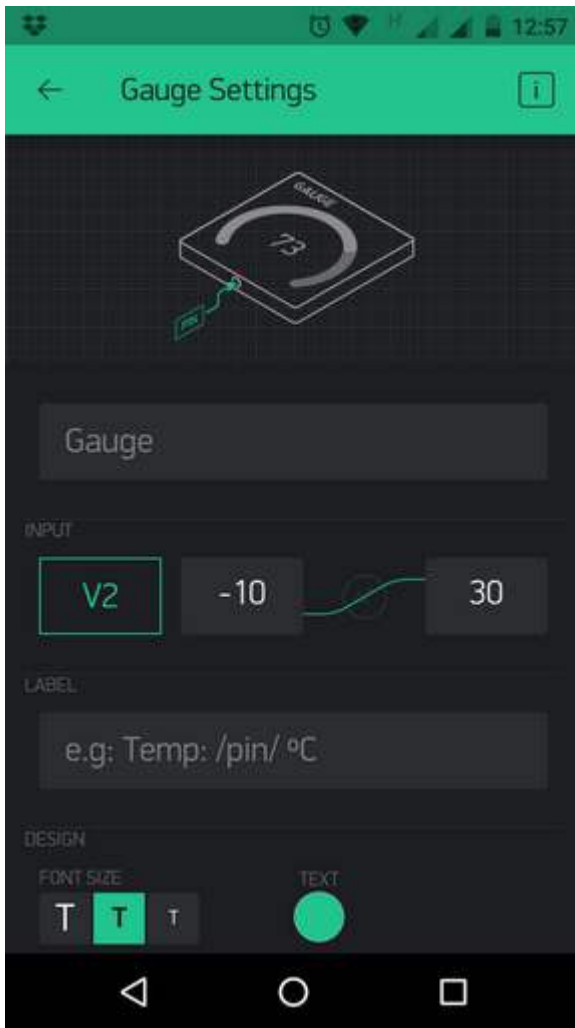
Da igual. Nosotros tenemos preferencia por el segundo método, nos parece más elegante y más controlable.

En la APP de Blynk

Y en Blynk incorporamos un Gauge que sea al PIN VIRTUAL V2.



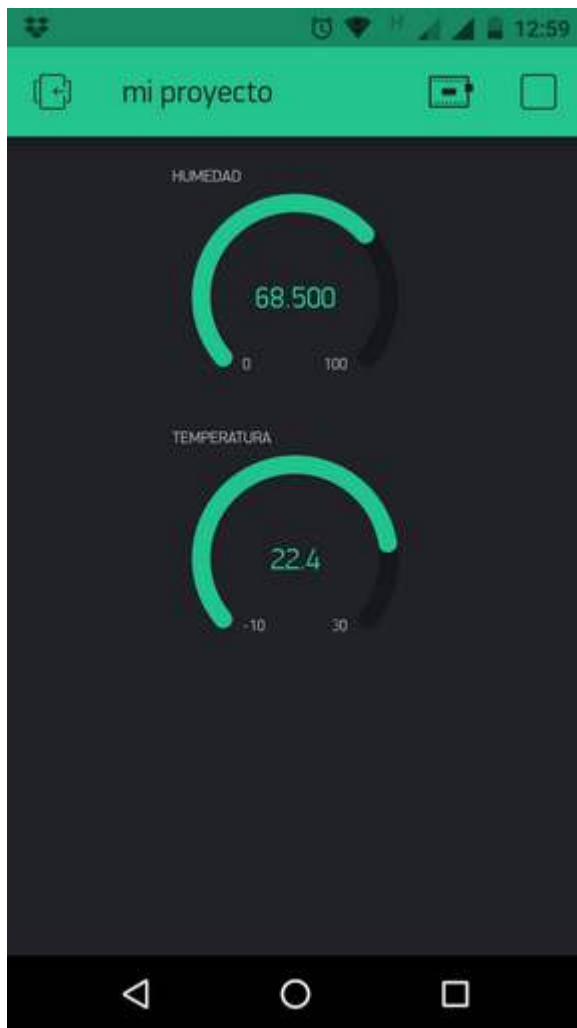
Modificamos también los límites, pues por defecto sale 0 a 1023 y se vería muy bajo la temperatura, ponemos -10 a 30. Si es para el Rover marciano de Arduino, allí se puede llegar a - 50°C y si es para el TDR Steam será en Aragón, el récord lo tiene [Fuentes Claras con -30°C](#).



Para la humedad hacemos lo mismo:

- Pin virtual V3
- Límites 0% a 100%

Resultado :



Avanzado: Blynk Legacy local

Blynk Legacy: 5 programa: Lectura sensor LM35 y receptor IR

Podríamos hacer una lectura de estos sensores, sin necesidad de tratamiento ninguno, tal y como hemos hecho en la lectura de la luz con LDR, simplemente se añade un gadget a la APP de Blynk que :

- Lea el pin D11 que es donde está el sensor de infrarrojos
- Lea el pin A2 que es donde está el sensor de temperaturas LM35

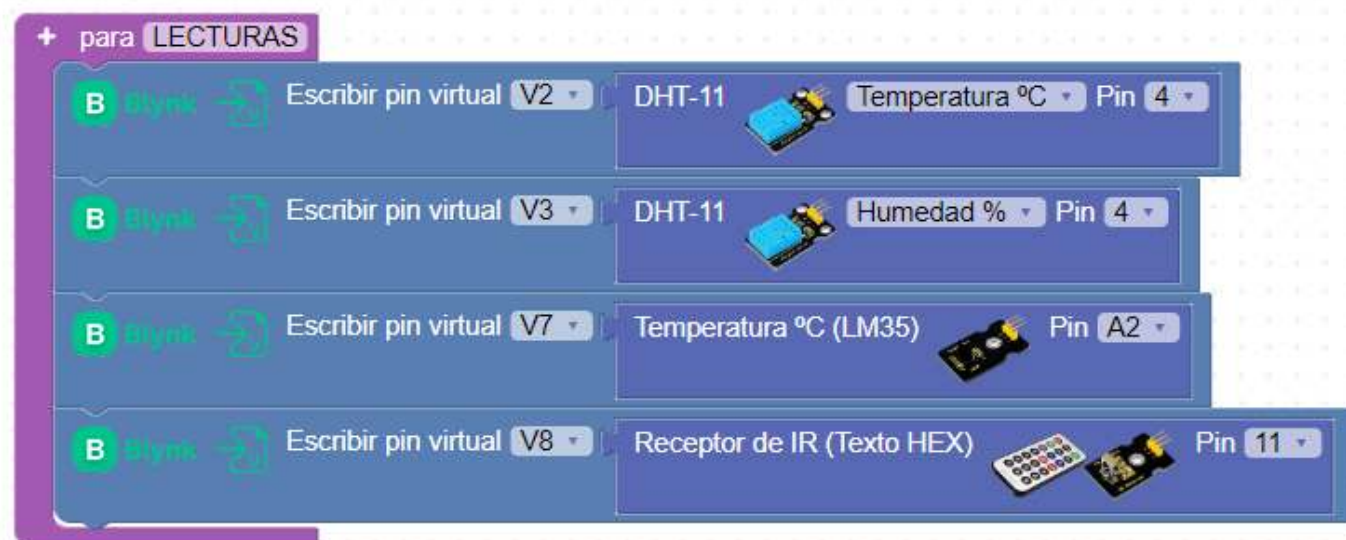
Los resultados serían totalmente incorrectos !!!

- El sensor de infrarrojos lanza un código que al leerlo el pin D11 en la APP aparecería encendidos y apagados sin poder leer qué código es lo que dice
- En el sensor de temperaturas, es un sensor conectado al A2 por lo que en la APP mediría desde 0 hasta 1023, y esto no sólo habría que mapearlo a temperaturas, sino tratarlo correctamente pues el LM35 da lecturas de dos decimales.

En **Arduinoblocks** tenemos dos bloques específicos para tratar estas lecturas, las llevamos a **pinos virtuales** y que los gadgets de la APP visualicen los valores de estos pines virtuales. Otro uso de los pines virtuales.

En Arduinoblocks

Ponemos dentro de la función LECTURAS los siguientes dos bloques :



Como vemos, Arduinoblocks procesa la lectura de estos dos sensores, y simplemente se almacenan en los pines virtuales V7 y V8

En la APP de Blynk

Ponemos para la lectura de la temperatura, un display que visualice el número V7



Para el sensor de infrarrojos igual pero que visualice V8



Lo **subimos** el programa Arduinoblocks al Arduino TDR Steam, pulsamos el **play** en la APP, esperamos a que se conecte y el sensor de temperatura muestra su valor perfectamente

Curiosamente dan unas lecturas algo diferentes el DHT11 y el LM35, el correcto es el LM35 pues el DHT11 no es un sensor muy preciso.

Para ver los códigos que se leen en Infrarrojos, hay que pulsar los números del mando a distancia.

Financiado por el Ministerio de Educación y Formación Profesional y por la Unión Europea - NextGenerationEU



Financiado por
la Unión Europea
NextGenerationEU



Plan de Recuperación,
Transformación
y Resiliencia



GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN
Y FORMACIÓN PROFESIONAL



GOBIERNO
DE ARAGON

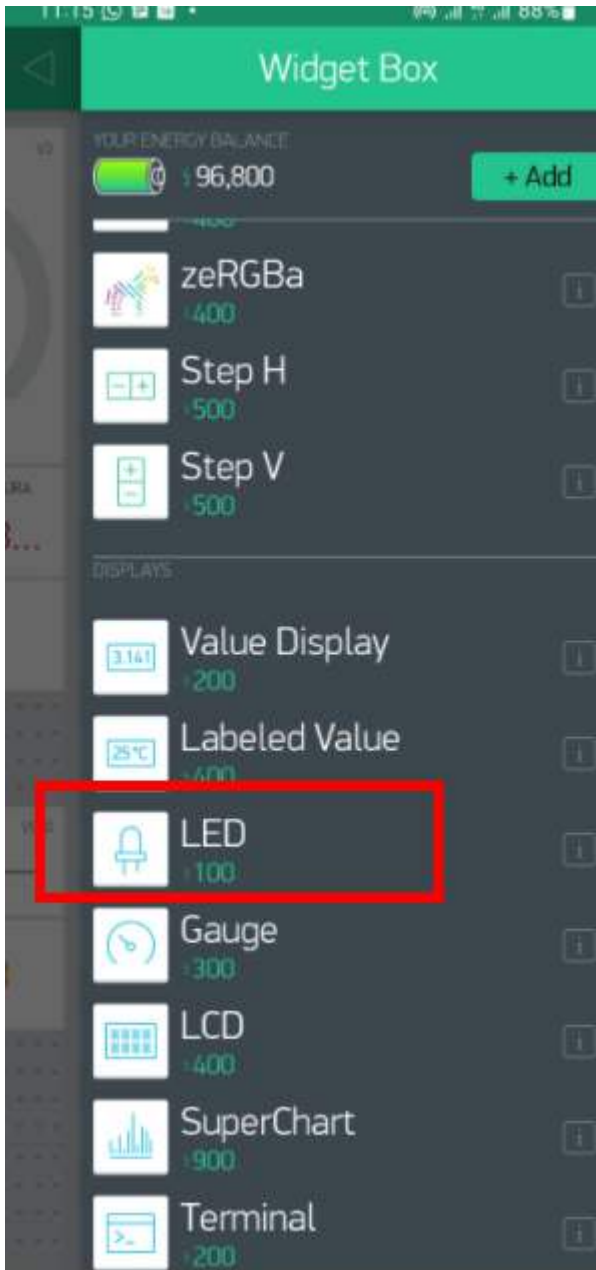
Avanzado: Blynk Legacy local

Blynk Legacy: 6 programa Leer eventos del TDR Steam

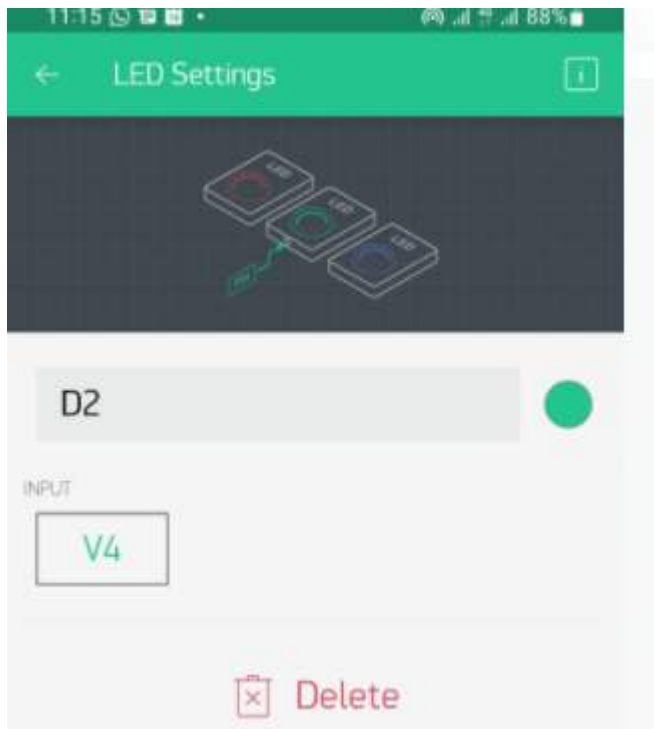
Hasta ahora hemos enviado eventos al TDR Steam, y leer sensores del TDR Steam pero ¿y leer un evento? por ejemplo leer si se pulsa los pulsadores D2 y D7

En la APP de Blynk

Añadimos un gadget que se llama LED

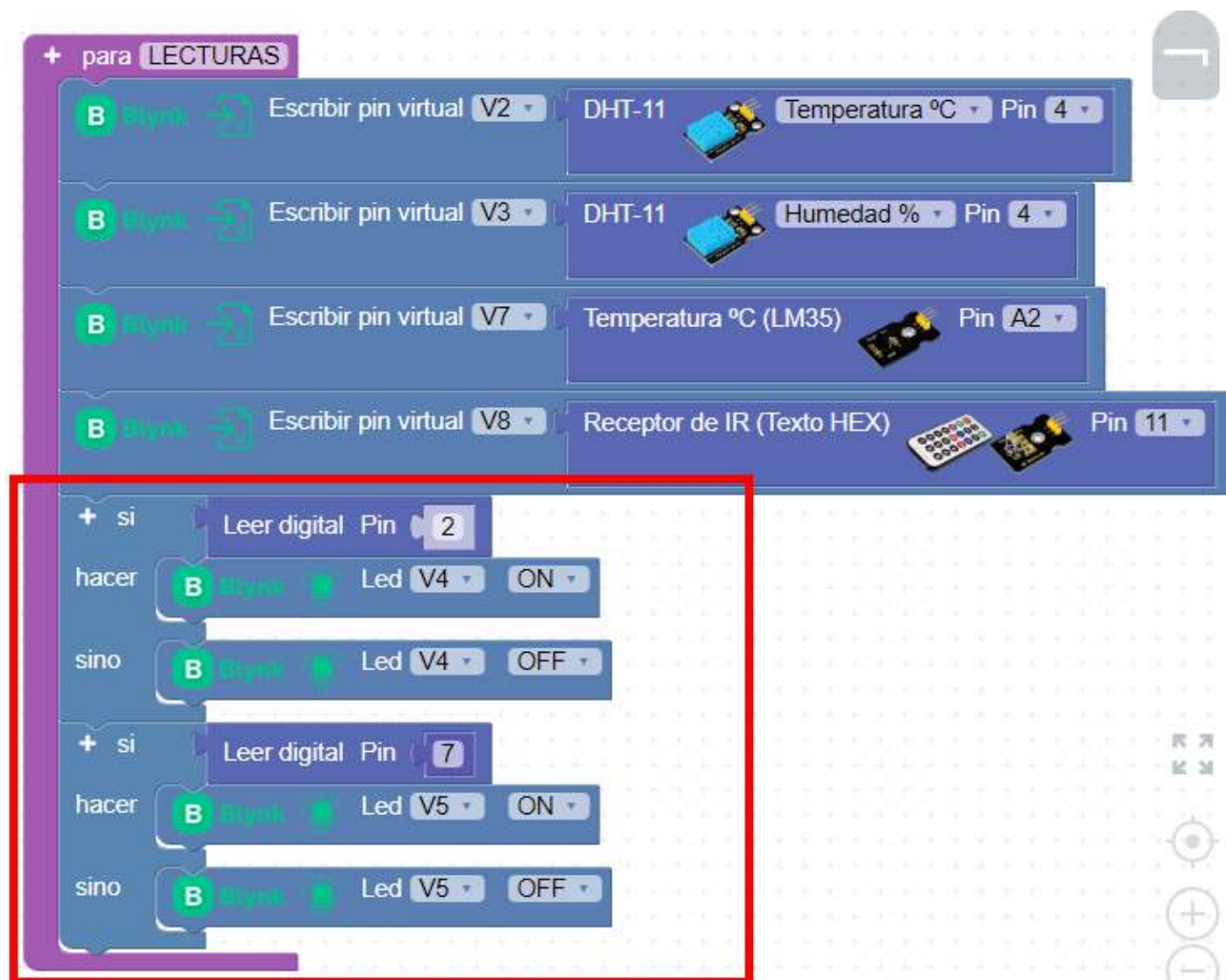


y leerá de un pin virtual pin V4 y otro gadget para V5



En ARDUINOBLOCKS

Pondremos el siguiente código, simplemente lee que si se pulsa pues que encienda el led en el APP



Financiado por el Ministerio de Educación y Formación Profesional y por la Unión Europea - NextGenerationEU



Financiado por
la Unión Europea
NextGenerationEU



Plan de Recuperación,
Transformación
y Resiliencia



GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN Y FORMACIÓN PROFESIONAL



GOBIERNO
DE ARAGON

Avanzado: Blynk Legacy local

Blynk Legacy: 7 programa: El timbre

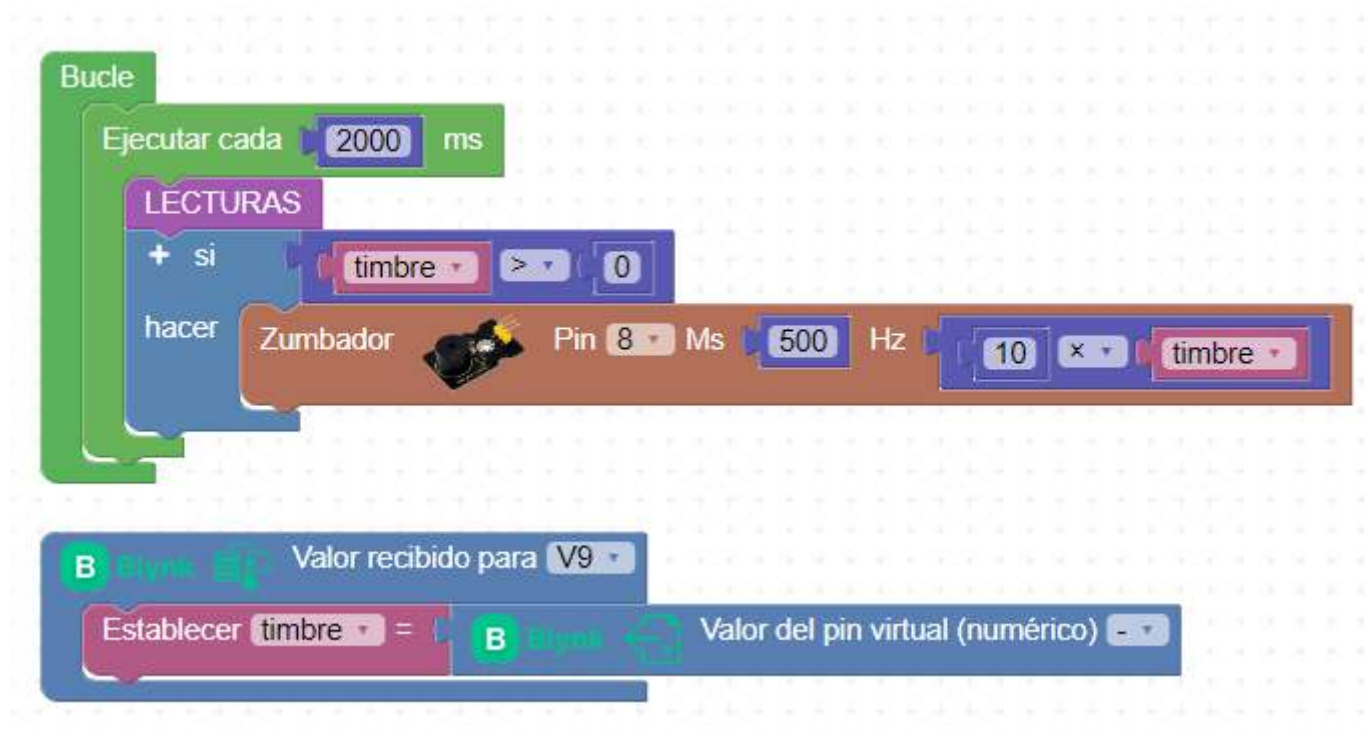
El timbre es algo especial, si ponemos una música, el Arduino está tan ocupado y el código engorda que da muchas desconexiones, si es el ESP32 lo aguanta bien.

En la APP ponemos un slider asociado a un pin virtual, por ejemplo a V9



Y en Arduinoblocks añadimos un bucle que lea ese pin virtual V9 y que lo almacene en una variable numérica, que lo hemos llamado *timbre*

Si esta variable, ha cambiado el valor, que haga un tono. si te fijas el valor máximo del slider se ha puesto a 2000, el máximo tono audible es 20.000, pero el slider no lo permite, por lo tanto, el truco es multiplicarlo por 10



Financiado por el Ministerio de Educación y Formación Profesional y por la Unión Europea - NextGenerationEU



Avanzado: Blynk Legacy local

Blynk Legacy: ¿Todo junto?

Si que se puede pero con la placa Arduino alguna vez se desconecta, pues no tiene un microprocesador muy potente. Con ESP32 sí que lo aguanta bien.

En la APP el dashboard o proyecto queda:

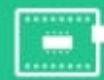
11:34



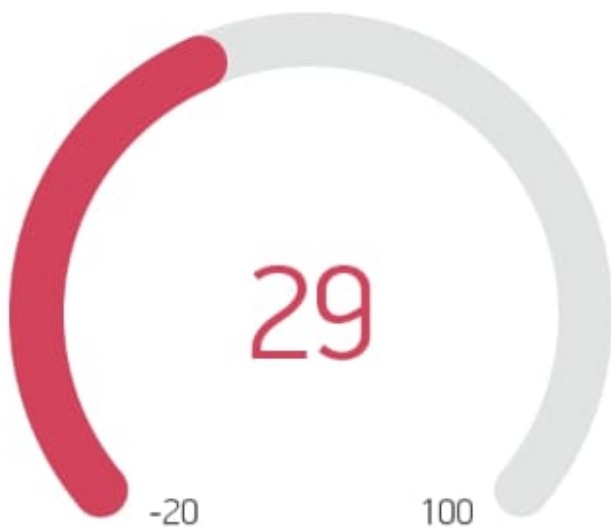
86%



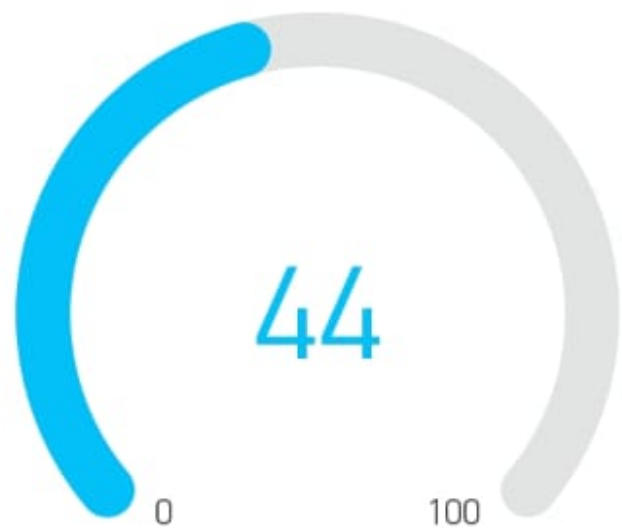
New Project



TEMPERATURA



HUMEDAD



BUTTON



BUTTON



D2



D7



TEMPERATURA

32.26

POTENCIOMETRO

327

ZERGBA

251

0

0

LUZ

821

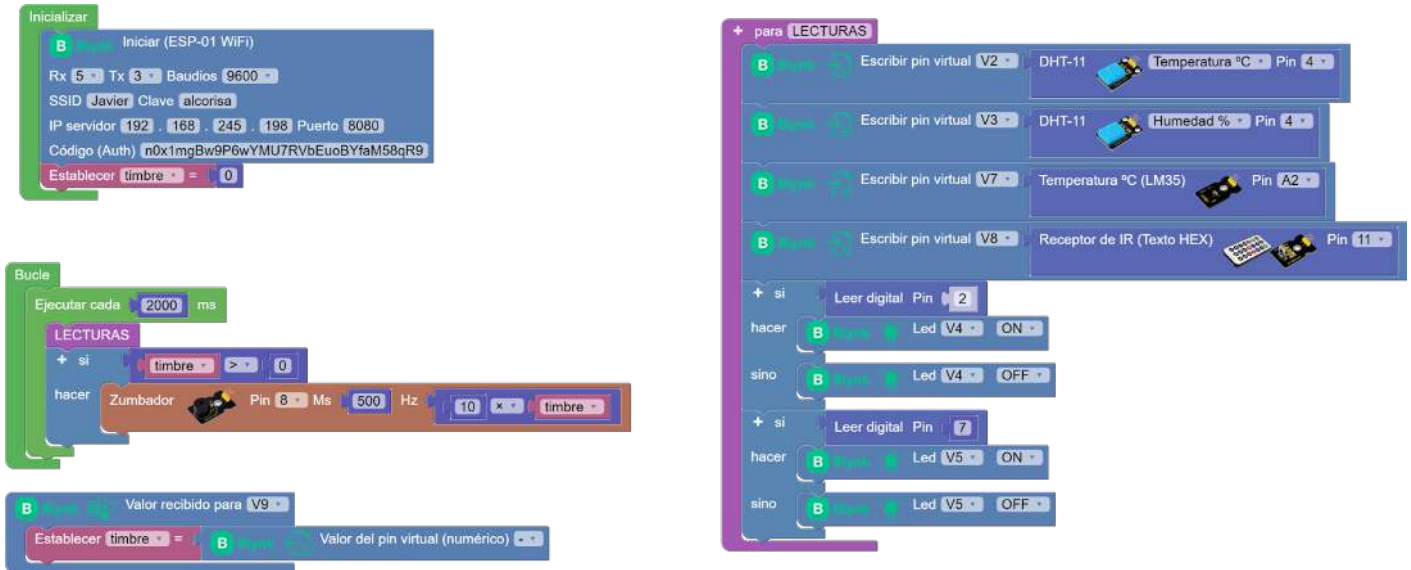


TIMBRE

0



Y en ARDUINOBLOCKS



Financiado por el Ministerio de Educación y Formación Profesional y por la Unión Europea - NextGenerationEU



Muro

<https://padlet.com/embed/nnk8myhkdoze1cn8>

Hecho con Padlet

Para saber más ...

PROYECTOS

<https://content.innovadidactic.com/>

PROGRAMAR ESP32 STEAMAKER TDR STEAM CON MICROBLOCKS

<https://pedroruizf.github.io/steamakersmicroblocks>



APP ARDUTALLER

App [Ardutaller WB Control](https://www.ardutaller.com.es/apps-by-ardutaller/manuales-app-ardutaller-wb-control) y manual en <https://www.ardutaller.com.es/apps-by-ardutaller/manuales-app-ardutaller-wb-control>

MALETA DE LA INNOVACIÓN 4.0 IOT

Tarjeta micro SD, HID, servidor HTTP, cliente HTTP, Alexa, Telegram Bot, Wifimesh ...

<https://drive.google.com/file/d/1XZrXIEV3WdbZwsugKRka3MhwACDjvYUg/view>



Maleta de la Innovación 4.0 IoT



Proyectos IoT con ESP32 STEAMakers y ArduinoBlocks



Documento resumen 6 sesiones formativas:

Repaso conceptos básicos
 Conexión modular
 Conexión WiFi: STA y AP
 Almacenamiento en tarjeta SD
 Bluetooth y BLE HID: teclado, ratón
 Protocolo MQTT: dashboards, Adafruit, ThingSpeak
 Protocolo HTTP cliente/servidor
 Dispositivo compatible Alexa
 Integración con Telegram Bot
 WiFiMesh

Formador:

Juanjo López
arduinoblocks@gmail.com

ESP32 PLUS STEAMMaker Imagina TDR STeam ArduinoBlocks

- <https://drive.google.com/file/d/11OVVUzdMhZLwt6nxMWmzSioArvtdOict/view>
- <https://fgcoca.github.io/ESP32-STEAMakers/>
- [Descarga desde el drive del auto tutorial Manual de Actividades ESP32 SteamMaker 2022 Junio](#)

Physical Computing

ESP32 Plus STEAMakers

Imagina TDR STEAM

ArduinoBlocks



Versión 4.0
(Junio 2022)



Carpeta con los programas, circuitos, etc... <https://drive.google.com/file/d/1HObvv4gBn-TB4pDG4W-JYIBScPXaXpMs/view>



-  ApplInventor2
-  ArduinoBlocks Projects
-  Esquemas Fritzing
-  fritzing.0.9.3b.64.pc
-  Manual Actividades ESP32 SteaMakers 20.

Arduino Free Book

Servidor HTTP, cliente HTTP, WifiMesh... https://docs.google.com/document/u/1/d/e/2PACX-1vQSrOKHpbLQHVBGFdAvp7DcndofoHDI20nvwGMaxu_7bGc1bUCmi4U6DZrJWRSudc2iXBg43QMuzCT/pub#h.35h2u46mzlaq

arduinoblocks - Free Book



Free Book

Juan José López Almendros

arduinoblocks@gmail.com

Manuales EducaconTic

En este manual, puedes encontrar prácticas de MQTT usando **THINGSPEAK** Y **THINGVIEW**

<https://sites.google.com/view/educacont/documentaci%C3%B3n/hist%C3%B3rico/manuales?pli=1>

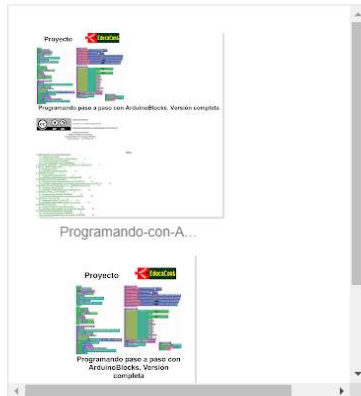


EducaCont

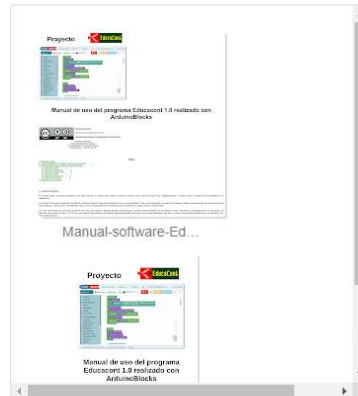
Inicio · Introducción · ¿CÓMO PARTICIPAR? · Documentación ▾ · LOS INICIOS

Manuales de programación y uso del software

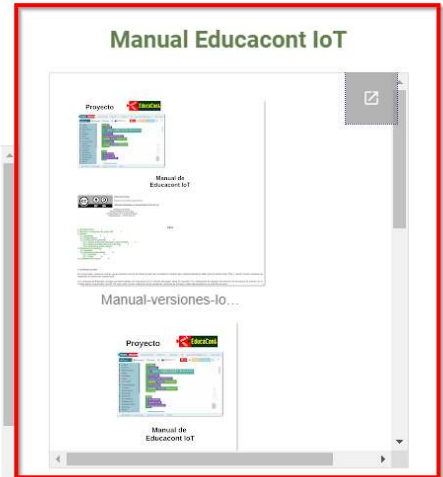
Programando paso a paso con ArduinoBlocks. Versión completa



Manual de utilización del programa creado con ArduinoBlocks



Manual Educacont IoT



Créditos

Autoría y licencias

- **Federico Coca** [GitHub](#) [Twitter](#) [Instagram](#) [Mas información](#).

Autor de los siguientes capítulos con licencia Creative Commons Atribución-

Compartir Igual 4.0 International ("[CC BY-SA](#)"). Ver Notas sobre ESP32 STEAMakers -

Miscelanea-Licencias <https://fgcoca.github.io/ESP32-STEAMakers/Miscelanea/lic/>

- **Prácticas con sensores digitales**
- **Prácticas con sensores analógicos**
- **Prácticas con otros elementos del kit**
- **Javier Quintana Peiró** CATEDU.
Maquetador de todo el libro y autor de los siguientes capítulos con licencia Attribution-NonCommercial-ShareAlike 4.0 International ([CC BY-NC-SA 4.0](#))
 - **Introducción**
 - **Bluetooth**
 - **IoT-Wifi-MQTT**
 - **IoT-Wifi - Blynk**
 - **IoT-Wifi - Blynk Legacy**
 - **Cómo montar un servidor Blynk Legacy Local**

Cualquier observación o detección de error en soporte.catedu.es



GOBIERNO DE ARAGON

Departamento de Educación,
Cultura y Deporte



CATEDU 

CENTRO ARAGONÉS de TECNOLOGÍAS para la EDUCACIÓN